
ANALYST TO ARCHITECT

How to Use This Book

How each chapter works, how to read the code and its output, the exercises and answers, how the parts climb, and where the companion files are.

Contents

How to Use This Book	2
Who this book is for, and how it's built	2
How each chapter is laid out	2
The four exercise groups, and the answers	3
How to read code and its output	4
How the parts climb	4
A rough sense of time	5
The companion files and a tidy folder	5

How to Use This Book

Who this book is for, and how it's built

This book starts from "what is data?" and ends with the work of a data architect: the person who decides how a whole company's data is collected, stored, checked, and put to use. It assumes no technical knowledge at all. If you can use a phone and add up a shopping bill, you can start at Chapter 1.

One company runs through every chapter: **Riverstone Supplies**, a fictional company that makes and sells plastic storage boxes, kitchenware, crates, and furniture to shops, hotels, and wholesalers. You'll meet its people, follow its orders from enquiry to cash, and answer its questions, first by hand, then with each new tool. Because the company stays the same, each new skill lands on a problem you already understand, and you can check a new answer against one you worked out before.

Tools arrive one at a time, in the chapter that first needs them. Parts 0 and I need no software at all. From Chapter 10 on, each chapter that brings a new tool starts by installing it and checking it works with one small first step. Every idea follows the same order: the plain idea first, then worked by hand, then in a tool you already know, and only then in a new tool, one line at a time.

How each chapter is laid out

Every teaching chapter has the same parts, in the same order, so you always know where you are.

- **Chapter at a glance.** A box at the top with five lines: *You will learn to* (what you'll be able to do by the end), *Before you start* (the chapters it builds on), *Time needed* (an honest estimate, including the exercises and the project), *Tools* (what you need, often only a pen and paper), and *Practice data* (the files the chapter uses).
- **Why this matters.** Why the chapter's skill is worth your hours, in a work setting.
- **In plain English.** The chapter's main idea through an everyday picture, such as a kitchen, a hospital, or a shop receipt, before any technical words.
- **Numbered sections** (1.1, 1.2, and so on). The teaching itself, one idea at a time, each worked through with Riverstone's data.
- **Boxes** inside the sections, each with a label that tells you what kind it is:
 - **Watch out:** a mistake that's easy to make, and how to avoid it.
 - **Try it:** a short task to do right now, before reading on.
 - **Simplification note:** where the book has simplified something on purpose, and what the fuller picture is.
 - **Real-life example:** the idea at work in a real kind of job or company.
 - **Tool note:** a detail about one tool or version, such as a feature that only some versions have.
 - **Interview extra point:** something that sets an answer apart in a job interview.

- **Common mistakes and how to spot them.** A table of the usual errors, the symptom each one shows, and the fix.
- **In the real world.** A short story from Riverstone in which someone uses the chapter's ideas on a real problem.
- **Tools.** The software and files the chapter used, and where to get them.
- **The project.** A larger piece of work that uses the whole chapter, with a clear goal and a deliverable.
- **You've got it when...** A checklist. If you can tick every line, you're ready for the next chapter.
- **Recap.** The chapter's main points in a few lines, to revise from.
- **Practice exercises.** Four groups of exercises (see the next section).
- **Key terms.** Every new term the chapter introduced. Use it to test yourself: can you explain each one out loud?
- **Where this leads.** Which later chapters build on this one, and which parts of the interview chapters it prepares you for.
- **Answers to practice exercises.** Worked answers to every exercise.

A few chapters, such as the capstones and the interview chapters, adapt this layout to their job. They say so at the start.

The four exercise groups, and the answers

Every teaching chapter ends with practice exercises in four groups:

Group	What it's for	How to use it
Warm-up	checks you understood the basics	do them right after reading; they should take a few minutes each
Core	the skills the chapter exists to teach	do all of them; this is where most of the learning happens
Stretch	harder, closer to real work	do at least one; come back to the rest on a second pass
Think about it	judgment, with no single right answer	write a short answer, then compare it with the model answer

Each chapter's answers follow its exercises. To get the most from them:

- **Write your answer before you look.** Even a partial answer. Reading an answer you haven't attempted feels like learning and isn't.
- **Compare the reasoning, not only the number.** Many answers show a method or a check (like "✓ reconciles to the total"); copy the method.
- **Mark the ones you got wrong or guessed,** and redo them on Sunday from a blank page.
- **For code exercises, run your version.** If it gives the same result by a different route, that's often fine; if it gives a different result, find out why before moving on.

How to read code and its output

Parts 0 and I contain no code. You'll work everything out by hand, with a pen, a calculator, and the tables on the page.

From Chapter 10 on, you'll meet **code**: short instructions you type for a spreadsheet, a database, or a programming language. The book shows code the way you'll use it, one small step at a time. Each step has three parts, always in this order.

1. The code block. The instructions you type, in a shaded box with a coloured bar down its left edge:

[Placeholder: from Chapter 10 on, the instructions you type appear in a box like this one.]

2. The output block. What the computer printed when that code was run, in a plain white box with a dashed border, directly below it:

[Placeholder: the output appears here, exactly as the computer printed it.]

3. How it works. The explanation, line by line: what each line does, and what each part of it means.

Four promises come with every code block.

- **The output is real.** Every block was run, and the output is what it actually printed. Nothing was typed in by hand.
- **One new idea per block.** A block adds one thing to what you already know, never five at once.
- **Every line is explained.** If a line appears, the text says what it does and what each part of it means.
- **Differences are shown side by side.** The book uses two databases, called PostgreSQL and MySQL (Chapter 12 introduces both). Where they need different code for the same job, you'll see both versions next to each other, labelled.

The best way to read a code block is to type it yourself, run it, and compare your output with the book's. If yours is different, the difference is worth finding before you move on: it's usually a typo, and sometimes it's a lesson.

How the parts climb

The book has nine parts and a closing chapter. Each part assumes the ones before it.

- **Part 0 — First Principles: Data from Zero** (Chapters 1–6). What data is, how computers store and move it, how a business runs on it, numbers without fear, thinking like an analyst, and planning your learning.
- **Part I — The Map** (Chapters 7–9). The data jobs, how skills unlock them, and how expertise forms.
- **Part II — The Analyst** (Chapters 10–27). Spreadsheets, SQL (the language for asking a database questions), cleaning data, charts, Power BI dashboards, the Python programming language, statistics, business skills, and a portfolio. **The end of Part II is where "job-ready" ends:** it covers the skills of a first analyst job.

- **Part III — Advanced Analytics & Analytics Engineering** (Chapters 28–34).
- **Part IV — Machine Learning & Data Science** (Chapters 35–44).
- **Part V — Data Engineering, Integration & Scale** (Chapters 45–52).
- **Part VI — Production ML, Generative AI & MLOps** (Chapters 53–59).
- **Part VII — Architecture, Governance & Leadership** (Chapters 60–67).
- **Part VIII — The Interview Playbook** (Chapters 68–82). How data hiring works, and question banks for each role.
- **Closing** (Chapter 83). The long game: what the whole path costs, and how to keep going.

Parts III to VII are branches, not a ladder you must climb to the top. Which ones you need depends on the job you want, and Chapter 8 shows which parts lead to which roles. Complete beginners should start at Chapter 1 and read in order. If you already work as an analyst, skim Parts 0 to II and start fully at Part III. Chapter 8, section 8.7, has a route for each role.

A rough sense of time

Adding up every chapter's *Time needed* line, the book's own estimates put the end of Part II at **317 to 394 hours**: at six hours a week, about 12 to 15 months. Chapter 6 turns that into a plan for your week.

The companion files and a tidy folder

The **companion files** hold every dataset, spreadsheet, and script the book uses. Each chapter's *Tools* and *Practice data* lines name the files that chapter needs, so you never have to guess. Appendix E gives the address to download them from and lists every file.

Two sets of Riverstone data run through the early chapters. The **mini database** has Riverstone's first three months of 2026: twelve orders, small enough to check by hand. The **one-year database** has all of 2025: 173 orders, big enough to show real patterns. Every table of Riverstone numbers in the book says which one it comes from.

Set up one home for everything on your computer, and keep it that way:

```
analyst-to-architect/
  companion/          the downloaded companion files, unchanged
  work/
    ch10/             your own files for each chapter
    ch12/
    ...
  notes/              your study log, plans, and questions
```

Two rules keep it useful. **Never edit the companion files themselves**: copy what you need into `work/` first, so you can always start again from a clean copy. And **name your own files the way Chapter 2 (section 2.4) will show you**: dates as year-month-day, lower-case words joined with hyphens or underscores, and no "final".

Three habits with files save hours. Chapter 2 explains the ideas behind them; set them up now.

- **Show file extensions.** An extension is the short ending of a file's name, such as `.csv` or `.xlsx`, that says what kind of file it is. Many computers hide it, so `orders.csv` appears as plain `orders`. Turn that off in your computer's file settings.
- **Know how to copy a file's full path.** A path is a file's complete address, from the drive down through each folder to the file. On Windows, hold `Shift`, right-click the file, and choose `Copy as path`. On a Mac, select the file and press `Option+Cmd+C`.
- **Unzip a download before opening what's inside it.** Opening a file directly from inside a zip file is a common cause of "my changes disappeared".

ANALYST TO ARCHITECT · PART 0

First Principles: Data from Zero

Chapters 1–6: what data is, how computers store and move it, how a business runs on it, numbers without fear, thinking like an analyst, and planning your learning.

Contents

- Part 0 — First Principles: Data from Zero 2
- Chapter 1. What Is Data? 3
- Chapter 2. How Computers Store, Move and Protect Data 25
- Chapter 3. How a Business Runs on Data 50
- Chapter 4. Numbers Without Fear 73
- Chapter 5. Thinking Like an Analyst 96
- Chapter 6. Planning Your Learning 117

Part 0 — First Principles: Data from Zero

Part 0 starts from nothing. It assumes no technical knowledge, no math beyond school arithmetic, and no tools. By the end of it, you'll understand what data is, where it lives, how a business creates and uses it, how to work with numbers confidently, how to think through a business question, and you'll have a plan for how and when you'll learn the rest.

Every example uses **Riverstone Supplies**, the fictional manufacturer you'll follow through the whole book, and every number was checked against its practice databases.

Before Chapter 1, read **How to Use This Book** at the front: it shows how each chapter is laid out, how to read the code and its output, and where the answers and companion files are.

Chapter	What you'll be able to do	Time needed
1. What Is Data?	tell data from information and insight; name types and levels of measurement; judge data quality	3–4 hours
2. How Computers Store, Move and Protect Data	work with files and formats from CSV to Parquet; explain databases, the cloud, and APIs; keep data safe	3–4 hours
3. How a Business Runs on Data	follow an order from enquiry to cash; tell bookings from billings from collections; define KPIs; find manual work	3–4 hours
4. Numbers Without Fear	calculate percentages, points, growth, and CAGR; choose the right average; read charts; estimate	4–5 hours
5. Thinking Like an Analyst	turn vague requests into precise questions; build issue trees; test hypotheses; spot bias; decide with data	3–4 hours
6. Planning Your Learning	estimate your hours honestly; set a weekly rhythm; know which chapter brings each tool; read documentation; learn with AI assistants; plan your first 90 days	2–3 hours

In total, allow 18–24 hours, including the exercises and projects. After Part 0, Part I (Chapters 7–9) shows the map of data careers, and Part II (Chapters 10–27) builds the analyst's toolkit.

Chapter 1. What Is Data?

Chapter at a glance

You will learn to: explain what data is, and how it differs from information, knowledge, and insight · turn everyday records, like a shop receipt, into rows and columns · name the type of any value (number, text, date, true/false) and spot numbers that aren't really numbers · tell qualitative from quantitative data, and discrete from continuous · use the four levels of measurement to decide which calculations make sense · recognize structured, semi-structured, and unstructured data · read and write metadata · say where data comes from · judge whether a small dataset is fit to use.

Before you start: nothing. This chapter assumes no technical knowledge at all.

Time needed: 3–4 hours, including the exercises and the project.

Tools: a pen and a notebook are enough. If you already have Excel or Google Sheets, you can use it for the project, but you don't need to.

Practice data: your own week of spending, plus small examples from Riverstone Supplies, the fictional company you'll follow through this book.

Why this matters

Every job in this book, from a junior analyst's first report to an architect's design for a whole company, is built on one raw material: **data**. Tools change every few years. Spreadsheets gave way to databases, databases to cloud warehouses, and now AI assistants can write the formulas for you. The raw material stays the same.

That's why this chapter comes before any tool. People who skip it can learn the buttons in Excel or the words in SQL, the language for asking a database questions, and still produce reports that are confidently wrong: an "average" of PIN codes, a customer counted twice because their name was typed in capitals, a sales total that silently left out every blank row. These mistakes aren't about software. They come from not asking basic questions about the data first: *What does each value mean? What kind of value is it? What calculations does it allow? Where did it come from? Can I trust it?*

By the end of this chapter you'll ask those questions automatically. It's a small habit, and it's the one that experienced analysts are quietly relying on every time they say, "Wait, that number doesn't look right."

In plain English

Picture the owner of a small grocery shop in your neighborhood. Next to the till is a thick notebook. Every time a regular customer buys on credit, the owner writes a line: the date, the customer's name, what they bought, and how much they owe. When the customer pays, another line goes in.

- Each **line** in that notebook is a small, recorded fact. That's **data**.
- At the end of the month, the owner adds up what each customer owes and sees that Mrs. Joshi owes ₹2,400. That total is **information**: data organized to answer a question.
- Over the years, the owner notices that customers who owe more than ₹3,000 often stop coming, because they feel embarrassed. That's **knowledge**: an understanding of *why* things happen.
- So the owner decides to send a friendly reminder whenever anyone's balance passes ₹2,500, before it becomes awkward. That's **insight** turned into action.

Now notice two more things about the notebook. First, the owner writes every line **the same way**: date first, then name, then items, then amount. That consistent layout is what makes adding things up possible. Second, some lines are hard to use: a smudged amount, a name written as "Joshi aunty" one day and "Mrs. Joshi" the next. That's a **data quality** problem, and every business, from this shop to the largest bank, has the same problem at a bigger scale.

Everything in this chapter is a version of that notebook.

Meet Riverstone. Riverstone Supplies is the fictional company you'll follow through this book. It makes and sells plastic storage boxes, kitchenware, industrial crates, and a small range of furniture to shops, hotels, and wholesalers. It has two plants, at Taloja and Chakan, and one warehouse, at Bhiwandi.

You'll practice on two of its datasets:

- **The mini database:** 12 orders from January to March 2026, small enough to check by eye. Chapters 1–5 and 12 use it.
- **The one-year database:** all of 2025, 173 orders plus 2 that were cancelled. You'll use it from Chapter 4 onward.

Every table and figure of Riverstone numbers says which of the two it comes from. These are the people you'll meet:

Who	Role	First met in
Meera Iyer	sales coordinator	Chapter 1
Anita Rao	Sales Head	Chapter 1
Vikram Singh	Sales Manager	Chapter 3
Neha Kulkarni, Rahul Mehta, Farah Khan	sales executives	Chapters 1–2
Suresh Menon	Finance Manager	Chapter 3
Imran	ran sales operations before Meera joined	Chapter 2

1.1 What data actually is

Data is a recorded observation about the world. Something happened, someone or something noticed it, and a note was made that can be looked at later.

Three parts of that definition matter:

1. **Observation.** Data describes something: a sale, a temperature, a step, a click, an opinion.
2. **Recorded.** A thought in your head isn't data until it's written, typed, photographed, or saved. The recording is what lets other people, and computers, use it.
3. **Later use.** Data is kept because someone expects to look at it again: to add it up, compare it, check it, or prove something.

You create and use data all day, usually without calling it that:

Everyday thing	The data in it
A shop receipt	what you bought, how many, at what price, when, and how you paid
Your phone's step counter	a count of steps for each hour and each day
A train timetable	train numbers, stations, and arrival and departure times
A class attendance register	each student, each date, present or absent
A food-delivery app	the restaurant, the items, the delivery address, the time taken, your rating
A photo on your phone	the picture itself, plus the date, time, and often the place it was taken

In the singular, one recorded value is sometimes called a **datum**, but almost everyone, including this book, simply says "data" for one value or many.

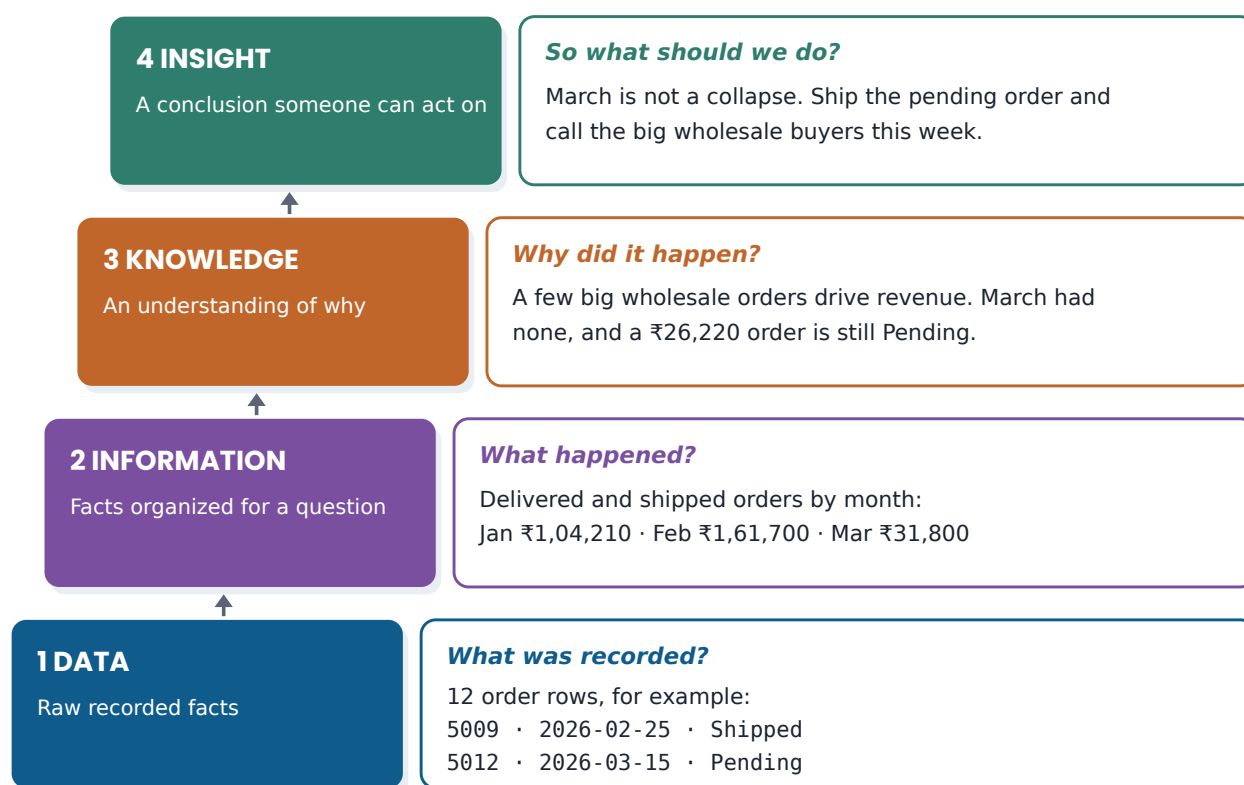
Try it. Look at your phone for thirty seconds. Name five kinds of data it recorded about you today without you typing anything. (Some ideas: steps, screen time, battery use, location, photos, missed calls.)

1.2 From data to information, knowledge, and insight

People often use *data* and *information* as if they mean the same thing. In data work, the difference matters, because your job is to move up from one to the next. A common way to picture it is a four-step ladder, sometimes called the **DIKW** model (data, information, knowledge, wisdom). This book uses *insight* for the top step, because that's the word businesses use.

Step	What it is	The question it answers
Data	raw recorded facts	What was recorded?
Information	data organized, summarized, or put in context	What happened?
Knowledge	an understanding of patterns and causes	Why did it happen?
Insight	a conclusion that points to an action	So what should we do?

Here's the ladder with real numbers from Riverstone's mini database.



Each step up needs more context and more thinking, and is worth more to the business.

Figure 1.1 — The same twelve orders at four levels of usefulness. Mini database (Jan–Mar 2026).

- **Data.** Riverstone's system holds twelve orders from January to March 2026. Each has an order number, a customer, a date, and a status such as *Delivered*, *Shipped*, or *Pending*. On their own, twelve rows tell a manager nothing.
- **Information.** Add up the value of delivered and shipped orders for each month: January ₹1,04,210, February ₹1,61,700, March ₹31,800. Now there's a clear message: March fell by 80% compared with February.
- **Knowledge.** Look closer and you learn *why*. Most of Riverstone's revenue comes from a few big wholesale orders, and none were placed in March. One more March order, worth ₹26,220, exists but is still *Pending*, so it isn't counted yet.
- **Insight.** So March is not the collapse the headline number suggests. The useful actions are to ship the pending order before the quarter closes, and to call the big wholesale buyers this week to find out why they haven't reordered.

Notice what made each step possible. Information needed a **question** ("how much did we sell each month?") and a **rule** ("count only delivered and shipped orders"). Knowledge needed a closer look at **detail and context**. Insight needed **judgment about the business**. Tools help most with the first step; your value as an analyst grows as you climb.

Watch out: a number is not an insight. "March revenue: ₹31,800" is information. Reports that stop there leave the reader to guess what it means, and people often guess wrong. Add the "why" and the "so what" before you send it.

1.3 Records, fields, and datasets

Data becomes easy to use when it's arranged in a **table**: a grid of rows and columns. Almost every tool in this book, from spreadsheets to databases to Python, a programming language, works with tables.

Here's a receipt from a neighborhood store, and the same receipt as a table.

SAI KRUPA GENERAL STORE		
Kothrud, Pune		
Bill No: 4417	14-09-2026 18:42	

Toor dal 1 kg	1 × 165	165
Basmati rice 5 kg	1 × 540	540
Milk 500 ml	4 × 28	112
Bath soap	3 × 45	135
Tea 250 g	1 × 140	140

TOTAL	₹1,092	
Paid by: UPI		
Thank you! Visit again		

What happens to the receipt

- Each item line on the receipt (shaded) becomes one row: a record.
- Each kind of detail becomes one column: a field.
- The bill number, date and payment method repeat on every row.
- The total ₹1,092 is not a row: it can be calculated from the amount column.



The same bill as a table: one row per item

bill_no, bill_date and paid_by repeat on every row

bill_no	bill_date	item	qty	unit_price	amount	paid_by
4417	2026-09-14	Toor dal 1 kg	1	165	165	UPI
4417	2026-09-14	Basmati rice 5 kg	1	540	540	UPI
4417	2026-09-14	Milk 500 ml	4	28	112	UPI
4417	2026-09-14	Bath soap	3	45	135	UPI
4417	2026-09-14	Tea 250 g	1	140	140	UPI

Figure 1.2 — Each line on the receipt becomes a row; each kind of detail becomes a column.

The vocabulary:

- A **row** (also called a **record**) holds the details of one thing. Here, one row is **one item on one bill**.
- A **column** (also called a **field**, **variable**, or **attribute**) holds one kind of detail for every row: `item`, `qty`, `unit_price`.
- A **value** is what sits where a row and a column meet: 4 in the `qty` column of the milk row.
- A **dataset** is a collection of related data. It might be one table (every bill from the shop this year) or several tables that belong together.

Two ideas from this small example will stay with you for the whole book.

1. Decide what one row means. In Figure 1.2, one row is one *item* on a bill, not one *bill*. That's why `bill_no`, `bill_date`, and `paid_by` are repeated on all five rows: each row needs to stand on its own. The level of detail one row represents is called the **grain** of a table. When two people disagree about a number, they're very often counting at different grains: "5 sales" (items) versus "1 sale" (bill).

2. Store the ingredients, calculate the results. The receipt prints a total of ₹1,092, but the table has no "total" row. You can always calculate it: $165 + 540 + 112 + 135 + 140 = ₹1,092$. A total typed in as a row would get added in again by anyone who sums the column, and it would be wrong the moment someone corrected a line. Keep the detailed facts; calculate summaries when you need them.

Two more habits make a table easy to work with:

- **One kind of fact per column.** A column called `item` shouldn't also hold "Toor dal 1 kg, 2 packets" with the quantity mixed into the text. Split it: `item` and `qty`.
- **One value per cell.** "UPI + cash" in one cell can't be added up. If a bill was split between two payment methods, that's two payment rows.

Try it. Take any receipt from your wallet or your email. Write its first three lines as rows of a table with the same seven columns as Figure 1.2. What is the grain of your table?

1.4 Kinds of values: data types

Every column holds one **type** of value. The type tells you, and the computer, what you can do with it. Four types cover almost everything you'll meet:

Type	Holds	Examples	You can...
Number	amounts and counts, whole or with decimals	4, 28, 1450.50	add, average, compare
Text (also called a string)	letters, words, codes	Toor dal 1 kg, UPI, Pune	sort alphabetically, search, group
Date and time	a moment or a calendar day	2026-09-14, 18:42	sort in time order, find days between, group by month
True/false (also called Boolean)	yes-or-no facts	TRUE, FALSE	count how many are true, filter

Getting the type right matters more than beginners expect.

Numbers that aren't numbers. A PIN code like 411038 is made of digits, but you would never add two PIN codes together or calculate an average one. The same goes for phone numbers, employee codes, bank account numbers, and bill numbers. They're **identifiers** or **codes**, and they should be treated as text. Store a phone number as a number and a spreadsheet may drop its leading zero (022... becomes 22...) or display a long number as 9.82E+09. **Rule of thumb: if you'd never do arithmetic with it, it's text.**

Dates stored as text. 03/04/2026 means 3 April in India and the UK, but 4 March in the United States. A computer that reads it as text can't tell, and it will sort 10/01/2026 before 9/01/2026, because the character 1 comes before 9. The safest way to write a date is **year-month-day**, 2026-04-03, which sorts correctly and means the same thing everywhere. It's an international standard (ISO 8601), and it's how dates are written in this book's data.

Missing values. Sometimes a value simply wasn't recorded. A blank is not the same as zero: a blank "discount" might mean *no discount* or *nobody wrote it down*. Databases use a special marker, **NULL**, for "unknown", and Chapter 12 shows how it trips up calculations. For now, whenever you see a blank, ask: **does this mean zero, "not applicable", or "unknown"?**

Watch out: the type you see isn't always the type that's stored. A spreadsheet can show 14-09-2026 while storing it as text, and then refuse to sort or group it by month.

1.5 Qualitative and quantitative data

A second way to describe data is by what it measures.

Quantitative data is measured or counted in numbers, and the numbers mean an amount: 4 packets of milk, ₹540, 12,000 steps, 38.5 °C.

Qualitative data (also called **categorical data**) describes a quality or puts things into groups: the payment method (UPI, card, cash), a city, a product category, a customer's comment, "satisfied" or "not satisfied".

Quantitative data splits once more:

- **Discrete** data can only take separate, countable values, usually whole numbers: the number of orders, children, or items in a cart. You can have 3 orders or 4, but not 3.7.
- **Continuous** data can take any value in a range, limited only by how precisely you measure: weight, height, time taken, temperature. A parcel can weigh 2.4 kg, 2.43 kg, or 2.4318 kg.

Money is technically discrete (you can't pay a fraction of a paisa), but because it has so many possible values, it's usually treated as continuous in analysis.

Column from the receipt	Qualitative or quantitative	If quantitative
item	qualitative	—
qty	quantitative	discrete
unit_price	quantitative	continuous (in practice)
paid_by	qualitative	—
bill_no	qualitative (it's a label, not an amount)	—

This isn't just labeling. It decides which **chart** fits (a bar chart for categories, a histogram for continuous amounts) and which **summary** makes sense (a count of each payment method, but an average of prices).

1.6 Levels of measurement: which math is allowed

Not all numbers support the same calculations. A useful way to think about this is the four **levels of measurement**, described by the psychologist S. S. Stevens in the 1940s and still taught in every statistics course. Each level can do everything the level before it can, plus something new.

Nominal Level 1	Names or labels, no order <i>e.g. segment, city, payment method</i> Allows: count · most common value (mode)
Ordinal Level 2	Order matters, gaps are unknown <i>e.g. T-shirt size, rating 1-5, High/Medium/Low</i> Adds: rank · median · "higher than"
Interval Level 3	Equal gaps, but no true zero <i>e.g. temperature in °C, calendar dates</i> Adds: differences · mean
Ratio Level 4	Equal gaps and a true zero <i>e.g. revenue, quantity, weight, age</i> Adds: "twice as much" · % change

Each level keeps every calculation of the level before it and adds one more.

Figure 1.3 – From labels to true amounts: each level unlocks more calculations.

1. Nominal: names and labels. The values are categories with no natural order: city, product category, payment method, customer segment. You can **count** how many are in each group and find the most common one (the **mode**). You can't sort them in any meaningful way. Even when nominal data uses digits, like PIN codes or jersey numbers, the digits are just labels.

2. Ordinal: order matters, but the gaps don't. The values have a clear order, but the distance between them isn't defined: T-shirt sizes (S, M, L, XL), priority (Low, Medium, High), exam rank, or a satisfaction rating from 1 to 5. You can say one is **higher** than another and find the **median** (the middle value). But the gap from "Poor" to "Average" isn't necessarily the same as the gap from "Good" to "Excellent".

3. Interval: equal gaps, but no true zero. The distance between values is meaningful, but zero doesn't mean "none". Temperature in degrees Celsius is the classic example, and calendar dates are another. The **difference** between two values makes sense (it's 20 degrees warmer; the invoice is 30 days late), and so does the mean. But **ratios** don't: 0 °C isn't "no temperature", so 40 °C is not "twice as hot" as 20 °C.

4. Ratio: equal gaps and a true zero. Zero means *none* of it: revenue, quantity, weight, distance, age, time taken. Every calculation works here, including "twice as much" and percentage change. ₹2,000 of sales really is twice ₹1,000, and ₹0 really means no sales.

Most business amounts are ratio data, so most of the time every calculation is allowed. The levels matter at the edges, and those edges are where reports go wrong.

Worked example 1: the average rating that hides the story

Riverstone asks customers to rate two delivery partners from 1 (very poor) to 5 (excellent). Each partner receives five ratings:

Partner	Ratings	Mean	Median
Swift Movers	3, 3, 3, 3, 3	3.0	3
Rapid Wheels	5, 5, 5, 1, 1	3.4	5

Check the means by hand: $15 \div 5 = 3.0$, and $17 \div 5 = 3.4$. ✓

On the mean, Rapid Wheels looks slightly better. But look at the ratings themselves. Swift Movers is consistently average. Rapid Wheels delights most customers and badly fails two out of five, which, for a supplier delivering to hotels, might mean a banquet with no plates. The mean treats the gap between 1 and 5 as four equal steps, which a rating scale doesn't promise. For ordinal data, the **median** and the **count in each category** ("two of five customers gave a 1") are more honest summaries.

Worked example 2: twice as hot?

Riverstone's warehouse was 20 °C in the morning and 40 °C in the afternoon. A report says it was "twice as hot". It wasn't: 0 °C isn't "no heat", so Celsius numbers can't be divided like that. The honest sentence is "*the temperature rose by 20 degrees*".

Worked example 3: dates

An invoice dated 10 March and due on 9 April: you can subtract the dates to get 30 days, a perfectly meaningful difference. But "20 March is twice 10 March" is nonsense. Dates are interval data.

Watch out: averaging codes. Put a column of PIN codes or employee IDs into a spreadsheet and it will happily calculate their average. The result is meaningless. Before you summarize any column, ask which level of measurement it is.

1.7 Structured, semi-structured, and unstructured data

The last way to describe data is by its **shape**: how organized it is when you receive it. Here's the same Riverstone order, for 20 storage boxes and 50 water bottles, in three shapes.

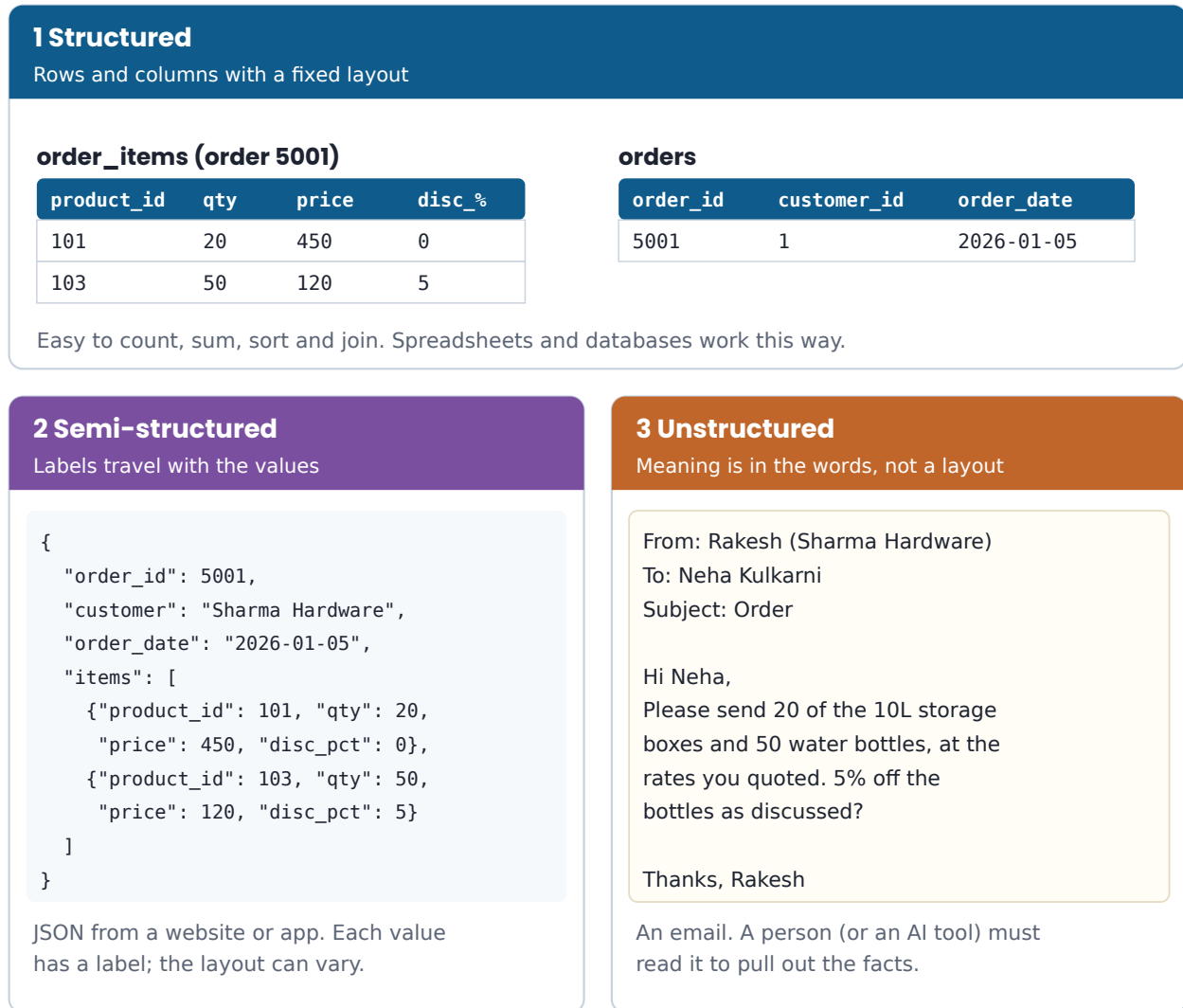


Figure 1.4 – Same facts, three shapes. The less structure, the more work before you can count anything. Mini database (Jan–Mar 2026), order 5001.

Structured data fits a fixed layout of rows and columns, where every row has the same fields and every column has one type. Sales tables, bank statements, attendance registers, and stock lists are structured. It's the easiest to add up, sort, filter, and combine, and it's what spreadsheets and databases are built for. Much of a data analyst's day is spent with structured data.

Semi-structured data has labels that travel with the values, but no fixed table layout. The most common format is **JSON** (say "jay-son"), which websites and apps use to send data to each other. In the middle panel of Figure 1.4, every value has a name ("qty": 20), and the `items` list can hold one item or fifty. Two orders don't have to have exactly the same fields. Other examples are XML files and the logs that apps write. Semi-structured data is easy for computers to read, and usually needs a step of **flattening** into tables before analysis. Chapter 2 introduces these formats.

Unstructured data has no predefined layout at all. The meaning is in the words, pixels, or sounds: emails, WhatsApp messages, PDF contracts, product photos, call recordings, customer reviews. A person can read Rakesh's email in Figure 1.4 and understand the order instantly. A computer has to work out that "the 10L storage boxes" means product 101, and that "the rates you quoted" means ₹450

and ₹120. Most of the information inside organizations is unstructured, and for a long time most of it went unused. Modern AI tools have made it much easier to pull structured facts out of unstructured text, and that's one of the big changes in data work in recent years.

Shape	Examples
Structured	spreadsheets, database tables, CSV files
Semi-structured	JSON, XML, app and website logs
Unstructured	emails, PDFs, images, audio, free-text reviews

Real-life example: the order that arrives by email. In many companies, including plenty of manufacturers and distributors, customers still send orders by email or WhatsApp, and someone re-types them into the billing system. Each re-typing is a chance for a mistake: 50 bottles becomes 500, or the discount is missed. Turning that unstructured message into a structured order automatically is a classic automation project.

1.8 Metadata: data about data

Metadata is data that describes other data. Your phone already shows you plenty:

- A photo's metadata includes the date, time, camera model, and often the GPS location where it was taken.
- A file's metadata includes its name, size, type, and when it was last changed.
- A song's metadata includes the title, artist, album, and length.

In data work, the most important kind of metadata is a **data dictionary**: a short document that explains what each column in a dataset means. Here's one for the receipt table from Figure 1.2:

Column	Type	Meaning	Allowed values or format	Can be blank?
bill_no	text	the number printed on the bill	digits only	no
bill_date	date	the date the bill was printed	YYYY-MM-DD	no
item	text	the product name as printed	any text	no
qty	number (whole)	how many units were bought	1 or more	no
unit_price	number (₹, 2 decimals)	price of one unit, before any discount	more than 0	no
amount	number (₹, 2 decimals)	qty × unit_price	more than 0	no
paid_by	text	how the bill was paid	UPI, Card, Cash	no

It looks like a small thing. It's one of the most valuable things a data team produces. Without it, people guess: Is *amount* before or after discount? Does "date" mean the order date or the delivery date? Is the price in rupees or thousands of rupees? Guesses become arguments, and arguments become two

departments reporting different numbers for the same month. Other useful metadata includes **where the data came from**, **when it was last updated**, and **who owns it**, meaning who to ask when something looks wrong.

Try it. Pick one column from any table you use at work or college. Could a new colleague understand exactly what it means from its name alone? If not, write the one-line meaning they'd need.

1.9 Where data comes from

Every piece of data was created by someone or something. Knowing the source tells you a lot about how far to trust it.

Source	How it's created	Examples	Typical problems
People	typed, written, selected, or spoken	order forms, surveys, a salesperson updating a CRM (the sales team's contact system), the shop owner's notebook	typos, blanks, inconsistent spelling, fields filled in "just to get past the screen"
Machines and sensors	measured automatically	step counters, temperature sensors on a production line, GPS in delivery vans, electricity meters	faulty or drifting sensors, gaps when a device goes offline, huge volumes
Business systems	recorded as a side effect of doing work	billing software, a website's shopping cart, a bank's payment system, attendance swipes	only as good as the process around them; changes when the system is upgraded
Outside sources	collected by someone else	government statistics, weather data, market prices, data bought from partners	different definitions, delays, unclear collection methods

Two more distinctions you'll hear:

- **Primary data** is collected by you or your organization for your own purpose, like Riverstone's order records or a survey you run. **Secondary data** was collected by someone else for another purpose, like census figures. Secondary data is quicker to get, but you have to check that its definitions match your question.
- **First-party data** is what an organization collects directly from its own customers and operations. **Third-party data** is bought or obtained from outside. The terms come up often in marketing.

Notice where the problems cluster. **Whenever a person types data by hand, and especially when the same data is re-typed from one place into another, errors creep in.** Data captured automatically, at the moment something happens, is usually more complete and more consistent. That single observation explains why so much of a data team's work, and a whole thread of this book from Chapter 3 to the architecture chapters, is about capturing data once, at the source, and letting it flow automatically to reports, dashboards, and other systems.

Watch out: personal data. Some data describes people: names, phone numbers, addresses, health details, salaries, locations. Most countries now have laws about how it may be collected, stored, and shared, including India's Digital Personal Data Protection Act, 2023, and the European Union's GDPR. As a beginner, follow two rules: never copy personal data from work to your own devices or accounts, and use only what you need for the task. Chapter 64 covers privacy and governance properly.

1.10 Data quality in one page

Data is **fit for use** when it's good enough for the decision it supports. A rough count of website visitors can be a little off; the amount on a tax invoice can't. Data quality is usually judged on six dimensions:

Dimension	Question to ask	Example of a problem
Accuracy	Does the value match reality?	a signup date in the year 2062
Completeness	Is anything missing?	a customer with no city
Consistency	Is the same thing recorded the same way everywhere?	"Mumbai", "mumbai", and "Bombay"
Validity	Does the value follow the agreed format and rules?	an email address with no @
Uniqueness	Is each real thing recorded only once?	the same customer entered twice
Timeliness	Is it up to date for the decision?	a stock report from last month used to promise delivery today

Here's a small extract from a customer list that was kept by hand in a spreadsheet for a few months. Look at it for a minute before reading on.

customer_name	city	email	signup_date
Sharma Hardware	Mumbai	sharma.hardware@example.com	2025-11-04
Metro Mart	mumbai	metromart@example.com	2026-01-22
METRO MART	Mumbai	METROMART@EXAMPLE.COM	22/01/2026
Coastal Foods	Chennai	coastalfoods.example.com	2025-10-20
Sunrise Caterers			2026-02-03
Western Logistics	Bombay	westernlogistics@example.com	2062-04-01

There are at least six problems, one or more for every dimension except timeliness:

1. **Uniqueness.** Metro Mart appears twice, once in capitals. It's the same customer, with the same email address in different letters.
2. **Consistency.** Mumbai is written three ways: "Mumbai", "mumbai", and "Bombay", the city's former name. (The third row also has an invisible space after "Mumbai", which you can't see here but a computer can.)
3. **Validity.** The third row's date is written 22/01/2026 while every other row uses year-month-day.
4. **Validity.** Coastal Foods' email address has no @, so no email will ever reach it.
5. **Completeness.** Sunrise Caterers has no city and no email.
6. **Accuracy.** Western Logistics signed up on 1 April **2062**, forty years in the future. Someone almost certainly meant 2026.

Now try a simple question: **how many customers are in Mumbai?** A computer that matches the exact text "Mumbai" finds **one**: Sharma Hardware. ("mumbai" has a small *m*, "Bombay" is different text, and the third row's "Mumbai" has a trailing space.) The real answer is **three different customers**: Sharma Hardware, Metro Mart, and Western Logistics. And for Sunrise Caterers, the honest answer is "unknown".

Nothing in that table looks dramatic. Each problem is one small slip by a busy person. Together they turn a simple question into a wrong answer, and nobody sees an error message. **This is why analysts check data before they trust it.**

Common mistakes and how to spot them

Mistake	Symptom	Fix
Treating codes as numbers	an "average PIN code", phone numbers showing as 9.82E+09, lost leading zeros	Store identifiers as text; ask "would I ever do arithmetic with this?"
Not deciding what one row means	two people report different counts for the same thing	State the grain: "one row = one item on one bill"
Storing totals as rows	column sums come out double	Keep the detail; calculate totals when needed
Treating a blank as zero, or zero as blank	totals or averages silently wrong	Ask whether a blank means zero, not applicable, or unknown
Writing dates in local formats	3 April read as 4 March; months sort in the wrong order	Use year-month-day: 2026-04-03
Averaging ordinal ratings without looking at the spread	a polarizing supplier looks "above average"	Report the median and the count in each category
Saying "twice as much" for interval data	"twice as hot", "twice as late in the month"	Describe the difference instead: "20 degrees warmer"
Stopping at information	a report full of numbers that nobody acts on	Add the "why" and the "so what"
Trusting data because it's in a system	wrong counts from duplicates, spelling variations, and typos	Check a small sample against the six quality dimensions first

Mistake	Symptom	Fix
Using a column without knowing its meaning	"amount" assumed to include tax when it doesn't	Read, or write, the data dictionary

In the real world: Meera's first week

Meera Iyer has just joined Riverstone Supplies as a sales coordinator. On her second day, Anita Rao, the Sales Head, stops by her desk. "We're planning a customer meet in Mumbai next month. How many customers do we have there? I need the number by lunch."

Meera opens the shared customer spreadsheet. It has 240 rows. She filters the city column for "Mumbai" and gets 31. It would be easy to send "31" and move on. Instead, she spends twenty minutes using what she learned in this chapter.

1. What does one row mean? She scrolls and notices some customers appear more than once. Asking around, she learns that the sales team adds a new row whenever a customer opens a new branch. So one row isn't one customer; it's one *customer location*. Anita wants to invite customers, not branches.

2. Is the city column consistent? She sorts the column and finds "Mumbai", "mumbai", "Bombay", "Mumbai " with a trailing space, "Navi Mumbai", "Thane", and "Mum". She groups the obvious variations of Mumbai together, and notes that Navi Mumbai and Thane are separate cities in the same metropolitan region. Whether to invite them is a business question, not a spreadsheet question.

3. What's missing? Eleven rows have no city at all. She checks their addresses, which are in a different column, and finds that four are in Mumbai.

4. Can the data be trusted? She spots two customers marked "closed" in a notes column.

At 12:30 she sends this:

"We have **38 active customers with locations in Mumbai** (44 locations in total). The filter on 'Mumbai' alone shows 31, because the city is spelled several ways and some rows have no city. Another **9 customers are in Navi Mumbai and Thane**; should we invite them too? I've excluded 2 customers marked closed. I've also listed the spelling issues I found, in case we want to clean up the sheet."

Look at what Anita received: a number she can use, the reason it differs from the "obvious" number, a decision only she can make, and a small fix that will help everyone next time. None of it needed an advanced tool. It needed Meera to ask *what does a row mean*, *what does each value mean*, and *can I trust it*. That's the whole of this chapter, applied in twenty minutes.

Tools

- **A notebook and pen.** Enough for every exercise in this chapter. Sketching a table by hand is still one of the best ways to think about data.
- **Excel or Google Sheets** (optional). Chapter 10 teaches both from the beginning. If you already have either, use it for the project; Google Sheets is free with a Google account.
- **Your phone.** It's full of data about you: steps, screen time, photos, payments. It's the most convenient practice dataset you own.

Each tool is installed in the chapter that first uses it; Chapter 6 shows when.

The project: a week of your own spending

Goal: collect a real dataset, describe it the way a data professional would, and turn it into one piece of insight about your own life.

Step 1. Collect. For seven days, record every payment you make: cash, card, and UPI. Write one row per payment with these columns:

Column	What to write
date	year-month-day, e.g. 2026-09-07
time	24-hour clock, e.g. 19:30
item	what you paid for
category	Food, Transport, Groceries, Bills, Entertainment, Shopping, Health, or Other
amount_rs	the amount in rupees (or your currency)
payment_method	UPI, Card, or Cash
shop	where you paid
necessary	Yes or No: would you have been fine without it?

Here's how the first three days looked for Kavya, a college student in Mumbai:

date	time	item	category	amount_rs	payment_method	shop	necessary
2026-09-07	08:40	Tea and poha	Food	60	UPI	Station stall	No
2026-09-07	09:15	Metro card top-up	Transport	500	UPI	Metro station	Yes
2026-09-07	19:30	Vegetables	Groceries	240	Cash	Local market	Yes
2026-09-08	13:05	Lunch thali	Food	180	Card	Canteen	Yes
2026-09-08	21:10	Movie ticket	Entertainment	350	UPI	Online	No
2026-09-09	08:45	Tea and poha	Food	60	UPI	Station stall	No
2026-09-09	18:20	Mobile recharge	Bills	299	UPI	Online	Yes
2026-09-09	20:00	Auto rickshaw	Transport	90	Cash	Street	Yes

Step 2. Describe the dataset. Write down:

- the **grain** (what one row means);
- for every column: its **type**, whether it's **qualitative or quantitative** (and discrete or continuous), and its **level of measurement**;
- a **data dictionary**, using the table in section 1.8 as a model;
- the **source** of the data, and which rows you're least sure about (a cash payment you only remembered the next day, for example).

Step 3. Check the quality. Go through the six dimensions. Did you miss any payments? Did you write a category two different ways ("Food" and "Snacks")? Is every date in the same format?

Step 4. Climb the ladder. Answer these, by hand or in a spreadsheet:

1. *Information:* What did you spend in total, and in each category? Which payment method did you use most, by amount?
2. *Knowledge:* What explains your biggest category? Is it one large payment or many small ones?
3. *Insight:* What is one specific change you'd make next week, and how much would it save?

For Kavya's three days: she spent ₹1,779 in total, about ₹593 a day. Transport was her largest category at ₹590, mostly one metro card top-up, which will last her weeks, so it isn't really a daily cost. UPI covered ₹1,269, or 71.3% of her spending. She marked ₹470 (26.4%) as not necessary, and ₹120 of that was two identical breakfasts at the station stall. Her insight: *"Carrying breakfast from home three days a week would save about ₹180 a week, and the metro top-up shouldn't count as three days of spending."*

Stretch goals:

- Keep the log for a month and compare weeks.
- Add a **mood** column (happy, neutral, stressed) and see whether your "not necessary" spending changes with it. What level of measurement is **mood**?
- Download a month of your UPI or bank statement and compare it with your hand-kept log. Which payments did you forget to write down? That gap is a completeness problem, and a very common one.

You've got it when...

- ☐ I can explain the difference between data, information, knowledge, and insight with an example from my own life or work.
- ☐ I can turn a receipt, register, or form into a table and state its grain.
- ☐ I name a column's type before I use it, and I treat codes and phone numbers as text.
- ☐ I can say whether a column is qualitative or quantitative, and which level of measurement it is.
- ☐ I don't average ratings or codes without thinking, and I don't say "twice as hot".
- ☐ I can tell structured, semi-structured, and unstructured data apart.
- ☐ I can write a data dictionary for a small table.
- ☐ I check a small dataset against the six quality dimensions before trusting a number from it.
- ☐ I've logged a week of my own spending and turned it into one insight.

Recap

- **Data** is a recorded observation. **Information** is data organized to answer a question. **Knowledge** explains why. **Insight** points to an action. Analysts earn their value by climbing that ladder.
- Data is easiest to use in **tables**: **rows** (records), **columns** (fields), and **values**. Always decide what one row means: the **grain**.
- Store detailed facts and **calculate** totals, rather than typing totals into the data.
- The four common **data types** are number, text, date/time, and true/false. Codes that look like numbers are text. Write dates as year-month-day. A blank isn't automatically zero.
- **Quantitative** data measures amounts (discrete or continuous); **qualitative** data describes categories.
- The **levels of measurement**, nominal, ordinal, interval, and ratio, decide which calculations make sense.
- Data comes in three shapes: **structured** (tables), **semi-structured** (JSON, logs), and **unstructured** (emails, PDFs, images).
- **Metadata** describes data. A **data dictionary** says what every column means, and saves hours of argument.

- Data comes from **people, machines, business systems, and outside sources**. Every manual re-typing adds errors, which is why capturing data once, at the source, matters.
- **Data quality** is judged on accuracy, completeness, consistency, validity, uniqueness, and timeliness. Check before you trust.

Practice exercises

Warm-up

1. Which of these are data? For each "no", say what would make it data. (a) A train timetable on a station wall. (b) A tune you're humming. (c) The number of likes on a post. (d) A thought that you should call your aunt. (e) A doctor's handwritten prescription.
2. Label each statement as data, information, knowledge, or insight: (a) "Order 5012 was placed on 15 March and is Pending." (b) "We should give customers who haven't ordered in 60 days a call before the festive season." (c) "Riverstone sold ₹1,61,700 in February." (d) "Hotels order less during the monsoon, because fewer events are held."
3. Give the data type (number, text, date/time, or true/false) of each: an Aadhaar-style ID number, a delivery date, the weight of a parcel, whether an invoice is paid, a customer's PIN code, the number of items in a cart.

Core

4. State the level of measurement (nominal, ordinal, interval, or ratio) of each: T-shirt size, monthly revenue, customer segment, temperature in °C, a student's rank in class, the number of years a customer has been with Riverstone, a calendar year such as 2026, and a star rating from 1 to 5.
5. Classify each as structured, semi-structured, or unstructured: a CSV file of bank transactions, a recorded customer-support phone call, the JSON a weather app receives, a scanned PDF invoice, an Excel sheet of attendance with one row per student per day, and a set of Google reviews.
6. Which of these calculations make sense? Explain each in one sentence. (a) The average PIN code of your customers. (b) The median T-shirt size sold. (c) "Revenue in March was 20% lower than in February." (d) "15 March is 50% later than 10 March." (e) The most common payment method.
7. Write a data dictionary entry (type, meaning, allowed values, can it be blank) for these four columns of an attendance register: `student_id`, `class_date`, `status`, `minutes_late`.

Stretch

8. Find every data quality problem in this extract, and name the dimension each one belongs to:

order_id	customer_name	order_date	quantity	unit_price
7001	Green Leaf	2026-04-02	10	450.00
7002	Patel Kitchen	2026-04-02	-5	780.00
7002	Patel Kitchen	2026-04-02	-5	780.00
7003	Blue Bay Cafe	2026-13-01	20	
7004	blue bay cafe	2026-04-05	15	120.00

9. Using the delivery-partner ratings from section 1.6 (Swift Movers: 3, 3, 3, 3, 3; Rapid Wheels: 5, 5, 5, 1, 1), Riverstone's operations manager asks, "Which partner should we use for hotel banquets?" Write a three-sentence answer that uses the right summaries for ordinal data.
10. Kavya's log has a `necessary` column (Yes/No) and an `amount_rs` column. What is the level of measurement of each? Name two summaries you could calculate from `necessary` and one you couldn't.

Think about it (no calculation needed)

11. Your phone's step counter says you walked 12,000 steps yesterday. How could that number be wrong? What metadata would help you decide how much to trust it?
12. The sales head says, "Our customer data is bad." You've been asked to fix it. What three questions would you ask before touching the data?
13. On an order form, the discount box is blank. List three different things the blank could mean, and say who in the company should decide how blanks are treated in reports.

Key terms

data · datum · information · knowledge · insight · DIKW · table · row / record · column / field / variable / attribute · value · dataset · grain · data type · number · text / string · date and time · Boolean · identifier · ISO 8601 date · missing value / NULL · quantitative · qualitative / categorical · discrete · continuous · levels of measurement · nominal · ordinal · interval · ratio · mode · median · mean · structured data · semi-structured data · JSON · unstructured data · flattening · metadata · data dictionary · primary data · secondary data · first-party data · third-party data · personal data · data quality · accuracy · completeness · consistency · validity · uniqueness · timeliness · fit for use

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 2, How Computers Store, Move and Protect Data**, shows where data lives: files and formats (CSV, Excel, JSON, Parquet), databases, the cloud, and APIs.
- **Chapter 3, How a Business Runs on Data**, follows one Riverstone order from enquiry to cash, and shows every system that records data along the way.
- **Chapter 4, Numbers Without Fear**, builds the everyday math for working with quantitative data: percentages, growth, and averages.
- **Chapters 10 and 12** put this chapter's ideas into tools: data types and tables in spreadsheets, including how to check what a cell really contains, then in databases, including how getting the grain wrong makes totals double-count.
- **Chapter 14, Data Cleaning & Preparation**, fixes the quality problems from section 1.10 at scale, and **Chapter 47** shows how data teams catch them automatically before a report goes out.
- **Chapter 15** matches each kind of data to the chart that fits it, and **Chapter 21, Descriptive Statistics & Probability**, explains which averages suit each level of measurement.

- **Chapter 18** flattens semi-structured data such as JSON into tables.
- **Chapter 24** shows how to write the "so what" that turns a number into a decision.
- **Chapters 41 and 55** work with unstructured text, and in **Chapter 58** you'll build the email-order automation from section 1.7.
- **Interview preparation:** questions on data types, levels of measurement, and data quality appear in the Statistics bank (Chapter 73) and the Business Analyst bank (Chapter 76), with model answers.

Answers to practice exercises

1. (a) Yes: it's recorded information about trains and times. (b) No, not until it's recorded, for example as an audio file or written music. (c) Yes: a recorded count. (d) No, until you write it in a to-do list or set a reminder; then it's a recorded note. (e) Yes: it's recorded, even though it's handwritten and unstructured, which makes it harder to use.
2. (a) Data: a single recorded fact. (b) Insight: an action based on understanding. (c) Information: orders summarized into a monthly total. (d) Knowledge: an explanation of a pattern.
3. ID number: text (a code; you'd never add two). Delivery date: date. Parcel weight: number. Invoice paid: true/false. PIN code: text. Items in a cart: number (a whole number).
4. T-shirt size: ordinal. Monthly revenue: ratio. Customer segment: nominal. Temperature in °C: interval. Rank in class: ordinal (the gap between 1st and 2nd isn't the same as between 2nd and 3rd). Years as a customer: ratio (zero means none). Calendar year: interval (the year 0 isn't "no time", so 2026 isn't twice 1013). Star rating: ordinal.
5. CSV of bank transactions: structured. Recorded phone call: unstructured. Weather app JSON: semi-structured. Scanned PDF invoice: unstructured (it's a picture of a document, even though the invoice itself had a layout). Attendance sheet: structured. Google reviews: unstructured text (though each review sits in a semi-structured record with a date and a star rating).
6. (a) No: PIN codes are nominal labels, so an average has no meaning. (b) Yes: sizes are ordinal, so the middle value is meaningful. (c) Yes: revenue is ratio data, so percentage change works. (d) No: dates are interval data; you can say "5 days later", but not "50% later". (e) Yes: counting and finding the most common value works for nominal data.
7. One good answer:

Column	Type	Meaning	Allowed values	Can be blank?
student_id	text	the school's ID for the student	the ID format used by the school	no
class_date	date	the day of the class	YYYY-MM-DD, a school day	no
status	text	attendance for that day	Present, Absent, Late, Leave	no
minutes_late	number (whole)	minutes after the start time the student arrived	0 or more	yes: blank when status isn't Late

The last row is the kind of detail that prevents arguments later: a blank `minutes_late` means "not applicable", not zero.

8. Problems and dimensions:

- Order 7002 appears twice: **uniqueness**.
- Quantity -5 isn't possible for an order (unless the business records returns this way, which should be stated in the data dictionary): **validity**.
- 2026-13-01 has a month 13: **validity** (and it can't be trusted for **accuracy**).
- Order 7003 has no unit price: **completeness**.
- "Blue Bay Cafe" and "blue bay cafe": **consistency**.
- "Green Leaf" and "Patel Kitchen" may be shortened names of Green Leaf Hotels and Patel Kitchenware, which would be a **consistency** problem if other tables use the full names. Worth checking.

9. A sample answer: *"The average ratings are close (3.0 and 3.4), but they hide an important difference: every Swift Movers delivery was rated 3, while two of Rapid Wheels' five deliveries were rated 1, the lowest possible. For hotel banquets, where one failed delivery can ruin an event, consistency matters more than the occasional excellent delivery, so I'd use Swift Movers. With only five ratings each, we should keep collecting ratings and review the choice in a quarter."*

10. `necessary` is nominal (two categories, Yes and No; you could argue it's ordinal if "Yes" is seen as higher, but it's usually treated as nominal). `amount_rs` is ratio. From `necessary` you can calculate the **count** (or share) of Yes and No rows, and the **total amount** in each group, such as Kavya's ₹470 marked No. You can't calculate an **average of the Yes/No values** themselves, unless you deliberately code Yes as 1 and No as 0, in which case the average is simply the share of Yes rows.

11. The counter could count arm movements as steps (while cooking or on a bumpy bus ride), miss steps while the phone sat on a desk, or double-count if both a watch and a phone were synced. Useful metadata: which device recorded it, whether the phone was carried all day, the hourly breakdown (12,000 steps between 2 and 3 a.m. would be suspicious), and whether data from several devices was combined.

12. Good questions include: *"Bad for what decision? What were you trying to do when the data let you down?"* (fit for use), *"Which fields matter most: names, contact details, cities, or something else?"* (scope), and *"Who enters and updates customer data today, and how?"* (the source, because fixing the data without fixing how it's captured means it will be bad again in six months).

13. The blank could mean *no discount was given, a discount was given but not recorded, or the discount wasn't decided yet* (for example, pending approval). The treatment should be decided by the people who own the rule: usually finance, together with sales. The analyst's job is to raise the question, document the answer in the data dictionary, and apply it consistently in every report.

Chapter 2. How Computers Store, Move and Protect Data

Chapter at a glance

You will learn to: explain how a computer stores letters and numbers as bits and bytes · read file sizes from bytes to terabytes, and work out how long a download takes · tell memory from storage · work with files, folders, paths, and extensions without surprises · choose between CSV, Excel, JSON, XML, PDF, and Parquet for a job, and avoid each format's traps · explain what a database, a server, the internet, and the cloud are · describe what an API does and read its replies · protect data with good passwords, encryption, access rules, and backups.

Before you start: Chapter 1 (what data is: rows, columns, types, and quality).

Time needed: 3–4 hours, including the exercises and the project.

Tools: any computer. For the project: a spreadsheet program (Excel or Google Sheets) and a plain-text editor (Notepad on Windows, TextEdit on a Mac, or any code editor).

Practice data: four Riverstone orders from the mini database (Jan–Mar 2026), saved in five formats in the companion files (Appendix E), and the replies of a small demonstration API. Every file size and every reply in this chapter is real.

Why this matters

In Chapter 1 you learned what data is. This chapter is about where it *lives*, how it *travels*, and how it's *kept safe*. That sounds like IT's job, and some of it is. But these everyday situations all land on a data person's desk:

- A colleague emails a CSV export, you open it in Excel, and every product code that started with zero has quietly lost it.
- A report that took two seconds to open last year now takes two minutes, and the file is 80 MB.
- The finance team's figures in the monthly deck don't match yours, because they're working from `Sales_Report_FINAL_v3.xlsx` and you're working from `Sales_Report_FINAL_v4.xlsx`.
- Your manager asks whether the new CRM "has an API" so the weekly report can update itself.
- A laptop with a customer list on it is left in a taxi.

None of these needs you to be an engineer. Each needs you to understand, in plain terms, how computers store, move, and protect data. People who do understand it waste less time, lose less data, and are trusted with more of it, which is exactly the path this book is about.

In plain English

Imagine Riverstone's old paper office before any computers arrived.

- Everything was written in an **alphabet** of letters and digits. A computer has an even smaller alphabet: just 0 and 1.
- Records were kept on **shelves** in a storage room, where they stayed even when the lights went off. That's a computer's **storage**: its disk.
- To work on a record, a clerk carried it to a **desk**. The desk was fast to work at but small, and cleared every evening. That's the computer's **memory**.
- Records sat in **folders with labels**, and the label said what kind of paper was inside. Those are **files**, **folders**, and **extensions**.
- The same information could be written as a **ledger** with neat columns, a **letter**, or a **form** with labeled boxes. Those are **file formats**.
- Documents traveled between branches by **courier**. That's the **network** and the **internet**.
- Other companies didn't walk into the storage room. They came to a **counter window**, filled in a request slip, and a clerk brought back exactly what they asked for. That's an **API**.
- The storage room had **locks**, only some staff had **keys**, and important ledgers were **copied** and kept in another building in case of fire. That's **security** and **backups**.

Every section below is one part of that office, in modern form.

2.1 Bits and bytes: how computers store everything

Everything is 0s and 1s

Deep down, a computer can only store and move **two states**: on or off, charged or not charged, magnetized one way or the other. We write those states as **1** and **0**. One of these on/off values is a **bit** (short for *binary digit*).

One bit can only tell two things apart: yes or no. But put bits together and the number of possibilities doubles with each bit you add:

Bits	Possible patterns	Enough for...
1	2	yes/no
2	4	the four seasons
8	256	every English letter, digit, and punctuation mark
16	65,536	a whole number up to 65,535
32	4,294,967,296	a whole number up to about 4.3 billion

A group of **8 bits** is a **byte**, and the byte is the basic unit for measuring data. When someone says a file is 364 bytes, they mean it takes $364 \times 8 = 2,912$ on/off switches to store.

How text is stored

To store text, computers agree on a table that gives every character a number, and store the number. The capital letter A is number 65, which in 8 bits is 01000001. The digit 7 is number 55. A space is number 32. This agreement, first made for English as **ASCII** in the 1960s, fits every English letter, digit, and common symbol into one byte each.

English isn't the world's only language, of course. **Unicode** is the modern table: it gives a number to almost every character in every writing system, including ₹, Hindi, Tamil, Chinese, and emoji. The most common way to store Unicode as bytes is **UTF-8**, which keeps plain English letters at one byte each and uses two, three, or four bytes for everything else. Here's what some characters really take:

Text	Characters	Bytes in UTF-8
A	1	1
Pune	4	4
é	1	2
₹	1	3
न (Hindi <i>na</i>)	1	3
नमस्ते	6	18
😊	1	4
₹14,700	7	9

So "one character is one byte" is only true for plain English text. It's why a report with a lot of Hindi or Tamil text is bigger than the same report in English, and why a column limited to "100 characters" in one system can overflow a "100 bytes" limit in another.

When the table is wrong: garbled text

If a file is saved in one encoding and opened as if it were another, the bytes are read with the wrong table, and you get nonsense. Here's what happens when the UTF-8 bytes for ₹14,700 are opened as an older Windows encoding:

```
â, ¹14,700
```

And Café becomes CafÃ©. If you've ever seen Ã© or â€™ in a report, a mismatched encoding is almost always the cause. The fix is to open or import the file with the correct encoding, which is usually UTF-8. Section 2.5 shows where this bites in practice.

How numbers are stored

Whole numbers are stored exactly, as binary. Numbers with decimals are trickier. Most software stores decimals approximately, in a format called **floating point**, which is fast. Add 0.1 and 0.2 in many programs and the true stored answer is 0.30000000000000004, a hair more than 0.3. That's why systems that handle money store it in an exact decimal type.

So text, numbers, photos, music, and video are all, in the end, long rows of 0s and 1s. What makes them different is the **agreement** about how to read those bits: the encoding or format.

Try it. Open any plain `.txt` file, count roughly how many characters it has, then check its size in bytes (right-click → *Properties* on Windows, or *Get Info* on a Mac). For English text, the two numbers should be close.

2.2 How big is big? From kilobytes to terabytes

Because data sizes range from a few bytes to many billions, we use prefixes, just as we say kilometers instead of thousands of meters.

B	1 byte = one English letter, digit, or space 'A' is stored as 01000001
KB	1 kilobyte = 1,000 bytes a short plain-text email; the 4-order CSV in section 2.5 is 364 bytes
MB	1 megabyte = 1,000 KB a phone photo is typically a few MB
GB	1 gigabyte = 1,000 MB a feature-length film in HD is typically a few GB
TB	1 terabyte = 1,000 GB a typical laptop or external hard drive
PB	1 petabyte = 1,000 TB a large company's data warehouse, or a video platform's library

Each step is 1,000 times the one before.

(Some software counts in steps of 1,024; see the Watch out box below.)

Figure 2.1 – Each step is a thousand times bigger. The examples are typical sizes, not fixed rules.

A few sizes worth remembering: a page of plain English text is about 2–3 KB; a phone photo is typically a few MB; and an hour of HD video is typically a few GB.

Watch out: why your 1 TB drive shows 931 GB. A drive sold as 1 TB holds a trillion bytes. Windows divides by 1,024 at each step instead of 1,000, so it shows about 931 GB. Nothing is missing.

Bits for speed, bytes for size

Internet speeds are quoted in **megabits per second (Mbps)**, with a small *b*. File sizes are in **megabytes (MB)**, with a capital *B*. Since a byte is 8 bits, divide by 8:

- A 100 Mbps connection moves at most $100 \div 8 = 12.5$ **MB per second**.
- Downloading a 5 GB file on it takes at least $5,000 \text{ MB} \div 12.5 \text{ MB/s} = 400$ **seconds**, about **6.7 minutes**. In practice it takes longer, because the connection is shared and rarely runs at full speed.

Watch out: b and B. "My file is 80 Mb" and "my file is 80 MB" differ by a factor of eight. In data work, sizes are almost always bytes (MB, GB); speeds are almost always bits (Mbps, Gbps). When a number looks eight times too big or too small, check the letter.

Size limits you'll actually hit

- **Excel** holds at most **1,048,576 rows** and **16,384 columns** per sheet. Long before that, a large workbook becomes slow to open and save. A dataset with millions of rows belongs in a database (section 2.6) or a format such as Parquet (section 2.5).
- **Email attachments** are usually limited to a few tens of megabytes, which is one reason teams share large files by link instead.
- **Cloud storage plans, phone storage, and laptop drives** are all limited, and duplicate copies of large files (`report_v1`, `report_v2`, `report_final...`) fill them quickly.

2.3 Memory and storage: the desk and the shelves

Every computer has two very different places to keep data.

	Memory (RAM)	Storage (SSD or hard disk)
Office analogy	the desk you're working at	the shelves in the storage room
What it holds	whatever you have open right now	every file, program, and photo you've saved
Speed	very fast	slower (SSDs are much faster than older spinning hard disks)
Size on a typical laptop	about 8-32 GB	about 256 GB-2 TB
When the power goes off	everything in it is lost	everything stays

Three everyday consequences:

1. **Unsaved work lives only in memory.** If Excel crashes before you save, the changes since your last save were never on the shelves. Save often, or turn on your program's autosave.
2. **Opening a file copies it into memory.** A 200 MB workbook needs well over 200 MB of RAM to work with, often several times more once formulas and formatting are loaded. When a big spreadsheet slows your whole computer, memory is usually what's run out.
3. **Where "saved" means.** Your file might be saved on your own computer's drive, on a **shared network drive** at the office, or in **cloud storage** such as Google Drive or OneDrive, which keeps a copy on the provider's computers and **syncs** it to your devices. Knowing which one matters when you ask, "Can my colleague see this?" or "Is this backed up?"

2.4 Files, folders, paths, and extensions

A **file** is a named chunk of data on storage. A **folder** (or *directory*) is a container for files and other folders. Every file has a **path**, its full address, which lists every folder you pass through to reach it:

```
Windows: C:\Users\Kavya\Documents\Finance\2026-09_spending_log.xlsx
macOS:    /Users/kavya/Documents/Finance/2026-09_spending_log.xlsx
```

Windows separates folders with a backslash \ and starts with a drive letter such as C: . macOS and Linux use a forward slash /. You'll type paths when you load files into SQL, Python, or Power BI, and most "file not found" errors are a path with one folder or one letter wrong.

Extensions: what kind of file is this?

The letters after the last dot in a file name are its **extension**. The extension tells the computer which program should open the file:

Extension	What it usually is
.txt	plain text
.csv	comma-separated values: a table as plain text
.xlsx	an Excel workbook (.xls is the older format; .xlsm can contain macros)
.json, .xml	structured text, often exchanged between systems
.pdf	a document laid out for reading and printing
.jpg, .png	images
.parquet	a compressed, column-based data file (section 2.5)
.sql	a text file of SQL statements
.py	a Python program
.zip	a compressed bundle of other files
.exe	a Windows program that runs when you open it

Renaming `sales.csv` to `sales.xlsx` doesn't convert it into an Excel file. It only changes the label, and Excel will warn that the file doesn't match its name. To convert, open the file and use *Save As*.

Watch out: hidden extensions. Windows hides known extensions by default, so a file named `invoice.pdf.exe` appears as `invoice.pdf`, with a harmless-looking name and a program hidden inside. It's a classic trick in phishing emails. Turn on *File name extensions* in File Explorer's *View* menu, and never open an unexpected attachment that turns out to end in `.exe`, `.js`, `.bat`, `.scr`, or `.xlsm` from someone you don't know.

Naming files so people can find them

Compare these two folders:

Report final.xlsx	2026-01-31_monthly_sales_report.xlsx
Report final (2).xlsx	2026-02-28_monthly_sales_report.xlsx
Report FINAL v3 use this one.xlsx	2026-03-31_monthly_sales_report.xlsx

The right-hand names follow four habits: **a date first, written year-month-day**, so files sort in time order automatically (Chapter 1); **the same pattern every time**; **lower-case words joined with underscores or hyphens**, which avoids problems in code and web links; and **no words like "final"**, which are always eventually wrong. If you need versions, use the version history in Google Drive, OneDrive, or SharePoint (section 2.9), or, for queries and code, a version-control tool called Git.

2.5 Data file formats: the same data, packed five ways

Riverstone's sales coordinator exports four February orders. Here is exactly the same data saved in five formats (and a note on PDF). Look at what each one looks like inside; the differences explain when to use each.

CSV: a table as plain text

Mini database (Jan–Mar 2026): orders 5006 to 5009.

```
order_id,customer_name,order_date,status,sales_rep,net_revenue,delivery_note
5006,Metro Mart,2026-02-06,Delivered,Rahul Mehta,14640.00,
5007,Coastal Foods,2026-02-11,Delivered,Rahul Mehta,32625.00,Call before delivery
5008,Sunrise Caterers,2026-02-19,Delivered,,23325.00,
5009,Northgate Distributors,2026-02-25,Shipped,Farah Khan,76560.00,"Gate 2, Okhla Phase II"
```

CSV (comma-separated values) is the simplest data format there is: one line per row, commas between values, and usually a first line of column names. Any spreadsheet, database, and programming language can read it, which is why "export to CSV" is the universal language between systems. This file is **364 bytes**.

Look closely and you'll find three of CSV's limits:

1. **Commas inside values need quotes.** Northgate's delivery note contains a comma, so it's wrapped in double quotes. Without them, a program would see an extra column.
2. **A blank is just nothing.** Order 5008's missing sales rep is two commas in a row. CSV can't tell "unknown", "not applicable", and an empty text value apart (Chapter 1).
3. **There are no types.** Everything in a CSV is text. When a program reads this file back, 2026-02-06 arrives as text, not a date, unless the program guesses or you tell it.

Watch out: opening a CSV in Excel changes it. When Excel opens a CSV directly, it guesses a type for every value, and some guesses silently change your data: product codes like 00451 lose their leading zeros and become 451; long ID numbers turn into 4.52E+13; codes like 3-4 or 1/2 can become dates. If you then save, the damage is written back into the file. Instead, use *Data → From Text/CSV* (Excel) or *File → Import* (Google Sheets), and set code and ID columns to **Text** before loading. If ₹ or accented names appear garbled, choose **UTF-8** as the file's encoding in that same import window (section 2.1).

Excel (.xlsx): a workbook for people

An .xlsx file looks like a grid in Excel, but it isn't plain text. It's actually a **zip file** holding a small collection of XML files. Unzip the four-order workbook and you find ten files inside, including:

```
[Content_Types].xml
xl/workbook.xml
xl/worksheets/sheet1.xml
xl/sharedStrings.xml
xl/styles.xml
```

That structure lets a workbook hold far more than a CSV: several sheets, real dates and numbers, formulas, formatting, charts, and pivot tables. It also means the file isn't meant to be read as text, and needs software that understands the format. This tiny workbook is **5,674 bytes**, more than fifteen times the CSV, because the structure itself takes space. Excel is the right choice when **people** will open, read, and work with the file.

JSON: data with labels, for systems

```
[
  {
    "order_id": 5006,
    "customer_name": "Metro Mart",
    "order_date": "2026-02-06",
    "status": "Delivered",
    "sales_rep": "Rahul Mehta",
    "net_revenue": 14640.0,
    "delivery_note": null
  },
  {
    "order_id": 5008,
    "customer_name": "Sunrise Caterers",
    "order_date": "2026-02-19",
    "status": "Delivered",
    "sales_rep": null,
    "net_revenue": 23325.0,
    "delivery_note": null
  }
]
```

(Two of the four orders shown; the file has all four.)

JSON (JavaScript Object Notation) is the semi-structured format from Chapter 1. Each record is a set of **"name": value** pairs inside curly braces { }, and a list of records goes inside square brackets []. Notice what JSON can express that CSV can't: text is in quotes and numbers aren't, so some types survive; and a missing value is written as `null`, clearly different from an empty text value `"`. A record can also contain a list, such as an order with its order lines inside it. JSON is how most **apps and APIs** exchange data (section 2.8). Because every record repeats every column name, it's larger: **894 bytes** here.

XML: labels as tags

```
<order id="5009">
  <customer_name>Northgate Distributors</customer_name>
  <order_date>2026-02-25</order_date>
  <status>Shipped</status>
  <sales_rep>Farah Khan</sales_rep>
  <net_revenue>76560.00</net_revenue>
  <delivery_note>Gate 2, Okhla Phase II</delivery_note>
</order>
```

XML (eXtensible Markup Language) does the same job as JSON with **tags**: every value sits between an opening `<name>` and a closing `</name>`, and an empty value can be written `<sales_rep/>`. It's older and wordier (the four-order file is **1,118 bytes**), and you'll meet it in banking and government systems, e-invoicing, older business software, and inside Office files like the workbook above.

PDF: made for reading, not for data

A **PDF** fixes exactly how a document looks on screen and on paper, which makes it ideal for invoices, contracts, and reports people read. It's poor for data: a table in a PDF is stored as text placed at positions on a page, not as rows and columns, so copying it into a spreadsheet often scrambles the columns. When someone offers you data "as a PDF", ask whether the CSV or Excel export behind it exists.

Parquet: built for large-scale analysis

Parquet is a format built for analyzing large datasets, and it's the standard format of cloud data platforms. You can't read it as text: it's compressed binary data.

CSV and Excel: stored row by row

date	customer	product	qty		
2025-08-30	2	105	77	row 1	To add up qty, the computer still reads every value in every row, then throws most away.
2025-02-24	21	107	62	row 2	
2025-11-03	7	101	15	row 3	
2025-05-19	2	103	140	row 4	

Parquet: stored column by column

date	2025-08-30	2025-02-24	2025-11-03	2025-05-19	To add up qty, it reads only the qty column. Similar values sit together, so they compress well.
customer	2	21	7	2	
product	105	107	101	103	
qty	77	62	15	140	

Figure 2.2 — CSV stores data row by row; Parquet stores it column by column. The four sales lines are an example, not from either Riverstone database.

A CSV stores data **row by row**; Parquet stores it **column by column**, so a program that totals one column reads only that column, and similar values sitting together **compress** very well. Parquet also stores each column's **type**, so dates come back as dates. For four orders its advantages are invisible (the file is **4,872 bytes**, bigger than the CSV, because it also describes its own structure); they appear when a file holds millions of rows.

Compression

Compression makes files smaller by writing repeated patterns more efficiently. There are two kinds:

- **Lossless** compression (ZIP, gzip, and the compression inside Parquet and `.xlsx`) gives back **exactly** the original bytes when you unpack it. Data files must only ever use lossless compression.
- **Lossy** compression (JPEG photos, MP3 music, most video) throws away detail people won't notice: that's how photos and music are made so small. It's fine for a photo, and unacceptable for a sales ledger.

Choosing a format

Format	People can read it as text?	Keeps types?	Size	Best for
CSV	yes	no	small	moving tables between systems; simple exports
Excel	no (needs Excel or similar)	yes	medium	files people will read, filter, and work in
JSON	yes	partly	large	apps and APIs; nested records
XML	yes	partly	large	older systems, e-invoicing, Office internals
PDF	no	no	varies	documents for reading and printing
Parquet	no	yes	smallest	large datasets for analysis

Real-life example: why the data team asks for "the raw export". A manager sends a PDF of last quarter's sales, laid out beautifully. To analyze it, the analyst has to copy 40 pages of tables by hand or with a converter, and check every row. The same data exported from the billing system as CSV takes ten seconds to load. The rule most data teams follow: **PDF and formatted Excel for people who read; CSV, JSON, or Parquet for machines that process.**

2.6 Databases: data that many people can use at once

Files are fine for one person and one moment in time. Business data needs more. Picture Riverstone's orders kept as a shared Excel file: three salespeople edit it at once, two copies drift apart, someone sorts one column without the others and scrambles every order, and the file grows until it no longer opens.

A **database** is software built to avoid those problems. It keeps data in **tables** (rows and columns, like a strict spreadsheet), and it:

- lets **thousands of people and programs** read and write at the same time without overwriting each other;
- enforces **rules**: no order for a customer who doesn't exist, no negative quantities, no duplicate invoice numbers;
- answers **questions** in a language called **SQL**, even across millions of rows, in seconds;
- controls **who may see or change** each table;
- keeps a record of changes so it can recover after a crash.

Most business databases run as a **database server**: a program on a computer, usually in a data center or the cloud, that other programs connect to over the network. (A few, like SQLite, which runs inside many phone apps, store the whole database in a single file.) Underneath, a database still saves its data in files on storage; you just never touch those files directly.

This one page is only a preview; later chapters teach databases and SQL properly, from creating your first table onward.

2.7 Servers, the internet, and the cloud

Clients and servers

When you open Gmail, two computers are involved. Your laptop or phone is the **client**: it asks for something. A computer in one of Google's data centers is the **server**: it listens for requests and answers them. A server isn't a special kind of machine; it's a computer whose job is to serve requests, usually running all day in a **data center**, a building full of such computers with reliable power, cooling, and network connections.

How a request finds its way

The **internet** is a worldwide network of networks that carries data between clients and servers. Three ideas explain most of it:

- **IP address**. Every device on a network has an address, such as `203.0.113.14`, so data knows where to go, like a postal address.
- **DNS** (Domain Name System). People remember names like `riverstone.example`, not numbers. DNS works like a phone book, turning a name into an IP address before your request is sent.
- **HTTP and HTTPS**. These are the rules for how a browser or program asks a web server for something and gets a reply. **HTTPS** is the secure version: everything sent is **encrypted**, so people along the route can't read or change it. The padlock icon in a browser means the connection uses HTTPS. It does *not* mean the website itself is trustworthy.

The cloud

"The cloud" sounds mysterious. It's simpler than it sounds: **the cloud is computers in someone else's data center that you rent over the internet**, instead of buying and running your own. The largest providers include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud.

Companies rent anything from bare computers to finished applications like Gmail or a CRM. The finished-application kind is called **SaaS**, software as a service, and it's the kind you'll meet first at work.

Why companies move to the cloud: they **pay for what they use** instead of buying servers upfront, they can **grow or shrink** in minutes, and the provider handles power, hardware failures, and much of the security. The trade-offs: bills that grow quietly if nobody watches them, dependence on one provider, and questions about **where the data is physically stored**. Many companies, and some laws, require certain data to stay in a particular country, so cloud services let customers choose a region for their data.

2.8 APIs: how systems talk to each other

Riverstone's sales system, its accounting software, its website, and its payment gateway are separate programs, often from separate companies. For a website order to appear in the sales system, or for a weekly report to pull the latest invoices, the programs need a way to ask each other for data. That way is an **API** (Application Programming Interface).

Like a restaurant: you (the program) never walk into the kitchen (the database).
You give the waiter (the API) a clear order, and it brings back a dish.

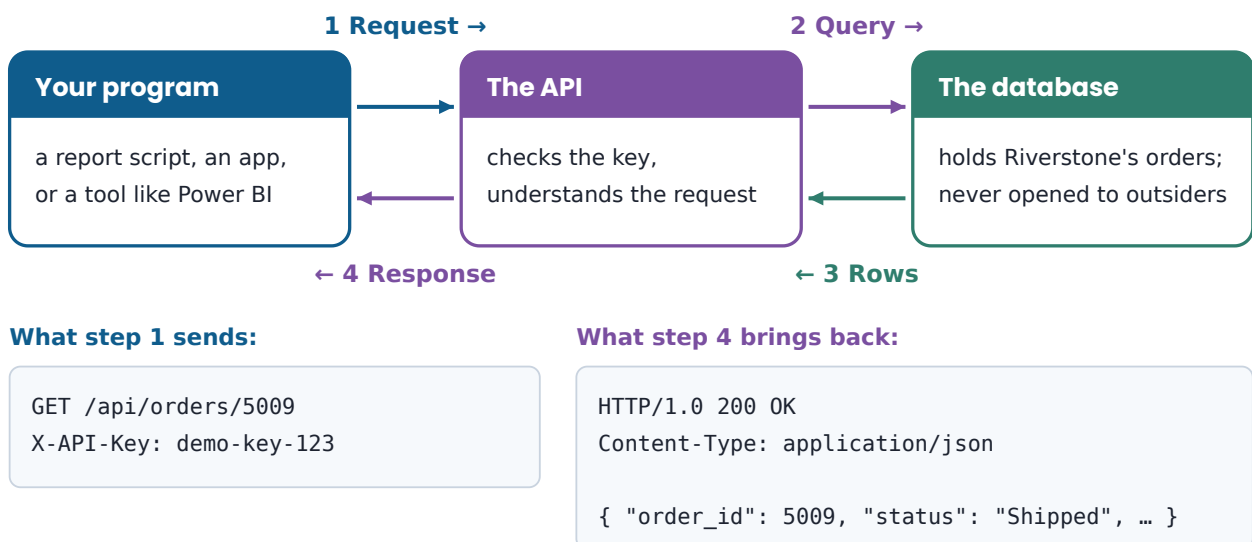


Figure 2.3 — An API is the waiter between your program and someone else's kitchen.

The restaurant analogy is the classic one. You don't walk into the kitchen and take food off the stove; you'd get in the way, and you might take someone else's dish. You give a **waiter** a clear order from the **menu**, and the waiter brings back what you asked for. An API is the waiter: it offers a menu of requests

it will accept, checks you're allowed to ask, fetches what's needed from the system behind it, and brings back a reply in an agreed format, usually JSON. The system's database (the kitchen) is never opened up to outsiders.

A real request and response

Riverstone has a small demonstration API that serves orders from the mini database (Jan–Mar 2026). A **request** asks for one order by its address, and includes an **API key**, a secret code that proves the caller is allowed in. Here's the full reply the demonstration API sends back when a program asks for order 5009:

```
HTTP/1.0 200 OK
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 147

{
  "order_id": 5009,
  "customer_name": "Northgate Distributors",
  "order_date": "2026-02-25",
  "status": "Shipped",
  "net_revenue": 76560.0
}
```

Every **response** has two parts. The top is the **header**: a **status code** (200 OK) that says how the request went, plus details such as the type of content coming back. After a blank line comes the **body**: the data itself, here in JSON.

Ask for an order that doesn't exist, and the status code changes:

```
HTTP/1.0 404 Not Found
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 38

{
  "error": "order 5099 not found"
}
```

Leave out the API key, and the API refuses before it even looks for the order:

```
HTTP/1.0 401 Unauthorized
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 44

{
  "error": "missing or invalid API key"
}
```

Status codes worth knowing

The first digit tells you who's responsible: **2** means it worked, **4** means the request has a problem (your side), **5** means the server has a problem (their side).

Code	Meaning	What to do
200 OK	it worked	read the body
401 Unauthorized	no valid key or login	check the API key
404 Not Found	no such record or address	check the ID and the address
429 Too Many Requests	you've hit the API's rate limit	slow down and retry later
500 Internal Server Error	the server failed	retry later; tell the API's owner if it persists

Why APIs matter to data people

Most modern business software, including CRMs, accounting tools, payment gateways, e-commerce platforms, and ad platforms, offers an API. That's what makes automation possible: a script can pull yesterday's invoices every morning without anyone logging in and clicking *Export*, and a dashboard can refresh itself.

Two more terms you'll hear:

- A **webhook** is an API in reverse. Instead of your program asking "anything new?" every five minutes, the other system calls *your* address the moment something happens, such as "payment received".
- **API keys are passwords.** Anyone who has the key can do whatever the key allows. Never paste one into a spreadsheet, a shared document, a chat, or code you publish online.

2.9 Keeping data safe

The three goals: confidentiality, integrity, availability

Security professionals describe what they protect with three words, often called the **CIA triad**:

Goal	Means	Example of failure at Riverstone
Confidentiality	only the right people can see the data	the customer price list is emailed to a competitor by mistake
Integrity	the data is accurate and hasn't been changed without authorization	someone edits invoice amounts in a shared spreadsheet, and nobody can tell
Availability	the data is there when people need it	ransomware locks the sales server during the busiest week of the year

Every habit below protects one or more of the three.

Accounts and passwords

Most data breaches start with a person, not a clever technical attack: a reused password, a shared login, or a click on a fake email.

The habits that prevent most of them:

- **Long and unique.** A long passphrase is harder to guess than a short password full of symbols, and **every account needs a different one.** When one website is breached, attackers try the same email and password everywhere else.
- **Use a password manager.** Nobody can remember fifty unique passphrases. A password manager remembers them, fills them in, and warns you about reused ones.
- **Turn on multi-factor authentication (MFA).** A second step, such as a code from an authenticator app or a prompt on your phone, stops most attacks even when a password leaks.
- **Never share logins.** A CRM password shared in a team chat means you can't tell who changed what, and you can't remove one person's access without locking everyone out. Each person gets their own account.
- **Be suspicious of urgency.** Phishing emails pressure you to act now: "Your account will be closed", "Pay this invoice today". Check the sender's real address, hover over links before clicking, and confirm unusual payment requests by phone.

Access: the principle of least privilege

Give each person and program only the access they need, and nothing more. An analyst who builds sales reports needs to *read* the sales tables, not change them, and doesn't need salary data at all. That's why many companies give analysts **read-only** accounts. Least privilege limits the damage from a mistake or a stolen password.

Encryption

Encryption scrambles data with a key so that only someone with the right key can read it. You rely on it in two places:

- **In transit**, while data moves: HTTPS websites, secure email, and encrypted connections to databases.
- **At rest**, while data sits on storage: an encrypted laptop drive (BitLocker on Windows, FileVault on a Mac) means a stolen laptop's files are unreadable without your login. For any laptop that carries work data, this should be switched on.

Integrity checks: fingerprints for files

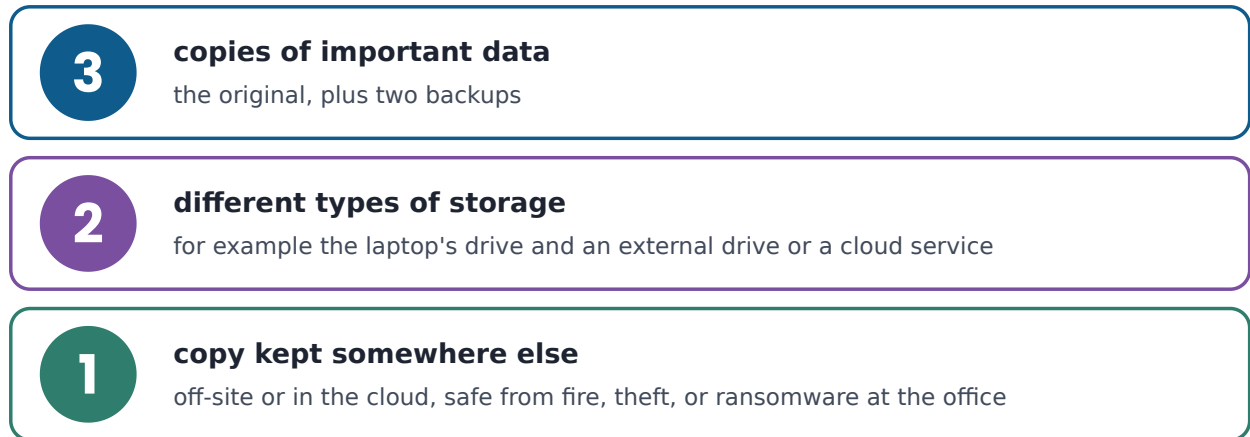
How do you know a file hasn't been changed, even by one character? Computers calculate a **hash**, a "fingerprint" of the data: the same data always gives the same fingerprint, and the smallest change gives a completely different one. Here are the first 12 characters of the fingerprints of two payment lines that differ only in the order of two digits:

Pay Rs 14,700 to Riverstone Supplies	f4251ff3fb71...
Pay Rs 17,400 to Riverstone Supplies	9d2f842c5011...

Software uses hashes to check that downloads arrived undamaged, that backups match the original, and that passwords are stored safely (systems store a hash of your password, not the password itself).

Backups and versions

Hardware fails, laptops are stolen, files are deleted by mistake, and **ransomware** (malicious software that locks files and demands payment) can encrypt everything a computer can reach. The only complete protection is a copy you can restore. The widely used rule of thumb is **3-2-1**:



**A backup you have never restored is a hope, not a backup:
test a restore regularly.**

Figure 2.4 – Three copies, two kinds of storage, one kept elsewhere.

Two points people often miss:

- **Syncing is not backing up.** Cloud storage such as Google Drive or OneDrive **copies every change** to every device, including deleting a file or saving over it with a mistake. Version history and a recycle bin help, but only for a limited time. A true backup is a separate copy that your everyday mistakes don't touch.
- **Test your restore.** Many organizations discover their backups were incomplete only on the day they need them.

Version history is backup's everyday cousin. Google Drive, OneDrive, and SharePoint keep earlier versions of a file, so you can see who changed what and roll back a bad edit. Use it instead of saving `report_v7_FINAL.xlsx`. For SQL queries and code, Git does the same job more precisely.

Personal data

Some data is about **people**: names, phone numbers, addresses, ID numbers, health information, salaries. Handle it with extra care. India's Digital Personal Data Protection Act, 2023, the European Union's GDPR, and similar laws elsewhere set rules for collecting, storing, and sharing it. The everyday habits: collect and keep only what you need; don't copy it to personal devices, personal email, or personal cloud accounts; share it only with people entitled to see it; and use anonymized or sample data for practice and demos. Chapter 64 covers privacy and governance in depth.

If you make a mistake, say so immediately. Emailed a customer list to the wrong person? Clicked a suspicious link? Lost a laptop? Tell your manager and IT **straight away**. Minutes matter: a password can be reset, a sent email sometimes recalled, a laptop wiped remotely. Organizations forgive a reported mistake far more easily than a hidden one.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Opening a CSV directly in Excel and saving it	leading zeros gone, IDs as 4.52E+13, codes turned into dates	Import with <i>Data</i> → <i>From Text/CSV</i> ; set ID columns to Text
Wrong text encoding	â, ¹ , Ã©, â€™ in names and amounts	Import as UTF-8
Confusing Mbps with MB/s	downloads "eight times slower than expected"	Divide Mbps by 8 for MB/s
Renaming a file's extension to "convert" it	the file won't open, or opens with a warning	Open the file and use <i>Save As</i>
Using "final", "new", or "(2)" in file names	nobody knows which file is current	Date-first names plus version history
Sending data as PDF	the receiver re-types or mis-copies tables	Share the CSV or Excel export
Keeping millions of rows in Excel	files take minutes to open; rows cut off at 1,048,576	Use a database or Parquet
Pasting API keys or passwords into files and chats	anyone with the file can use the account	Store secrets in a password manager or secure settings
Sharing one login among a team	no record of who changed what	One account per person, with only the access they need
Treating cloud sync as a backup	a deletion or a bad save spreads to every copy	Keep separate, tested backups (3-2-1)
Unencrypted work laptop	a lost laptop becomes a data breach	Turn on drive encryption
Hiding a security mistake	a small incident becomes a large one	Report it immediately

In the real world: the Friday file

Before Meera (Chapter 1) joined, Riverstone's weekly sales report worked like this. Every Friday afternoon, Imran, who ran sales operations, exported orders from the billing system as CSV, opened the file in Excel, added a few formulas and a pivot table, and saved it as `Weekly Sales FINAL.xlsx` on his laptop. Then he emailed it to eleven people. Over one bad month, four things went wrong:

1. **Product codes broke.** Riverstone's new product range used codes such as `00731`. Excel turned them into `731`, and the lookup to the price list failed for every new product. Two weeks of reports understated new-product sales.
2. **Nobody knew which file was current.** Imran sent a corrected report on a Monday, as `Weekly Sales FINAL (2).xlsx`. Half the recipients kept using the Friday version, and the regional

managers argued about numbers that came from two different files.

3. **The file grew until it choked.** Each week's data was pasted under the last. At 40 MB the workbook took four minutes to open, and some recipients' email rejected it.
4. **The laptop was stolen** from a car. It wasn't encrypted, and the workbook held two years of customer names, phone numbers, and prices.

Riverstone's fix used nothing more advanced than this chapter:

- The export was **imported** with codes set to text, and the billing system's **API** later replaced the manual export entirely.
- The report moved to **one shared location** with version history, and the email became a **link** to it, with the date in the file name. Everyone saw the same, current file.
- History moved out of the workbook into a **database**, and the report kept only the latest weeks, so it opened in seconds.
- Every company laptop was **encrypted**, customer phone numbers were removed from reports that didn't need them, and the IT team set up **tested 3-2-1 backups**.

None of those changes required a data engineer. They required someone who understood files, formats, APIs, and basic security well enough to notice what was wrong. Later in the book you'll automate a report of exactly this kind, Riverstone's Daily Sales Flash, end to end.

Tools

- **A plain-text editor.** Notepad (Windows), TextEdit in plain-text mode (Mac), or a free code editor such as Visual Studio Code. Opening a CSV or JSON file in a text editor shows you what's really inside, without a spreadsheet's guesses.
- **Excel or Google Sheets**, for the project. Learn the *import* routes (*Data* → *From Text/CSV* in Excel; *File* → *Import* in Google Sheets), not just double-clicking.
- **A password manager and an authenticator app.** Set them up for your own accounts this week.
- **The companion files** (Appendix E): `orders_feb_2026` in five formats, for the project.

The project: one dataset, five formats

Goal: see with your own eyes what each format stores, what it loses, and how each program treats it.

Option A: use the companion files `orders_feb_2026.csv`, `.xlsx`, `.json`, and `.xml`.

Option B: use your spending log from Chapter 1's project, and create the formats yourself: save it from your spreadsheet as `.xlsx` and as CSV (UTF-8), then type a JSON version of the first three rows by hand in a text editor, using section 2.5 as your model.

Steps:

1. **Open each file in a plain-text editor.** Which ones can you read? For the `.xlsx`, what do you see, and why?

2. **Find the missing values.** How does each format show a blank sales rep or delivery note? Can you tell "unknown" from "empty text" in each?
3. **Find the types.** In which formats can you tell that `net_revenue` is a number and `order_date` is a date, just by looking?
4. **Import the CSV into Excel or Google Sheets twice:** once by double-clicking, once through the import screen with every column set to Text. Add a row with a code like `00731` to your copy first. What differs?
5. **Compare sizes.** Record each file's size in bytes. Which is smallest? Why is the Excel file larger than the CSV for so little data?
6. **Break a CSV on purpose.** Remove the quotes around "Gate 2, Okhla Phase II", save, and reimport. What happens to the columns?
7. **Write it up.** A one-page comparison table: format, readable as text, keeps types, shows blanks as, size, and "I would use it for...". Finish with three sentences of advice for a colleague who is about to email a CSV.

Stretch goals:

- Save the same data as a ZIP file and compare its size with the CSV.
- Type ₹ into one value in your spreadsheet, save it once as CSV UTF-8 and once as plain CSV (*Comma delimited*), then open both files in a text editor. Does the symbol survive in each?
- Check which of your own accounts have MFA switched on, and switch it on for the rest.

You've got it when...

- ☐ I can explain bits, bytes, and why ₹ takes more space than R.
- ☐ I can convert between KB, MB, GB, and TB, and between Mbps and MB/s, and I know why a 1 TB drive shows 931 GB.
- ☐ I know the difference between memory and storage, and what "saved" means for my work.
- ☐ I name files so they sort by date and never need the word "final".
- ☐ I can say which format I'd use for a person, an app, or a large dataset, and why.
- ☐ I import CSV files instead of double-clicking them when codes or IDs are involved.
- ☐ I can explain a server, the cloud, and an API to a non-technical colleague.
- ☐ I can read an API response and tell from its status code whether the problem is mine or the server's.
- ☐ My own accounts use unique passwords from a password manager, with MFA.
- ☐ I can explain the 3-2-1 rule and why syncing isn't a backup.

Recap

- Computers store everything as **bits** (0s and 1s), grouped into **bytes**. Text is stored through an **encoding** such as **UTF-8**; the wrong encoding produces garbled characters. Decimals are usually

stored approximately, which is why money needs exact types.

- Sizes go **KB** → **MB** → **GB** → **TB** → **PB**, each 1,000 times the last (or 1,024 in some software). Speeds are in **bits**, sizes in **bytes**: divide Mbps by 8.
- **Memory** is fast and temporary; **storage** is slower and permanent. Unsaved work lives only in memory.
- Files have **paths** and **extensions**. Name them date-first, never "final", and watch for hidden extensions.
- **CSV** is universal but has no types; **Excel** is for people; **JSON** and **XML** carry labeled data between systems; **PDF** is for reading; **Parquet** is compact, typed, and fast for large-scale analysis. Import CSVs carefully.
- **Databases** let many people use data at once, with rules and SQL.
- A **server** answers a **client's** requests over the **internet**; **HTTPS** encrypts the connection; the **cloud** is rented computers and services, up to finished applications (**SaaS**).
- An **API** is the waiter between programs: a request goes in, a response with a **status code** and usually JSON comes back. API keys are passwords.
- Protect **confidentiality, integrity, and availability**: unique passwords, MFA, least privilege, encryption, hashes, tested **3-2-1 backups**, version history, careful handling of personal data, and reporting mistakes immediately.

Practice exercises

Warm-up

1. Convert: (a) 3.5 GB to MB; (b) 250,000 KB to MB; (c) 2 TB to GB.
2. How many bytes does each take in UTF-8: Riverstone, ₹500?
3. A friend's new laptop has "512 GB" of storage, but Windows shows about 476 GB. Is anything wrong? Explain in two sentences.
4. What file type is each of these, and which program would normally open it:
2026-03-31_stock.csv, invoice_9007.pdf, po_template.xlsm, export.json, setup.exe?

Core

5. How long, at best, does it take to download a 2 GB file on a 50 Mbps connection?
6. Choose the best format for each job, and say why in one line: (a) a monthly summary your manager will open and filter; (b) a website sending a new order to the sales system; (c) two years of sensor readings from Riverstone's machines, to be analyzed in the cloud; (d) a tax invoice sent to a customer; (e) a one-off list of 3,000 customers to import into a new email tool.
7. This line comes from a CSV file with seven columns. What's wrong with it, and how should it be written? 5010,Green Leaf Hotels,2026-03-03,Delivered,Farah Khan,20100.00,Leave at reception, back gate
8. A script calls Riverstone's API and gets these replies on different days: 401, 404, 429, 503. For each, say whether the problem is probably on the script's side or the server's, and what you'd do.

9. Section 2.9 shows the hashes of two payment lines. A colleague sends you a backup file and its hash. You calculate the hash of the file you received and get a different value. What does that tell you, and what doesn't it tell you?

Stretch

10. Kavya keeps her only copy of her spending logs in a Google Drive folder that syncs to her laptop. Does her setup meet the 3-2-1 rule? Describe one thing that could lose her data anyway, and a change that would fix it.
11. Riverstone's new product codes look like 00731. Describe exactly how a code can lose its leading zeros between the billing system's CSV export and the price lookup in a workbook, and give two ways to prevent it.
12. Why must a sales ledger only ever be compressed losslessly? Give one example of what lossy compression would do to it.

Think about it (no calculation needed)

13. A team shares one login for the company's CRM, and the password is pinned in a group chat. List three risks, and propose a better setup.
14. Your manager asks, "Should we keep our sales database on our own server in the office, or move it to the cloud?" Give two advantages of each option, and one question you'd ask before deciding.
15. You receive an email from "Riverstone Accounts" with an attachment called `Payment_Details.pdf` and the message "Urgent: confirm today or the supplier contract will be cancelled." What do you check before opening it?

Key terms

bit · byte · binary · ASCII · Unicode · UTF-8 · encoding · garbled text (mojibake) · floating point · kilobyte (KB) · megabyte (MB) · gigabyte (GB) · terabyte (TB) · petabyte (PB) · megabits per second (Mbps) · memory (RAM) · storage (SSD, hard disk) · file · folder / directory · path · extension · CSV · Excel workbook (.xlsx) · JSON · XML · PDF · Parquet · compression · lossless · lossy · database · database server · client · server · data center · internet · IP address · DNS · HTTP / HTTPS · cloud · SaaS · API · request · response · header · body · status code · API key · rate limit · webhook · CIA triad · confidentiality · integrity · availability · password manager · multi-factor authentication (MFA) · phishing · least privilege · encryption in transit · encryption at rest · hash · ransomware · backup · 3-2-1 rule · sync · version history · personal data

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 3, How a Business Runs on Data**, follows one Riverstone order through every system that stores and passes along its data.
- **Chapters 10 and 11** teach spreadsheets properly, including importing CSV files without damage.

- **Chapter 12, Databases & SQL Foundations**, turns the one-page preview in section 2.6 into a full, hands-on skill, including the exact decimal type databases use for money, and read-only accounts for analysts.
- **Chapter 17** shows the $0.1 + 0.2$ surprise from section 2.1 in Python, with the code you run yourself.
- **Chapter 18** reads CSV, Excel, JSON, and Parquet files in Python, and calls real APIs, including the demonstration API from section 2.8.
- **Chapter 20** automates a report of exactly the Friday file's kind, Riverstone's Daily Sales Flash, including storing API keys safely.
- **Chapter 26** uses Git to keep versions of queries and code.
- **Part V (Chapters 45–52)** builds on formats, compression, the cloud, and APIs at company scale: hashes that detect changed files (Chapter 45), Parquet and columnar storage tested at scale (Chapter 49), how whole systems exchange data (Chapter 51), and the levels of cloud service (Chapter 52). **Chapter 58** extracts tables from PDFs with AI tools, with checks. **Chapter 64** covers security, privacy, and governance in depth, and **Chapter 65** keeps cloud bills under control.
- **Interview preparation:** file formats, APIs, and data security questions appear in the Data Engineering bank (Chapter 77) and the Automation & Integration bank (Chapter 78).

Answers to practice exercises

- (a) $3.5 \times 1,000 = \mathbf{3,500\ MB}$. (b) $250,000 \div 1,000 = \mathbf{250\ MB}$. (c) $2 \times 1,000 = \mathbf{2,000\ GB}$. (Counting in steps of 1,024 instead, as Windows does, the answers would be 3,584, about 244, and 2,048. Either is acceptable if you say which you used.)
- Riverstone is 10 plain English letters: **10 bytes**. ₹500 is **6 bytes**: 3 for ₹ and 1 for each digit.
- Nothing is wrong. The maker counts 512 GB as 512,000,000,000 bytes, while Windows divides by 1,024 at each step ($1,024 \times 1,024 \times 1,024$) and shows about 476.8 GB; it's the same storage counted a different way.
- 2026-03-31_stock.csv: a CSV table as plain text; a spreadsheet or text editor. invoice_9007.pdf: a PDF document; a PDF reader or browser. po_template.xlsm: an Excel workbook that can contain macros; Excel (be careful enabling macros from unknown senders). export.json: JSON data; a text editor, code editor, or a program that reads JSON. setup.exe: a Windows program that runs when opened; only open it if you trust exactly where it came from.
- $50\ \text{Mbps} \div 8 = 6.25\ \text{MB per second}$. $2,000\ \text{MB} \div 6.25 = \mathbf{320\ seconds, about 5.3 minutes}$, and in practice a little longer.
- (a) **Excel**: people will read and filter it, and it keeps formatting and types. (b) **JSON through an API**: it's what apps exchange, and it can carry an order with its lines nested inside. (c) **Parquet**: large, typed, compressed, and fast to analyze by column. (d) **PDF**: a fixed document the customer reads and keeps, which looks the same everywhere. (e) **CSV**: nearly every tool imports it; set the ID and phone columns to text.

7. The delivery note contains a comma but isn't in quotes, so the line splits into **eight** values instead of seven: `Leave at reception and back gate` become separate columns, and every program reading it will either reject the row or shift the data. Correct version: `5010,Green Leaf Hotels,2026-03-03,Delivered,Farah Khan,20100.00,"Leave at reception, back gate"`

8. `401 Unauthorized`: the script's side. The API key is missing, wrong, or expired, so check how the key is stored and whether it has been changed. `404 Not Found`: usually the script's side: the ID or the address is wrong, or the record really doesn't exist. Check the ID before assuming the API is broken. `429 Too Many Requests`: the script's side. It's calling too often, so add a pause between calls and retry later. `503` (a 5xx code): the server's side. Retry after a delay, and if it continues, check the provider's status page or contact the API's owner.

9. Different hashes mean **the two files are not identical**: the file was changed or damaged somewhere between your colleague and you, or they calculated the hash on a different version. It doesn't tell you **what** changed, **where**, or **whether** the change was accidental or deliberate. Ask for the file again, compare the hashes, and don't use the backup until they match.

10. No. She has one copy that appears in two places, not three independent copies, and nothing is kept separately from her synced accounts. If she deletes a file, saves over it by mistake, or ransomware encrypts her laptop, the change **syncs to Drive** too. Version history might rescue her, but only for a limited time. A fix: a regular backup to an external drive kept at home, plus a second copy in a separate cloud backup service or account that doesn't sync changes automatically, and an occasional test that she can restore a file.

11. The billing system writes `00731` correctly into the CSV as text. When someone **double-clicks** the CSV, Excel guesses that `00731` is a number and stores **731**. If the file is saved, the damage is written back. The price list still says `00731`, so a lookup for `731` finds nothing. Prevention: (1) import the CSV with *Data → From Text/CSV* and set the product code column to **Text** before loading; (2) better still, get the data from the source system through a connection or an API that keeps the column's type, so no one opens the raw CSV at all. (Adding a letter prefix to codes, such as `P00731`, also prevents it, but changing codes is a business decision.)

12. A sales ledger can't lose a single digit: every amount, date, and code must come back exactly as it was written, so only **lossless** compression, such as ZIP, which restores exactly the original bytes, is safe. **Lossy** compression throws detail away, which is fine for a photo but not for data. On a ledger it might, for example, round ₹14,640.00 to ₹14,600 or turn product code `00731` into something close but different, and nobody could get the original values back. Ledgers compress well losslessly anyway, because columns such as status, product, and customer repeat the same values many times.

13. Risks: (1) nobody can tell who made a change or deleted a record, so mistakes and misuse can't be traced; (2) when someone leaves the team, they still know the password, and removing their access means changing it for everyone; (3) anyone who can read the chat, on any device, including a lost phone, can log in; and a single leaked password exposes the whole CRM. Better: one account per person, with permissions matched to each role (least privilege), MFA switched on, and access removed promptly when people leave.

14. *Own server*: direct physical control of where the data is; can work on the office network even when the internet is down. *Cloud*: no hardware to buy or maintain, with backups, updates, and failover handled by the provider; easy to scale up as data grows, and reachable securely from anywhere. Useful questions include: "What happens if our office server fails at 2 a.m., and who fixes it?", "Are there rules about which country our customer data must stay in?", and "What would the cloud cost each month at our size, compared with buying and running our own server?"

15. Check the **sender's real email address**, not just the display name; check whether the attachment is really a PDF, with extensions visible, because `Payment_Details.pdf.exe` is a program; notice the **pressure** ("urgent", "today", "cancelled"), a classic phishing sign; and **confirm by phone**, using a number you already have rather than one in the email, before opening the attachment or acting on it. If in doubt, report it to IT without opening it.

Chapter 3. How a Business Runs on Data

Chapter at a glance

You will learn to: name a company's departments and the data each creates · follow one order from enquiry to cash, and say which system records each step · explain what ERP, CRM, HRMS, POS, e-commerce, and support systems do · tell a transaction from a report, and bookings from billings from collections · define a KPI so two people calculate the same number · describe how dashboards, meetings, and decision rights turn data into decisions · find where people copy, re-type, and email data by hand, and estimate the cost.

Before you start: Chapter 1 (rows, columns, grain, data quality) and Chapter 2 (files, databases, and APIs).

Time needed: 3–4 hours, including the exercises and the project.

Tools: a pen and a notebook. A spreadsheet or a free diagram tool is useful for the project, but optional.

Practice data: Riverstone Supplies' first quarter of 2026: twelve orders, ten invoices, and ten payments from the mini database you'll query yourself in Chapter 12. Every number in this chapter was calculated from that database and checked by script.

Why this matters

In Chapter 1 you learned what data is. In Chapter 2 you learned where it lives. This chapter answers the question that sits between those and every job in this book: **why does a business have all this data in the first place, and who uses it?**

The short answer is that a company is a chain of people handing work to each other. A salesperson hands an order to the warehouse. The warehouse hands a delivery to finance. Finance hands a number to the managing director. Every handover leaves a record, and those records are the data you'll clean, query, chart, and model for the rest of your career.

Analysts who understand this chain are far more useful than analysts who only know the tools. When a manager asks, "*What were sales last month?*", the analyst who knows the chain asks one question back: "*Orders placed, invoices raised, or cash received?*" Those are three different numbers. At Riverstone Supplies in January 2026 they were ₹1,16,210, ₹1,04,210, and ₹0. By the end of this chapter you'll know exactly why, and you'll never again hand over a "sales" number without saying which one it is.

This chapter also starts the book's **automation** thread. Before you can automate anything, you have to see where people work by hand. Section 3.7 shows you how.

In plain English

Think of a relay race. Four runners, one baton. Each runner takes the baton, runs their stretch, and hands it on. Races are often lost at the handovers.

A business works the same way, and **data is the baton**. The salesperson writes down what the customer wants and passes it on. The warehouse packs the goods, records what it shipped, and passes that on. Finance sends a bill, records what was paid, and passes it on to the people who write reports.

Three things follow from the picture:

- **Each runner only sees their own stretch.** The warehouse doesn't care how long the customer took to decide; sales doesn't see when the cash arrives. That's why different departments quote different numbers for the "same" thing.
- **Handovers are where things go wrong.** An order typed twice can be typed wrong once. A file emailed around can be out of date by the time it's opened.
- **Someone has to watch the whole race.** Managers need to know how the team is doing, not just each runner. That's what reports, KPIs, and dashboards are for, and it's where analysts come in.

3.1 The departments of a company and the data they create

Meet the company. **Riverstone Supplies** makes and distributes plastic storage boxes, kitchenware, industrial crates, and a small range of furniture. Its customers are retailers (like Sharma Hardware in Mumbai), hotels and caterers (like Green Leaf Hotels in Pune), and wholesalers (like Coastal Foods in Chennai). It has two plants:

- **Plant 1, Taloja** (Navi Mumbai), which makes storage boxes and kitchenware.
- **Plant 2, Chakan** (Pune), which makes industrial crates and furniture.

Finished goods go to the central warehouse, **Bhiwandi Main**, near Mumbai, and are shipped to customers from there.

Each department exists to do a job, and creates data as a side effect of doing it. Figure 3.1 shows the eight you'll meet most often.

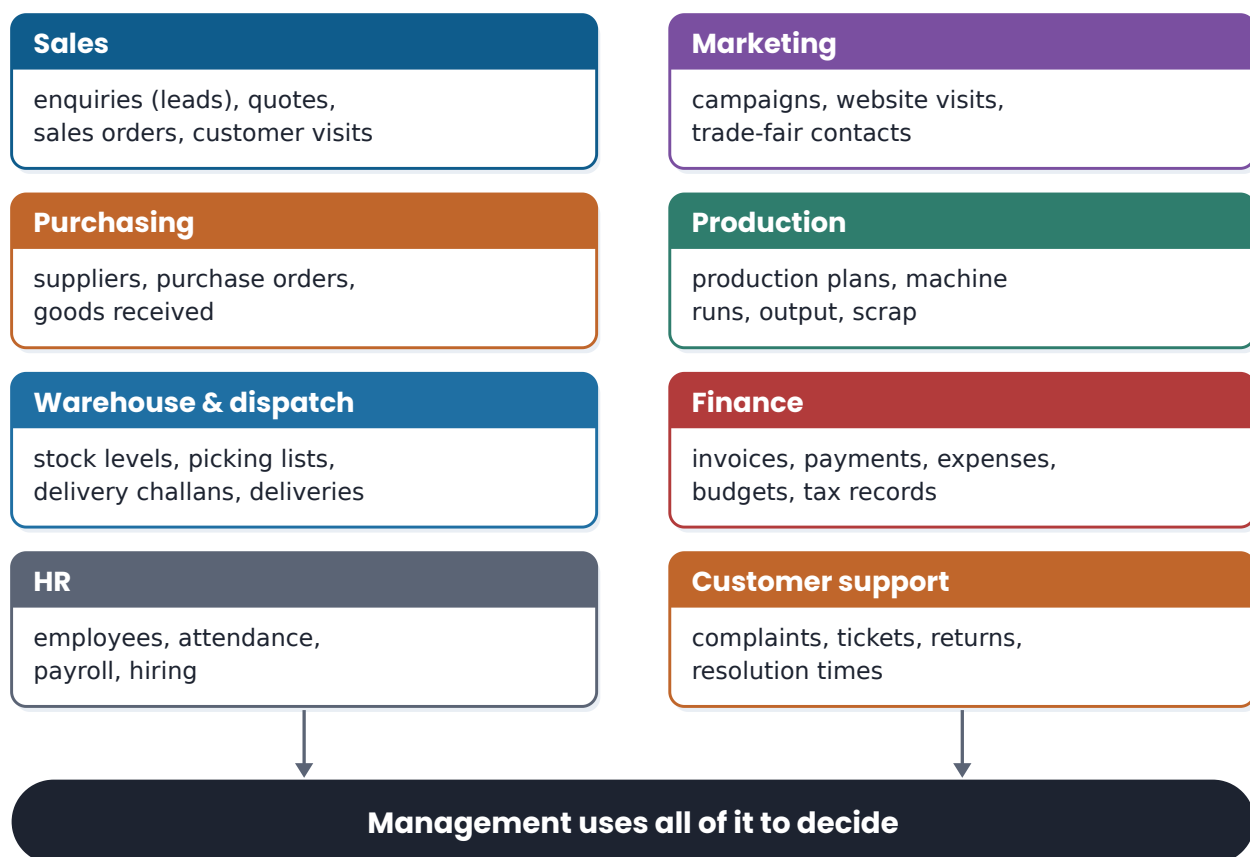


Figure 3.1 – Every department creates data while doing its own job. Management needs all of it to decide.

Department	Its job	Data it creates	A question the data answers
Sales	win and keep customers	enquiries (leads), quotes, orders, visit notes	<i>Which customers are ordering less than last year?</i>
Marketing	make the right people aware of the products	campaigns, website visits, trade-fair contacts	<i>Which campaign brought in enquiries that became orders?</i>
Purchasing	buy raw materials and packaging	suppliers, purchase orders, goods received	<i>Which supplier delivers late most often?</i>
Production	make the goods	production plans, machine runs, output, scrap	<i>How much plastic did we waste last month, and on which machine?</i>
Warehouse and dispatch	store, pick, pack, and ship	stock levels, picking lists, delivery challans , proofs of delivery	<i>How many orders left the warehouse late?</i>
Finance	bill customers, collect cash, pay bills, keep the books	invoices, payments, expenses, budgets, tax records	<i>How much are customers overdue, and who?</i>
HR	hire, pay, and look after employees	employee records, attendance, payroll, hiring	<i>How long does it take to fill a job?</i>
Customer support	fix problems after the sale	complaints, tickets , returns, resolution times	<i>Which product causes the most complaints?</i>

A **lead** is a possible customer who has shown interest but hasn't bought. A **delivery challan** is the document that travels with goods in India, listing what's in the truck (elsewhere, a delivery note).

Finance needs sales' orders to raise invoices. Production needs them to plan what to make.

Management needs everything to decide where to spend money. **Data created in one department is almost always used in another**, and that is the root of most of the problems, and most of the jobs, in this book.

Try it. Pick a company you've dealt with recently: a food delivery app, your bank, a hospital. Name three departments it must have and one record each creates about you.

3.2 Following one order from enquiry to cash

To see how departments depend on each other's data, follow one piece of business all the way through. This journey is called **order to cash**, or **lead to cash** when it includes winning the customer.

Here is a real order from Riverstone's database: **order 5001**, from Sharma Hardware, a retail customer in Mumbai. The order has two lines:

Product	Quantity	List price	Discount	Line value
Storage Box 10L	20	₹450	0%	₹9,000
Water Bottle 1L	50	₹120	5%	₹5,700
Total				₹14,700

Source: Mini database (Jan–Mar 2026).

Check it by hand: $20 \times ₹450 = ₹9,000$. $50 \times ₹120 = ₹6,000$, less 5% is ₹5,700. $₹9,000 + ₹5,700 = ₹14,700$.

✓ (As in the rest of this book, tax is left out to keep numbers simple.)

The order row only says *who ordered what, when*. But it has a history before it and a future after it, and every step was recorded somewhere. Figure 3.2 shows all ten steps.

	Step and date	What happened	Record it leaves
1	Enquiry 22 Oct 2025	Rakesh fills in the website form	CRM: new lead
2	New customer 4 Nov 2025	Credit check; 30-day terms	ERP: customer 1
3	Quote 22 Dec 2025	Neha sends prices by email	CRM: quote Q-2025-118
4	Order 5 Jan 2026	Rakesh emails; Neha re-types it	ERP: order 5001
5	Stock check 5 Jan 2026	Boxes and bottles made to stock at Taloja	ERP: stock reserved
6	Dispatch 6 Jan 2026	Bhiwandi Main picks, packs, ships	ERP: delivery challan
7	Invoice 6 Jan 2026	Raised when the order ships	ERP: invoice 9001
8	Delivery 8 Jan 2026	Rakesh signs the proof of delivery	Status: Delivered
9	Payment 2 Feb 2026	Bank transfer; finance matches it	ERP: payment 1
10	Report 3 Feb 2026	January sales report	Report: Jan ₹1,04,210

Ten steps and more than three months from a website enquiry to one line in a monthly report.

Source: Mini database (Jan-Mar 2026) for order 5001, invoice 9001 and payment 1.

Figure 3.2 – One order, ten steps. Each box shows the date, what happened, and the record it left behind.

1. Enquiry (22 October 2025). Rakesh, the buyer at Sharma Hardware, fills in the enquiry form on Riverstone's website: name, shop, city, phone, and "interested in storage boxes for our shop". The form creates a **lead** in the CRM, the sales team's system for tracking deals (section 3.3). Sharma Hardware isn't a customer yet; it's a row with a status of *New*.

2. New customer (4 November 2025). Neha Kulkarni, a sales executive, visits the shop. Rakesh wants to buy on credit, so finance runs a credit check and agrees **payment terms** of 30 days: Sharma Hardware can pay up to 30 days after each invoice. Finance creates the customer in the ERP, Riverstone's main business system, where it gets **customer ID 1** and a signup date of 2025-11-04. Every later order, invoice, and payment carries that ID.

3. Quote (22 December 2025). Rakesh asks for prices. Neha builds a quote, Q-2025-118, in a spreadsheet, emails it as a PDF, and logs it against the lead in the CRM. A **quote** is an offer: *these products, at these prices, until this date*.

4. Order (5 January 2026). Rakesh replies to the quote email: "Please send 20 of the 10-litre boxes and 50 bottles." Neha opens the ERP and **types the order in**: customer 1, two lines, prices, a 5% discount on the bottles that she agreed on the phone. The ERP creates **order 5001** with a status of *Pending*,

dated 2026-01-05, with Neha as the sales rep.

5. Stock check (5 January 2026). Both products are **made to stock**: Plant 1 at Taloja makes them in advance and sends them to Bhiwandi Main. But Bhiwandi Main updates the ERP's stock figures once a day from its own spreadsheet, so Neha phones the warehouse to be sure. The stock is **reserved** for order 5001.

6. Dispatch (6 January 2026). The warehouse prints a **picking list**, picks and packs the boxes and bottles, and ships them with a delivery challan. The ERP marks the order *Shipped*.

7. Invoice (6 January 2026). When goods ship, the ERP raises **invoice 9001** for ₹14,700, dated 2026-01-06, **due** on 2026-02-05, 30 days later, because of the terms agreed in step 2. An **invoice** is the formal request for payment: the sale becomes money the customer owes.

8. Delivery (8 January 2026). The truck arrives. Rakesh signs a paper **proof of delivery** (POD). Someone at Bhiwandi Main scans it, emails it to finance, and updates the order to *Delivered*.

9. Payment (2 February 2026). Sharma Hardware pays ₹14,700 by bank transfer. The bank statement shows a short reference, "SHARMA HW JAN". A finance assistant works out that it pays invoice 9001 and records **payment 1** against it.

10. Report (3 February 2026). The team prepares the **January sales report**. Invoice 9001 is one of the three behind the line "January: ₹1,04,210".

It took a long time. From enquiry to cash was 103 days. Once the order existed, things moved faster: order to invoice 1 day, invoice to payment 27 days, order to cash 28 days. Each gap comes from subtracting two dates recorded by two different departments.

The data lives in several places. The lead and quote are in the CRM; the customer, order, invoice, and payment in the ERP; the proof of delivery is a scanned image in someone's email; the report is an Excel file. To answer "How long does it take to turn an enquiry into cash?" you need two systems that don't share an ID for Sharma Hardware: the CRM knows it by a lead number, the ERP as customer 1.

People carried the baton by hand at several handovers. Neha re-typed the order and phoned about stock; someone scanned paper, matched a bank line by eye, and built the report. Section 3.7 puts a cost on these steps.

Simplification note. Real processes have more branches: partial shipments, back orders, credit holds, returns, and credit notes. Order 5001 is a clean run. You'll meet messier cases in the data itself: order 5004 was cancelled, invoice 9002 was paid in two instalments, and invoice 9007 hasn't been paid at all.

3.3 Business systems: ERP, CRM, HRMS, POS, and e-commerce

A **business system** is software a department uses for its daily work, storing the records in a database (section 2.6). You'll hear these abbreviations in your first week of any data job.

System	What it's for	Typical records	Main users	Examples of real products
ERP (enterprise resource planning)	running the core of the business: sales orders, stock, production, purchasing, invoicing, and accounts, all in one system	customers, products, orders, invoices, payments, stock movements, purchase orders	sales operations, warehouse, production, purchasing, finance	SAP, Oracle NetSuite, Microsoft Dynamics 365, Odoo; TallyPrime is common in smaller Indian firms
CRM (customer relationship management)	winning and keeping customers	leads, contacts, calls, meetings, quotes, deals and their stages	sales, marketing, customer success	Salesforce, HubSpot, Zoho CRM
HRMS (human resource management system)	managing employees	employee records, attendance, leave, payroll, hiring	HR, managers, every employee for leave and payslips	Workday, Zoho People, Keka
POS (point of sale)	recording sales at a shop counter	bills, items scanned, payment method, cashier, time	shop staff	the billing app at Sai Krupa General Store (Chapter 1); supermarket tills
E-commerce platform	selling online	product listings, carts, online orders, payments, website visits	online sales, marketing	Shopify, WooCommerce, marketplaces
Support desk (ticketing system)	tracking customer problems until they're fixed	tickets, messages, categories, status, time to resolve	customer support	Zendesk, Freshdesk

Riverstone's set-up is typical for a mid-sized manufacturer. This book names its systems by type, because the lessons don't depend on the brand:

- **The ERP** holds customers, products, employees (as sales reps), orders, order lines, invoices, and payments, plus stock, production, and purchasing. People at Riverstone still call its invoicing module "the billing system"; that's where Imran exported his Friday file from in Chapter 2. **The mini database behind this chapter's numbers is a small copy of the ERP's sales tables.**
- **The CRM** holds leads, contacts, quotes, and the sales pipeline.
- **The website** shows the catalog and feeds enquiries to the CRM (customers don't order online). **The support desk** records complaints and returns. **The HRMS** holds employees and payroll.
- **Spreadsheets and email** fill every gap between them: quotes, the warehouse's stock sheet, the monthly report.

Riverstone has no POS, because it sells to businesses. But Sharma Hardware's POS knows exactly which Riverstone boxes sell on a Saturday afternoon, and Riverstone never sees that unless Sharma shares it: **the most useful data about your products often sits in someone else's system.**

Systems of record and the gaps between systems

Every important fact should have one official source, its **system of record** (or **source of truth**). At Riverstone, that's the ERP for orders and invoices, the CRM for leads, and the HRMS for employees. When the CRM and the ERP disagree about what Sharma Hardware bought, the ERP wins.

The trouble is the gaps. Riverstone's CRM and ERP aren't connected: when a quote becomes an order, someone must mark the deal Won in the CRM, and a cancellation in the ERP never reaches the CRM unless someone remembers. Connecting systems so data flows between them automatically is **integration**.

Watch out: a spreadsheet can become the system of record by accident. If the warehouse's stock sheet is more current than the ERP, the sheet is now the source of truth for stock, with no access control or history (section 2.9).

3.4 Transactions and reports: booked, billed, and collected

Everything recorded in Figure 3.2 is a **transaction**: a record of one business event, written when it happens. *Invoice 9001 was raised for ₹14,700*. Transactions are the rows in a business system's database.

A **report** summarizes many transactions to answer a question ("*How much did we invoice in January?*"), at a point in time, using rules someone chose.

	Transaction	Report
What it is	one event	a summary of many events
Grain (Chapter 1)	one order, one invoice, one payment	one month, one customer, one product
Created	by the business system, as work happens	by a person or a scheduled job, after the fact
Changes?	should be corrected, not rewritten; a cancellation is a new status or a new record	changes whenever the data or the rules change
Example	invoice 9001: ₹14,700, due 2026-02-05	"January billings: ₹1,04,210"

Systems for transactions are tuned to write one record quickly and safely; reporting needs to read millions and add them up. That's why growing companies move reporting into a separate **data warehouse**.

Three numbers called "sales"

Because reports apply rules, **two reports built from correct transactions can give different answers to the same question**. The classic case is "sales", which can mean three things, each counted at a different step of Figure 3.2:

- **Bookings**: the value of orders placed (step 4). The earliest sign of sales work.
- **Billings**: the value of invoices raised (step 7). Money customers owe.
- **Collections**: cash that arrived (step 9). What pays salaries and suppliers.

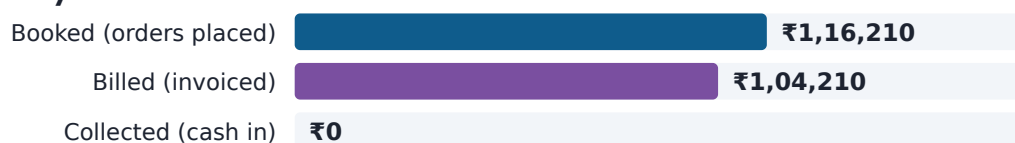
Here are all three for Riverstone's first quarter of 2026:

Month	Booked (orders placed)	Billed (invoiced)	Collected (cash in)
January	₹1,16,210	₹1,04,210	₹0
February	₹1,61,700	₹1,61,700	₹64,700
March	₹58,020	₹31,800	₹1,32,550
Quarter	₹3,35,930	₹2,97,710	₹1,97,250

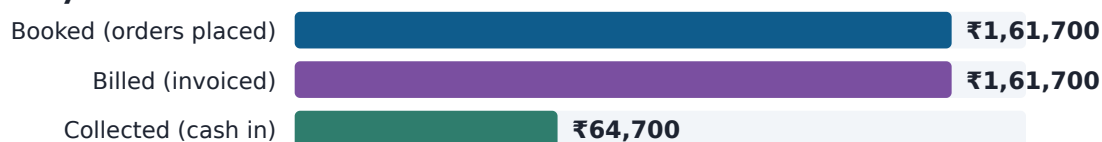
Source: Mini database (Jan–Mar 2026).

Riverstone, first quarter of 2026: three different numbers for “sales”

January 2026



February 2026



March 2026



Same scale for every bar (full grey track = ₹1,80,000). Source: Mini database (Jan–Mar 2026).

Figure 3.3 – Same company, same quarter, three honest answers to “what were sales?”

Every gap traces to specific transactions:

- **January booked ₹1,16,210 but billed ₹1,04,210.** The difference, ₹12,000, is order 5004 from Green Leaf Hotels: 100 water bottles, placed on 20 January and cancelled. Figure 3.3 and the table count bookings the way Riverstone's sales team reports them from the CRM, as every order placed, so the cancelled order is in. It was never shipped or invoiced.
- **January collected ₹0.** Every January invoice had 30-day terms, so none was due until February. Sharma Hardware's ₹14,700 arrived on 2 February.
- **February's three numbers.** All five February orders shipped and were invoiced in February, so bookings and billings match at ₹1,61,700. The ₹64,700 collected was for *January's* invoices: ₹14,700 from Sharma Hardware, ₹40,000 of Coastal Foods' ₹73,260, and ₹10,000 of Patel Kitchenware's ₹16,250.
- **March booked ₹58,020 but billed ₹31,800.** Order 5012 from Metro Mart, worth ₹26,220, is still *Pending* on 31 March: booked, not yet shipped, so not invoiced. ₹58,020 – ₹26,220 = ₹31,800. ✓

- **March collected more than it billed**, mostly February's invoices being paid. Cash lags billings by about the payment terms.

And the quarter reconciles:

- Bookings ₹3,35,930 – cancelled ₹12,000 – pending ₹26,220 = billings ₹2,97,710. ✓
- Billings ₹2,97,710 – collected ₹1,97,250 = **₹1,00,460 still owed by customers**, which finance calls **receivables** (or accounts receivable). ✓

Watch out: "sales" without a definition. If you don't know which of the three is meant, ask. If you can't, give the name and rule with the number: "Billed sales (invoices raised) in January: ₹1,04,210." Otherwise two departments argue about who is wrong when both are right.

Simplification note. Accountants recognize **revenue** under formal accounting standards, which decide exactly when a sale counts. This book uses invoices as a stand-in for revenue, as the rest of the book does. In a real company, ask finance which rule applies before publishing a revenue number; this is general information, not accounting advice.

When a report runs matters too

Timing also makes reports differ. Run on 31 March, a report shows order 5012 as *Pending*; run on 10 April, after it ships, it shows the order billed in April. Good reports state their run date and **cut-off** ("orders placed up to 31 March 2026"), which is why this book fixes "today" for each dataset.

3.5 What a KPI is

Managers can't read thousands of transactions or dozens of reports. They need a few numbers that show at a glance whether the business is on track.

A **metric** is any number you measure. A **KPI** (key performance indicator) is a metric chosen as one of the few that matter most, with a target, an owner, and a regular review.

Here are nine KPIs Riverstone's management could track, calculated for the first quarter of 2026 as of 31 March:

KPI	Definition	Q1 2026	Owner
Bookings	value of orders placed, excluding cancelled orders	₹3,23,930 (includes ₹26,220 pending)	Sales Head
Billings	value of invoices raised	₹2,97,710	Finance Manager
Collections	cash received from customers	₹1,97,250 (66.3% of billings)	Finance Manager
Gross margin	(billings – cost of the products sold) ÷ billings	22.6%	Finance Manager
Average order value (AOV)	billings ÷ number of invoiced orders	₹29,771	Sales Head

KPI	Definition	Q1 2026	Owner
Cancellation rate	cancelled orders ÷ all orders placed	8.3% (1 of 12)	Sales Head
Overdue receivables	unpaid amounts on invoices past their due date	₹88,760 of ₹1,00,460 owed	Finance Manager
Average days to collect	days from invoice to final payment, for fully paid invoices	31.6 days	Finance Manager
Active customers	customers with at least one non-cancelled order in the period	7 of 8	Sales Head

Source: Mini database (Jan–Mar 2026).

Check two of them by hand. **Gross margin:** the products on invoiced orders cost Riverstone ₹2,30,450 to make, so the margin is ₹2,97,710 – ₹2,30,450 = ₹67,260, and ₹67,260 ÷ ₹2,97,710 = 22.6%. ✓ **Average days to collect:** five invoices are fully paid, taking 27, 51, 26, 33, and 21 days; they add up to 158, and 158 ÷ 5 = 31.6 days. ✓ (Invoice 9001, the one from Figure 3.2, is the 27.)

A KPI needs a definition, not just a name

"Average order value" sounds precise. It isn't. Divide bookings by orders and you get ₹3,23,930 ÷ 11 non-cancelled orders = ₹29,448. Divide billings by invoiced orders and you get ₹29,771. Both are reasonable; they're different KPIs with the same name. You've already met the same problem with bookings: the table in section 3.4 counts the cancelled order (₹3,35,930 for the quarter), and the KPI table above leaves it out (₹3,23,930). A usable KPI definition answers six questions:

1. **Formula:** exactly what's divided by what?
2. **Inclusions and exclusions:** are cancelled orders in? Pending? Returns? Tax?
3. **Time:** which date decides the month: order date, invoice date, or payment date?
4. **Source:** which system and table? (The ERP's `invoices` table, not the CRM.)
5. **Owner:** who decides when the definition changes?
6. **Target and review:** what's good, and who looks at it how often?

Together, those answers are a **KPI definition**: Chapter 1's data dictionary one level up, saying what a number on a dashboard means.

Watch out: a KPI that can be improved without improving the business. If reps earn a bonus on bookings, a large order booked at quarter-end and later cancelled raises bookings and gains nothing. Pair KPIs that keep each other honest: bookings *with* cancellation rate, billings *with* collections.

Leading and lagging

Lagging indicators, like collections and gross margin, report what already happened: accurate, but too late to change. **Leading indicators**, like new leads, quotes, and bookings, move first: less certain, but early enough to act on. In Figure 3.2, the early steps lead and the late steps lag. A sales head who watches only collections learns about a bad quarter three months late.

3.6 Dashboards, meetings, and who decides what

Data changes nothing until someone uses it to decide, usually on **dashboards** and in **meetings**.

A **dashboard** is a screen of a few KPIs and charts, usually refreshed automatically from the systems of record, that answers the questions its viewer asks every week.

Companies look at their numbers on a rhythm. Riverstone's is typical:

When	Meeting	Who attends	What they look at	Decisions made
Every morning	dispatch stand-up at Bhiwandi Main	warehouse supervisor, dispatch team	orders to ship today, stock shortfalls	which orders ship first; what to chase from the plants
Every Monday	weekly sales review	Anita Rao (Sales Head), Vikram Singh (Sales Manager), sales executives	bookings, quotes sent, top open deals	which deals to push; which customers to visit
First week of each month	monthly business review	managing director, department heads	booked, billed, and collected; margin; overdue receivables	where to spend; which problems to fix; whether to change prices
Every quarter	board review	managing director, board of directors	results against the annual plan	strategy, investment, targets for next year

The closer a meeting is to daily work, the more detailed its data; the more senior, the more summarized and lagging. A common first analyst job is building the monthly business review pack.

Decision rights: who decides what

Written rules for who owns which decision are called **decision rights**. Riverstone's discount rules are an example: the bigger the discount, the more senior the approver.

Discount on an order line	Who approves	Data they need to decide
up to 5%	the sales executive	the customer's order history
over 5% and up to 10%	the Sales Manager (Vikram Singh)	order history, the margin on the product at that price
over 10%	the Sales Head (Anita Rao), after checking with the Finance Manager (Suresh Menon)	margin at that price, the customer's payment record, overdue amounts

The data follows the rules: Neha's 5% on order 5001 needed no approval. Rahul's 10% on Coastal Foods' orders 5002 and 5007 went to Vikram. Farah's 12% on Northgate Distributors' 60 industrial crates (order 5009) needed Anita.

Anita needed margin and payment data from finance and order history from the ERP. **The person who decides is rarely the person who holds the data.** Getting the right data to them, in time, in a form they can read in a minute, is the job this book trains you for.

3.7 Where manual work hides

Go back to Figure 3.2 and ask one question at every step: *did a person copy, re-type, check, or carry data by hand here?* Figure 3.4 marks the answers.

Step and date	What happened	Record it leaves
1 Enquiry 22 Oct 2025	Rakesh fills in the website form	CRM: new lead
2 New customer 4 Nov 2025	Credit check; 30-day terms	ERP: customer 1
3 Quote 22 Dec 2025	Neha sends prices by email	CRM: quote Q-2025-118 !
4 Order 5 Jan 2026	Rakesh emails; Neha re-types it	ERP: order 5001 !
5 Stock check 5 Jan 2026	Boxes and bottles made to stock at Taloja	ERP: stock reserved !
6 Dispatch 6 Jan 2026	Bhiwandi Main picks, packs, ships	ERP: delivery challan
7 Invoice 6 Jan 2026	Raised when the order ships	ERP: invoice 9001
8 Delivery 8 Jan 2026	Rakesh signs the proof of delivery	Status: Delivered !
9 Payment 2 Feb 2026	Bank transfer; finance matches it	ERP: payment 1 !
10 Report 3 Feb 2026	January sales report	Report: Jan ₹1,04,210 !

! A person copies, re-types, or checks data by hand at this step (6 of 10 steps)

Source: Mini database (Jan-Mar 2026) for order 5001, invoice 9001 and payment 1.

Figure 3.4 – Six of the ten steps depend on someone moving data by hand.

Step	What a person does by hand	What can go wrong
3. Quote	builds the quote in a spreadsheet, saves a PDF, emails it, and logs it in the CRM	old price list used; quote never logged, so the CRM undercounts
4. Order	reads the customer's email and re-types the order into the ERP	wrong quantity, wrong product code, discount forgotten
5. Stock check	phones the warehouse, because the ERP's stock figures are updated once a day from a spreadsheet	promises stock that's already gone
8. Delivery	scans a signed paper POD, emails it to finance, updates the order status	lost paper; order left as <i>Shipped</i> for weeks
9. Payment	reads the bank statement and matches each payment to an invoice	payment matched to the wrong invoice; customer chased for money they've paid
10. Report	exports data, pastes it into Excel, fixes formulas, emails the file	the Friday file problems from Chapter 2: wrong version, broken formulas, a report on one laptop

These steps have names:

- **Re-keying**: typing data that already exists in one place into another (step 4), inviting the typing errors from Chapter 1.
- **Copy-paste integration**: moving data between spreadsheets or systems by copying it (step 10).
- **Emailing files around** (steps 3, 8, and 10): the data stops updating the moment it's attached.
- **Manual matching**, or **reconciliation**: pairing up two sets of records by eye (step 9).
- **Shadow systems**: personal spreadsheets that hold data the official system should hold, like the warehouse's stock sheet (step 5).

Why it matters

Time. Manual work is small per item and large in total. Suppose a sales team re-types 40 emailed orders a week at about 6 minutes each: 240 minutes, or 4 hours a week, which over a 50-week year is 200 hours, about five working weeks of one person's time. (Round numbers for illustration; exercise 9 works a similar estimate.)

Errors, delay, and people. Mistyped orders are found when the customer receives the wrong goods. A report built by hand on the third of the month can't be seen on the first. And when only one person knows how the report is put together, like Imran and his Friday file in Chapter 2, it stops when they're on leave.

How to find manual work in any process

Ask five questions at every step:

1. **Where does this data first enter a system?** Ideally once, at the source (Chapter 1).
2. **Is it typed again anywhere later?** If yes, that's re-keying.
3. **Does it travel by email, chat, phone, or paper?** Each of those is a handover with no record in a system.
4. **Does anyone keep their own spreadsheet of it?** That's a shadow system.
5. **Does someone check or match things by eye?** That's reconciliation, and usually the slowest step.

Then estimate *how many times a week, how many minutes each*. A list of manual steps with hours attached is where every automation project starts.

Not every manual step should be automated

Anita approving a 12% discount is a judgment, and it should stay with a person; what can be automated is sending her the margin and payment data. And automating a broken process gives you a fast broken process: fix the process first. The book returns to each of Riverstone's manual steps later, and shows how to automate the ones that should be.

Interview extra point. When an interviewer asks, "What were sales last month?", or gives you a case with a "revenue" figure, say which definition you're using (booked, billed, or collected) before you calculate. It shows in one sentence that you understand the business, not only the tools. Chapters 75 and 76 have practice questions.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Quoting "sales" without saying which	two managers argue about a ₹12,000 difference	Name the number: booked, billed, or collected, with its rule
Treating a difference between reports as an error	hours spent hunting for a bug that isn't there	Reconcile first: cancellations, pending orders, timing, and payment terms explain most gaps
Assuming systems agree	CRM "won deals" total doesn't match ERP orders	Find the system of record for each fact and use it
Reporting a KPI without a written definition	the same KPI name gives different values in different files	Write the formula, inclusions, date rule, source, and owner
Rewarding a single KPI	bookings rise while cancellations rise too	Pair KPIs that keep each other honest
Ignoring the report's run date	the same report shows a different March on 10 April	State the cut-off and the date the report was run
Automating before fixing	wrong numbers delivered faster	Fix the process and the data, then automate
Automating away a judgment	a discount approved by a rule nobody questions	Keep decisions with people; automate getting them the data

In the real world: three numbers for January

It's Tuesday, 3 February 2026. The managing director's monthly business review is on Thursday, and two numbers for January have just reached the MD's inbox. Anita's weekly sales summary, pulled from the CRM's pipeline, says **January sales: ₹1,16,210**. Suresh Menon, the Finance Manager, sent his month-end report from the ERP: **January sales: ₹1,04,210**, and underneath it, **cash received in January: ₹0**.

The MD replies to both: "Which one is right? And why did we collect nothing?"

Anita asks Meera Iyer to find out. Meera doesn't start by deciding who's wrong. She asks: *which step of the order does each report count?*

1. What does each report count? Anita's summary adds up deals marked **Won** in the CRM, which happens when an order is placed: that's **bookings**. Suresh's report adds up invoices in the ERP: **billings**. His second line is **collections**.

2. What's in one and not the other? Meera lists January's orders from the ERP and January's won deals from the CRM side by side:

Order	Customer	ERP status	Value	In CRM "won"?	Invoiced?
5001	Sharma Hardware	Delivered	₹14,700	yes	yes, 9001
5002	Coastal Foods	Delivered	₹73,260	yes	yes, 9002
5003	Patel Kitchenware	Delivered	₹16,250	yes	yes, 9003
5004	Green Leaf Hotels	Cancelled	₹12,000	yes	no

Source: Mini database (Jan–Mar 2026).

The CRM total is ₹1,16,210. The invoiced total is ₹1,04,210. The difference is exactly order 5004: Green Leaf Hotels cancelled its 100 water bottles in the ERP, and nobody updated the deal in the CRM, because the systems aren't connected.

3. Why no cash? All three invoices had 30-day terms. The earliest, 9001, was due on 5 February. Sharma Hardware paid ₹14,700 yesterday, 2 February, three days early. Coastal Foods' due date is 9 February and Patel Kitchenware's is 14 February. Every invoice raised in January falls due in February, and no older invoices were waiting to be paid, so zero cash in January is exactly what 30-day terms predict.

4. Does it reconcile? ₹1,16,210 – ₹12,000 = ₹1,04,210. ✓

On Wednesday afternoon, Meera sends Anita and Suresh a half-page note:

"Both reports are correct; they count different steps. **Booked** in January (orders placed, per the CRM): ₹1,16,210. **Billed** (invoices raised, per the ERP): ₹1,04,210. The ₹12,000 difference is Green Leaf Hotels' order 5004, cancelled in the ERP but still marked Won in the CRM; I've asked Neha's team to update it. **Collected**: ₹0, because January's invoices aren't due until 5–14 February; ₹14,700 has already arrived. Suggestion: the monthly pack shows all three lines with a one-line definition under each, and the ERP is the source for billed and collected."

On Thursday the pack has three lines instead of one, and the meeting discusses what the numbers mean instead of which is true.

Meera used no tool or formula, only the order's journey, systems of record, three definitions of "sales", and a reconciliation to the rupee. She also found a failed manual handover and fixed it. That's this chapter, applied in an afternoon.

Tools

- **A notebook and pen.** Enough for every exercise, and the best way to draw your first process map.
- **A spreadsheet** (Excel or Google Sheets, optional). Useful for the project's step table and time estimates. Chapter 10 teaches both from the beginning.
- **A diagram tool** (optional). diagrams.net (also called draw.io) is free and runs in a browser; PowerPoint, Google Slides, and Google Drawings work too. Boxes and arrows are all you need.
- **The Riverstone mini database.** Not needed yet; Chapter 12 installs it and queries the orders, invoices, and payments you followed here.

The project: map the data flow of one process

Goal: map one real process the way Figure 3.2 maps order 5001, find its manual work, and propose one improvement.

Step 1. Choose a process you can observe or ask about: an expense claim, a customer return, a monthly report, or outside work, how a local shop restocks or a clinic books appointments.

Step 2. List 6 to 12 steps in order, recording six things for each:

Column	What to write
step	a number and a two-word name
who	the role (not a person's name)
what_happens	one sentence
data_created_or_used	the record: a form, a bill, a row in a system
where_it_lives	system, spreadsheet, paper, email, someone's memory
how_it_moves_on	automatic, typed again, emailed, phoned, carried

Step 3. Draw it. Boxes for steps, arrows for handovers, the record written under each box, as in Figure 3.2.

Step 4. Mark the manual work. Ask the five questions from section 3.7 at every step. Mark each manual handover with a symbol, as Figure 3.4 does with "!".

Step 5. Put a number on it: hours per month for each manual step, with your assumptions.

Step 6. Name one KPI that shows whether the process works (for example, "days from return request to refund"), defined with section 3.5's six questions.

Step 7. Propose one improvement in three sentences: what changes, what it saves, and what could go wrong.

Here is a short worked start, for the restocking process at Sai Krupa General Store from Chapter 1:

step	who	what_happens	data_created_or_used	where_it_lives	how_it_moves_on
1 Spot low stock	shop assistant	notices a shelf is nearly empty	item name	memory	told to the owner
2 Write list	owner	writes items and quantities to reorder	reorder list	paper notebook	photographed and sent by chat
3 Place order	distributor's salesperson	types the list into the distributor's app	purchase order	distributor's system	automatic
4 Receive goods	shop assistant	checks delivered items against the paper bill	delivery bill	paper	carried to the owner
5 Update stock	owner	adds received quantities in the billing app	stock levels	billing app (POS)	typed again

Steps 1, 2, and 5 are manual, and step 5 re-keys what the bill already says. If the billing app flagged low stock, step 1 wouldn't depend on someone noticing.

Deliverable: the table, drawing, hours estimate, KPI definition, and proposal. Keep it for Chapter 25.

You've got it when...

- ☐ I can name the main departments of a company and one kind of data each creates.
- ☐ I can walk through lead to cash for one order and say what record each step leaves, and in which system.
- ☐ I can explain what an ERP, CRM, HRMS, POS, e-commerce platform, and support desk are for.
- ☐ I can tell a transaction from a report, and I know what a system of record is.
- ☐ I never say "sales" without saying booked, billed, or collected, and I can reconcile the three.
- ☐ I can write a KPI definition that two people would calculate the same way.
- ☐ I can find re-keying, copy-paste, emailed files, manual matching, and shadow systems in a process, and estimate their cost in hours.
- ☐ I've mapped the data flow of one real process and proposed one improvement.

Recap

- A company is a chain of departments handing work to each other. **Every handover leaves data**, and data created in one department is almost always used in another.

- **Lead to cash** follows one piece of business from enquiry to payment. Riverstone's order 5001 took ten steps: 103 days from enquiry to cash, 28 from order to cash.
- **Business systems** record daily work: **ERP** (orders, stock, invoices, accounts), **CRM** (leads, quotes, deals), **HRMS** (employees, payroll), **POS** (shop sales), **e-commerce** (online orders), and **support desks** (tickets). Spreadsheets and email fill the gaps between them.
- Each fact should have one **system of record**. Systems that aren't **integrated** drift apart.
- A **transaction** records one event; a **report** summarizes many, using rules, at a point in time.
- "Sales" can mean **bookings** (orders placed), **billings** (invoices raised), or **collections** (cash received). For Riverstone's January: ₹1,16,210, ₹1,04,210, and ₹0, all correct. Reconcile them with cancellations, pending orders, and payment terms.
- A **KPI** is a chosen metric with a **definition**, an owner, a target, and a review. Pair KPIs so none can be gamed alone; watch **leading** as well as **lagging** indicators.
- **Dashboards and meetings** turn data into decisions on a rhythm. **Decision rights** say who decides; the decider rarely holds the data.
- **Manual work hides** in re-keying, copy-paste, emailed files, manual matching, and shadow systems. Find it with five questions, count it in hours, fix the process first, and keep judgment with people.

Practice exercises

Warm-up

1. Match each record to the department that creates it: (a) a delivery challan, (b) a purchase order to Western Polymers for plastic granules, (c) a payslip, (d) a complaint about a cracked crate, (e) a quote, (f) a record of how much plastic was scrapped on a machine, (g) a list of people who visited Riverstone's stall at a trade fair.
2. Transaction or report? (a) Payment 3: ₹33,260 against invoice 9002 on 2026-03-02. (b) "Overdue receivables by customer, as of 31 March 2026." (c) Order 5012, placed 2026-03-15, status Pending. (d) "Average days to collect in Q1: 31.6." (e) Ticket 208: "lid doesn't fit", opened 2026-03-18.
3. Which Riverstone system would you look in first to answer each question? (a) How many enquiries came through the website last week? (b) Has invoice 9007 been paid? (c) How many days of leave does Neha have left? (d) How many storage boxes are in stock at Bhiwandi Main? (e) Which quotes are still waiting for a customer's reply?
4. For each event, say whether it changes bookings, billings, collections, or none of them: (a) Metro Mart places an order. (b) Coastal Foods pays ₹20,000 against invoice 9006. (c) A customer asks for a quote. (d) Order 5011 ships and the ERP raises an invoice. (e) Order 5004 is cancelled before shipping. (f) A lead is marked *Qualified* in the CRM.

Core

5. February 2026 at Riverstone: orders 5005 to 5009 were placed (₹14,550, ₹14,640, ₹32,625, ₹23,325, ₹76,560), all shipped and invoiced in February, and none was cancelled. Payments received in February: ₹14,700 (invoice 9001), ₹40,000 (invoice 9002), ₹10,000 (invoice 9003).

Calculate February's booked, billed, and collected amounts, and explain in two sentences why collections are so much lower than billings.

6. A department head writes this KPI definition: *"Sales growth: sales this month compared with last month."* List at least four things that are missing or ambiguous, then rewrite it as a complete definition using the six questions in section 3.5.
7. In March 2026, three orders were placed: 5010 (₹20,100, Delivered), 5011 (₹11,700, Shipped), and 5012 (₹26,220, Pending). Only 5010 and 5011 were invoiced. Calculate (a) the cancellation rate for March, (b) average order value on billings, and (c) average order value on bookings. Explain why (b) and (c) differ.
8. List every manual handover in this process and name its type (re-keying, copy-paste, emailed file, manual matching, or shadow system): *"An employee fills in a paper expense form with receipts, and their manager signs it. The employee scans and emails it to finance. A finance assistant types each line into the ERP and checks the total against the receipts. Every Friday, the assistant copies the week's claims into the Finance Manager's budget spreadsheet."*
9. Riverstone's finance team types 25 supplier invoices a day into the ERP, taking about 3 minutes each, on 22 working days a month. How many hours a month is that? Show your working, and name two costs of this work that the hours don't capture.

Stretch

10. It's 31 March 2026. Northgate Distributors placed order 5009 on 25 February: 60 industrial crates at ₹1,450 with a 12% discount. Invoice 9008 was raised on 26 February with 30-day terms, due 28 March. Northgate paid ₹30,000 on 20 March. The order's status is *Shipped*. Answer: (a) Check the order value by hand. (b) How much is booked, billed, and collected for this order? (c) How much does Northgate still owe, and is it overdue? By how many days? (d) What would you check before calling Northgate about the balance?
11. Build a simple **KPI tree** for collections: write "Collections" at the top and break it into the two or three things that drive it, then break each of those down one more level. For each box at the bottom, name the department that can influence it.
12. Farah wants to offer Northgate Distributors 15% off another order of industrial crates, which cost ₹1,100 to make and list at ₹1,450. (a) Who approves it? (b) Calculate the gross margin per crate at list price, 12% off, and 15% off. (c) What other data should the approver look at, and what should they ask before agreeing?

Think about it (no calculation needed)

13. Riverstone is considering paying sales executives a commission of 1% of the bookings they bring in. Describe two ways this could reward the wrong behavior, and suggest a better basis for the commission, or a second KPI to pair with it.
14. At the monthly business review, the Sales Head wants "sales" to mean bookings and the Finance Manager wants it to mean billings. Who should decide, and how would you present the numbers so that the meeting can move on?
15. The Sales Manager makes ten CRM fields mandatory before a lead can be saved, including the customer's annual turnover. What will happen to the quality of those fields, and what would you suggest instead?

Key terms

department · lead · quote / quotation · order · delivery challan / delivery note · picking list · proof of delivery (POD) · invoice · due date · payment terms · payment · made to stock · lead to cash · order to cash · business system · ERP · CRM · HRMS · POS · e-commerce platform · support desk / ticketing system · ticket · system of record / source of truth · integration · transaction · report · data warehouse · bookings · billings · collections · receivables / accounts receivable · revenue · cut-off · metric · KPI · KPI definition · leading indicator · lagging indicator · gross margin · average order value (AOV) · cancellation rate · overdue · dashboard · decision rights · re-keying · copy-paste integration · reconciliation / manual matching · shadow system · KPI tree

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 4, Numbers Without Fear**, teaches the percentages, averages, and growth rates behind every KPI in section 3.5.
- **Chapter 5, Thinking Like an Analyst**, turns vague questions like "why is January low?" into precise ones, the way Meera did.
- **Chapters 10 and 12** put the order-to-cash records into tools: a spreadsheet sales tracker, then the ERP's orders, invoices, and payments tables in SQL, where you'll calculate booked, billed, and collected yourself.
- **Chapters 15 and 16** design and build dashboards like the ones in section 3.6.
- **Chapters 19 and 20** automate reports: macros and Apps Script first, then scheduled email reports and alerts. Chapter 20 automates a report of exactly this kind, Riverstone's Daily Sales Flash.
- **Chapter 23, Business Acumen, KPIs & Metrics**, builds a full KPI tree for Riverstone and adds finance and operations metrics such as days sales outstanding.
- **Chapter 25, The Business Analyst Track**, maps Riverstone's order-to-cash process formally and writes requirements for an improvement.
- **Chapters 45, 49, and 51** connect the systems: moving data from the ERP and CRM into a data warehouse (Chapter 49 explains how warehouses are built), and sending results back into them.

- **Chapter 58** automates the re-typing of emailed purchase orders with AI, with a person checking uncertain cases.
- **Interview preparation:** metric definitions, KPI trees, and business-process questions appear in Chapter 75 (product sense, metrics, and case studies) and Chapter 76 (the Business Analyst question bank), with model answers.

Answers to practice exercises

- (a) Warehouse and dispatch. (b) Purchasing. (c) HR (payroll). (d) Customer support. (e) Sales. (f) Production. (g) Marketing.
- (a) Transaction: one payment event. (b) Report: a summary of many invoices and payments at a cut-off date. (c) Transaction: one order. (d) Report: an average over several invoices. (e) Transaction: one support ticket.
- (a) The CRM, where the website form creates leads. (b) The ERP, where payments are recorded against invoices. (c) The HRMS. (d) The ERP, but its stock is updated once a day (section 3.2), so for an urgent answer also check with Bhiwandi Main. (e) The CRM.
- (a) Bookings. (b) Collections. (c) None: a quote isn't a sale. (d) Billings. (e) Bookings go down if you count bookings net of cancellations (the KPI definition in section 3.5); nothing is billed or collected. (f) None: it's a leading indicator, not a sale.
- Booked: ₹14,550 + ₹14,640 + ₹32,625 + ₹23,325 + ₹76,560 = **₹1,61,700**. Billed: all five were invoiced in February, so also **₹1,61,700**. Collected: ₹14,700 + ₹40,000 + ₹10,000 = **₹64,700**. Collections are lower because all three payments were for January's invoices; February's invoices have 30-day terms and weren't due until March.
- Missing: which "sales" (booked, billed, or collected); whether cancellations, returns, and tax count; whether "compared" means rupees or percent; which date decides the month; the source system; the owner and target. One good rewrite:

Question	Answer
Name	Monthly billings growth
Formula	(billings this month – billings last month) ÷ billings last month, as a percentage to one decimal
Inclusions and exclusions	invoices raised; net of discount; tax excluded; credit notes subtracted in the month they're raised
Time	invoice date decides the month; calendar months
Source	ERP, invoices table
Owner, target, review	Finance Manager; target set in the annual plan; reviewed at the monthly business review

- (a) 0 cancelled out of 3 placed: **0%**. (b) ₹20,100 + ₹11,700 = ₹31,800, divided by 2 invoiced orders = **₹15,900**. (c) ₹58,020 ÷ 3 orders placed = **₹19,340**. They differ because order 5012 (₹26,220), the largest of the three, is booked but not yet invoiced. Neither is wrong; a report must say which it uses.

8. (1) Paper form and receipts: data created on paper instead of in a system. (2) Scanning and emailing the form to finance: **emailed file**. (3) Typing each line into the ERP: **re-keying**. (4) Checking totals against receipts: **manual matching**. (5) Copying the week's claims into the Finance Manager's spreadsheet: **copy-paste** into a **shadow system** (the budget tracker lives outside the ERP).

9. $25 \times 3 = 75$ minutes a day. $75 \times 22 = 1,650$ minutes a month. $1,650 \div 60 = 27.5$ **hours a month**, more than three working days. Costs the hours don't show: typing errors (paying a supplier the wrong amount), delay (untyped invoices make reports of what Riverstone owes out of date), and dependence on the people who do it.

10. (a) $60 \times ₹1,450 = ₹87,000$; 12% off is ₹10,440; $₹87,000 - ₹10,440 = ₹76,560$. ✓ (b) Booked ₹76,560 (in February); billed ₹76,560 (in February); collected ₹30,000 (in March). (c) $₹76,560 - ₹30,000 = ₹46,560$ owed. It was due on 28 March, so on 31 March it's **3 days overdue**. (d) Whether the goods were delivered (the status is still *Shipped*, so Northgate may be waiting for them), whether a payment arrived but isn't recorded, and whether there's an open complaint. Three days overdue after a part-payment calls for a polite reminder, not an escalation.

11. One good tree (other sensible splits are fine):

- **Collections**

- **Billings**: orders shipped (sales, warehouse) · average order value (sales) · invoicing accuracy (finance)
- **Share paid on time**: payment terms and credit checks (sales, finance) · disputes from wrong deliveries or missing proof of delivery (warehouse, support)
- **Overdue recovered**: reminders and escalation (finance, sales)

When collections drop, you can ask which branch moved, and each branch has an owner.

12. (a) Over 10%, so the Sales Head, Anita Rao, after checking with the Finance Manager, Suresh Menon. (b) At list price: $(₹1,450 - ₹1,100) \div ₹1,450 = 24.1\%$. At 12% off: price ₹1,276; $(₹1,276 - ₹1,100) \div ₹1,276 = 13.8\%$. At 15% off: price ₹1,232.50; $(₹1,232.50 - ₹1,100) \div ₹1,232.50 = 10.8\%$. 15% off leaves less than half the list-price margin. (c) Northgate's payment record (₹46,560 overdue, exercise 10), the volume the discount would win, and competitors' prices. A sensible question before agreeing: "Can we settle the overdue balance first, or tie the discount to paying on time?"

13. (1) Reps can book large orders that are later cancelled: bookings rise, no cash arrives. (2) Reps are pushed toward customers who pay late, or heavy discounts at poor margins. Better: pay on **collections**, or on billings less cancellations and long-unpaid invoices; or pair bookings with **cancellation rate** and **gross margin**.

14. The managing director, because the definition affects everyone who reads the pack (often finance proposes and management approves). Meanwhile, don't pick a winner: show **booked, billed, and collected** as labeled lines with definitions and sources, reconciled, as Meera did.

15. People fill mandatory fields with anything that gets past the screen: "0", "NA", or a guess at turnover. The fields become full but untrustworthy, which is worse than blank (Chapter 1 lists this problem for data typed by people). Better: require only what's known at that stage (name, contact, interest), collect the rest later, use pick-lists, and offer an "unknown" option.

Chapter 4. Numbers Without Fear

Chapter at a glance

You will learn to: calculate percentages forward and backward, and see why a rise and an equal fall don't cancel · tell percentage points from percent change · choose the right denominator for a ratio or rate · measure growth month by month and year by year, and calculate compound growth and CAGR · choose between mean, median, and mode, and use weighted averages · round without changing the story · read tables and charts without being fooled · think about probability as "how often, out of how many" · estimate quickly and sanity-check any number · spot the number tricks in business news.

Before you start: Chapter 1 (especially section 1.6, levels of measurement) and Chapter 3 (bookings, billings, KPIs).

Time needed: 4–5 hours, including the exercises and the project.

Tools: a calculator with a power key (your phone's, turned sideways), a pen, and a notebook.

Practice data: Riverstone's 2025 sales from the one-year database: monthly revenue, targets, and margins, and 173 orders. Every number was checked by script.

Why this matters

Data work is mostly arithmetic. Not advanced math: percentages, averages, growth rates, and a little probability. The difficulty isn't the calculation. It's knowing *which* calculation answers the question, and noticing when a number that looks right is wrong.

Here are three sentences a manager at Riverstone could say in a meeting, each built on a real 2025 number:

- "Revenue grew 13.1% a month on average." True arithmetic, badly misleading. The steady monthly rate that gets from January to December is 7.3%.
- "Margin fell 3.9% in November." It fell 3.9 percentage points, which is a 14.0% fall.
- "Our average order is ₹24,243." That's an average of monthly averages. The real average order is ₹25,061, and half of all orders are below ₹21,375.

None of these is a lie. Each is a small slip that changes what people decide. By the end of this chapter you'll catch all three on sight, and you'll never be the person who makes them.

This chapter has no code. Every idea is worked by hand with real numbers, because being able to check a number on the back of an envelope is what makes you trustworthy when the tools do the work later.

In plain English

A shop has a sign: "50% off, and an extra 20% off at the till!" How much do you save?

It's tempting to add them: 70%. But the second discount is taken from the *already reduced* price. On a ₹1,000 item, 50% off leaves ₹500. Then 20% off ₹500 is ₹100 off, leaving ₹400. You pay 40% of the original price, so you save **60%**, not 70%.

That one example holds the most important idea in this chapter: **every percentage is a percentage of something**. When the "something" changes, the same percentage means a different amount. Most number mistakes in business, and most number tricks in the news, come from losing track of what the percentage is of.

Keep asking two questions and you'll be right far more often than wrong:

1. **Of what?** What's the base, the denominator, the starting point?
2. **Compared with what?** Last month, last year, the target, another group?

4.1 Percentages: of, back, and change

A **percentage** is a fraction out of 100. 12% means 12 out of every 100, or 0.12. There are three everyday calculations, and it helps to name them.

1. A percentage of a number. Multiply by the percentage as a decimal.

"What is a 12% discount on an Industrial Crate listed at ₹1,450?" $0.12 \times ₹1,450 = ₹174$. The crate sells for ₹1,450 – ₹174 = ₹1,276, which is the same as $₹1,450 \times 0.88$. (That's order 5009's price per crate, from Chapter 3.)

2. What percentage one number is of another. Divide the part by the whole.

"What share of 2025 revenue came from Sharma Hardware?" $₹5,02,775 \div ₹43,35,471 = 0.116$, or **11.6%**.

3. Percent change. The change, divided by the *starting* value:

$$\text{percent change} = (\text{new} - \text{old}) \div \text{old}$$

"How did November's revenue compare with October's?" October was ₹6,81,071, November ₹6,33,408. $(₹6,33,408 - ₹6,81,071) \div ₹6,81,071 = -0.070$, or **-7.0%**.

Working backward

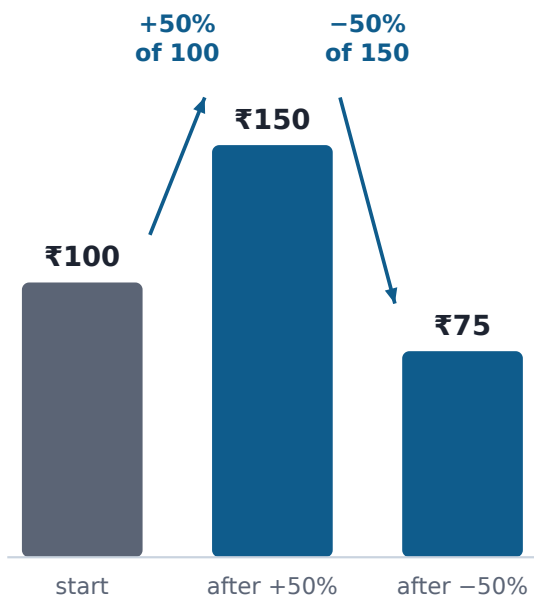
"Northgate paid ₹1,276 per crate after a 12% discount. What was the list price?"

The tempting wrong answer is to add 12% back: $₹1,276 \times 1.12 = ₹1,429.12$. That's wrong because the 12% was taken off ₹1,450, not off ₹1,276. The right way: ₹1,276 is 88% of the list price, so the list price is $₹1,276 \div 0.88 = ₹1,450$. ✓

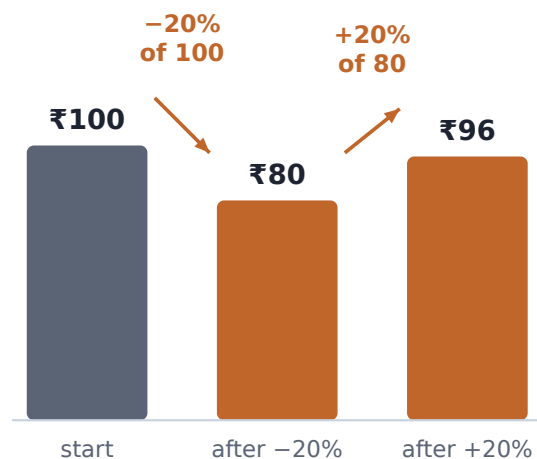
Watch out: undoing a percentage. To reverse "x % off", divide by $(1 - x \%)$. To reverse "x % added", divide by $(1 + x \%)$. Never add or subtract the same percentage again.

Rises and falls don't cancel

Up 50%, then down 50%



Down 20%, then up 20%



Each percentage is taken of a different starting number, so equal-looking rises and falls don't cancel out.

Figure 4.1 – A 50% rise followed by a 50% fall leaves you at 75, not 100.

Figure 4.1 shows why. Up 50% takes ₹100 to ₹150. Down 50% is half of ₹150, which is ₹75. Down 20% then up 20% leaves ₹96. The fall and the rise are each calculated on a different base.

Two consequences come up all the time:

- **Recovering from a fall needs a bigger rise.** After a 50% fall, you need a 100% rise to get back. After a 20% fall, you need 25%.
- **Discounts stack by multiplying.** A 10% trade discount followed by an extra 5% for early payment is $0.90 \times 0.95 = 0.855$ of the price, a total of **14.5% off**, not 15%.

4.2 Percentage points and percent change

When the thing that changes is *itself* a percentage, there are two different ways to describe the change, and mixing them up is one of the most common errors in business writing.

Riverstone's gross margin (Chapter 3, section 3.5) was **27.9%** in October 2025 and **24.0%** in November.

- The difference is $27.9 - 24.0 = 3.9$ **percentage points**. A **percentage point** is the unit for the plain difference between two percentages.
- The **percent change** is $(24.0 - 27.9) \div 27.9 = -14.0\%$. The margin is 14.0% smaller than it was.

Both are correct. They answer different questions, and they sound very different. "Margin fell 3.9%" is wrong: it mixes the size of one with the unit of the other, and it makes the fall sound less than a third as big as it was.

Here are three more from Riverstone's data:

Measure	Before	After	Change in points	Percent change
Gross margin, January → February 2025	29.2%	26.0%	−3.1 points	−10.8%
Revenue as % of target, August → September 2025	102.9%	146.9%	+44.0 points	+42.8%
Win rate, each enquiry counted once → duplicate enquiries counted too	20.0%	14.0%	−6.0 points	−30.2%

Source: One-year database (2025).

(The first row shows −3.1 points, not $29.2 - 26.0 = 3.2$, because it's calculated from the unrounded margins, 29.18% and 26.04%. Section 4.6 is about exactly this kind of rounding.)

Which should you report? Use **points** when you talk about the level of a rate: "conversion fell 6 points, from 20% to 14%". Use **percent change** when you want the relative size: "a 30% drop in conversion". Whenever you can, give the before and after values too, and the reader can see both.

Watch out: "percent" for a change in a percentage. When a rate, share, margin, or interest rate moves, say "percentage points" (or "points") for the difference. If a loan's interest rate goes from 8% to 9%, it rose 1 point, which is a 12.5% increase in what you pay in interest.

4.3 Ratios and rates: always ask about the denominator

A **ratio** compares two quantities by dividing one by the other. A **rate** is a ratio where the bottom is a unit of something else: per day, per customer, per 1,000 people. The bottom number is the **denominator**, and choosing it is the whole skill.

Here are Riverstone's three customer segments in 2025:

Segment	Revenue	Orders	Customers	Revenue per customer	Average order value
Wholesale	₹17,02,658	55	6	₹2,83,776	₹30,957
Retail	₹14,88,774	59	8	₹1,86,097	₹25,233
Hospitality	₹11,44,039	59	9	₹1,27,115	₹19,390
Total	₹43,35,471	173	23	₹1,88,499	₹25,061

Source: One-year database (2025).

Hospitality has the most customers and ties for the most orders, but the least revenue. Which segment is "biggest" depends entirely on the denominator you pick:

- **By customers:** Hospitality (9).
- **By revenue:** Wholesale (₹17,02,658).
- **Per customer:** Wholesale again, and by a lot. A wholesale customer brings in ₹2,83,776 ÷ ₹1,27,115 = **2.23 times** as much as a hospitality customer.

Rates also make different-sized periods comparable. ₹43,35,471 in a year is about **₹11,878 per day** (÷ 365), and 173 orders is about **3.3 orders per week** (÷ 52). Neither is a number anyone at Riverstone reports, but both are useful sanity checks: if someone claims a single ordinary day brought in ₹2 lakh, you know to ask what happened that day.

A **share** (or proportion) is a ratio where the part is inside the whole, like Sharma Hardware's 11.6%. Shares of a whole add up to 100%. Ratios between separate groups, like 2.23 times, don't add up to anything.

Try it. Riverstone collected ₹1,97,250 of ₹2,97,710 billed in the first quarter of 2026 (Chapter 3; mini database, Jan–Mar 2026). What's the collection rate? If next quarter it's 75.0%, how many points is that up, and what percent change?

4.4 Growth, compounding, and CAGR

Month-over-month growth

Here is Riverstone's revenue for every month of 2025, with the percent change from the month before.

Month	Revenue	Change from previous month
January	₹2,02,640	
February	₹2,53,664	+25.2%
March	₹2,78,008	+9.6%
April	₹2,10,282	−24.4%
May	₹3,29,359	+56.6%
June	₹1,86,928	−43.2%
July	₹2,32,692	+24.5%
August	₹3,29,282	+41.5%
September	₹5,58,315	+69.6%
October	₹6,81,071	+22.0%
November	₹6,33,408	−7.0%
December	₹4,39,824	−30.6%
Year	₹43,35,471	

Source: One-year database (2025). Monthly figures are rounded to the rupee, so they add to ₹43,35,473 (₹2,02,640 + ₹2,53,664 + ₹2,78,008 + ₹2,10,282 + ₹3,29,359 + ₹1,86,928 + ₹2,32,692 + ₹3,29,282 + ₹5,58,315 + ₹6,81,071 + ₹6,33,408 + ₹4,39,824); the exact annual total is ₹43,35,471.

The monthly changes swing wildly, from −43.2% to +69.6%. Most of that is the calendar: monsoon months are slow and the festive season is busy. Month-over-month percentages exaggerate seasonal patterns, which is why businesses with more than a year of data also compare each month with the same month last year.

The average of growth rates is a trap

"What was the typical monthly growth in 2025?"

The tempting method is to average the eleven monthly changes. They add up to 143.8, and $143.8 \div 11 = 13.1\%$. It sounds reasonable. It's wrong, and you can prove it: start with January's ₹2,02,640 and grow it by 13.1% eleven times. You get **₹7,82,621** for December. The real December was ₹4,39,824.

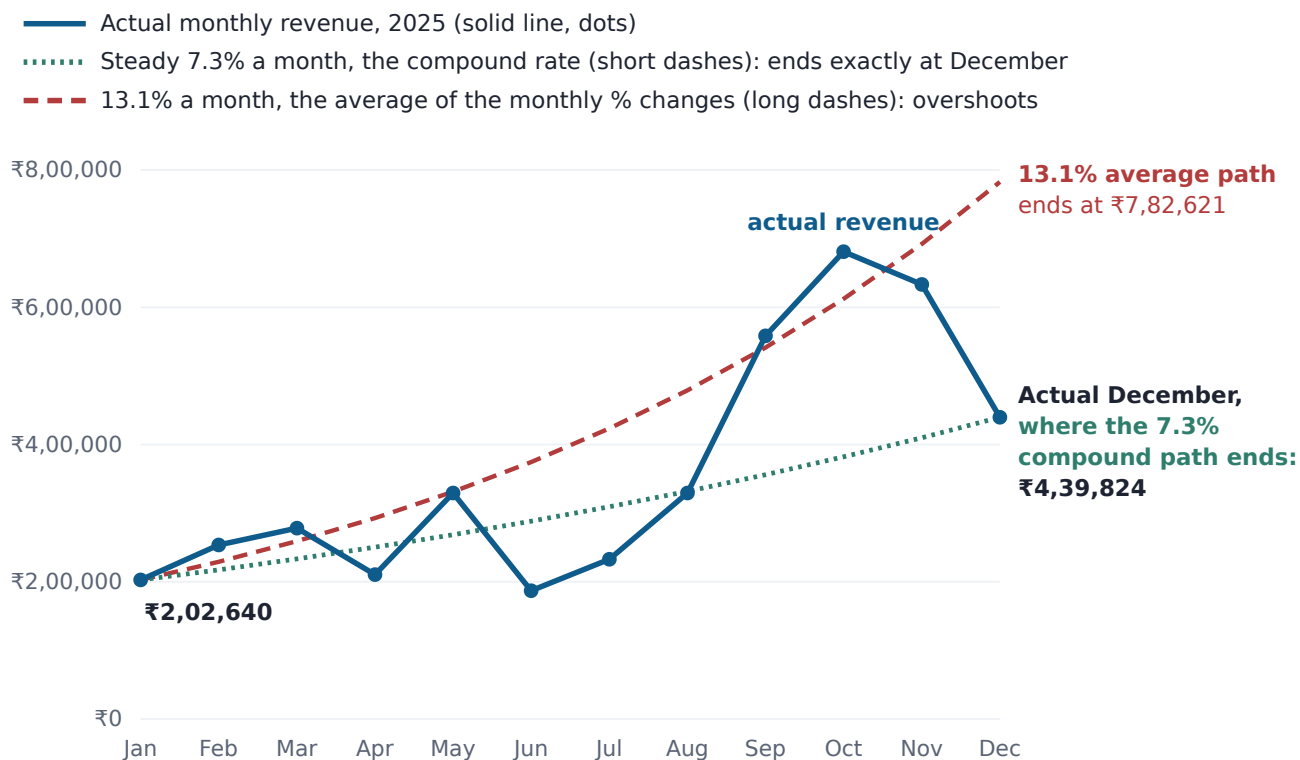
The problem is Figure 4.1 again. A +56.6% month and a −43.2% month don't cancel, because each is a percentage of a different base. Averaging percentages that compound on each other always overstates growth when the numbers bounce around.

The right question is: *what single, steady monthly rate would take ₹2,02,640 to ₹4,39,824 in eleven steps?* That's the **compound growth rate**:

$$\text{compound growth rate} = (\text{end} \div \text{start})^{1 \div \text{number of periods}} - 1$$

$₹4,39,824 \div ₹2,02,640 = 2.17$. The eleventh root of 2.17 is 1.073. So the compound monthly growth rate is **7.3%**. Grow ₹2,02,640 by 7.3% eleven times and you land exactly on December.

"The eleventh root" sounds hard, but a root only undoes a power. 1.073 multiplied by itself 11 times gives 2.17, so 1.073 is the "eleventh root" of 2.17. You never work it out by hand: type 2.17, press the power key (x^y), then $(1 \div 11)$.



Source: One-year database (2025).

Figure 4.2 – The 13.1% path, built from the average of the monthly changes, overshoots December by ₹3,42,797. The 7.3% compound path connects the real start and end.

Watch out: start and end points drive compound growth. The compound rate only uses the first and last values. From June (₹1,86,928, the lowest month) to October (₹6,81,071, the highest), revenue grew **264.3%**, a number that's true and tells you almost nothing about the year. Always ask why a growth figure starts and ends where it does.

Compounding

Compounding means growth that builds on previous growth. ₹100 growing 10% a year becomes ₹110, then ₹121, then ₹133.10 after three years. Simple growth, adding ₹10 a year, would give ₹130. The extra ₹3.10 is growth on growth. Over short periods the gap is small. Over long periods it dominates: that's why a loan's interest and a company's growth rate both matter so much over ten years.

A handy shortcut is the **rule of 72**: a quantity growing at $r\%$ a year doubles in about $72 \div r$ years. At 12%, about 6 years (the exact answer is 6.12). At 8%, about 9 years (exactly 9.01). It's for quick thinking in meetings, not for reports.

CAGR

CAGR (compound annual growth rate) is the compound growth rate when the periods are years. It's the standard way to describe growth over several years in company reports, investor decks, and interviews.

"Anita wants the business in the one-year database to reach ₹60 lakh (₹60,00,000) of revenue by 2028, three years after 2025's ₹43,35,471. What growth rate does she need each year?"

1. Ratio of end to start: $₹60,00,000 \div ₹43,35,471 = 1.384$.
2. Three years, so take the cube root: $1.384^{1/3} = 1.1144$.
3. Subtract 1: **CAGR = 11.4% a year.**

Check it by growing forward: $₹43,35,471 \times 1.1144 = ₹48,31,418$ in 2026, ₹53,84,098 in 2027, and ₹60,00,000 in 2028. ✓

Two tempting shortcuts both get the plan wrong:

- **Splitting the gap evenly.** $₹60,00,000 - ₹43,35,471 = ₹16,64,529$, or ₹5,54,843 a year. That's 12.8% of 2025's revenue, but a fixed rupee amount is a smaller percentage each year, so it isn't a growth rate at all.
- **Rounding down to a nice number.** 10% a year for three years reaches ₹57,70,512, which is ₹2,29,488 short. Over several years, one percentage point matters.

4.5 Averages: mean, median, mode, and weighted

An **average** is one number that stands for many. There are three common kinds, and Chapter 1 (section 1.6) showed that the level of measurement decides which ones make sense. This section shows how to choose between them for amounts, like order values, where all three are allowed.

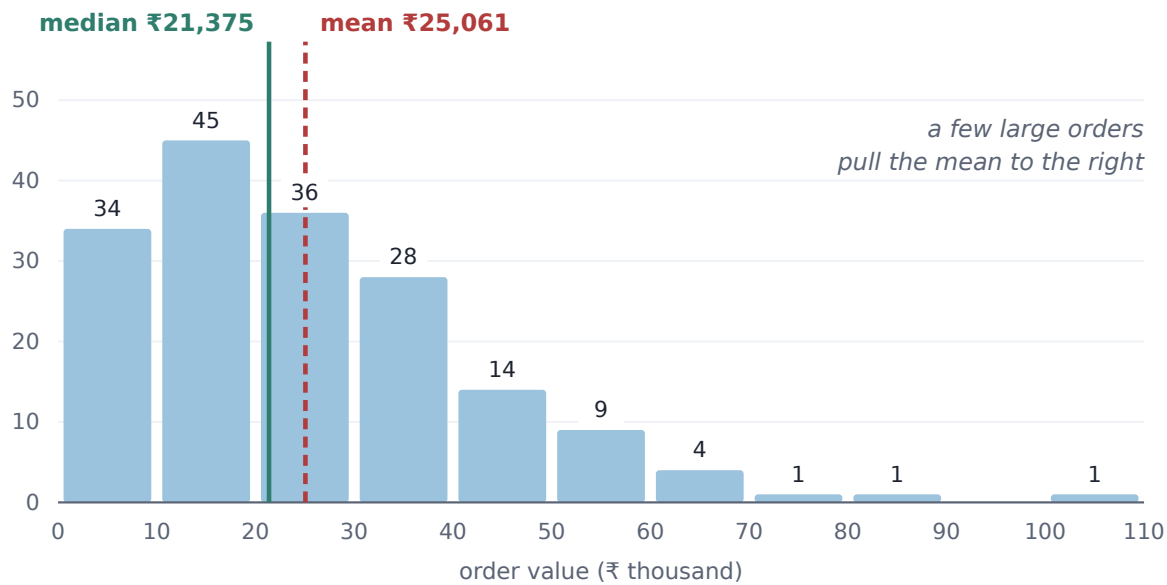
- The **mean** adds up the values and divides by how many there are. It's what most people mean by "average".
- The **median** is the middle value when the values are sorted. Half are below it, half above.
- The **mode** is the most common value.

Riverstone received 173 orders in 2025 (not counting the two that were cancelled), worth ₹43,35,471 in total.

- **Mean:** $₹43,35,471 \div 173 = ₹25,061$.
- **Median:** sort the 173 order values; the 87th is **₹21,375**.
- **Mode:** order values are almost never exactly equal, so the mode of the values themselves is useless here. The mode is useful for categories and repeated counts instead: the most commonly ordered product is the Storage Box 10L (on 76 order lines), the most common quantity on a line is 15, and the most common discount is 0% (140 of 326 lines).

173 orders in 2025, grouped by value

Height of each bar: number of orders in that ₹10,000 band



Source: One-year database (2025).

Figure 4.3 – Most orders are small; a few large ones stretch the tail to the right and pull the mean above the median.

Why is the mean higher than the median? Look at Figure 4.3. Most orders are under ₹40,000, but a handful are much larger: the biggest five are ₹65,818, ₹68,875, ₹74,218, ₹82,250, and ₹1,00,278. Large values pull the mean toward them; they don't move the median at all, because the median only cares about which value is in the middle. Data with a long tail on the high side is called **right-skewed**, and it's everywhere in business: order values, salaries, house prices, time to resolve a ticket.

The result: only **70 of the 173 orders (40.5%)** are above the "average" order. If a sales executive is told the average order is ₹25,061, most of their orders will feel below average.

Which to use?

Situation	Use	Why
Amounts that add up to a total you care about (revenue, cost, hours)	mean	mean × count = total, so it connects to budgets and plans
Amounts with a long tail (order values, salaries, delivery times)	median, alongside the mean	tells you what's <i>typical</i> ; the gap between them shows the skew
Categories (products, payment methods, segments)	mode	the only average that works for nominal data (Chapter 1)
Ratings and ranks (ordinal data)	median, plus the count in each category	the gaps between ratings aren't equal (Chapter 1)

Weighted averages

"What's Riverstone's average discount?"

Every order line has a discount of 0%, 5%, 8%, 10%, or 12%. The simple average of the discount on all 326 lines is **4.06%**. But a 12% discount on a ₹50,000 line costs far more than 12% on a ₹1,000 line. To know how much discounting costs the business, each line's discount must count in proportion to its value. That's a **weighted average**:

weighted average = $\text{sum of (value} \times \text{weight)} \div \text{sum of weights}$

Weighted by each line's value before discount, the average discount is **4.69%**. You can check it without a formula: the lines were worth ₹45,48,725 at list price and ₹43,35,471 after discounts, so discounts took ₹2,13,254, and $₹2,13,254 \div ₹45,48,725 = 4.69\%$. ✓ Larger lines tend to get larger discounts (5.3% on average for lines worth ₹20,000 or more, 3.8% for smaller ones), which is why the weighted figure is higher.

The same trap appears with prices. Riverstone's four product categories sold at average net prices of ₹435 (Storage), ₹341 (Kitchen), ₹1,273 (Industrial), and ₹1,121 (Furniture) per unit. The simple average of those four prices is ₹793. But the company sold 9,475 units for ₹43,35,471, an average of **₹458 per unit**, because Storage and Kitchen sold thousands of units and Furniture sold 30. **An average of averages ignores how many items stand behind each one.**

The average of averages

The monthly business review pack often shows a row of monthly averages. Averaging that row gives the wrong annual figure:

- Average of the twelve monthly average order values: **₹24,243**.
- Actual average order value for the year: $₹43,35,471 \div 173 \text{ orders} = \textbf{₹25,061}$.

The busy months (September to November, with 18 to 21 orders each and high order values) count the same as quiet January with 8 orders. To combine averages, go back to the totals: add up all the revenue, add up all the orders, then divide.

4.6 Rounding and significant figures

Rounding makes numbers readable. It also creates small puzzles that make careful readers distrust a report.

Shares that don't add up to 100%

Riverstone's 2025 revenue by segment, rounded to whole percentages:

Segment	Share (whole %)	Share (one decimal)
Wholesale	39%	39.3%
Retail	34%	34.3%
Hospitality	26%	26.4%
Total	99%	100.0%

Source: One-year database (2025).

Nothing is missing. The unrounded shares are 39.27%, 34.34%, and 26.39%, and each one rounded down a little. Rounding to one decimal happens to fix it here, but not always: the same thing can happen at any precision (exercise 8). You have three honest options: show one more decimal place, add a note ("shares may not add to 100% because of rounding"), or leave it and expect the question. Never quietly change one number to force the total, because then the table no longer matches the data.

Round at the end, not in the middle

Rounding in the middle of a calculation carries the error forward. Section 4.2's margin example showed it: from rounded margins, the January to February fall looks like 3.2 points; from the real ones, it's 3.1. Keep full precision in the spreadsheet or calculator, and round only what you show.

Significant figures and false precision

The **significant figures** of a number are the digits that carry meaning. ₹43,35,471 has seven. Does the reader need all seven? In a finance reconciliation, yes. In a meeting, "about ₹43 lakh" or "₹4.3 million" says the same thing and is easier to remember.

The opposite mistake is **false precision**: more digits than the data can support. "Customers take an average of 31.6 days to pay" (Chapter 3) is correct arithmetic from five invoices. With five invoices, "about a month" is more honest. The number of digits you write is a claim about how sure you are.

A rough guide for reports:

- **Money in tables:** whole rupees in detail tables; thousands or lakhs in summaries, with the unit stated in the header.
- **Percentages:** one decimal place, unless the numbers are small samples (then whole numbers or "about").
- **Averages of small counts:** round more, and give the count: "31.6 days (5 invoices)".

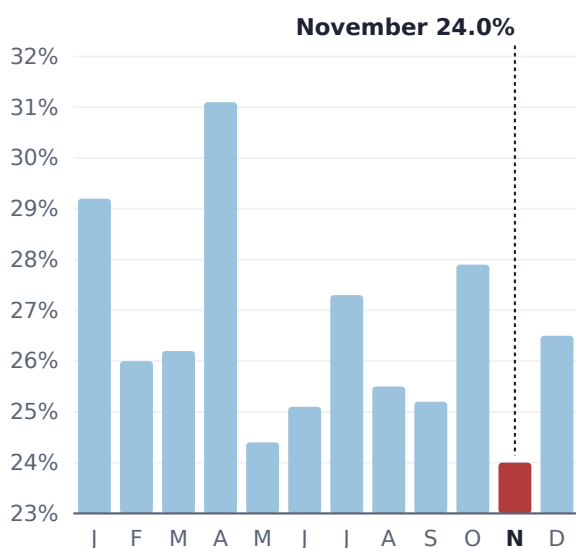
4.7 Reading tables and charts correctly

Before you read any number in a table or chart, read the frame around it. Five checks catch most misreadings:

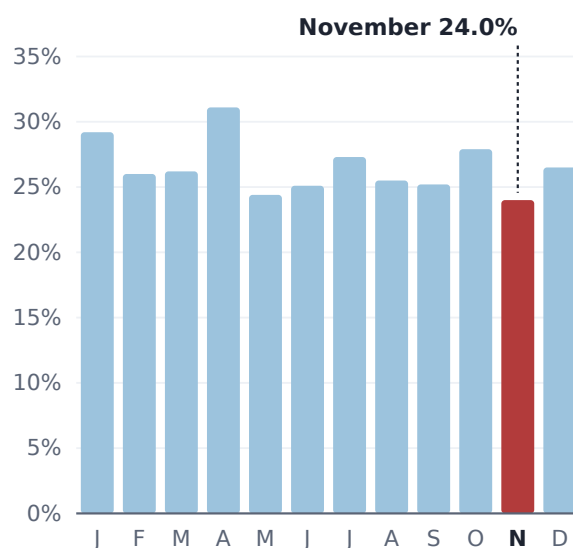
1. **Units.** Rupees, thousands, lakhs, crores, or millions? Percent or percentage points? A column headed "Revenue (₹ lakh)" with a value of 43.4 means ₹43,40,000.
2. **Period and cut-off.** A month, a quarter, year to date? Chapter 3 showed that the same report run on different dates gives different numbers.
3. **Per-period or cumulative.** A **cumulative** (running total) line always goes up as long as the values are positive, even in a terrible month. December 2025's revenue fell 30.6%, but the year-to-date line still rose.
4. **Definition.** Booked, billed, or collected? With or without cancellations? (Chapter 3.)
5. **The axis.** Where does it start?

The last one deserves a picture.

**Axis starts at 23%:
November looks like a collapse**



**Axis starts at 0%:
a dip of 3.9 points**



Riverstone's monthly gross margin in 2025. Both charts show exactly the same twelve numbers. Source: One-year database (2025).

Figure 4.4 — The same twelve numbers. A bar chart whose axis doesn't start at zero turns a 3.9-point dip into what looks like a collapse.

In the left chart of Figure 4.4, the axis starts at 23%, so November's 24.0% bar is about a fifth the height of October's 27.9%. The eye reads "November's margin was a fifth of October's". In the right chart, starting at zero, the bars are 24 and 28 units tall, and the dip looks like what it is.

The rule: **bar charts must start at zero**, because readers compare bar *lengths*. Line charts can start elsewhere, because readers compare *positions* and *slopes*, but the axis should be labeled clearly. Chapter 15 covers chart choice and design in depth.

Watch out: one chart, two axes. A chart with revenue on a left axis and margin on a right axis lets the designer stretch either line to make them appear to move together. When you see two vertical axes, read each line against its own axis before believing the pattern.

4.8 Probability: how often, out of how many

A **probability** is how often something happens, out of how many chances. Written as a fraction, a decimal, or a percentage, it's always between 0 (never) and 1 (always), or 0% and 100%. You don't need formulas to use it well; you need to keep asking "out of how many?"

Counting what happened

In 2025, **2 of 175 orders** were cancelled. As a rate: $2 \div 175 = 1.1\%$, or about 1 order in 88. In the first quarter of 2026 (the mini database), 1 of 12 orders was cancelled: **8.3%**.

Did Riverstone's customers get seven times more likely to cancel? Almost certainly not. With 12 orders, a single cancellation moves the rate by 8.3 points. **Small counts make rates jumpy.** Before comparing two rates, look at how many cases each is based on; Chapter 22 shows how to judge whether a difference is bigger than chance.

"At least one"

"If each order has a 1.1% chance of being cancelled, what's the chance that at least one of the next 20 orders is cancelled?"

It's easier to work out the chance that *none* is cancelled and subtract from 1. The chance one order goes through is $1 - 0.0114 = 0.9886$. The chance all 20 go through is 0.9886 multiplied by itself 20 times, which is 0.795. So the chance of at least one cancellation is $1 - 0.795 = 20.5\%$. Using the first quarter's 8.3% rate instead gives **82.5%**. The rate you assume changes the answer completely.

(This assumes one order's cancellation doesn't affect another's, which is called **independence**. If one big customer cancels several orders at once, the assumption breaks.)

Chaining stages

Riverstone's 2025 sales funnel took 30 unique leads to 22 contacted, 14 quoted, and 6 won. Each step has its own rate: 22 of 30 (73.3%), 14 of 22 (63.6%), and 6 of 14 (42.9%). The chance that a new lead is eventually won is the rates multiplied: $0.733 \times 0.636 \times 0.429 = 0.200$, or 20%, which matches 6 of 30. ✓

That makes planning concrete. To win 10 new customers at a 20% win rate, the sales team needs about $10 \div 0.20 = 50$ **leads**. To win more with the same number of leads, improve the weakest step.

The order of "given" matters

Of the 173 orders, 16 were worth more than ₹50,000, and 10 of those came from wholesale customers, who placed 55 orders in all.

- Chance an order is over ₹50,000: $16 \div 173 = 9.2\%$.

- Chance a **wholesale** order is over ₹50,000: $10 \div 55 = 18.2\%$. This is a **conditional probability**: the chance of one thing *given* another. The denominator shrinks to wholesale orders only.
- Chance a large order is **from wholesale**: $10 \div 16 = 62.5\%$.

The last two sound alike and differ by a factor of more than three, because the denominators differ. Mixing them up is one of the most common errors in medicine, law, and business alike: "most large orders are wholesale" is not the same as "most wholesale orders are large".

4.9 Orders of magnitude and quick estimation

The **order of magnitude** of a number is its rough size in powers of ten: thousands, lakhs, millions, crores. Getting the order of magnitude right catches more errors than any formula, because the most damaging mistakes are the ones that are off by a factor of 10 or 100: a misplaced decimal, a figure in thousands read as rupees, an extra zero.

Indian and international units

Indian business writing uses lakhs and crores; international writing uses thousands, millions, and billions. You'll switch between them constantly.

Indian	Written the Indian way	Written the international way	International
1 lakh	1,00,000	100,000	100 thousand
10 lakh	10,00,000	1,000,000	1 million
1 crore (100 lakh)	1,00,00,000	10,000,000	10 million
100 crore	1,00,00,00,000	1,000,000,000	1 billion

The Indian way puts a comma after the thousands and then after every two digits; the international way puts one after every three. Riverstone's 2025 revenue in the one-year database, ₹43,35,471, is about ₹43.4 lakh, ₹0.43 crore, or ₹4.3 million. This book writes rupee amounts the Indian way, as you've just seen. Numbers that aren't money, such as counts of orders or units, keep the international grouping.

Sanity checks: does the number fit?

When a number arrives, check it against another number you already trust. Two examples from Riverstone:

- **Revenue per day.** ₹43 lakh a year is about ₹11,878 a day. Separately, $173 \text{ orders} \div 52 \text{ weeks} \div 7 \text{ days} \times ₹25,061$ an order is about ₹11,911 a day. (Keep $173 \div 52 = 3.327$ orders a week unrounded: rounded to 3.3, it gives ₹11,814, the drift section 4.6 warns about.) Two different routes land within ₹50 of each other, so both numbers are probably sound.
- **Units from revenue.** October's revenue was ₹6,81,071, and the year's average price was ₹458 a unit. So October probably shipped about $₹6,81,071 \div ₹457.57 \approx 1,488$ units. The database says **1,625**. The estimate is 8.4% low, because October sold relatively more low-priced kitchen items and fewer ₹1,450 industrial crates than the year as a whole, but it's the right order of magnitude. If the database had said 16,250 or 162, you'd know something was wrong before looking at a single row.

Estimating from nothing

Sometimes there's no data yet: "Is it worth building a report for this?" or, in interviews, "How many plastic storage boxes are sold in Mumbai each year?" These are called **Fermi estimates** or **guesstimates**. The method is always the same:

1. Break the unknown into smaller pieces you can guess: households, share that buy boxes, boxes per purchase, purchases per year.
2. Write each assumption down with a round number.
3. Multiply, and round the answer to one or two significant figures.
4. Sanity-check the answer against anything you know, and say which assumption matters most.

Being close matters less than being clear. A reader can replace a bad assumption; they can't fix reasoning they can't see.

4.10 Number tricks in business news

Most misleading numbers in headlines, press releases, and sales decks are true. They mislead through what they leave out. Here are the common tricks, with invented examples, and the question that exposes each one.

Trick	Example	Question to ask
Tiny base	"Profits up 300%!" (from ₹2 lakh to ₹8 lakh)	<i>300% of what?</i>
Relative without absolute	"New process doubles the risk of a defect" (from 1 in 10,000 to 2 in 10,000)	<i>What are the actual numbers before and after?</i>
Cherry-picked period	"Revenue up 264% since June" (the slowest month to the busiest)	<i>Why start and end there? What about the same period last year?</i>
"Up to"	"Up to 70% off" (on one item)	<i>How many items, and what's the typical discount?</i>
Percent for points	"Interest rates rise 1%" (from 8% to 9%)	<i>Points or percent?</i>

Trick	Example	Question to ask
Average hiding the spread	"Average order ₹25,061" (most orders are smaller)	<i>What's the median?</i>
Truncated axis	a bar chart starting at 23% (Figure 4.4)	<i>Where does the axis start?</i>
Mixed units	"₹43 lakh" next to "₹4.3 million" in the same slide	<i>Are these in the same unit?</i>
Record without context	"Our best month ever!" (October, in a seasonal business)	<i>Best compared with the same month last year?</i>
Survivors only	"Our customers grew 40% on average" (counting only customers who stayed)	<i>Who was left out?</i>

A good habit is to rewrite a headline number into a plain sentence with both numbers and the base: "Profit rose from ₹2 lakh to ₹8 lakh, a fourfold increase from a small base." If the rewritten sentence sounds much less exciting, the original was relying on a trick.

Interview extra point. When a case interview gives you a growth figure or a percentage, restate it with its base and period before you use it: "So revenue went from ₹20 lakh to ₹32 lakh over four years, which is about 12.5% a year compounded." It shows you check numbers before trusting them, which is what interviewers for analyst roles are testing. Chapter 73 (statistics and probability) and Chapter 75 (metrics, case studies, and guesstimates) have practice questions.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Adding percentages that apply one after another	"50% + 20% off = 70% off"	Multiply what remains: $0.5 \times 0.8 = 0.4$, so 60% off
Reversing a percentage by adding it back	list price from a discounted price comes out too low	Divide by $(1 - \text{discount})$
Saying "percent" for a change in a percentage	"margin fell 3.9%" when it fell 3.9 points	Use points for the difference; give before and after
Averaging growth rates that compound	typical monthly growth of 13.1% when the steady rate is 7.3%	Use $(\text{end} \div \text{start})^{1/\text{periods}} - 1$
Quoting growth between hand-picked months	"up 264%" from the slowest to the busiest month	Compare like with like: full years, or same month last year
Reporting only the mean for skewed data	most orders feel "below average"	Report the median too
Averaging averages	₹24,243 instead of ₹25,061	Go back to the totals and divide once
Using a simple average when items differ in size	average discount 4.06% instead of 4.69%	Weight by value, units, or count
Rounding in the middle of a calculation	small errors that grow, points that don't match	Keep full precision; round only what you show

Mistake	Symptom	Fix
Too many digits	"31.6 days" from five invoices	Match the digits to how sure you are; give the count
Bar charts with a truncated axis	a small dip looks like a collapse	Start bars at zero
Comparing rates from very different sample sizes	1.1% vs 8.3% cancellation read as a real change	Give the counts; be cautious with small samples
Confusing "A given B" with "B given A"	"most large orders are wholesale" read as "most wholesale orders are large"	Say the denominator out loud

In the real world: the board slide

It's the first week of January 2026. The managing director is presenting Riverstone's year to the board, and Vikram Singh's team has drafted the sales slide. Before it goes to the MD, Anita asks Meera Iyer to check the numbers. "Nothing fancy. Make sure nothing on it will embarrass us."

The draft slide has six claims. Meera takes them one at a time, and for each one asks: *of what, and compared with what?*

1. "2025 revenue up 117%." Meera finds where it comes from: December's ₹4,39,824 against January's ₹2,02,640. That's true, but it compares two single months of a seasonal business, and it isn't "2025 revenue" at all. The slide has no 2024 figures to compare with, so it can't support any year-on-year growth claim. What the board can use: *full-year revenue ₹43.4 lakh (₹43,35,471), 102.3% of the annual target.*

2. "Average monthly growth: 13.1%." The average of the eleven monthly changes. Meera grows January by 13.1% eleven times and gets ₹7,82,621 for December, almost double the real figure. The compound rate is 7.3%, but she recommends dropping the line entirely: in a year that peaks in October and dips in the monsoon, a "monthly growth rate" describes nothing real.

3. "Gross margin down 3.9% in November." It was 27.9% in October and 24.0% in November: down 3.9 points, a 14.0% fall. More useful for a board: *full-year gross margin 26.2%; November was the lowest month.*

4. "Average order value: ₹24,243." Meera recognizes the number from the monthly pack: it's the average of the twelve monthly averages. The real figure is $₹43,35,471 \div 173 = ₹25,061$. She adds the median, ₹21,375, because the MD is likely to be asked what a typical order looks like.

5. The segment pie chart: Wholesale 39%, Retail 34%, Hospitality 26%. They add to 99%. A board member will notice. One decimal fixes it: 39.3%, 34.3%, 26.4%.

6. The margin bar chart. Its axis starts at 23%, so November's bar is a stub. Meera redraws it from zero (Figure 4.4).

There's also a note under the slide's goal for 2028, ₹60 lakh: "That's about ₹5.5 lakh more each year." Meera adds the growth rate that the plan really needs, **11.4% a year**, because the board will think in percentages and "₹5.5 lakh a year" gets harder every year as a percentage.

She sends the corrected slide back with a four-line note explaining each change and the check behind it. Vikram's reply: "The 117% was the best number on the slide." Anita's: "It was also the one the board would have questioned first."

Nothing Meera did needed more than a calculator and two questions. What it needed was the habit of never passing a number on without knowing what it's a percentage of.

Tools

- **A calculator.** Your phone's calculator in landscape (scientific) mode has a power key (x^y or $^$) for compound growth.
- **A pen and a notebook.** Write each calculation out in full, with its units, so you can check it later. When spreadsheets arrive in Chapter 10, you'll redo this chapter's numbers there.

The project: check five statistics

Goal: take five numbers from the real world and check each one the way Meera checked the board slide.

Step 1. Collect five statistics. Find them in news articles, company annual reports or investor presentations, advertisements, or presentations at your workplace. Aim for variety: at least one growth figure, one percentage or share, one average, one chart, and one comparison between two groups. Copy the exact wording and note the source and date.

Step 2. Record each one in a table.

Column	What to write
claim	the exact words, e.g. "Sales up 40% in three years"
source_and_date	where and when it was published
base	what the number is a percentage <i>of</i> , or an average <i>of</i>
comparison	what it's compared with: which period, group, or target
type	percent change, points, share, ratio, average, probability, chart
check	your recalculation, or the missing information you'd need
trick_found	none, or which trick from section 4.10
rewrite	an honest one-sentence version with both numbers and the base

Step 3. Recalculate whatever you can from the numbers given. For growth over several years, work out the CAGR. For a percentage, find both the part and the whole. For a chart, check the axis and the units.

Step 4. Name what's missing. Many claims can't be checked because the base, the period, or the sample size isn't given. Saying exactly what's missing is a result.

Step 5. Rewrite each claim as one honest sentence.

Step 6. Write a three-line summary: how many of the five held up, the most common problem, and the one question you'll ask next time you see a number like it.

Deliverable: your table and summary. Keep it; Chapter 5 asks you to turn one of these claims into an analyst's question.

You've got it when...

- ☐ I can calculate a percentage of a number, a share, a percent change, and a list price from a discounted price.
- ☐ I know that a rise and an equal fall don't cancel, and that discounts stack by multiplying.
- ☐ I say "percentage points" for the difference between two percentages.
- ☐ I ask what the denominator is before comparing ratios or rates.
- ☐ I calculate compound growth and CAGR, and I never average growth rates that compound.
- ☐ I choose between mean, median, and mode for a purpose, and use weighted averages when items differ in size.
- ☐ I don't average averages.
- ☐ I round only at the end, and I don't write more digits than the data supports.
- ☐ I check a chart's units, period, and axis before reading its bars.
- ☐ I can work out "at least one" and conditional probabilities by counting.
- ☐ I sanity-check numbers by estimating them another way.
- ☐ I've checked five real statistics and rewritten each one fairly.

Recap

- **Every percentage is a percentage of something.** Ask of *what?* and *compared with what?*
- Percent change is $(\text{new} - \text{old}) \div \text{old}$. Reverse a discount by dividing by $(1 - \text{discount})$. Rises and falls don't cancel, and discounts stack by multiplying.
- The difference between two percentages is measured in **percentage points**. Riverstone's margin fell from 27.9% to 24.0%: 3.9 points, a 14.0% fall.
- **Ratios and rates** depend on the denominator. Wholesale earned 2.23 times as much per customer as Hospitality, though Hospitality had more customers.
- **Compound growth** = $(\text{end} \div \text{start})^{1/\text{periods}} - 1$. Riverstone's 2025 compound monthly rate was 7.3%; averaging the monthly changes gives a misleading 13.1%. **CAGR** is the yearly version: ₹43.4 lakh to ₹60 lakh in three years needs 11.4% a year. The **rule of 72** estimates doubling time.
- The **mean** (₹25,061 per order) is pulled up by large values; the **median** (₹21,375) shows what's typical; the **mode** suits categories. Use **weighted averages** when items differ in size (average discount 4.69%, not 4.06%), and never average averages.

- **Round at the end.** Shares may not add to 100%; say so or add a decimal. Don't claim more precision than the data has.
- **Read the frame first:** units, period, cumulative or not, definition, and where the axis starts. Bar charts start at zero.
- **Probability** is how often, out of how many. Small counts make rates jumpy. "At least one" = 1 – chance of none. Chained stages multiply. "A given B" isn't "B given A".
- **Estimate** to catch errors of 10× or 100×, and convert confidently between lakhs, crores, and millions.
- **Number tricks** leave out the base, the absolute numbers, the period, or the spread. Rewrite the claim with both numbers and the base.

Practice exercises

Warm-up

1. Calculate: (a) a 5% discount on an order worth ₹14,550; (b) what percentage ₹26,220 is of ₹58,020 (order 5012's share of March 2026 bookings, Chapter 3); (c) the price of a ₹780 Storage Box 25L after 8% off.
2. From Riverstone's 2025 revenue by quarter and category (one-year database): Kitchen revenue was ₹1,33,888 in the first quarter of 2025 and ₹4,61,146 in the fourth; Industrial was ₹2,75,450 in the third quarter and ₹2,92,040 in the fourth. Calculate each percent change.
3. Wholesale's share of revenue was 39.3% in 2025. If it's 42.0% next year, what's the change in percentage points, and what's the percent change? Write one correct sentence for each.
4. The 11 non-cancelled orders in the mini database (first quarter of 2026) were worth: ₹14,700, ₹73,260, ₹16,250, ₹14,550, ₹14,640, ₹32,625, ₹23,325, ₹76,560, ₹20,100, ₹11,700, ₹26,220. Find the mean, the median, and the mode. Which better describes a typical order, and why?

Core

5. (a) Northgate paid ₹76,560 for an order after a 12% discount. What was the order worth at list price? (b) Riverstone raises a price by 8%, then gives a customer 8% off the new price. Is the customer paying more or less than before, and by what percent?
6. A distributor's revenue grew from ₹25 lakh to ₹40 lakh over five years. (a) What was its CAGR? (b) Using the rule of 72, roughly how long would it take to double at that rate? (c) Why is "it grew 12% a year" ($60\% \div 5$) wrong?
7. Riverstone's 2025 gross margin by category was: Storage 26.9% on ₹22,97,974 of revenue; Kitchen 33.2% on ₹12,08,030; Industrial 13.6% on ₹7,95,830; Furniture 24.2% on ₹33,638. (a) What's the simple average of the four margins? (b) What's the margin weighted by revenue? (c) Which one is the company's gross margin, and why are they different?
8. The categories' shares of 2025 revenue, to one decimal place, are Storage 53.0%, Kitchen 27.9%, Industrial 18.4%, and Furniture 0.8%. A manager says the table is wrong. Explain what's happening and write the note you'd put under the table.

9. Riverstone wins 20% of its unique leads. (a) How many leads does it need to win 15 new customers? (b) If three new leads arrive this week and each has a 20% chance of being won, what's the chance at least one is won? (c) What assumption does (b) make, and when might it be false?

Stretch

10. A local newspaper profile of Riverstone runs the headline "Riverstone revenue soars 264%", based on June 2025 (₹1,86,928) and October 2025 (₹6,81,071). Explain in two sentences why the headline misleads, and write an honest replacement using numbers from this chapter.
11. November 2025's revenue was ₹6,33,408. Using the year's average price of ₹458 per unit, estimate how many units Riverstone shipped in November. The database says 1,355. How far off is your estimate, in percent? Is that good enough for a sanity check, and why?
12. In 2025, Riverstone received 29 orders worth ₹7,34,312 in the first quarter and 59 orders worth ₹17,54,302 in the fourth. (a) Calculate each quarter's average order value. (b) Calculate the average order value for the two quarters combined. (c) Explain why the average of your two answers in (a) isn't the answer to (b).

Think about it (no calculation needed)

13. A company's press release says, "Our employees earn an average of ₹18 lakh a year." What else would you want to know before believing that this describes a typical employee?
14. A sales dashboard shows only a year-to-date revenue line, and it has gone up every month. Explain how this chart could hide a bad month, and suggest a better chart to put next to it.
15. A sales manager announces, "Our win rate went up 50% this quarter!" Write the two questions you'd ask before sharing the news.

Key terms

percentage · percent of · share / proportion · percent change · reverse percentage · percentage point · ratio · rate · denominator · month-over-month growth · compound growth rate · compounding · rule of 72 · CAGR (compound annual growth rate) · average · mean · median · mode · right-skewed · weighted average · average of averages · rounding · significant figures · false precision · cumulative / running total · truncated axis · probability · independence · conditional probability · order of magnitude · lakh · crore · sanity check · Fermi estimate / guesstimate · base effect · relative vs absolute change

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 5, Thinking Like an Analyst**, turns the questions of *what?* and *compared with what?* into a method for breaking down any business question.

- **Chapters 10 and 11** build these calculations into spreadsheets: percentages, weighted averages, and pivot tables of shares and averages.
- **Chapter 13** calculates month-over-month growth, running totals, and moving averages in SQL on the same 2025 data you used here.
- **Chapter 15, Data Visualization Principles**, goes deeper into honest charts: axes, chart choice, and the visual tricks from section 4.7.
- **Chapter 21, Descriptive Statistics & Probability**, adds spread, percentiles, distributions, and Bayes' rule.
- **Chapter 22, Statistics Without Fooling Yourself**, shows how to tell whether a difference between two rates is real or chance.
- **Chapter 23, Business Acumen, KPIs & Metrics**, applies growth rates, margins, and ratios to reading a company's financial statements.
- **Interview preparation:** percentage, growth, averages, and probability questions appear in Chapter 73 (statistics, probability, and experimentation), and guesstimates and metric questions in Chapter 75, with model answers.

Answers to practice exercises

- (a) $0.05 \times ₹14,550 = ₹727.50$. (b) $₹26,220 \div ₹58,020 = 0.452$, so **45.2%**. (c) $₹780 \times 0.92 = ₹717.60$.
- Kitchen: $(₹4,61,146 - ₹1,33,888) \div ₹1,33,888 = +244.4\%$; revenue more than tripled. Industrial: $(₹2,92,040 - ₹2,75,450) \div ₹2,75,450 = +6.0\%$.
- $42.0 - 39.3 = 2.7$ **percentage points**: "Wholesale's share rose 2.7 points, from 39.3% to 42.0%." $(42.0 - 39.3) \div 39.3 = 6.9\%$: "Wholesale's share of revenue grew by 6.9%." The first is clearer for most readers.
- Total $₹3,23,930 \div 11 = \text{mean } ₹29,448$. Sorted: ₹11,700, ₹14,550, ₹14,640, ₹14,700, ₹16,250, **₹20,100**, ₹23,325, ₹26,220, ₹32,625, ₹73,260, ₹76,560; the sixth value is the **median, ₹20,100**. No value repeats, so there's **no mode**. The median describes a typical order better: two large wholesale orders (₹73,260 and ₹76,560) pull the mean up, and 8 of the 11 orders are below it.
- (a) $₹76,560 \div 0.88 = ₹87,000$. (Check: 12% of ₹87,000 is ₹10,440, and $₹87,000 - ₹10,440 = ₹76,560$. ✓)
(b) $1.08 \times 0.92 = 0.9936$. The customer pays **0.64% less** than the original price, because the 8% discount is taken from a bigger number than the 8% rise was.
- (a) $₹40 \text{ lakh} \div ₹25 \text{ lakh} = 1.6$. $1.6^{1/5} = 1.0986$, so the CAGR is **9.9% a year**. (b) $72 \div 9.9 \approx 7.3$ **years** (the exact answer is 7.4). (c) $60\% \div 5 = 12\%$ ignores compounding: each year's growth is calculated on a bigger base, so a steady 9.9% a year is enough to add 60% in five years. Growing ₹25 lakh by 12% a year for five years would reach about ₹44 lakh, not ₹40 lakh.
- (a) $(26.9 + 33.2 + 13.6 + 24.2) \div 4 = 24.5\%$. (b) Weighted by revenue, the margin is **26.2%**: total revenue ₹43,35,471 minus total product cost ₹31,98,250, divided by revenue. (c) The weighted figure is the company's gross margin. The simple average gives tiny Furniture (under 1% of revenue) the same say as Storage (more than half), and it gives low-margin Industrial (18.4% of revenue) a quarter of the weight, which drags the simple average down.

8. Nothing is wrong. The unrounded shares (53.00%, 27.86%, 18.36%, 0.78%) add up to 100%, but three of them rounded up, so the rounded shares add to 100.1%. Note: "Shares are rounded to one decimal place and may not add to exactly 100%." Don't adjust one share to force the total.

9. (a) $15 \div 0.20 = 75$ **leads**. (b) Chance none is won: $0.8 \times 0.8 \times 0.8 = 0.512$. Chance at least one is won: $1 - 0.512 = 48.8\%$. (c) It assumes the leads are **independent** and each has the same 20% chance. It's false if, for example, all three come from the same company or from a trade fair where one bad impression affects everyone, or if one lead is far warmer than the others.

10. It compares the slowest month with the busiest month of a seasonal business, so most of the "growth" is the calendar, and it says nothing about the year. A single-month comparison can't be called revenue growth for the company. Honest replacement: "Riverstone's 2025 revenue was ₹43.4 lakh, 2.3% above its annual target, with a festive-season peak of ₹6.8 lakh in October."

11. $₹6,33,408 \div ₹458 \approx 1,383$ **units** (using the unrounded ₹457.57, about 1,384). Against the actual 1,355, that's about **2.1% too high**. That's good enough: a sanity check is looking for errors of 10 times or 100 times, and an estimate within a few percent confirms the order of magnitude. Differences in the product mix explain the rest.

12. (a) First quarter: $₹7,34,312 \div 29 = ₹25,321$. Fourth quarter: $₹17,54,302 \div 59 = ₹29,734$. (b) $(₹7,34,312 + ₹17,54,302) \div (29 + 59) = ₹24,88,614 \div 88 = ₹28,280$. (c) The average of the two quarterly figures, ₹27,528, gives each quarter equal weight, but the fourth quarter had twice as many orders. The combined figure must come from the totals.

13. Whether "average" is the mean or the median (a few very high salaries, such as senior leaders', pull the mean up); the median and the range; who is included (full-time only? contractors? leaders?); whether it's salary alone or includes bonuses and benefits; and the date and location. The median, with the number of employees, describes a typical employee far better.

14. A cumulative line rises whenever a month's revenue is positive, so a month that fell sharply (like December 2025, down 30.6%) still shows as the line going up, only less steeply. Put a monthly bar chart next to it (starting at zero), ideally with each month's target or the same month last year, so a bad month is visible as a short bar.

15. (1) *From what to what?* A rise from 10% to 15% is "up 50%" but only 5 points; a rise from 2% to 3% is also "up 50%". (2) *Out of how many leads, and were they counted the same way?* With a small number of leads, one or two extra wins can move the rate a lot, and a change such as removing duplicate leads raises the win rate without any change in selling.

Chapter 5. Thinking Like an Analyst

Chapter at a glance

You will learn to: ask the questions that turn a request into useful analysis · rewrite a vague request as a precise question tied to a decision · write hypotheses that data can prove wrong · break a problem into an issue tree whose branches are MECE (no overlaps, no gaps) · tell facts from opinions and assumptions · check a claim or chart before believing it, including correlation that isn't causation · recognize the biases that bend how people read data, including your own · structure a decision so that data can inform it.

Before you start: Chapter 1 (the data-to-insight ladder), Chapter 3 (booked, billed, and collected), and Chapter 4 (percentages, averages, and small numbers).

Time needed: 3–4 hours, including the exercises and the project.

Tools: a pen and paper, or a whiteboard. Issue trees are easiest to draw by hand first.

Practice data: Riverstone's first quarter of 2026 (the mini database) for the worked example, and its 2025 CRM leads (the one-year database) for the story. Every number was checked by script.

Why this matters

Two analysts get the same message from the Sales Head: *"March was terrible. Can you look into it?"*

The first opens the database, pulls every number about March, builds twelve charts, and sends a 15-slide deck two days later. It's accurate. Nobody knows what to do with it.

The second replies with one question: *"Do you mean billed revenue, and is this for the plan you're making with the team next week?"* Then she lists five possible reasons revenue could have fallen, checks each against the data in an afternoon, rules out three, and sends a half-page note: *"Most of the fall is three customers who ordered in February and not in March. All three owe us money. I'd call them before offering anything."*

Both know the same tools. The second knows how to **think about the problem before touching the data**, and that's what separates analysts people ask for by name from analysts who produce reports.

It's a method, not a talent: ask what the question is for, list the possible answers before looking, break the problem into pieces that don't overlap, test each piece, and notice when your own expectations are steering you. Every later chapter gives you a new tool. This chapter decides whether you point it at the right thing.

In plain English

Think about a good doctor.

You walk in and say, "I feel terrible." A bad doctor prescribes something for "feeling terrible". A good doctor asks questions. *Since when? Where does it hurt? Any fever? Did you eat anything unusual?* In their head, they're listing possible causes and ruling them out one by one. Only then do they decide on treatment, and they tell you what to watch for in case they're wrong.

An analyst does the same thing with a business problem:

- **"I feel terrible"** is the vague request: *"Sales are down."*
- **The doctor's questions** turn it into a precise question: *which sales, compared with when, and what are you trying to decide?*
- **The list of possible causes** is a set of hypotheses, organized into an issue tree.
- **The tests** are queries, counts, and comparisons.
- **The diagnosis and treatment** are the conclusion and the recommendation.

A doctor who orders every test on every patient isn't thorough; they're lost. The same is true of an analyst who pulls every number.

5.1 Curiosity and asking good questions

Four kinds of question

Chapter 1 (section 1.2) introduced the ladder from data to insight. Business questions climb the same ladder, and knowing which rung a question is on tells you what kind of work it needs.

Kind	The question	Riverstone example	What it needs
Descriptive	What happened?	<i>What was billed revenue in March 2026?</i>	Counting and summarizing
Diagnostic	Why did it happen?	<i>Why did March fall 80.3% from February?</i>	Breaking down, comparing, testing hypotheses (this chapter)
Predictive	What will happen?	<i>What will April's revenue be?</i>	Patterns over time and models
Prescriptive	What should we do?	<i>Should sales offer a discount to customers who didn't reorder?</i>	Options, criteria, and judgment (section 5.8)

Most requests arrive as descriptive questions but are really diagnostic or prescriptive underneath. "What was March revenue?" usually means "Is March a problem, and what should we do about it?" Answering only the surface question is the most common way to do correct work that doesn't help.

The habit questions

Keep these in your head, or on a card next to your screen:

1. **What is this for?** What decision will the answer inform, and who makes it?
2. **Of what?** What exactly is being counted or measured? (Chapter 4)
3. **Compared with what?** Last month, last year, the target, another group? (Chapter 4)
4. **How do we know?** Where does the number come from, and can we trust it? (Chapter 1)
5. **So what?** If the answer is X, what changes? If nothing changes whatever the answer, why are we asking?
6. **What else could explain it?** Before accepting the first explanation, name at least one other.

The last one is the one people skip, and it catches the most mistakes.

5.2 From a vague request to a precise question

Real requests are almost always vague, because the people asking are busy and know what they mean. Your first job is to turn the request into a question precise enough that two analysts would answer it the same way.

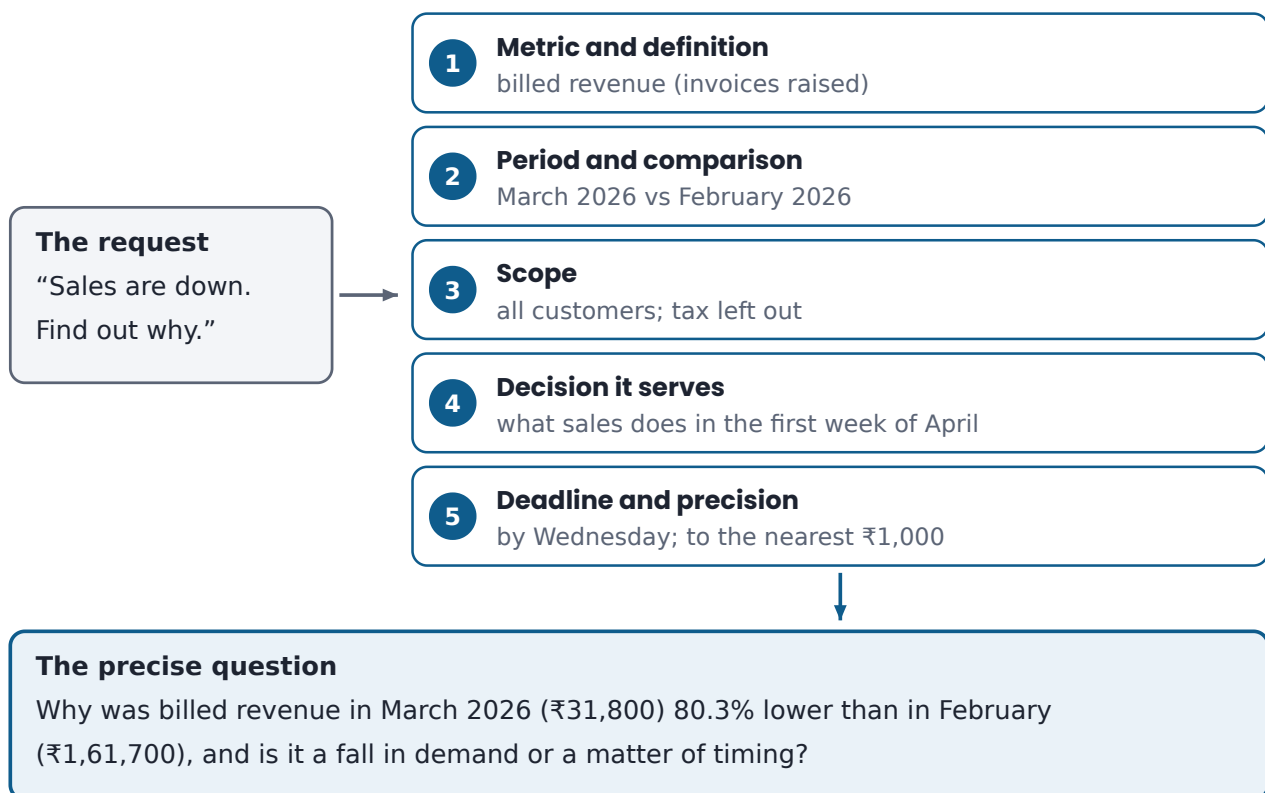


Figure 5.1 – Five checks turn a vague request into a question with one answer.

A precise question pins down five things:

1. **The metric and its definition.** "Sales" could be booked, billed, or collected (Chapter 3). Name one.
2. **The period and the comparison.** March 2026 compared with February? With January? With the quarter's average? Each gives a different story: March was 80.3% below February, 69.5% below January, and 68.0% below the quarter's monthly average of ₹99,237.
3. **The scope.** All customers? One segment? Including the pending order or not?
4. **The decision it serves.** "What should the sales team do in the first week of April?" needs a different depth from "What do we tell the board?"
5. **The deadline and precision.** An answer by Wednesday to the nearest ₹1,000 is a different job from a perfect answer next month.

You don't always need to ask all five. Often you can choose sensibly, state it, and let the requester correct you: "I'm reading 'sales' as billed revenue and comparing March with February; tell me if you meant something else." That one sentence prevents most wasted work.

Chapter 3's question, made precise

In Chapter 3's story, the managing director asked, "Which one is right? And why did we collect nothing?" about January. Meera's first move was to turn "January sales" into three precise questions: What was booked in January? What was billed? What was collected? Once the questions were precise, the "disagreement" disappeared, because the two reports were answering different ones. **A surprising number of business arguments are two people answering different questions with correct numbers.**

Try it. Take this request from a store manager: "Our customers aren't happy. Can you check?" Write down three different precise questions it could mean, each with a metric, a period, and a comparison.

5.3 Hypotheses: possible answers you can test

A **hypothesis** is a possible answer to a question, stated clearly enough that data could show it's wrong. It's a guess with a test attached.

For "Why did March 2026 billed revenue fall 80.3% from February?" here are some hypotheses:

- **H1.** Fewer customers placed orders in March.
- **H2.** Customers placed orders of the same number but smaller size.
- **H3.** Orders were placed in March but haven't been invoiced yet.
- **H4.** Wholesale customers, who place the largest orders, didn't order in March.
- **H5.** March is always a slow month for this business.
- **H6.** A competitor cut prices and took our customers.

What makes a hypothesis useful

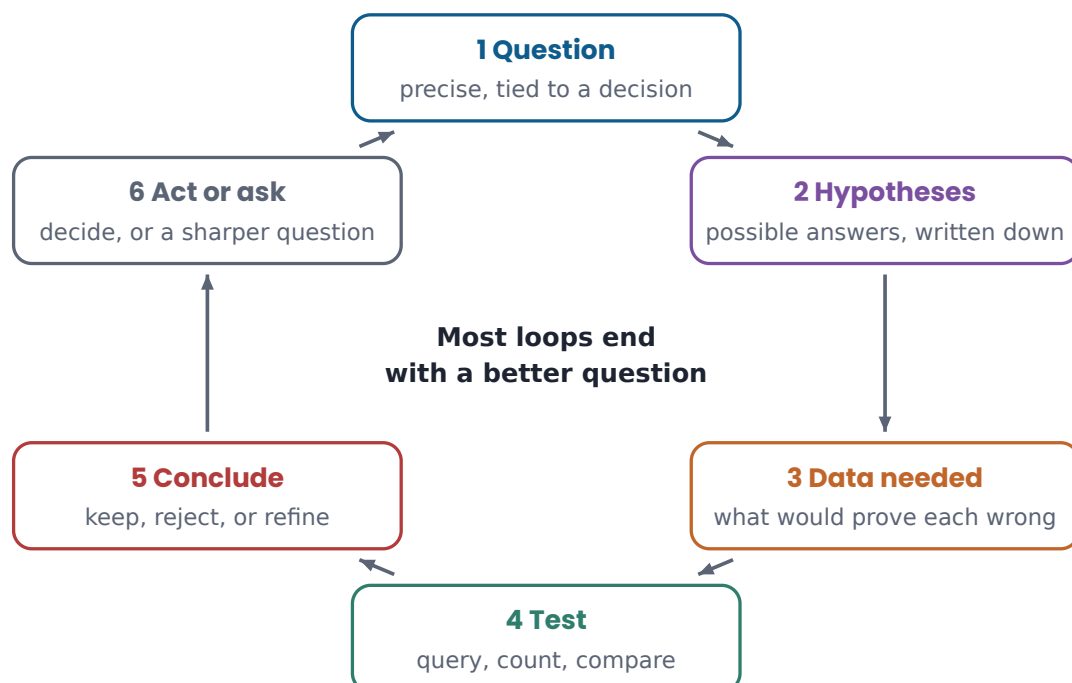
Weak	Strong	Why the strong one is better
"Something happened with customers."	"Customers who ordered in February didn't order in March."	It says which customers and what they did, so you can check it.
"Pricing is a problem."	"Customers who didn't reorder were offered smaller discounts than those who did."	It points to specific data (discounts by customer) that could prove it wrong.
"The market is tough."	"March is always slow: March revenue is lower than February in previous years too."	It names the evidence: March in earlier years.

A strong hypothesis is **specific** (who, what, when), **testable** with data you have or could get, and **falsifiable**: you can say in advance what result would prove it wrong. "The market is tough" can't be proved wrong by any number, which is exactly why people like saying it.

Write them down before you look

List your hypotheses before you open the data. If you look first, you'll find a pattern, build a story around it, and stop looking. Writing them first makes you check the ones you didn't expect, and records what you ruled out, which is often as valuable as what you found.

Some hypotheses can't be tested with the data you have. H5 needs March in earlier years, and the mini database only holds the first quarter of 2026. H6 needs information about competitors that no Riverstone table contains. Say so plainly, and name what would test them (last year's data from the ERP; a conversation with customers). **"This data can't answer that" is a legitimate finding.**



Write hypotheses before you look at the data, so the data can prove you wrong.

A rejected hypothesis is progress: it removes a branch of the issue tree.

Stop when the answer is good enough for the decision, not when it's perfect.

Figure 5.2 – The hypothesis loop. Most loops end with a better question, which starts the next loop.

Every analysis in this book follows the loop in Figure 5.2, whether the test is a pivot table, a SQL query, or a machine learning model.

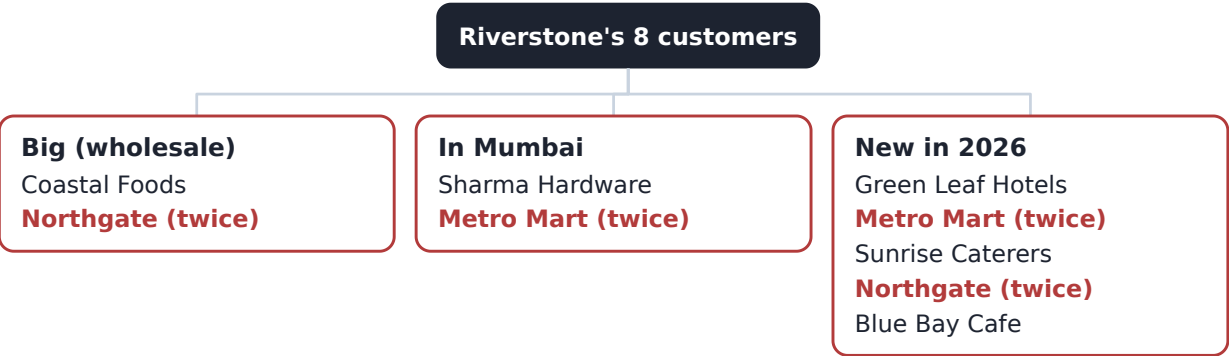
5.4 Breaking problems down: issue trees and MECE

A list of six hypotheses is a start. But lists get long, overlap, and miss things. An **issue tree** organizes a question into branches, each branch into smaller branches, until every leaf is small enough to check with one piece of data.

MECE: no overlaps, no gaps

The branches of a good tree are **MECE** (pronounced "mee-see"): **mutually exclusive** (no two branches cover the same thing) and **collectively exhaustive** (together they cover everything). The term comes from management consulting, but the idea is plain logic.

✗ Not MECE split: big, in Mumbai, new in 2026



In no group: Patel Kitchenware

Overlaps: Metro Mart and Northgate appear twice. Gap: Patel Kitchenware is in no group.
Totals by group won't add up to the whole.

✓ MECE split: segment



Every customer is in exactly one group: nothing is counted twice, nothing is left out.
Totals by group add up to the whole. Source: Mini database (Jan–Mar 2026).

Figure 5.3 – The top split double-counts two customers and misses one. The bottom split is MECE.

In Figure 5.3, splitting customers into "big", "in Mumbai", and "new in 2026" puts Metro Mart and Northgate in two groups each and Patel Kitchenware in none, so group totals won't match the company's revenue, and a conclusion like "the problem is new customers" might really be about

Mumbai. Split by segment instead, and every customer sits in exactly one group.

Four reliable ways to get MECE branches:

Split by	Example	Why it's MECE
A formula	billed revenue = number of invoiced orders × average order value	math can't overlap or leave gaps
A category with one value per record	segment, product category, region	each customer or product has exactly one
A process	lead → quote → order → invoice → payment (Chapter 3)	each step follows the last
Opposites	inside our control / outside it; new customers / existing	"not A" covers everything that isn't A

Splits like "people, process, technology" feel MECE but usually overlap. When in doubt, use a formula or a single-valued category.

Working through March

Here is the question from section 5.2, answered with an issue tree. All the numbers come from the mini database.

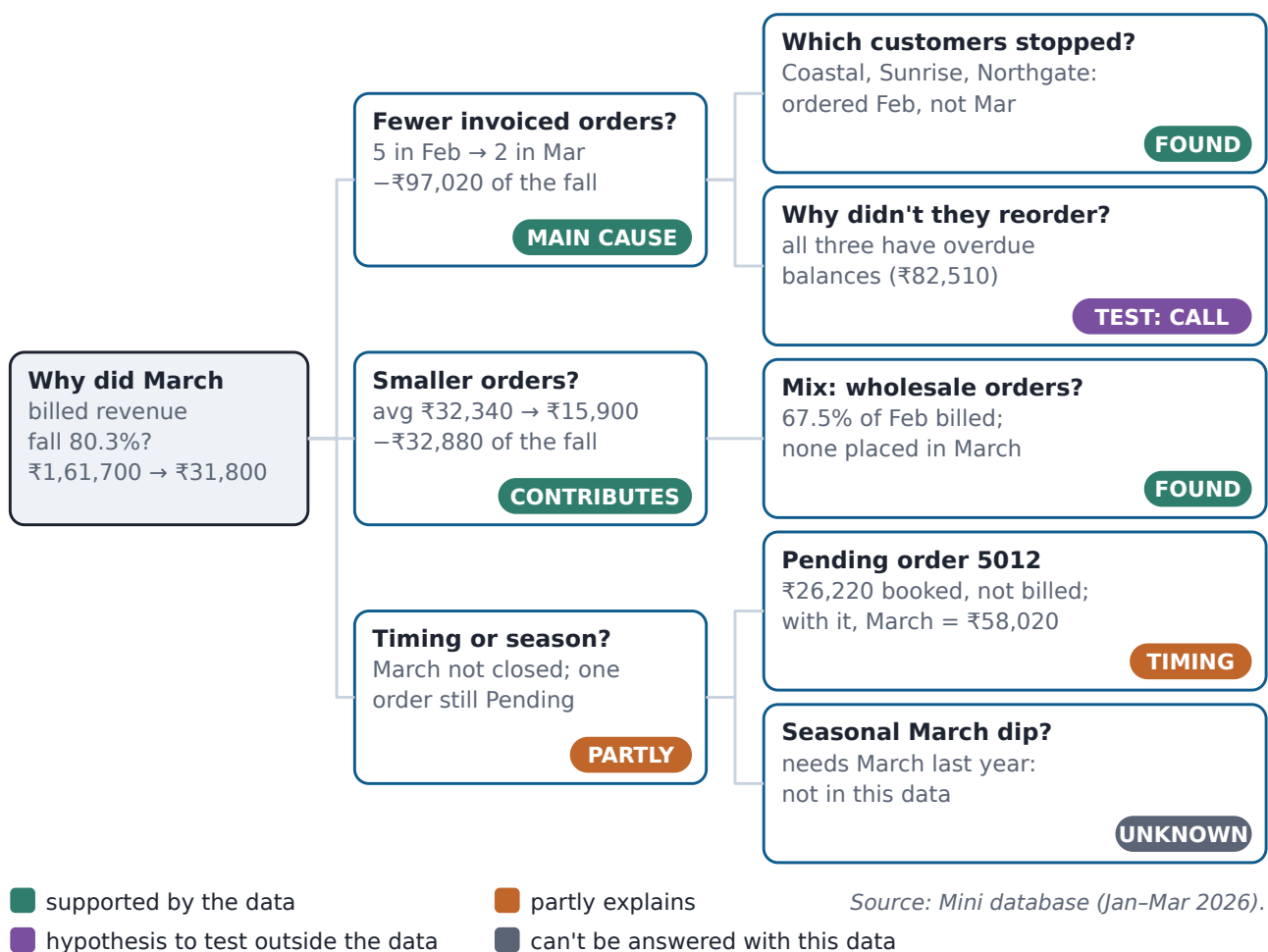


Figure 5.4 – The March issue tree, with what the data says about each branch.

Step 1. Split with a formula. Billed revenue = number of invoiced orders × average invoiced order value.

	Invoiced orders	Average order value	Billed revenue
February 2026	5	₹32,340	₹1,61,700
March 2026	2	₹15,900	₹31,800

Source: Mini database (Jan–Mar 2026).

Both parts fell. How much of the ₹1,29,900 fall does each explain? If March had kept February's average order value, 2 orders would have brought ₹64,680. So the drop in the **number** of orders accounts for ₹1,61,700 – ₹64,680 = **₹97,020** (74.7% of the fall), and the smaller **size** of March's orders accounts for the remaining ₹64,680 – ₹31,800 = **₹32,880** (25.3%). ✓ ₹97,020 + ₹32,880 = ₹1,29,900.

Simplification note. The shares depend slightly on whether you change count or size first; the ranking doesn't.

Step 2. Fewer orders: which customers? Five customers had orders invoiced in February: Sharma Hardware, Metro Mart, Coastal Foods, Sunrise Caterers, and Northgate Distributors. In March, only Sharma Hardware and Green Leaf Hotels were invoiced. Metro Mart ordered in March, but its order is still pending. So three customers ordered in February and not at all in March: **Coastal Foods, Sunrise Caterers, and Northgate Distributors.**

Is that unusual? Coastal Foods ordered on 9 January and 11 February, 33 days apart; by 31 March it's been 48 days. Worth noticing, though two orders are a thin pattern (Chapter 4).

Step 3. Timing: is something stuck? Order 5012 from Metro Mart, worth ₹26,220, was placed on 15 March and is still *Pending*. It's booked, not billed. With it, March would be ₹58,020, still 64.1% below February. **H3 is true but explains only a part.**

Step 4. Smaller orders: the mix. February's two wholesale orders (Coastal Foods ₹32,625 and Northgate ₹76,560) were ₹1,09,185, or 67.5% of February's billed revenue. No wholesale customer ordered in March. Wholesale orders are the largest, so losing them shrinks both the count and the average. **H4 is supported.**

Step 5. Why didn't they reorder? The ERP's orders can't answer "why", but the invoices and payments can add a clue. On 31 March:

Customer	Ordered in March?	Owed	Overdue
Northgate Distributors	no	₹46,560	₹46,560
Sunrise Caterers	no	₹23,325	₹23,325
Coastal Foods	no	₹12,625	₹12,625
Patel Kitchenware	no (last order in January)	₹6,250	₹6,250
Sharma Hardware	yes	₹11,700	₹0 (not yet due)
Green Leaf Hotels	yes	₹0	₹0
Metro Mart	yes (pending)	₹0	₹0

Source: Mini database (Jan–Mar 2026).

Every customer with an overdue balance placed no order in March, and no customer who ordered in March had anything overdue. The three customers from step 2 owe ₹82,510 between them, all of it overdue.

That's a striking pattern, and it's exactly the moment to slow down. It doesn't say *which way* the connection runs, or whether there is one. A customer short of cash might stop ordering until it pays. A customer unhappy with a delivery might both withhold payment and stop ordering, so a single cause would explain both (section 5.6). Or, with seven customers, it could be coincidence. One detail points to a specific question: Northgate's February order still shows *Shipped*, not *Delivered*. If the crates never arrived, Northgate isn't a late payer; it's a customer waiting for its goods.

This branch ends as a **hypothesis to test outside the data**: a phone call to each of the three customers, and a check of the delivery records for Northgate.

Step 6. The branches the data can't reach. H5 (March is always slow) needs last year's March, which isn't in the mini database. H6 (a competitor) needs information from customers. Both go on the list of open questions, not in the conclusion.

What to tell Anita. A good answer is short, ranked, and honest about confidence:

"March billed revenue was ₹31,800, down 80.3% from February. About three-quarters of the fall is fewer orders: Coastal Foods, Sunrise Caterers, and Northgate didn't reorder in March, and February's two wholesale orders alone were 67.5% of that month. Metro Mart's ₹26,220 order is still pending; shipping it brings March to ₹58,020. All three customers who didn't reorder have overdue balances (₹82,510 in total), and Northgate's order still shows as not delivered. I'd call all three this week, starting with Northgate, before offering any discounts. I can't tell yet whether March is seasonally slow; that needs last year's data."

5.5 Fact, opinion, and assumption

Meetings mix three kinds of statement, often in the same sentence. Separating them is one of the quickest ways to make a discussion productive.

- A **fact** is a statement that can be checked against a record, and has been. "March billed revenue was ₹31,800." (It still depends on a definition, so a good fact states it.)
- An **opinion** is a judgment. It may be wise, but it isn't checkable as stated. "Northgate is a difficult customer."
- An **assumption** is something treated as true without checking, usually because checking is hard or slow. "Northgate will pay next week."

Here are statements from Riverstone's Monday sales review, sorted:

Statement	Type	What to do with it
"March billed revenue was ₹31,800."	fact	Use it, with its definition.
"Coastal Foods orders about once a month."	a claim from very little data	Treat as a hypothesis: two orders, one gap of 33 days.
"Our prices are too high for wholesalers."	opinion	Turn it into a testable hypothesis: <i>wholesale customers who didn't reorder were quoted higher prices than those who did.</i>
"Northgate will pay next week."	assumption	Write it down, give it an owner, and check it by a date.
"Northgate is a difficult customer."	opinion	Ask what experience it's based on; check the delivery status first.

Source: Mini database (Jan–Mar 2026).

Businesses run on opinions and assumptions, because there's never time to check everything. The danger is when they're **presented as facts**, or an old assumption quietly becomes "what we know". List your assumptions in one place, so anyone can challenge them.

5.6 Checking claims and charts

Chapter 4 (section 4.10) covered number tricks. This section is about the reasoning behind a claim. Before accepting one, from anyone including yourself, ask five questions:

1. **Who says so, and how do they know?** A measured count, a survey, a guess, or an anecdote?
2. **Compared with what?** A number with no comparison can't be "high" or "low".
3. **How many cases, and which ones?** Five customers or five thousand? Chosen how?
4. **What's the mechanism?** Is there a believable reason why A would cause B?
5. **What else could explain it?** Chance, reverse causation, a common cause, or selection.

Correlation is not causation. Two things that move together are **correlated**. That doesn't mean one causes the other. There are three common alternatives:

- **Reverse causation: B causes A.** "Order lines with bigger discounts are bigger lines, so discounts make customers buy more." In the mini database, the four order lines worth ₹20,000 or more had an average discount of 10.5%; the other fifteen averaged 1.67%. But Riverstone's own rules (Chapter 3, section 3.6) give larger discounts to larger orders. The size comes first; the discount follows. The data can't show that discounts grow orders.
- **A common cause: C causes both.** In section 5.4, overdue balances and missing reorders went together. A delivery problem could cause both: the customer won't pay for goods it hasn't received, and won't reorder either. Chasing payment harder would then make things worse.
- **Selection: the cases were chosen in a way that creates the pattern.** "Customers who attend our trade fair order more." Maybe the ones who attend were already the most engaged customers.

And sometimes it's **chance**: with seven customers, patterns appear by accident. Chapter 22 shows how to judge whether a pattern is bigger than chance.

Watch out: your own analysis is a claim too. The five questions apply to what you're about to send. The note to Anita in section 5.4 states a measured fact ("about three-quarters of the fall is fewer orders") and a recommendation ("I'd call them"), but it's careful not to say "customers stopped ordering *because* they owe us money".

5.7 Bias in how we see data

A **cognitive bias** is a predictable way in which people's judgment drifts from the evidence. Everyone has them, including experienced analysts. You can't switch them off, but you can recognize the common ones and build habits that catch them.

Bias	What it looks like at Riverstone	Antidote
Confirmation bias: noticing evidence that fits what you already believe	Vikram is sure a competitor is undercutting prices, so he asks for lost deals that mention price, and not for the ones that don't	Write hypotheses first (section 5.3); look for evidence that would prove your favorite wrong
Anchoring: judging a number against the first number you saw	March looks like a disaster against February's ₹1,61,700. But February was unusual: Northgate's first order alone was ₹76,560, 47.3% of the month. Against the quarter's monthly average of ₹99,237, March is still weak, but the comparison is fairer	Compare against several baselines: the previous month, the average, the same month last year, the target
Regression to the mean: an unusually high or low value tends to be followed by a more ordinary one	A month boosted by one big first order is likely to be followed by a lower month, even if nothing went wrong	Before explaining a change, ask whether the starting point was unusual
Survivorship bias: studying only the cases that made it through	Studying won deals to learn "what works", without looking at the lost and never-contacted leads that went through the same steps	Always include the cases that dropped out

Bias	What it looks like at Riverstone	Antidote
Availability and recency: overweighting what's vivid or recent	One angry phone call from a customer on Friday becomes "customers are unhappy" on Monday	Count: how many complaints, out of how many customers, over what period?
Small numbers: trusting patterns from very few cases	"All the customers who owe us stopped ordering" is based on four customers	Give the counts; call it a hypothesis until more cases agree (Chapter 4, section 4.8)
The analyst's own bias: wanting an interesting finding	A clean story about overdue balances is more exciting than "one big order made February unusual", so it's tempting to lead with it	State the ordinary explanation first if it's the bigger one

Source for the Riverstone numbers: Mini database (Jan–Mar 2026).

The last row matters most. Analysts are rewarded for insights, so there's a pull toward the surprising story; a good reputation rests on being right, which often means reporting the ordinary explanation clearly.

5.8 Deciding with data

Analysis exists to help someone decide. Data rarely decides by itself; it narrows the options, estimates consequences, and shows where the uncertainty is.

Part	Question	For the three customers who didn't reorder
Decision	What exactly is being decided, and by whom?	What sales does about Coastal Foods, Sunrise Caterers, and Northgate this week (Anita decides)
Options	What are the choices, including doing nothing?	A. Wait. B. Call each customer to ask why, and check Northgate's delivery. C. Offer 5% off their next order. D. Put overdue accounts on hold until they pay.
Criteria	What matters in choosing?	Revenue, margin, cash owed, the customer relationship
Evidence	What does the data say about each option?	All three owe ₹82,510, all overdue. Northgate's order isn't marked delivered. A 5% discount on an Industrial Crate cuts its margin from 24.1% to 20.1%.
Reversibility	How costly is it to be wrong?	A call is cheap and can't do much harm. A discount sets a price expectation that's hard to take back. A hold could lose a customer whose goods never arrived.
Recommendation and confidence	What should we do, and how sure are we?	B this week; decide on C or D after the calls. Medium confidence: the pattern is clear, but it's four customers and the cause is unknown.
What would change my mind	Which new fact would change the recommendation?	If customers say price is the reason and their balances are paid, reconsider C. If goods weren't delivered, fix the delivery before anything else.

Source: Mini database (Jan–Mar 2026).

Three principles sit behind the table:

- **Match the effort to the decision.** A reversible, cheap decision (a phone call) needs less evidence than an irreversible, expensive one (a price change or a hire).
- **Include "do nothing".** It's often the real alternative, and it has costs too.
- **Say what would change your mind.** It shows where the uncertainty is and what to watch.

Interview extra point. In a case interview or a take-home question ("Revenue fell 20%. Why?"), don't start calculating. Spend the first minute restating the question precisely, then sketch a MECE split out loud (for example, number of orders × average order value, then by segment), and say which branch you'd check first and why. Interviewers are grading the structure of your thinking more than the final number. Chapter 75 (product sense, metrics, and case studies) and Chapter 76 (the Business Analyst question bank) have practice cases with model answers.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Answering the surface question	correct numbers nobody can act on	Ask what the answer is for, and who decides
Starting with the data instead of the question	a pile of charts and no conclusion	Write the precise question and hypotheses first
Hypotheses that can't be wrong	"the market is tough" survives any evidence	Make each one specific, testable, and falsifiable
Branches that overlap or leave gaps	group totals don't add up to the whole	Split by a formula, a single-valued category, or a process
Stopping at the first explanation	a confident story that's partly true	Name at least one other explanation and check it
Reading correlation as causation	"discounts make customers buy more"	Check reverse causation, common causes, selection, chance
Comparing against one unusual baseline	a normal month looks like a disaster	Use several baselines: average, target, same month last year
Over-analyzing a small, reversible decision	a week spent on something a phone call would settle	Match the effort to the cost of being wrong

In the real world: do we need another sales executive?

It's the first week of January 2026. Vikram Singh, the Sales Manager, has asked the managing director for a fourth sales executive. His reason: *"Leads are piling up. The CRM shows 43 enquiries last year, and my team can't keep up."* The MD forwards it to Anita with one line: *"Is this right?"* Anita asks Meera Iyer to look before Friday.

1. What's the decision, and what's the precise question? The decision is whether to hire. The question underneath: "Did Riverstone lose new business in 2025 because the sales team didn't have time to handle its leads?" A hire fixes capacity, not a slow process, poor routing, or poor leads, so the question must separate those.

2. The issue tree, following the lead's journey:

- **Too many leads?** How many real enquiries arrived, per executive?
- **Leads missed or handled slowly?** How many were never contacted, and how long did first contact take?
- **Leads contacted but lost?** What share of contacted leads were won?
- **Capacity or something else?** If the team is too busy, the busiest people should be the ones missing leads.

3. The tests. All from the one-year database.

- **Volume.** The 43 rows include duplicates: the same enquiry submitted two or three times. There were **30 real enquiries** in the year, about 2.5 a month, or fewer than one a month per sales executive. "Piling up" isn't about volume.
- **Missed and slow. 8 of the 30 (26.7%) were never contacted at all.** The other 22 waited an average of **9.7 days** for a first contact. One referral, Tulip Mart, has been waiting since 23 September, 99 days.
- **Lost after contact.** 6 of the 22 contacted leads were won (27.3%). Of the leads contacted within a week, 3 of 8 were won; of those contacted later, 3 of 14. That fits "faster is better", but with numbers this small it's a hypothesis, not a finding.
- **Capacity.** If the team were overloaded, the executives handling the most customer orders should miss the most leads. The data says the opposite:

Sales executive	Orders handled in 2025	Leads assigned	Never contacted	Average days to first contact
Farah Khan	64	5	0	7.7
Rahul Mehta	53	10	3	10.9
Neha Kulkarni	46	15	5	9.8

Source: One-year database (2025 CRM leads).

Farah, with the most orders, missed none of her leads. Neha, with the fewest orders, was assigned half of all leads, including 9 of the 14 website enquiries, and missed 5. Website leads were the most often missed: 5 of 14 were never contacted.

4. Checking herself. The CRM doesn't record time spent on visits, calls, or complaints, so orders aren't the whole workload. And reading the table as "Neha is the problem" would be unfair: she got as many leads as Rahul and Farah combined, including most website enquiries. The pattern points at **how leads are routed and followed up**, not at a person.

5. The recommendation. She sends Anita a half-page note:

"The data doesn't support hiring for lead volume: 30 real enquiries came in last year (the CRM's 43 includes duplicates), fewer than one a month per executive. The problem is follow-up: 8 enquiries were never contacted and the rest waited almost 10 days on average. Missed leads are concentrated among website enquiries and in the largest lead list, not with the busiest executive. I'd (1) call the 8 uncontacted leads this week, starting with the Tulip Mart referral, (2) spread website leads evenly, (3) set a two-working-day rule for first contact with a daily reminder, and (4) fix the duplicate website submissions. What would change my mind: if leads grow sharply, or if response times are still slow after a quarter of the new routing, a hire is worth revisiting. The CRM doesn't record time spent, so I can't rule out that the team is busy with work outside orders and leads."

Anita forwards it to the MD and Vikram. Vikram's first reaction is irritation. His second, after reading the table, is to ask Meera how to set up the daily reminder (you'll build that reminder yourself later in the book).

The request was a solution ("hire"). Meera turned it into a question about a cause, tested each branch, avoided blaming one person, stated what she couldn't see, and recommended cheap, reversible steps first, with a clear condition for revisiting the expensive one.

Tools

- **Pen and paper, or a whiteboard.** Issue trees are fastest by hand. Draw the first version in five minutes; tidy it later.
- **A spreadsheet or document** for the hypothesis log: one row per hypothesis, with the data needed, the result, and the status (supported, rejected, open).
- **A diagram tool** (optional): diagrams.net, PowerPoint, or Google Slides for sharing a tree.
- **SQL and spreadsheets** for the tests, from Chapter 10 onward. This chapter's numbers came from short queries on the Riverstone databases.

The project: an issue tree for a real question

Goal: take one real business question, make it precise, and build an issue tree that shows exactly which data would answer each branch.

Step 1. Choose a question from your work, a local business, or your Chapter 4 project: turn one checked claim into an analyst's question. "Sales up 40% in three years" becomes "Did the company's revenue grow faster than its market over those three years, and where did the growth come from?"

Step 2. Make it precise. Write the metric and its definition, the period and the comparison, the scope, the decision it serves, and the deadline (section 5.2).

Step 3. Write at least five hypotheses before looking at any data. Make each specific and testable (section 5.3).

Step 4. Build the tree two or three levels deep, starting with a MECE split (section 5.4). Check each split for overlaps and gaps.

Step 5. Add the data to every leaf. For each leaf, write:

Column	What to write
branch	the leaf's question
hypothesis	what you expect, stated so it could be wrong
data_needed	the table, file, report, or conversation that would test it
available	yes, no, or "needs to be collected"
test	the count, comparison, or query you'd run
result_that_rejects_it	what you'd have to see to drop this branch

Step 6. Label facts, opinions, and assumptions in anything the question's owner has said about it.

Step 7. Name one bias that could affect this analysis (yours or the requester's), and how you'll guard against it.

Deliverable: the precise question, the tree, the data table, and a three-sentence plan for which branch you'd test first and why. If you have access to the data, test one branch and write a half-page note like Meera's.

You've got it when...

- ☐ I ask what an analysis is for before I start it.
- ☐ I can turn a vague request into a precise question with a metric, period, comparison, scope, and decision.
- ☐ I write hypotheses before looking at the data, and each one could be proved wrong.
- ☐ I can build an issue tree with MECE branches, and I know four reliable ways to split.
- ☐ I label statements as fact, opinion, or assumption.
- ☐ I check claims for reverse causation, common causes, selection, and chance.
- ☐ I can name the common biases and the habit that counters each one.
- ☐ I can structure a decision with options, criteria, evidence, reversibility, confidence, and what would change my mind.
- ☐ I've built an issue tree for a real question, with the data for every branch.

Recap

- Questions are **descriptive, diagnostic, predictive, or prescriptive**. Most requests are diagnostic or prescriptive underneath.
- Keep asking: *what is this for? of what? compared with what? how do we know? so what? what else could explain it?*

- A **precise question** pins down the metric, the period and comparison, the scope, the decision it serves, and the deadline.
- A **hypothesis** is a possible answer that data could prove wrong. **Write hypotheses before you look.** "This data can't answer that" is a legitimate finding.
- An **issue tree** breaks a question into branches until each leaf can be tested. Branches should be **MECE**: no overlaps, no gaps. Split by a formula, a single-valued category, a process, or opposites.
- March 2026's 80.3% fall split into **fewer orders (₹97,020)** and **smaller orders (₹32,880)**, then into three customers who didn't reorder, the missing wholesale orders, and a pending order, ending in a hypothesis to test by phone.
- Separate **facts, opinions, and assumptions**, and list assumptions where everyone can see them.
- **Correlation isn't causation.** Check for reverse causation, a common cause, selection, and chance.
- **Biases** (confirmation, anchoring, regression to the mean, survivorship, availability, small numbers, and the analyst's own) bend everyone's judgment; counter them with habits.
- **Decide with data** by laying out options (including doing nothing), criteria, evidence, reversibility, a recommendation with confidence, and what would change your mind.

Practice exercises

Warm-up

1. Label each question as descriptive, diagnostic, predictive, or prescriptive: (a) *How many orders did Sunrise Caterers place in February?* (b) *Should we stop offering 12% discounts?* (c) *Why did Kitchen revenue grow every quarter of 2025?* (d) *How much will we collect in April?* (e) *Which customers owe us money?*
2. Label each statement as fact, opinion, or assumption, and say what you'd do with it: (a) "Invoice 9007 hasn't been paid." (b) "Sunrise Caterers is a bad customer." (c) "The new price list will be ready by Monday." (d) "Hotels don't care about price." (e) "Blue Bay Cafe signed up on 1 March 2026."
3. Is each split MECE? If not, say whether it overlaps, leaves gaps, or both. (a) Orders: Pending, Shipped, Delivered, Cancelled. (b) Customers: large, loyal, new. (c) Revenue: Storage, Kitchen, Industrial, Furniture. (d) Reasons for a late delivery: warehouse delay, truck problem, customer not available, bad weather. (e) Leads: from the website, from referrals, from Mumbai.
4. Rewrite each vague request as a precise question with a metric, period, comparison, and the decision it might serve: (a) "How are we doing with hotels?" (b) "Is the website worth it?" (c) "Check if customers are paying on time."

Core

5. For the question "Why did hospitality customers bring in less revenue in March 2026 than in February?", write three hypotheses that data could prove wrong, and for each, name the data you'd use. (Hospitality billed ₹23,325 in February and ₹20,100 in March in the mini database.)
6. Billed revenue rose from ₹1,04,210 in January 2026 (3 invoiced orders) to ₹1,61,700 in February (5 invoiced orders). (a) Calculate each month's average invoiced order value. (b) Split the ₹57,490

increase into the part from more orders (holding January's average order value) and the part from the change in order size. (c) Which explains more?

7. Chapter 3 showed that Riverstone collected ₹1,97,250 of ₹2,97,710 billed in the first quarter of 2026 (66.3%). Build a two-level MECE issue tree for "Why did we collect only two-thirds of what we billed?" For each leaf, name the data that would test it.
8. Name the bias in each situation and suggest one habit that would counter it: (a) After one customer complains about a cracked crate, a manager says, "Our quality has slipped." (b) An analyst studies the five best customers to learn why customers stay. (c) Revenue drops after a record month, and the team spends a week looking for what went wrong. (d) A manager who wanted a new CRM highlights only the figures that make the old one look bad.
9. A manager says: "Customers with overdue balances order less. So chasing payments hurts sales, and finance should send fewer reminders." Give three other explanations for the pattern, and describe what data would help decide between them.

Stretch

10. Using the table from the story: Farah handled 64 orders and missed none of her 5 leads; Neha handled 46 orders and missed 5 of her 15. (a) What does this data support? (b) What doesn't it support? (c) What additional data would you want before saying anything about individual performance?
11. In the story, leads contacted within a week were won 3 times out of 8 (37.5%); leads contacted later were won 3 times out of 14 (21.4%). (a) Why isn't this proof that faster contact wins more deals? (b) Name one alternative explanation. (c) How could Riverstone test the idea properly?
12. Blue Bay Cafe signed up on 1 March 2026 and hadn't placed an order by 31 March. Structure the decision "What should sales do about Blue Bay Cafe?" using the table in section 5.8: at least three options (including doing nothing), criteria, the evidence you'd look for, reversibility, and what would change your mind.

Think about it (no calculation needed)

13. When should you stop analyzing and give an answer? Describe two signals that you've done enough, and one signal that you haven't.
14. A senior manager says, "I've already decided to cut the Furniture range. Can you find me some numbers that support it?" How would you respond, and what would you offer to do instead?
15. You ask an AI assistant why March revenue fell, and it gives a fluent, confident explanation. Using this chapter, list three things you'd check before repeating it.

Key terms

descriptive question · diagnostic question · predictive question · prescriptive question · habit questions · precise question · hypothesis · testable · falsifiable · issue tree · MECE · mutually exclusive · collectively exhaustive · decomposition (count × size) · fact · opinion · assumption · claim · correlation · causation · reverse causation · common cause · selection · chance · cognitive bias · confirmation bias · anchoring · regression to the mean · survivorship bias · availability bias · recency · small-numbers bias · decision rights · reversibility · confidence · "what would change my mind"

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 6, Planning Your Learning**, turns the book's hours into a plan for your week, and shows which chapter brings each tool you'll use to test hypotheses.
- **Chapters 10–13** give you the tests: spreadsheets and SQL to count, compare, and break down numbers the way section 5.4 did.
- **Chapter 14, Data Cleaning & Preparation**, handles the "is the data even right?" branch that every issue tree should include.
- **Chapter 22, Statistics Without Fooling Yourself**, shows whether a pattern like "3 of 8 versus 3 of 14" is bigger than chance.
- **Chapter 23, Business Acumen, KPIs & Metrics**, builds full KPI trees for Riverstone and diagnoses a revenue dip with more careful breakdowns.
- **Chapter 24, Requirements, Storytelling & Stakeholders**, turns notes like Meera's into memos and presentations, and covers handling "can you find numbers that support this?"
- **Chapters 30 and 31** test cause and effect properly: experiments, and methods for when experiments aren't possible.
- **Chapters 36 and 40** take on predictive questions like "What will April's revenue be?": the machine learning workflow, and forecasting over time.
- **Interview preparation**: case questions ("revenue fell; why?"), structuring, and hypothesis-driven thinking appear in Chapter 75 (product sense, metrics, and case studies) and Chapter 76 (the Business Analyst question bank), with model answers.

Answers to practice exercises

1. (a) Descriptive. (b) Prescriptive. (c) Diagnostic. (d) Predictive. (e) Descriptive.
2. (a) Fact (checkable in the payments table); use it. (b) Opinion; ask what it's based on and turn it into something checkable, such as "Sunrise pays later than other customers". (c) Assumption (a plan); give it an owner and confirm it. (d) Opinion, and a sweeping one; turn it into a hypothesis, such as "hotel customers accept price increases as often as retailers". (e) Fact (the `signup_date` in the customers table); use it.

3. (a) MECE, as long as every order has exactly one of these four statuses. (b) Not MECE: it overlaps (a new customer can be large; a large customer can be loyal) and has gaps (a small, occasional, long-standing customer is none of them). (c) MECE: each product belongs to one category, and the four cover every product. (d) Not MECE: it can overlap (a truck problem caused by bad weather) and has gaps (a wrong address, missing stock, a paperwork delay). Add "other" and define each reason so that one is chosen. (e) Not MECE: "Mumbai" is a location, not a source, so a website lead from Mumbai is in two groups, and leads from trade fairs or cold calls are in none.

4. One good version of each: (a) "What was billed revenue from hospitality customers in the first quarter of 2026, compared with the previous quarter, and which customers drove the change? (For deciding on an April hotel promotion.)" (b) "How many 2025 website enquiries became customers, and how much revenue did they bring, compared with referrals and trade fairs? (For next year's website budget.)" (c) "What share of first-quarter 2026 invoices were paid by their due date, and which customers are most often late? (For deciding whom finance calls first.)"

5. Examples: (1) Fewer hospitality customers ordered in March than in February. Data: orders by customer and month (Sunrise Caterers ordered in February, Green Leaf Hotels in March). (2) Hospitality customers who ordered placed smaller orders. Data: order values by customer and month. (3) The hospitality order that would have made the difference is still pending or was cancelled. Data: order status for hospitality customers in March. Note that the gap is small (₹3,225) and each month has one order, so almost any difference is normal variation.

6. (a) January: $₹1,04,210 \div 3 = ₹34,737$. February: $₹1,61,700 \div 5 = ₹32,340$. (b) Holding January's average order value, 5 orders would bring $5 \times ₹34,736.67 = ₹1,73,683$, which is ₹69,473 more than January: that's the **count effect**, **+₹69,473**. The change in size is $5 \times (₹32,340 - ₹34,736.67) = -₹11,983$. Check: $₹69,473 - ₹11,983 = ₹57,490$. ✓ (c) The increase came entirely from **more orders**; the average order actually got a little smaller.

7. One good tree:

- **Collected only 66.3% of billings**
 - **Not yet due** (the amount billed recently, within 30-day terms). Data: invoices by due date as of 31 March (invoice 9010's ₹11,700 isn't due until 10 April).
 - **Due but unpaid (overdue)**
 - Customers can't or won't pay. Data: overdue by customer; payment history; credit checks.
 - Customers dispute the invoice. Data: support tickets, delivery status, proof of delivery.
 - Riverstone hasn't chased. Data: reminder logs from finance.
 - **Paid but not recorded yet.** Data: bank statement lines not yet matched to invoices.

The first split (not yet due / overdue / paid but unrecorded) is MECE for the money not collected. From Chapter 3: ₹1,00,460 is unpaid, of which ₹88,760 is overdue and ₹11,700 is not yet due.

8. (a) **Availability** (one vivid complaint). Habit: count complaints over a period, out of how many deliveries. (b) **Survivorship** (studying only customers who stayed). Habit: compare with customers who left. (c) **Regression to the mean** (after a record month, a lower one is normal). Habit: compare with the average and the same month last year before searching for a cause. (d) **Confirmation bias**. Habit: write down in advance what evidence would show the old CRM is fine, and look for it.

9. (1) Common cause: a delivery or quality problem makes customers both withhold payment and stop ordering. **(2) Reverse causation:** customers who order less have less reason to keep their account current. **(3) Cash-strapped customers** pay late and order less, whatever the reminders. Data that helps: complaints and delivery records for overdue customers; whether ordering fell before or after the balance became overdue; and a few customer conversations.

10. (a) Missed leads weren't concentrated with the executive handling the most orders, so "too busy with orders" isn't supported, and lead assignment was very uneven. **(b)** Nothing about individual effort or ability: the numbers are tiny, Neha got most website leads, and orders aren't all of anyone's work. **(c)** Time spent on other work, each person's lead sources, how leads were assigned, and absences over the year.

11. (a) The counts are very small (6 wins in total), so the difference could well be chance (Chapter 22), and the leads weren't assigned a response time at random. **(b)** Executives may contact the most promising leads first, such as referrals, so the leads contacted quickly were already more likely to be won (selection, or a common cause: lead quality affects both speed and winning). **(c)** Run a simple experiment (Chapter 30): for a quarter, randomly assign new leads to "contact within two working days" or "normal process", then compare win rates, or at least compare fast and slow contact within the same lead source.

12. Decision: what sales does about Blue Bay Cafe in April. **Options:** A. wait; B. call or visit to understand its needs; C. a first-order offer; D. send the catalog and a sample. **Criteria:** chance of a first order, cost, margin, fit. **Evidence:** 30 days since signup without an order; what it enquired about; how long other new customers took (Green Leaf Hotels signed up on 8 January and placed its first order on 20 January). **Reversibility:** a call is cheap; a discount sets an expectation. **Recommendation:** B first, then D if interest is confirmed. **What would change my mind:** a specific, large first order that depends on price would make C worth considering.

13. Enough: more precision wouldn't change the decision; the remaining branches can't be answered with available data, and you've said so. Not enough: you can't yet explain most of the change, or the recommendation would flip on a branch you haven't checked.

14. Don't cherry-pick; it risks your credibility and the manager's. Offer a fair picture instead: "I'll look at Furniture's revenue, margin, and trend. If it supports cutting the range, that's a stronger case; if not, better you hear it from me than from the board." Chapter 24 covers handling this kind of pressure.

15. (1) Are its numbers right? Check the metric, definition, and figures against the database. **(2) Is it presenting hypotheses as facts?** Order tables can't tell you *why* customers behaved as they did. **(3) What has it left out?** Compare with your own issue tree: timing, mix, and the limits of the data. Chapter 26 covers working with AI assistants.

Chapter 6. Planning Your Learning

Chapter at a glance

You will learn to: estimate honestly how many hours this book takes, and turn them into weeks at your real pace · build a weekly rhythm you can keep, and recover from a missed week · check whether your computer is ready, and know which chapter brings each tool · read an official page to settle a question for yourself · use AI assistants to learn faster without letting them do your thinking · plan your route and your first 90 days.

Before you start: Chapters 1–5, and "How to Use This Book" at the front of the book.

Time needed: 2–3 hours, including the exercises and the project.

Tools: a notebook or a notes app, and a calendar. No software to install.

Practice data: none; you plan with your own week.

Why this matters

From Chapter 10 onward, every chapter asks you to type, run, break, and fix things: formulas, queries, scripts, dashboards. Each of those chapters installs the tool it needs at its start and checks it works with one small first step, so there's nothing to install today.

This chapter sets up the other half: **you**. An honest number of hours, a plan you can keep, a weekly rhythm, a way to use AI assistants that helps you learn instead of replacing the learning, and the habit of checking the official source when two sources disagree. It all decides whether you finish.

In plain English

Think of a cook planning a week of meals. A good cook doesn't buy every utensil in the shop on Sunday and leave them in boxes. They decide what they'll cook each day, check how long each dish takes, and get each utensil when a recipe first calls for it.

Planning to learn is the same:

- **The meal plan** is your study plan: what you'll cook this week, and when.
- **The cooking times** are each chapter's *Time needed* line. Added up, they tell you honestly how long the whole menu takes.
- **The utensils** are the tools: a spreadsheet, a database, Power BI, Python. You get each one when a recipe first needs it.
- **The recipe book** is this book, plus the official documentation for each tool, which is where you look when the recipe is unclear.

You don't need every utensil on day one, only a plan, a rhythm, and the next recipe.

6.1 How long it really takes

Every chapter's *Time needed* line estimates its **study hours**: the reading, the exercises, and the project together. Adding up those lines for every teaching chapter gives the book's own answer to "how long will this take?":

	Hours	6 hours a week	8 hours a week	10 hours a week
Parts 0 and I (Chapters 1–9)	27–37	5–6 weeks	3–5 weeks	3–4 weeks
Part II (Chapters 10–27)	290–357	48–60 weeks	36–45 weeks	29–36 weeks
Job-ready: Parts 0 to II (Chapters 1–27)	317–394	53–66 weeks, or 12 to 15 months	40–49 weeks, or 9 to 11 months	32–39 weeks, or 7 to 9 months
Parts III to VII (Chapters 28–67)	445–584	1.4 to 1.9 years	1.1 to 1.4 years	0.9 to 1.1 years
All of it (Chapters 1–67)	762–978	2.4 to 3.1 years	1.8 to 2.4 years	1.5 to 1.9 years

These are reading-and-exercise hours. Fluency takes more practice on top (Chapter 9).

Read the bold job-ready row first. Parts 0 to II, the **job-ready path**, are what a first analyst job asks for. Parts III to VII are the rest of a career, and they're optional branches: Chapter 8 shows which of them each role needs.

To turn the table into your own plan, divide. Weeks = hours ÷ your hours a week. At 6 hours a week, the job-ready path is $317 \div 6 = 52.8$, about 53 weeks, at the low end and $394 \div 6 = 65.7$, about 66 weeks, at the high end: a year to fifteen months. Months are weeks $\times 12 \div 52$, so 53 weeks is about 12 months and 66 weeks is about 15. The same arithmetic works for any number of hours; use the hours you really have, not the hours you wish you had.

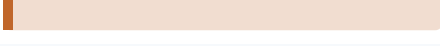
That's longer than many courses promise, and it's honest. A plan built on the real number survives a bad month; a plan built on a hopeful one fails in week three and takes your confidence with it.

One more thing about Parts 0 and I. Each of their nine chapters has a project, and together they're more than a month's work if you do them all at once. Do three of them properly as you go: Chapter 1's spending log (it runs for a week while you read on), this chapter's plan, and Chapter 8's door plan. Start Chapter 9's learning system and carry it alongside the book for its twelve weeks. Come back to the others (Chapters 2, 3, 4, 5, and 7) in a review week. Their hours are already in the table, so doing them later doesn't change your total.

6.2 A weekly rhythm you can keep

The book is long. The difference between finishing and stopping is rarely intelligence; it's a plan, a rhythm, and a way of keeping going after a missed week.

One week, about 8 hours

Hours		What you do
Mon	 1 h	Read a new section
Tue	 1 h	Exercises from that section
Wed	 1 h	Read the next section
Thu	 1 h	Exercises
Fri	 30 min	Review: recap and key terms from memory
Sat	 2.5 h	Project or lab work
Sun	 1 h	Redo missed exercises; plan next week

Total: 1 + 1 + 1 + 1 + 0.5 + 2.5 + 1 = 8 hours

Figure 6.1 – Short, regular sessions, one longer session for projects, and a review built into the week.

Figure 6.1 adds up to 8 hours: 1 + 1 + 1 + 1 + 0.5 + 2.5 + 1. Three ideas are built into it:

- **Alternate reading and doing.** A section read on Monday gets practiced on Tuesday, while it's fresh but no longer in front of you.
- **Review from memory.** On Friday, close the book and write down the chapter's key terms and main ideas before checking the recap. Pulling ideas out of your memory strengthens them far more than reading them again. Chapter 9 explains why this works and how expertise forms.
- **Plan at the end of the week.** Ten minutes on Sunday deciding exactly which sections you'll do on which days makes Monday simpler to start.

When you fall behind

You will miss weeks. Everyone does. When it happens, don't try to catch up by doubling the hours; that's how a missed week becomes a missed month. Instead: **move the plan back a week, do one short review session to reconnect, and start the next section.** Keep a one-line **study log** in your **notes/** folder (date, what you did, one thing you learned), so that when you return, you can see how far you've come.

6.3 What you'll need, and when

The computer

You don't need a new or high-end computer to become a data analyst. Almost everything in Parts 0 to II runs comfortably on an ordinary laptop from the last five or six years: Windows 10 or 11, or a recent macOS; **8 GB of memory (RAM)** to start, 16 GB to be comfortable; 20 GB of free disk space, 50 GB or more to be comfortable; and an internet connection good enough to download software. A second monitor helps, so the book or the documentation sits next to your work, but a laptop screen is enough.

Three situations need a little planning:

- **A Mac.** Everything in the book runs on a Mac except **Power BI Desktop** and **Excel's Power Pivot**, a data-modelling feature of desktop Excel (Chapter 11, section 11.8), which are Windows only. The usual answer in Microsoft's Q&A forums is to run Windows 11 in a virtual machine, a program that runs Windows in a window on your Mac (for example, Parallels Desktop). The browser version of Power BI can view and interact with reports but can't build the data models Chapter 16 teaches, and it needs a work or school account. You can do everything else on your Mac and handle those two separately.
- **A Chromebook, tablet, or phone.** These can run Google Sheets, Excel for the web, and online SQL practice sites, which is enough for the spreadsheet chapters and some early SQL practice. They can't run Python properly, the databases, or Power BI Desktop. Plan to borrow or buy a laptop before the Python chapters.
- **A work laptop you can't install software on.** Many companies lock their laptops, for good security reasons. Ask your IT team; learning tools like PostgreSQL, Python, and DBeaver are commonly approved. Some tools can be installed without IT's help: Microsoft's documentation notes, for example, that Power BI Desktop from the Microsoft Store doesn't need administrator rights. The story later in this chapter shows one way through.

Watch out: Windows 10. Microsoft stopped regular security updates for Windows 10 in October 2025. It still runs every tool in this book, but a computer that no longer gets security updates is a risk when you start connecting to company data. If you can upgrade to Windows 11, do.

The tools, and when each arrives

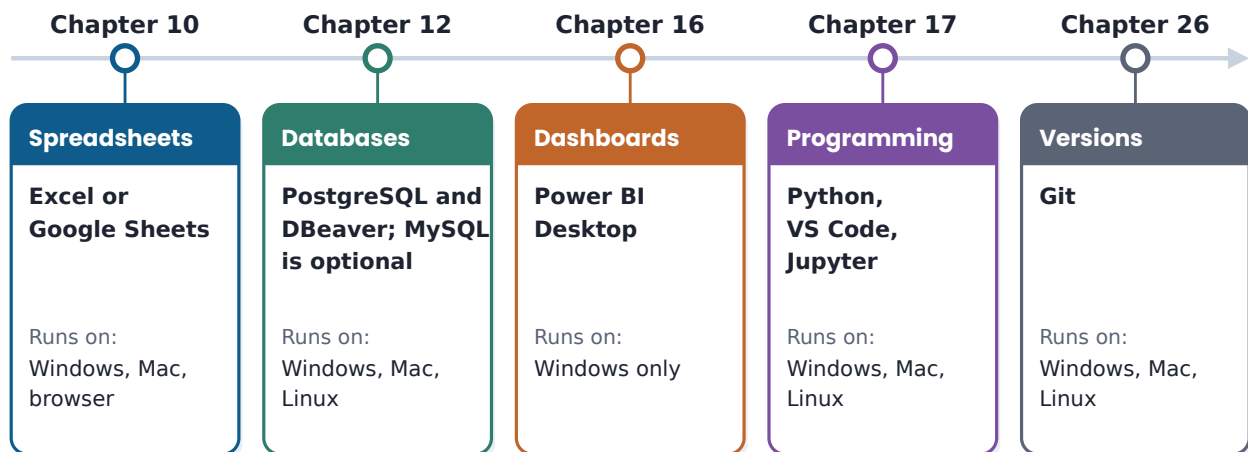
This is the book's **tool timeline**: which tool you'll need, and in which chapter you'll first need it.

Tool	First needed in	Cost	Runs on
A spreadsheet: Excel or Google Sheets	Chapter 10	Free on the web	Windows, Mac, browser
PostgreSQL and DBeaver (MySQL optional)	Chapter 12	Free	Windows, Mac, Linux
Power BI Desktop	Chapter 16	Free	Windows only
Python, VS Code, and Jupyter	Chapter 17	Free	Windows, Mac, Linux
Git	Chapter 26	Free	Windows, Mac, Linux

You'll install each tool at the start of the chapter that first uses it, and check it works with one small first step. Appendix B gathers all the install steps in one place, for when you set up a second computer.

Why wait? A tool installed months before you use it has usually been updated by the time you open it, and a problem is much easier to fix when the chapter in front of you explains what the tool is for. Installing in the chapter that needs it costs nothing and saves an evening.

The core toolkit, and the chapter that first needs each tool



Each tool is installed at the start of the chapter that first uses it. Later parts add their own tools when you reach them: dbt, cloud accounts, containers, and machine-learning libraries.

Figure 6.2 – Five groups of tools cover the whole analyst path, each installed in the chapter that first needs it. Later parts add their own tools when you reach them.

A few words on these choices. **Free tools only**, so nobody is locked out. **PostgreSQL and MySQL both**, because between them they cover most databases an analyst meets. **DBEaver**, because one app works with both databases. **Power BI**, because it's widely used and free to start with. **Python with VS Code**, because it's free, runs everywhere, and the same editor serves you from your first script to production code.

6.4 Reading documentation

Every tool in this book has **official documentation**: the manual written by the people who make it. When a tutorial, a colleague, and an AI assistant disagree, the documentation settles it. You'll use it in every tool chapter. You can practise the habit today, on a page you already have: the official page for your phone plan, or your bank's page of fees and charges.

You don't read an official page front to back. You go in with a question and look for five things:

1. **The exact rule:** the name of the thing you're asking about, and what it applies to. On a phone plan's page, that might be the line headed *Daily data*.
2. **The description, every word:** what the rule says, in one or two sentences. Read every word; the small details are usually the point. "2 GB per day" and "2 GB per day, unused data does not carry forward" are different plans.
3. **An example:** many pages show one, such as what happens on a day you use 2.5 GB. Work through it yourself before relying on it.
4. **The exceptions:** often in a box, a footnote, or the small print at the bottom. This is where "except on international roaming" or "after the fair-usage limit, speed drops" lives.
5. **The date or version:** official pages change, and old copies stay online. Check the "last updated" date or the plan's name and version, and make sure it's the one you're actually on.

If the page doesn't answer it, try the organization's official help pages or support, then a well-asked search.

Tools disagree with each other more often than you'd expect. For example, spreadsheets, databases, and Python don't all round a number that ends in exactly .5 the same way. Each tool's documentation says exactly which rule it uses, and you'll check it for yourself when you meet those tools.

6.5 Learning with AI assistants without letting them think for you

AI assistants can explain a concept five different ways, spot the typo in a query, and describe what an error message means. Used well, they make you learn faster. Used badly, they let you finish every exercise without learning anything, and you find out in an interview, or in your first week at work, when the assistant isn't the one being asked.

Rules that keep the learning yours

1. **Try first.** Spend real effort on an exercise before asking for help: at least fifteen to twenty minutes, or until you can say exactly where you're stuck. The struggle is where the learning happens.
2. **Ask for explanations, not answers.** "Why does my query return 21 rows instead of 19?" teaches you something. "Write the query for exercise 14" teaches you nothing.
3. **Check everything against a source you trust.** Assistants can be fluent and wrong at the same time. Section 6.4's method is how you check: find the official page and read the exact rule. A good test: ask an assistant what your phone plan does when you pass your daily data limit, then check the plan's own page.
4. **Never paste data you don't own.** Company data, customer names, invoices, and passwords don't belong in a chat with an outside service unless your employer has explicitly approved that tool for that data (Chapter 2, section 2.9, goes deeper). Practice data like Riverstone's is fine.

5. **Explain it back.** After the assistant helps, close the chat and explain the idea in your own words, or redo the exercise from a blank page. If you can't, you haven't learned it yet.
6. **Use it as a tutor, not a crutch.** Good requests: "Quiz me on percentage points versus percent change", "Give me a harder version of this exercise", "Explain this error message; don't fix it".

Common mistakes and how to spot them

Mistake	Symptom	Fix
Planning with the hours you wish you had	behind by week three; the plan quietly abandoned	Plan with last week's real hours and section 6.1's table
Doubling the hours to catch up after a missed week	a missed week becomes a missed month	Move the plan back a week, review once, start the next section
Installing every tool now "to get it out of the way"	forgotten passwords and out-of-date versions by the time you need them	Install each tool at the start of the chapter that first uses it
Trusting a tutorial or AI over the official source	answers that differ, unexplained	Check the official page, for the right date or version
Letting an AI assistant do the exercises	you can follow solutions but can't write them	Try first; ask for explanations; redo from a blank page
Reading answers before attempting exercises	answers "make sense" but skills don't stick	Write something first, always

In the real world: Meera makes a plan

Meera Iyer has done analyst work for months without the title: the customer counts in Chapter 1, the January reconciliation in Chapter 3, the board slide in Chapter 4. Along the way she has picked up some spreadsheet formulas and a little SQL on the job. Now she wants to go further: to learn it properly, from the ground up, and fill the gaps she knows are there. She has two computers to do it with. Neither is ideal.

Her work laptop runs Windows 11 and is locked: she can't install software. **Her home laptop** is a MacBook Air from 2020 with 8 GB of memory.

1. Which tool lives where. She reads section 6.3 and makes a list. She doesn't install anything today; she decides which tools will live on which computer when she reaches them. The Mac can run everything except Power BI Desktop and Power Pivot. The work laptop is where Power BI would be most useful, since the company already uses Microsoft 365. So: the book's tools go on the Mac at home, each at the start of its chapter; Power BI Desktop goes on the work laptop, if IT agrees. She puts a reminder in her calendar to ask IT two weeks before she reaches Chapter 16, with the request already written: *"I'd like to install Power BI Desktop from the Microsoft Store for learning and for building sales reports. It doesn't need admin rights. I won't connect it to any company data without approval."*

2. The 8 GB question. 8 GB is enough to start. When the Mac slows down later, with a database app, an editor, twenty browser tabs, and a video call all open, the fix will cost nothing: close what she isn't using while studying.

3. The hours. She has about 6 hours a week: an hour on three weekday evenings after work, and three hours on Sunday morning. Ninety days is about 13 weeks, so $6 \times 13 = 78$ hours. She adds up the *Time needed* lines from Chapter 7 onward, using the high end of each range so that one slow week doesn't break the plan:

Chapters	Hours (high end)	Running total
7, 8 (with its project), and 9	$3 + 7 + 3 = 13$	13
10	17	30
11	30	60
the next chapter in the book's order	18 of its hours	78

She writes her first 90 days in `notes/plan.md`. By day 30, about 26 hours: Chapters 7 to 9 and most of Chapter 10. By day 60, about 51 hours: two-thirds of the way through Chapter 11. By day 90, 78 hours: Chapter 11 finished in week 10, and the next chapter under way. At the top of the plan she writes the whole path, from the job-ready row of section 6.1: 317 to 394 hours, which at 6 hours a week is 12 to 15 months. At work, she'll rebuild the January reconciliation from Chapter 3 in a spreadsheet as her month 2 practice project, using the practice data rather than company files, so nothing sensitive leaves the company's systems.

4. The AI rule. She sets herself one rule, taped to the edge of her screen: "*Try for 20 minutes. Ask why, not what. Never paste Riverstone's real data.*"

She matched tools to computers without installing anything yet, turned hours into dates with arithmetic, and wrote a plan for her real week, not an ideal one.

Tools

- **A notebook or a notes app** for your plan (`notes/plan.md`) and your study log. A plain text file is enough.
- **A calendar**, paper or on your phone, for your study sessions as named appointments.
- **Each chapter's *Time needed* line**, which is where the hours in section 6.1 come from.

The project: plan your route and your first 90 days

Goal: a study plan you can keep, built from arithmetic, not hope.

Step 1. Count your hours. Write down the hours you really studied, or could have studied, last week. Using section 6.1's table, work out how many weeks the job-ready path takes at that pace, and the month you expect to finish Part II. Show your division.

Step 2. Put your rhythm in a calendar. Adapt Figure 6.1 to your hours and your days, and put each session in your calendar as a named appointment for the next four weeks.

Step 3. Set up your folder. Create the `analyst-to-architect` folder described in "How to Use This Book", with `companion/`, `work/`, and `notes/` inside it, and download the companion files into `companion/` (Appendix E gives the address).

Step 4. Write your AI rule. One or two lines, like Meera's, somewhere you'll see them.

Step 5. Read one official page. Pick one rule that affects you (your phone plan's data limit, a bank fee) and find its official page. Write down the exact rule, one example, one exception, and the date or version (section 6.4).

Step 6. Match tools to chapters. Using section 6.3, write which chapter you'll install each tool in, and on which computer.

Step 7. Write your 90-day plan in `notes/plan.md`:

- how many hours a week you can study, and on which days and times;
- which chapters you'll finish by day 30, day 60, and day 90, counted from the *Time needed* lines;
- one practice project for each month, using practice data;
- your rule for using AI assistants;
- what you'll do when you miss a week.

Step 8. Start your study log with today's entry.

Deliverable: `notes/plan.md` with your hours calculation, your 90-day plan, your tool-and-computer list, and your AI rule; your calendar with the next four weeks booked; and your notes on one official page.

You've got it when...

- ☐ I know how many hours the job-ready path takes, and how many weeks that is at my real weekly hours.
- ☐ My finish date comes from arithmetic, not hope.
- ☐ I have a weekly rhythm in my calendar, and I know what I'll do when I miss a week.
- ☐ I know whether my computer can run each tool, and I have a plan for any it can't.
- ☐ I know which chapter I'll install each tool in.
- ☐ I can find the exact rule, an example, the exceptions, and the date on an official page.
- ☐ I have rules for using AI assistants that keep the learning mine, and I never paste data I don't own.
- ☐ I have a written 90-day plan.
- ☐ I attempt exercises before reading the answers.

Recap

- **The book's hours are known.** Parts 0 to II, the job-ready path, are 317 to 394 hours; the whole book is 762 to 978. At 6 hours a week, job-ready is 12 to 15 months.

- **Divide to plan.** Weeks = hours ÷ your hours a week. Use the hours you really have. Chapter hours are reading-and-exercise hours; fluency takes more practice on top.
- **A rhythm finishes books.** Short regular sessions, one longer project session, and a weekly review from memory. A missed week moves the plan back; it never doubles the load.
- **You don't need a high-end computer.** 8 GB of memory is enough to start, 16 GB is comfortable. Power BI Desktop and Power Pivot are Windows only; Chromebooks cover spreadsheets and online SQL practice, not Python or Power BI.
- **Each tool arrives in the chapter that first uses it:** a spreadsheet in Chapter 10, the databases in 12, Power BI in 16, Python in 17, and Git in 26. Nothing needs installing before then.
- **Official sources settle disagreements.** Go in with a question and find the exact rule, every word, an example, the exceptions, and the date or version.
- **AI assistants are tutors, not substitutes:** try first, ask why, check against sources, never paste data you don't own, and explain it back.
- **Attempt every exercise before reading its answer,** and redo the ones you missed.

Practice exercises

Warm-up

1. For each computer, say which chapters you could do on it, and what the owner should do about the rest: (a) a Windows 11 laptop with 8 GB of memory; (b) a MacBook with 16 GB of memory; (c) a Chromebook; (d) a locked-down work laptop running Windows 11.
2. Sort these requests to an AI assistant into "helps you learn" or "does the learning for you", and rewrite the ones that don't help: (a) "Solve Chapter 4, exercise 6." (b) "Explain why averaging monthly growth rates overstates growth." (c) "My query returns 21 rows instead of 19. What kinds of mistake cause extra rows?" (d) "Write my 90-day plan."
3. Without looking back, name the chapter in which you'll install each of these: a spreadsheet, a database, Power BI Desktop, Python, and Git. Then check your answers against section 6.3.

Core

4. You can study 5 hours a week. Adapt Figure 6.1's weekly rhythm to 5 hours, keeping reading, exercises, review, and project work. Then use section 6.1's table to estimate how long the job-ready path would take at that pace.
5. Your company uses a customer list with names, phone numbers, and outstanding balances. You want an AI assistant's help writing a formula that flags overdue customers. Describe how you'd get the help without sharing the real data.
6. Using the table in section 6.1 and the hours you really studied last week, write the month you expect to finish Part II. Show your division.
7. Find the official page for one rule that affects you (your phone plan's data limit, a bank fee) and write down the exact rule, one example, one exception, and the date or version.

Stretch

8. Write a first version of your 90-day plan using the checklist in the project, with specific chapters for days 30, 60, and 90, and a plan for a missed week.
9. A friend plans to install all five groups of tools this weekend "to get it out of the way". Using section 6.3, give two reasons to wait, and one useful thing they could do this weekend instead.

Think about it (no calculation needed)

10. Why does this book recommend free tools, even though many companies use paid ones? What might you still need to learn on the job?
11. A friend says, "I'll skip the exercises and read the answers; it's faster." What would you tell them, using "How to Use This Book"?
12. When is it reasonable to ask an AI assistant for a complete answer, rather than an explanation? Give one example where it's fine and one where it isn't.

Key terms

study hours · job-ready path · weekly rhythm · review from memory · study log · study plan · RAM (memory) · tool timeline · official documentation · AI assistant

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 7, The Data Landscape, and Chapter 8, The Career Tree**, show where the skills you're planning for lead, and which roles use which tools.
- **Chapter 9, How Expertise Actually Forms**, explains the learning science behind section 6.2 (practice, review from memory, and feedback), and why chapter hours are not fluency hours.
- **The chapters that bring each tool**: Chapter 10 (a spreadsheet), Chapter 12 (the databases and DBeaver), Chapter 16 (Power BI Desktop), Chapter 17 (Python, VS Code, and Jupyter), and Chapter 26 (Git, and AI assistants in a professional setting). Each starts by installing its tool.
- **Chapter 83, The Long Game**, returns to the same hours at the end of the book, and to the pace that survives a bad month.
- **Appendix B** gathers every install step in one place.
- **Interview preparation**: Chapter 81 (behavioral, HR, and offer conversations) covers how to talk about how you learned and what you've built, and Chapter 68 explains how data hiring works.

Answers to practice exercises

1. (a) Every chapter, including Power BI Desktop in Chapter 16; 8 GB is enough to start, and closing unused apps helps. (b) Every chapter except the Power BI parts of Chapter 16 and the Power Pivot section of Chapter 11; for those, run Windows 11 in a virtual machine, or use a Windows computer at work or college. (c) Parts 0 and I, the spreadsheet chapters with Google Sheets or Excel for the web, and some early SQL practice on online practice sites; plan to use a laptop before the Python chapters and for the databases and Power BI. (d) Parts 0 and I straight away, since they need no software; for later chapters, nothing until IT approves. Ask for Power BI Desktop from the Microsoft Store (no admin rights needed) and for the other tools as you reach their chapters, and meanwhile use a home computer or the web tools with practice data.
2. (a) Does the learning for you. Better: *"I got a different answer for Chapter 4, exercise 6. Here's my working; where did my reasoning go wrong?"* (b) Helps you learn. (c) Helps you learn. (d) Does the planning for you. Better: *"Here's my 90-day plan and my available hours. What's unrealistic about it?"*
3. A spreadsheet in Chapter 10; the databases (PostgreSQL and DBeaver, with MySQL optional) in Chapter 12; Power BI Desktop in Chapter 16; Python, with VS Code and Jupyter, in Chapter 17; Git in Chapter 26.
4. One way: Monday 1 hour reading, Wednesday 1 hour exercises, Friday 30 minutes review from memory, Saturday 2 hours project, Sunday 30 minutes redoing missed exercises and planning: $1 + 1 + 0.5 + 2 + 0.5 = 5$ hours. The job-ready path is 317–394 hours; $317 \div 5 = 63.4$, about 63 weeks, and $394 \div 5 = 78.8$, about 79 weeks. That's 63–79 weeks, or about 15 to 18 months ($63 \times 12 \div 52 = 14.5$; $79 \times 12 \div 52 = 18.2$).
5. Describe the structure, not the data: *"I have a table with a customer name column, a due date column, and a balance column. How do I flag rows where the due date is before today and the balance is above zero?"* Or build a small made-up sample (three fake customers with invented numbers) and share that. Test the formula on the fake data, then apply it to the real file on your own computer. If your company has approved an AI tool for internal data, follow its rules instead.
6. Answers vary. A worked example: last week you studied 5 hours, and you've finished Chapters 1 to 6. What's left is Chapters 7 to 9 (9–13 hours, from their *Time needed* lines) plus Part II (290–357 hours): 299–370 hours. $299 \div 5 = 59.8$, about 60 weeks; $370 \div 5 = 74$ weeks. In months, $60 \times 12 \div 52 = 13.8$ and $74 \times 12 \div 52 = 17.1$: about 14 to 17 months. If you start Chapter 7 in October, you'd finish Part II between December of the following year and March of the year after. A good answer shows the division and gives a range, not a single hopeful date.
7. Answers vary. A complete answer names the rule exactly as the page does (for example, "Daily data: 2 GB"), gives one example worked from the page (what happens on a day you use 2.5 GB), quotes one exception or piece of small print (for example, that unused data doesn't carry forward, or that roaming is charged separately), and records the page's "last updated" date or the plan's name and version.
8. Answers vary. A good plan names specific hours ("Tuesday and Thursday, 8–9 p.m.; Saturday, 9–11 a.m."), specific chapters for each checkpoint, counted from the *Time needed* lines as Meera did, one project a month on practice data, an AI rule, and a concrete plan for a missed week ("move the plan

back a week; do one review session; start the next section").

9. Reasons to wait: a tool installed months early has usually been updated by the time it's used, so the versions and the steps may no longer match; problems are much easier to fix when the chapter in front of you explains what the tool is for; and passwords set months before they're needed get forgotten. Something useful instead: work through this chapter's project (count the hours, book the sessions, set up the folder and download the companion files), and start Chapter 7.

10. Nobody is locked out by cost, and PostgreSQL, MySQL, Python, and Git are widely used at work anyway. The ideas transfer: most SQL runs elsewhere, and Power BI's concepts map to other tools such as Tableau and Looker. On the job you may still need to learn a company's own data systems, its BI tool, its data definitions, and its security rules.

11. Reading answers you haven't attempted feels like learning because the answers make sense, but it doesn't build the skill of producing them. The exercises are where most of the learning happens, especially the core group. Suggest a compromise: attempt every warm-up and core exercise, even partially, before reading its answer, and redo the missed ones at the end of the week.

12. It's reasonable when the task isn't the skill you're trying to learn, or when you already understand it and are saving time, and you'll check the result. Fine: asking for a list of keyboard shortcuts for an app you use, or for a first version of an IT request that you then edit. Not fine: asking for the answer to a SQL exercise while you're learning SQL, because producing that answer is exactly the skill you need.

ANALYST TO ARCHITECT · PART I

The Map

Chapters 7–9: the data landscape, the career tree and how skills unlock roles, and how expertise actually forms.

Contents

- Part I — The Map** 2
- Chapter 7. The Data Landscape** 3
- Chapter 8. The Career Tree: How Skills Unlock Roles** 28
- Chapter 9. How Expertise Actually Forms** 54

Part I — The Map

Part I shows you the whole field before you start climbing it: who does which data work, how the roles connect, which skills open which doors, and how expertise is built over months and years. It's the part to reread whenever you're choosing what to learn next.

Chapter	What you'll be able to do	Time needed
7. The Data Landscape	place any data job under the four questions; describe the ten roles and three team structures; follow one request through every role	2–3 hours
8. The Career Tree: How Skills Unlock Roles	read the tiers and the skills matrix; decode a job description; understand pay sources; choose an entry route and plan backward from a role	3–4 hours
9. How Expertise Actually Forms	estimate your own timeline; practice deliberately; build portfolio pieces; get feedback; work through plateaus	2–3 hours

In total, allow 7–10 hours, including the exercises and projects. After Part I, Part II (Chapters 10–27) builds the analyst's toolkit, starting with spreadsheets.

Chapter 7. The Data Landscape

Chapter at a glance

You will learn to: name the four questions every data job answers, and place any job title under one of them · describe what each of the ten main data roles does, what it produces, and which tools it uses · explain why the field grows like a tree, with a shared trunk and branches that rejoin at the top · recognize the automation and integration track, and the automation hidden inside every other role · compare centralized, embedded, and hub-and-spoke data teams, and say when each fits · follow one business request through every role, from the first question to a running platform · judge what AI assistants change in each role, and what stays your responsibility.

Before you start: Chapter 1 (what data is) and Chapter 2 (files, databases, servers, and APIs). Chapter 3 (how a business runs on data) helps but isn't required.

Time needed: 2–3 hours, including the exercises and the project.

Tools: a pen and paper, or any free drawing tool for the project. No software to install.

Practice data: one result from the Riverstone one-year database (2025), so you can see what an analyst's answer looks like. Every number shown is real.

Why this matters

From the outside, "working in data" sounds like a country where every city has the same name. Data analyst, business analyst, BI developer, analytics engineer, data scientist, ML engineer, data engineer, AI engineer, integration engineer, data architect: the titles blur together, and the job descriptions overlap. People new to the field often spend months learning tools for a job they turn out not to want, or apply for roles whose real work they don't understand.

The first skill in a data career isn't SQL or Python. It's **orientation**: knowing what the roles are, what each one produces, how they depend on each other, and where you stand among them. Once you have that, every later decision gets easier. You can read a job description and see the real job behind the title. You can pick what to learn next for a reason. And in an interview, you can explain where your work fits in the team, which interviewers notice.

This chapter is the map. Chapter 8 turns it into a route, skill by skill, and Chapter 9 is honest about how long the route takes.

In plain English

Think about what happens when you walk into a hospital with a pain in your side.

- At reception, someone asks what's wrong and writes it down in a way the doctors can use. Vague ("it hurts") becomes precise ("sharp pain, lower right, since yesterday"). That's the work of a

business analyst: turning a vague request into a clear question.

- A doctor looks at your test results and says what's happening now. That's a **data analyst:** describing what happened, clearly and correctly.
- A specialist uses your history to judge what will probably happen next, and which treatment will work. That's a **data scientist:** prediction and cause.
- The lab technicians make sure every blood sample is collected, labeled, tested, and delivered to the right doctor on time. Nobody thanks them until a sample goes missing. That's a **data engineer.**
- The hospital's systems send your prescription to the pharmacy and a reminder to your phone, without anyone re-typing it. That's **automation and integration.**
- Someone designed the building: where the lab sits relative to the emergency ward, how records flow between departments, who is allowed to see what. That's a **data architect.**

One patient, one question, many specialists, all depending on each other. A data team works the same way. Keep this picture in mind; section 7.6 follows a single Riverstone request through every one of these roles.

7.1 The four questions every data job answers

Strip away the tools and the titles, and every data role exists to help answer one of four questions about an organization and its data.

1. **What happened?** Describing the past and present clearly: last month's sales, this week's late deliveries, which customers have stopped ordering, and the immediate reasons behind them.
2. **What will happen, and why?** Predicting what comes next, and finding out what causes what: which customers are likely to leave, how much stock to make for Diwali, whether a new discount really increased sales.
3. **How does data move, get served, and get put to work, reliably?** Building the machinery: collecting data from every system, checking it, storing it, and delivering it (as a report, an alert, or a task in another system) on time, every time, without someone doing it by hand.
4. **How should the whole system be designed?** Deciding how all the pieces fit together, who owns what, what it costs, and how it stays secure and trustworthy as the company grows.

Almost every data job is a specialization of one of these. When you meet an unfamiliar title, ask "Which of the four questions does this person spend most of their day on?" The title then stops being confusing.

QUESTION 1
What happened?

TRACK
Analytics & BI

ROLES

- Data analyst
- Business analyst
- BI developer

Taught in Parts II-III

QUESTION 2
What will happen, and why?

TRACK
Data science, ML & AI

ROLES

- Data scientist
- ML engineer
- AI engineer

Taught in Parts IV and VI

QUESTION 3
How does data move, and get put to work?

TRACK
Engineering & integration

ROLES

- Data engineer
- Analytics engineer
- Automation analyst, RPA developer, integration engineer

Taught in Parts II, III and V

QUESTION 4
How should the whole system be designed?

TRACK
Architecture

ROLES

- Data architect

Taught in Part VII

Underneath all four: governance and data quality (data stewards, data quality and governance analysts).
Shared by every role: SQL, business context, and some automation of repeated work.

Figure 7.1 – The four questions, the tracks that answer them, and the ten roles this book follows. Notice the band at the bottom: some skills belong to everyone.

Simplification note. Real jobs cross these lines. Analysts ask "why?" all the time, and a good analyst explains *why* sales fell in March, not only *that* they fell. The difference is depth: an analyst finds the likely reason in the data; a data scientist designs an experiment or a model to prove a cause or predict an outcome. Use the four questions to find a role's center of gravity, not its edges.

7.2 The tracks and the ten roles

A **track** is a family of related roles that answer the same question. A **role** is a job with a recognizable set of responsibilities, whatever a particular company calls it. The same role might be advertised as "MIS executive", "reporting analyst", or "insights analyst", and the same title might mean different work at two companies. That's why this book describes roles by what they produce, not by title.

Here are the ten roles this book follows, grouped by track in the same order as Figure 7.1.

Track	Role	Center of gravity	What they typically produce	Core tools	Taught in
Analytics & BI	Data analyst	What happened, and why it probably happened	Answers to business questions, reports, analyses, recommendations	Excel or Google Sheets, SQL, a BI tool, some Python	Part II
Analytics & BI	Business analyst	What the business needs, and how a process should change	Clear requirements, process maps, user stories, test plans, the case for a change	Spreadsheets, SQL, process-mapping and ticketing tools	Ch 24–25
Analytics & BI	BI developer	Making "what happened" available to everyone, all the time	Dashboards, data models behind them, scheduled reports	Power BI or similar, SQL, DAX	Ch 16, 28
Data science, ML & AI	Data scientist	What will happen, and what causes what	Models, forecasts, experiment results, recommendations	Python, statistics, scikit-learn, SQL	Part IV
Data science, ML & AI	ML engineer	Keeping models working in the real world	Models running as services, with monitoring and retraining	Python, cloud services, MLOps tools	Ch 56
Data science, ML & AI	AI engineer	Building useful, safe products on top of AI models	Assistants, AI features, and AI steps inside business workflows, with evaluation	Python, LLM APIs, retrieval, evaluation tools	Ch 54–58
Engineering & integration	Data engineer	Getting data from every source, correct and on time	Pipelines, warehouses, data-quality checks	SQL, Python, orchestration tools, cloud	Part V
Engineering & integration	Analytics engineer	One trusted, tested version of the business's numbers	Clean, documented, tested tables that analysts and dashboards use	SQL, dbt, Git	Ch 32
Engineering & integration	Automation analyst, RPA developer, integration engineer	Removing manual steps, and connecting systems so data flows between them	Automated reports, low-code flows, bots, API integrations, data written back into business systems	Spreadsheet automation, Python, Power Automate or n8n, APIs	Ch 19–20, 51, 58

Track	Role	Center of gravity	What they typically produce	Core tools	Taught in
Architecture	Data architect	How the whole data system should be designed	Designs, standards, decision records, platform roadmaps	All of the above, at the level of design	Part VII

A few terms in that table will be new. A **dashboard** is a screen of charts and numbers that refreshes itself from the data. A **model**, in data science, is a set of rules learned from past data that makes predictions about new data. A **pipeline** is an automated series of steps that moves data from one place to another and changes it along the way. **BI** stands for business intelligence: the tools and practices for turning data into reports and dashboards people use to run the business. **MLOps** and **LLM** will each get full chapters; for now, MLOps is "keeping machine learning models healthy in production", and an LLM (large language model) is the kind of AI model behind chat assistants.

The one skill every role shares

Look down the *Core tools* column and one name keeps appearing: **SQL**. It's the language for asking questions of a database, and every role on this map writes it, often every day. The analyst uses it to answer questions. The BI developer uses it to feed dashboards. The analytics engineer builds whole tested models of the business in it. The data scientist pulls training data with it. The data engineer moves and checks data with it. The integration engineer uses it to choose which records to push into the CRM. The architect reads it to understand what the system is really doing. That's why Part II teaches it at full depth, and why nothing you learn there goes to waste, whichever branch you climb.

Watch out: titles lie, outputs don't. A "data scientist" at one company builds dashboards all day; a "business analyst" at another writes Python pipelines. Before you decide a job is (or isn't) for you, read what the person will *produce* each week. Chapter 8 shows how to decode a job description line by line.

Roles beyond the ten

The map has more cities than this book can visit in depth. **Data governance analysts** and **data stewards** make sure data is defined, documented, protected, and used lawfully. **Data quality analysts** hunt for and fix errors at the source. Specialist scientists work on text (NLP), images (computer vision), or recommendations. **Analytics managers**, **heads of data**, and **chief data officers** lead the people. Many domain roles combine a business function with data work: sales operations, finance analyst, supply chain analyst, marketing analyst. Each of these sits on the same map, and the chapters listed in "Where this leads" show where the book touches them.

7.3 The shape of the field: a trunk that branches

Here is the most important idea in this chapter: **the roles share a trunk and branch upward, and the branches rejoin at the top.**

Everyone starts with the same foundations: understanding data (Part 0), a spreadsheet, SQL, and business sense. The analyst core grows straight up from there (Part II), and many people build a whole career on it. Then the field branches. After the advanced analytics and analytics engineering skills of Part III, one branch leads toward data science and machine learning (Part IV), and the other toward data engineering and integration (Part V). The two branches meet again when models and AI have to run inside a real business (Part VI), and they come together fully at the top, in architecture (Part VII). A data architect can't design a system without having walked at least partway down each branch.

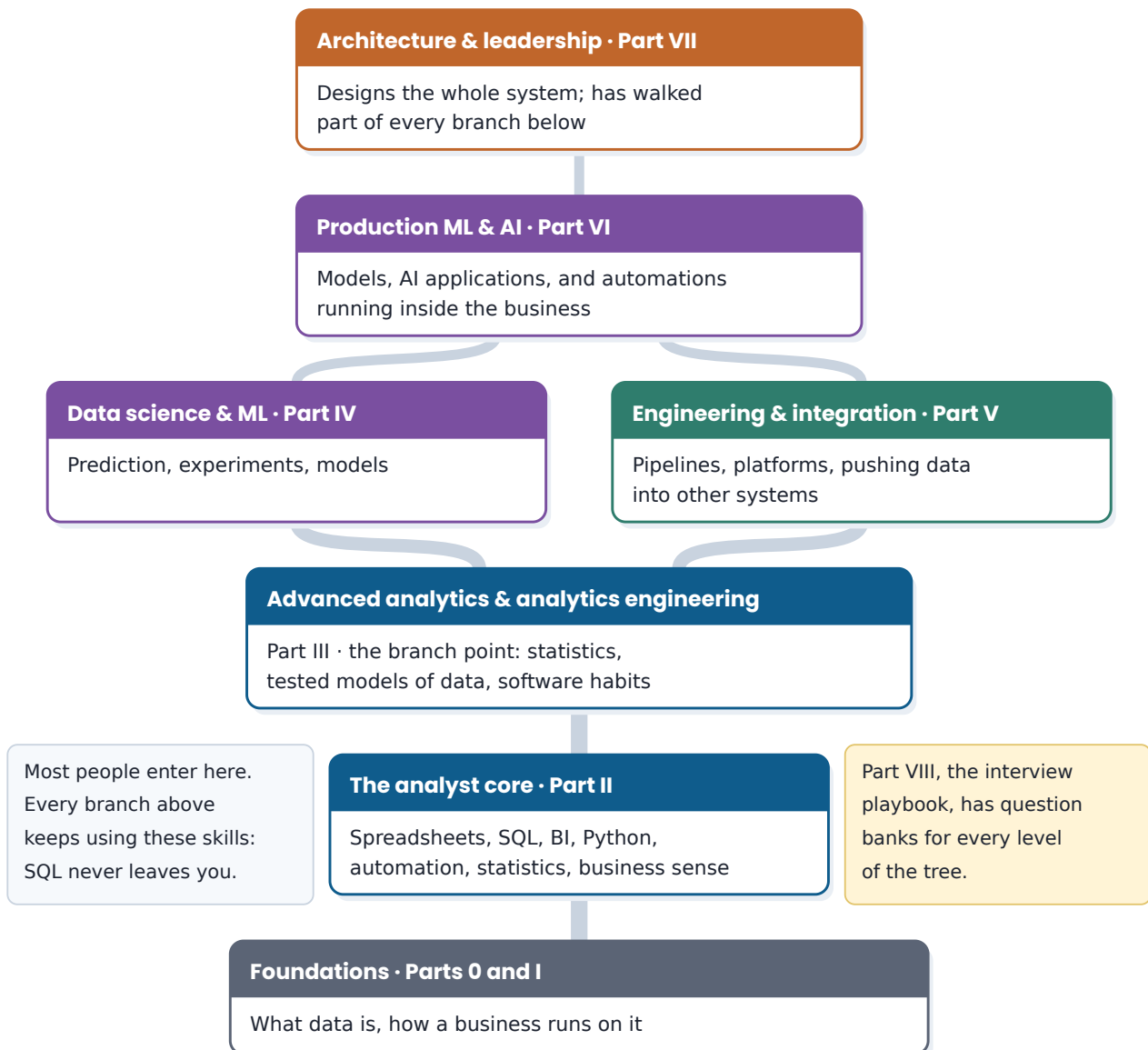


Figure 7.2 — The field grows like a tree, and this book follows it part by part. Read it from the bottom up: nobody starts at the top.

Two consequences follow, and they shape how you should use this book.

- **You don't abandon lower levels when you climb.** A data scientist still writes SQL every week, and an architect still reads spreadsheets. Skills accumulate.
- **Science and engineering are peers, not a ladder.** Neither branch is "above" the other. The book teaches data science (Part IV) before data engineering (Part V) because many analysts move

toward science first, but plenty of excellent careers go straight from the analyst core to engineering. Chapter 8 turns this tree into tiers you can plan with, in the same order as the book's parts.

7.4 The automation and integration track

At every level of the tree, the same practical question comes up: *once the data is right, how does it reach the people and systems that act on it, without someone doing it by hand?*

In most companies, a large share of data work is exactly this. Someone exports a file from one system, pastes it into a spreadsheet, fixes the columns, adds formulas, saves a copy, and emails it to twenty managers. Next week they do it again. Chapter 2's "Friday file" was a small example of that kind of routine. Removing those steps is the job of the automation and integration track.

The track has three common roles, and the lines between them are blurry.

- An **automation analyst** finds manual, repeated work in business processes and automates it, often with spreadsheet automation (macros, VBA, Office Scripts, Google Apps Script), low-code tools such as Power Automate or n8n, and Python scripts. The role often grows out of an analyst or operations job.
- An **RPA developer** builds software "robots" that click through screens the way a person would. **RPA** stands for robotic process automation. It's the right tool when an old system has no other way in, and the wrong tool when a proper connection exists, because a robot breaks whenever the screen changes.
- An **integration engineer** connects systems directly, usually through APIs (Chapter 2), so data flows between them: new orders from the website into the ERP, payment status from finance into the CRM, a risk flag from the data warehouse back into the sales team's tools. Writing data from the warehouse back into business systems is called **reverse ETL** or **data activation**.

Here's what makes this track different from the others: **every data role includes some automation**.

The analyst schedules a refreshable report instead of rebuilding it. The BI developer sets up a dashboard subscription. The data engineer's whole job is automated pipelines. The ML engineer automates retraining. The architect decides which automations the company should build, own, and eventually retire. The automation roles specialize in it, but nobody in data is exempt.

Take a simple example. Riverstone's **Daily Sales Flash**, a one-page summary of yesterday's orders, could start as a manual report that takes someone 40 minutes each working day. If it's produced on 250 working days a year, that's $40 \times 250 = 10,000$ minutes, or about 167 hours: more than four working weeks spent re-typing the same report. Over the course of this book, that report becomes a SQL query (Chapter 12), a Python script (Chapter 18), a VBA macro and an Apps Script project (Chapter 19), a scheduled email in every manager's inbox (Chapter 20), a monitored pipeline (Chapters 46–47), a source of flags pushed into the CRM (Chapter 51), and finally part of a designed reporting platform (Chapter 63). The business need never changes. The way it's met gets better at each level, which is exactly how the job grows in real life.

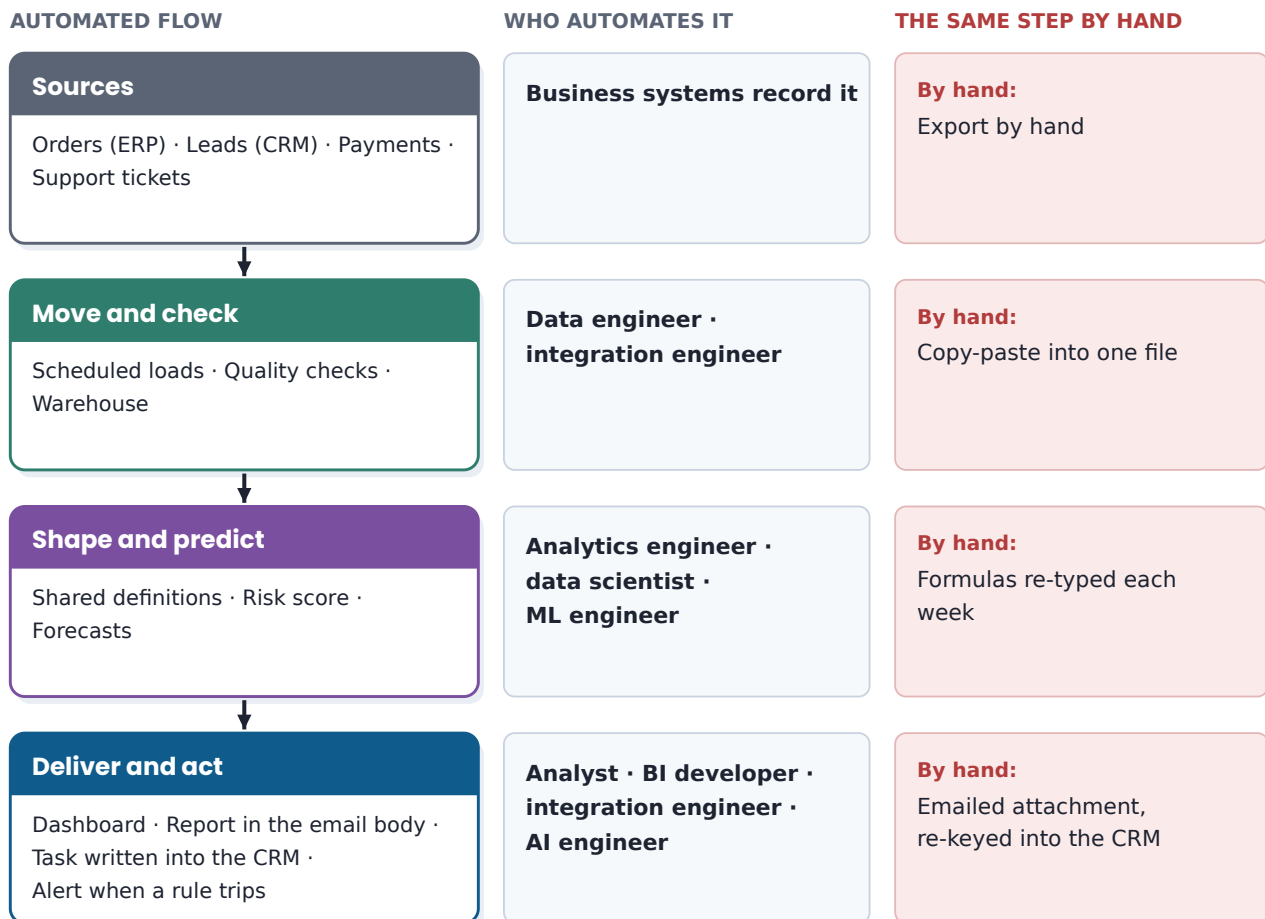


Figure 7.3 — The flow from source to action, with the roles that automate each stage. The red "by hand" column is what the same flow looks like when people do it by hand.

Real-life example: the report nobody owns. Many companies run an important report on a macro written years ago by someone who has since left. It works until the source file's columns change, and then it fails quietly or, worse, produces wrong numbers that look right. Automating a report is only half the job. Someone has to own it, monitor it, and know what to do when it breaks.

7.5 How data teams are organized

Knowing the roles is half the map. The other half is how companies arrange those people, because it changes your day, your boss, and your career path. There are three common structures.

In a **centralized** team, all the data people sit in one team, usually under a head of data or a technology leader, and serve every department. Sales, finance, and operations send requests to that team.

In an **embedded** (or **decentralized**) setup, data people sit inside the departments they serve and report to those departments' heads: a sales analyst in the sales team, a finance analyst in finance. There's little or no central data team.

In a **hub-and-spoke** (sometimes called **federated**) structure, a central hub owns the shared platform, the standards, and the common definitions of key numbers, while analysts in each department (the spokes) do the day-to-day work. Spokes often report to their department but follow the hub's standards and meet with the hub regularly.

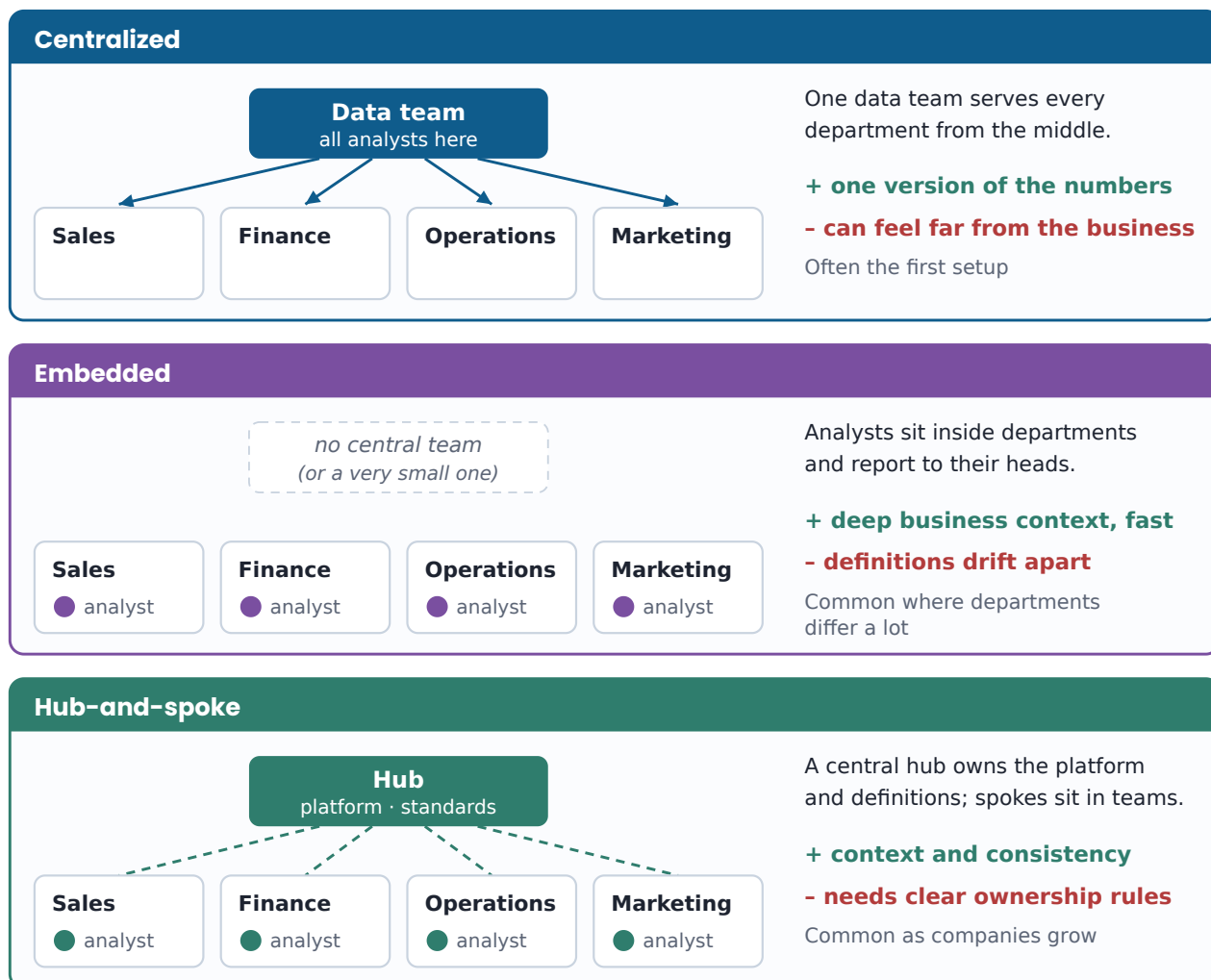


Figure 7.4 — Three ways to organize a data team. The same people can be arranged very differently, and each arrangement has a typical failure.

	Centralized	Embedded	Hub-and-spoke
Consistency of numbers	High: one team, one definition	Low: each department defines "revenue" its own way	High for shared numbers, if the hub enforces definitions
Understanding of the business	Can be weak; the team hears about problems secondhand	Strong; analysts sit in the meetings	Strong in the spokes
Speed for a department	Slower; requests wait in a queue	Fast	Fast for local work; shared changes take coordination
Duplicated work	Low	High: three teams build three versions of the same report	Medium
Career growth for data people	Clear; you learn from other data people	Harder; your manager may not understand your work	Clearer, through the hub's community and standards
Typical failure	The bottleneck: "the data team never gets to our request"	"Why do sales and finance report different revenue?"	Nobody is sure who owns a definition

No structure is right for every company. Small companies often start with one or two data people who behave like a tiny central team. As departments grow, analysts get hired into them, and inconsistent numbers start to hurt. Many companies then move toward hub-and-spoke to keep local speed while fixing definitions centrally.

Try it. Think of a company you know, or your college's administration. Where do the people who make reports sit? Which of the three structures is it closest to, and which typical failure have you heard people complain about?

7.6 One request, every role

Roles make the most sense in motion. So let's follow a single business request through all ten of them.

Simplification note. Riverstone is a mid-sized company, and in real life one or two people would cover most of these roles. To see each role clearly, imagine a larger version of Riverstone with one specialist in every seat. The data in step 2 is the real one-year database; the later steps describe what each role would build, and "Where this leads" names the chapters that teach you to build it.

It's the year-end review, and Anita Rao, the Sales Head, says:

"Some customers seem to have stopped ordering. Which ones, and what should we do about it?"

Step 1: The business analyst turns the request into a question

"Stopped ordering" can mean many things. The business analyst asks questions before anyone touches data: *Stopped since when? Does a cancelled order count? What about customers who signed up and never ordered at all? Who will act on the list, and what will they do? How often do you need it?*

The answers become a short requirement: "**List every customer with no non-cancelled order in the 60 days up to 31 December 2025, including customers who have never ordered. Sales reps will call them. Needed weekly.**" That sentence took twenty minutes of conversation, and it will save days of rework.

Step 2: The data analyst answers it

The data analyst turns the requirement into a method before touching the data. In plain words, it has four steps:

1. Start from every customer, including those who never ordered.
2. For each one, find the latest order that wasn't cancelled.
3. Count the days from that date to 31 December.
4. Keep those over 60 days, or with no order at all.

Run against the one-year database, the method gives five customers.

Customer	Segment	Latest order that wasn't cancelled	Days since that order
Home Plus	Retail	none	none
City Needs Store	Retail	22 March 2025	284
Sunrise Caterers	Hospitality	10 June 2025	204
Om Sai Provisions	Retail	22 July 2025	162
Tasty Tiffins	Hospitality	26 October 2025	66

One-year database (2025). The customer with no order comes first, then the longest silence.

Home Plus signed up on 18 June 2025 and has never ordered, so there's no date to count from. That's exactly the customer step 1 of the method is there to keep.

Check one row by hand. Tasty Tiffins last ordered on 26 October 2025. From 26 to 31 October is 5 days, November has 30, and December has 31: $5 + 30 + 31 = 66$ days. ✓

A good analyst doesn't stop at the list. Looking closer, City Needs Store and Tasty Tiffins have placed only two orders each, so it's hard to call them "regulars". Sunrise Caterers and Om Sai Provisions placed four orders each, roughly every five to six weeks, and then went silent. Together those two brought in ₹1,14,072.50 of revenue in 2025, about 2.6% of Riverstone's ₹43,35,471 for the year.

What to tell Anita. Five customers haven't ordered in 60 days. Two of them, Sunrise Caterers and Om Sai Provisions, used to order regularly, so they're the most urgent calls: find out what went wrong. Home Plus signed up in June and never ordered: a lead that was never converted. The other two ordered only twice, so they may be occasional buyers.

In Chapter 12 you'll write this yourself.

Step 3: The BI developer makes it available every day

Anita likes the answer and wants it every week without asking. The BI developer adds an "At-risk customers" page to the sales dashboard. It refreshes from the database each morning, each rep can filter it to their own customers, and Anita gets it by email every Monday through a dashboard subscription. The analyst no longer has to re-run the list by hand.

Step 4: The analytics engineer makes the definition trustworthy

A month later, finance mentions that *their* "inactive customers" report uses 90 days, not 60. The customer-support team uses a third rule. Three reports, three answers, and every meeting starts with an argument about whose number is right.

The analytics engineer agrees one definition with sales and finance, writes it once as a tested, documented table (a `customer_activity` model), and points every report at it. Tests run automatically: no customer can appear twice, every customer must have a status, and the total number of customers must match the source system. "Active customer" now means one thing everywhere.

Step 5: The data engineer makes sure the data arrives

Before anyone can predict, the data has to be complete: payments from finance, tickets from the support desk, and sales activity from the CRM, as well as orders, all fresh by 6 a.m. The data engineer builds pipelines that pull data from each system every night, check it (did yesterday's orders arrive? are there duplicate payments?), and load it into the warehouse. If a check fails, the pipeline stops and alerts someone *before* a wrong number reaches a sales rep.

Step 6: The data scientist asks what will happen next

The list tells Riverstone who *has* gone quiet. The data scientist asks a better question: *can we tell who is **about to** go quiet, while there's still time to help?* They look for early warning signs in the history: orders getting smaller, gaps getting longer, late payments, complaints to support. They build a model that gives each customer a risk score, and test it fairly against past data: does it beat the simple 60-day rule, or is it an expensive way to get the same list? If it does beat it, they suggest a fair test: call half the high-risk customers, leave the other half alone for a month, and compare what happens. That test is how you learn whether the calls work.

Step 7: The ML engineer keeps the model running

A model in a notebook helps nobody on Monday morning. The ML engineer turns it into a service that scores every customer each night, logs every score, and raises an alarm if the model's accuracy starts to slip, for example if customer behavior changes after a price increase. They also set up a way to retrain it safely.

Step 8: The automation or integration engineer puts it to work

A score sitting in a warehouse changes nothing. The integration engineer writes the high-risk customers into the CRM as call tasks for the right rep, making sure the same customer doesn't get a duplicate task every night. They also set up a Monday email to each rep with their own list in the body of the email, not as an attachment nobody opens. If the old finance system had no API, this is where an RPA bot might be considered, as a last resort.

Step 9: The AI engineer helps the rep prepare

Before calling Sunrise Caterers, a rep wants to know the story: last orders, open invoices, recent complaints. The AI engineer builds an assistant that drafts a short call brief from Riverstone's own data. The design matters more than the demo: the assistant may only use facts it retrieved from the company's systems, it shows where each fact came from, the rep reads and approves the brief before using it, and the team tests it regularly for invented facts.

Step 10: The data architect designs how it all fits

Stand back and look at everything that now exists: pipelines, a warehouse, a shared definition, a model service, a dashboard, CRM tasks, emails, an AI assistant. The data architect decides how these pieces should fit together, so the company doesn't end up with a tangle of scripts nobody understands. Where does the risk score officially live? Who owns the definition of "active"? Should reps see only their own customers? What happens if the CRM write-back fails halfway? What does the whole thing cost each month, and is it worth it? The architect writes these decisions down, with the reasons, so the next person can understand them.

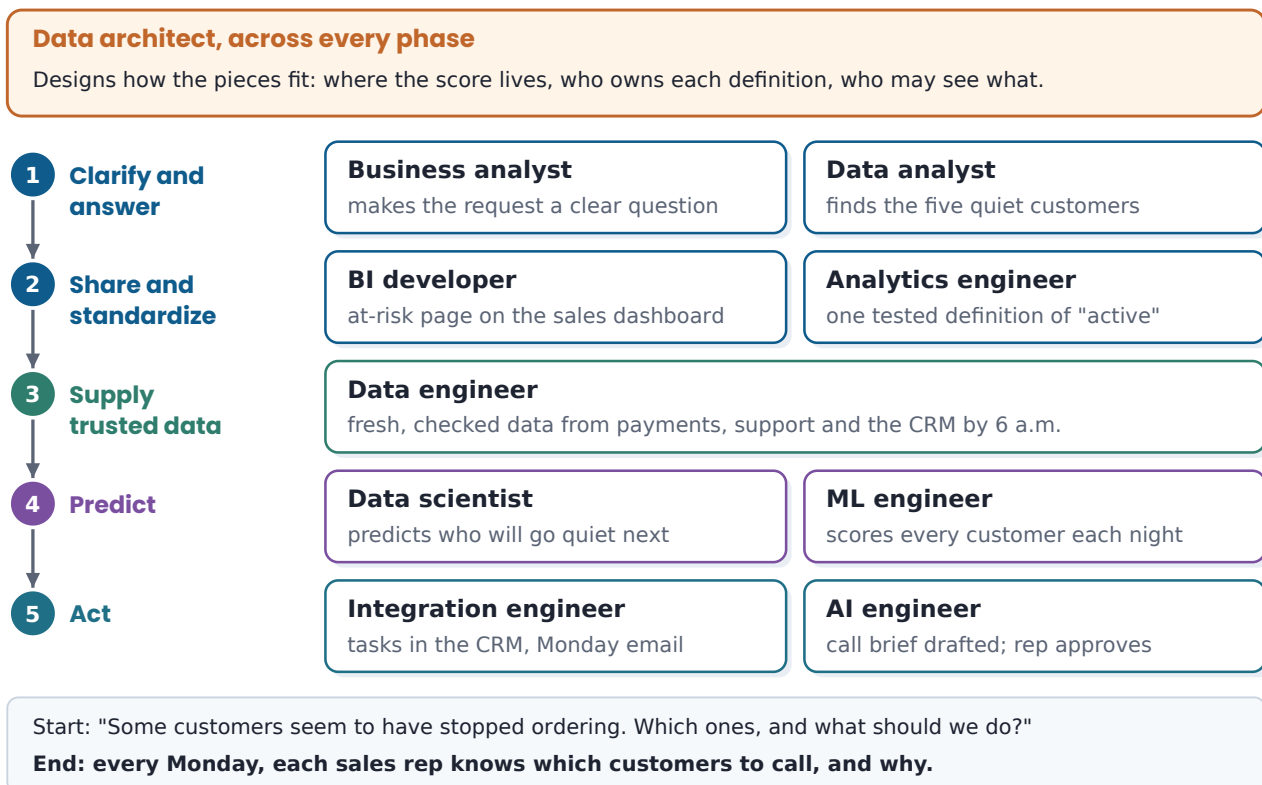


Figure 7.5 – One request, every role. Notice that the order matters: prediction is worth building only after the question is clear, the definition is agreed and the data is trusted.

What the walk-through shows

Three things stand out.

- **Most of the value came early.** Steps 1 and 2 answered Anita's question in an afternoon. Everything after that makes the answer faster, more consistent, more predictive, or more automatic. Those are real improvements, but a company should build them only when the simpler version is in use and its limits are hurting.
- **Every role depends on the ones before it.** A model built on an unclear question, or on data nobody checks, produces confident nonsense.
- **Automation appears at almost every step.** The subscription, the tests, the nightly scoring, the pipeline, the CRM tasks, the email: each one removes a manual step someone would otherwise repeat forever.

7.7 How AI assistants are changing each role

AI assistants can now draft formulas, SQL, Python, documentation, and summaries in seconds. It's natural to wonder whether that makes these roles, or the skills in this book, less needed. The pattern so far is more useful than either hype or fear.

Adoption is high, and trust is limited. In Stack Overflow's 2025 Developer Survey, a large survey of people who write code, 84% of respondents said they use or plan to use AI tools in their work, up from 76% the year before. In the same survey, more respondents said they distrust the accuracy of AI output (46%) than trust it (33%). The survey covers developers generally, not data roles specifically, but the message fits data work well: people use these tools constantly, and they check what the tools produce.

Demand for data skills is still growing. The World Economic Forum's *Future of Jobs Report 2025*, based on a survey of employers, lists big data specialists and AI and machine learning specialists among the fastest-growing jobs to 2030, and data entry clerks among the fastest-declining. Employers in the same report rate analytical thinking as the most sought-after core skill. Read the direction, not the exact rankings: the work that is shrinking is re-typing data; the work that is growing is understanding and using it.

Here's how that plays out role by role.

Role	What AI assistants speed up	What stays your responsibility
Data analyst	First drafts of formulas and queries; explaining an unfamiliar table; summarizing results	Knowing which question matters; checking the numbers; telling the manager what to do
Business analyst	Meeting notes, first drafts of requirements and user stories	Noticing what stakeholders didn't say; resolving disagreements; deciding what's in scope
BI developer	Suggesting charts; drafting measures	Choosing what the dashboard should help people decide; making sure the numbers match the source
Analytics engineer	Drafting tests and documentation	Getting departments to agree on one definition

Role	What AI assistants speed up	What stays your responsibility
Data scientist	Boilerplate code; lists of candidate features	Framing the problem; spotting data leakage; designing fair tests; saying "the model isn't worth it"
ML engineer	Deployment and monitoring scripts	Deciding what to monitor, and what to do when a model degrades
Data engineer	Pipeline code; translating SQL between databases	Designing for failure; agreeing data contracts with source-system owners
Automation / integration	Building flows from a plain-language description	Handling exceptions, permissions, and what happens when a step fails silently
AI engineer	Much of the code itself	Evaluating quality and safety; deciding where a human must approve
Data architect	Drafting design documents; comparing options	Making trade-offs and being accountable for them

The right column is the durable part of each job. Notice what those tasks have in common: they need context about the business, judgment about trade-offs, and someone who is accountable when things go wrong. The left column holds tasks with a checkable right answer, and those are what assistants do well, if someone who understands the work checks them.

That leads to the rule this book follows throughout: **you are responsible for every number you deliver, whoever or whatever typed the formula.** An assistant that writes a query you can't read hasn't saved you time; it has moved the risk to the moment your manager acts on a wrong number. So the book teaches you to read, write, and check the work yourself, then shows how to use assistants to go faster. Chapter 6 covers learning with AI assistants without letting them think for you, and Chapter 26 covers using them well at work.

Watch out: "almost right" is the dangerous kind of wrong. An AI-drafted query that fails with an error is harmless: you notice. One that runs and returns a plausible total, while quietly counting cancelled orders, is the one that reaches the board. Hand-check one row and reconcile to a known total, every time.

7.8 Where you are right now

Wherever you're reading from (a spreadsheet job, an operations role, a college course, or a complete standing start), you're somewhere on this map. Maybe you already answer Question 1 in Excel every week without calling it analysis. Maybe you're the person who automated a painful report and became the office's unofficial automation analyst.

Locate yourself truthfully, and you'll see that the next step is usually closer than it looks. Chapter 8 gives you the tool to plan it: a career tree that shows which skills unlock which roles, what each role's day looks like, how to read a job description, and which parts of this book to read for your goal.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Choosing a target role by title or by what sounds impressive	You're learning deep learning, but the jobs you apply for want SQL and dashboards	Pick a role by the question it answers and what it produces each week (sections 7.1–7.2)
Assuming a job title means the same thing everywhere	Two "data analyst" postings ask for completely different skills	Read the responsibilities and outputs, not the title; Chapter 8 shows how
Thinking data science is "the senior version" of analysis	You feel you must leave analysis to progress	Senior analysts are highly valued; science and engineering are branches, not promotions (section 7.3)
Skipping the analyst core to reach a "higher" branch	Models and pipelines built on questions you can't frame or data you can't check	Build the trunk first: spreadsheets, SQL, business sense
Treating automation as someone else's job	The same report takes you hours every week	Every role automates; start with refreshable reports and scheduled delivery (section 7.4)
Automating a report without owning it	It fails silently when a source file changes, and nobody notices for weeks	Assign an owner, add a check, and write down what to do when it breaks
Jumping to prediction before the basics work	A churn model nobody uses, while "active customer" still has three definitions	Clarify, answer, share, standardize, and supply trusted data before you predict (section 7.6)
Believing one team structure is always best	Endless reorganizations, same complaints	Match the structure to the company's size and pain; know each structure's typical failure (section 7.5)
Assuming AI assistants make core skills unnecessary	You can't tell when a drafted query is wrong	Learn to read and check the work; use assistants to go faster, not to replace understanding
Pasting AI output into a report unchecked	A plausible number that doesn't reconcile to any known total	Hand-check a row and reconcile to a total before anything leaves your hands
Picking RPA because it's quick to demo	A bot that breaks whenever a screen changes	Use an API or a direct integration where one exists; RPA only as a last resort
Waiting to "finish learning" before locating yourself	Months of study with no clear target	Place yourself on the map now; it will change, and that's fine

In the real world: Anita wants to hire a data scientist

Anita Rao came back from an industry conference with a clear idea. "Everyone is doing AI," she told Meera Iyer on Monday morning. "We need a data scientist. Someone who can predict which customers we're about to lose. Can you help me write the job advertisement?"

Meera had been at Riverstone long enough to inherit Imran's Friday file, learn SQL, and become the person everyone asked for numbers. She didn't want to say no to her boss. She also didn't think a data scientist was what Riverstone needed first.

"Before we write it," she said, "can I show you what the job would involve day to day? It'll take fifteen minutes."

She drew the four questions on the whiteboard and asked Anita which one she needed answered. Anita pointed at the second: *what will happen?* Then Meera opened the year's data and ran the query from her customer review. Five customers had gone 60 days without an order. Two of them, Sunrise Caterers and Om Sai Provisions, used to order every five or six weeks and had been silent for months. Nobody in sales had noticed.

"We didn't need a prediction to catch those two," Meera said. "We needed someone to look, every week. A model would have flagged them earlier, maybe. But right now the list isn't even on anyone's desk."

Anita frowned. "So you're saying we don't need a data scientist."

"I'm saying we don't need one yet," Meera said. She walked through what a data scientist would need to be useful: a clear definition of a lost customer, which Riverstone didn't have (sales and finance used different rules); clean history joined across orders, payments, and support complaints, which currently lived in three places, one of them an Excel file; and somebody to act on the scores every week. "Without those, a data scientist would spend their first six months doing my job and a data engineer's job, and they'd probably leave, because that's not the work they were hired for."

Then she listed the manual work she did every week: 40 minutes each morning on the Daily Sales Flash, most of a Friday on the sales pack, and copying overdue invoices from the finance system into emails for reps. "If we hire an analyst who can automate, those hours come back in the first three months. That person sets up the at-risk list as a Monday email to every rep, fixes the definition of 'active customer' with finance, and gets the data into one place. Once that's running and we have a couple of years of clean history, a data scientist has something to work with."

Anita was quiet for a moment. "The conference speaker made it sound like step one."

"For a company with millions of customers and a data platform, it might be," Meera said. "For us, it's step six."

They rewrote the advertisement together. The title became *Data Analyst (Sales Analytics and Automation)*. The responsibilities listed the Monday at-risk list, a sales dashboard, agreed definitions with finance, and automating recurring reports. The skills section asked for SQL, Excel or Google Sheets, a BI tool, and "experience automating a recurring report, with any tool". Under *Nice to have*, Meera added "interest in forecasting and customer analytics", so the new hire could grow toward the role Anita had first imagined.

What made this work.

- **Meera answered the real need, not the requested title.** Anita wanted to stop losing customers. A title was her guess at the solution.
- **She used the map.** Framing the conversation around the four questions and the order of roles from section 7.6 turned an opinion ("I don't think we need that") into a plan.
- **She showed evidence from the data.** Two lapsed regular customers nobody noticed made the case better than any argument.
- **She didn't kill the ambition.** The data-science path stayed in the plan, in the right place.

Tools

This chapter needs no software. For the project you'll need:

- **Pen and paper**, or a free diagram tool such as **diagrams.net** (also called draw.io) or **Excalidraw**, to draw your team map and a request's journey.
- **A few job postings** from any job portal, for the project's stretch goals. Save or print them; postings disappear.

The project: map the data work around you

Goal: a one-page map that shows who does which data work in an organization, how a request travels, and where manual work hides. It's a small portfolio piece, and it's excellent preparation for "tell me about how your team works" in an interview.

Option A: your own workplace or college. Use a real team you know.

Privacy reminder. Don't put colleagues' names, salaries, or confidential systems in anything you share publicly. Use role names ("sales coordinator", "finance team") instead of people's names, and leave out anything your employer wouldn't want outside the building.

Option B: Riverstone Supplies. Use the people and systems you've met in Chapters 1, 2, and this chapter: Anita, Vikram, the sales executives, Meera, the Friday file, and the Daily Sales Flash.

Steps

1. **List the recurring data requests.** Write down at least five reports, lists, or numbers people regularly ask for. For each, note who asks, how often, and in what form it arrives (a file, an email, a dashboard, a phone call).
2. **Tag each request with one of the four questions.** Most will be Question 1. That's normal.
3. **Name the roles doing the work today**, whatever their titles. Where one person covers several roles, say so ("Meera: analyst, BI developer, and automation analyst").
4. **Draw the team structure.** Is it centralized, embedded, hub-and-spoke, or "one person doing everything"? Sketch it in the style of Figure 7.4.

5. **Trace one request from start to finish**, in the style of Figure 7.5: who asks, who clarifies, who pulls data, who checks it, how it's delivered, who acts on it.
6. **Mark every manual step in red**: exports, copy-paste, re-typing, emailed attachments. Estimate the minutes each one takes, and multiply by how often it happens in a year.
7. **Write three sentences**: the biggest risk in this flow, the single automation that would save the most time, and which role you'd hire (or become) next, with one reason.

Stretch goals

- Add where AI assistants are used today (or could be), and what someone checks before the output is used.
- Collect three job postings for the role you picked in step 7 and underline what each would produce in its first three months.
- Redraw the map for the organization as it might look in three years, and write down which structure you'd recommend and why.

You've got it when...

- ☐ You can name the four questions and place any data job title under one of them.
- ☐ You can describe what each of the ten roles produces, in one sentence each, without using its title.
- ☐ You can explain why SQL is useful in every role on the map.
- ☐ You can sketch the tree in Figure 7.2 from memory and explain why science and engineering are peers.
- ☐ You can explain the difference between an automation analyst, an RPA developer, and an integration engineer, and when RPA is the wrong choice.
- ☐ You can give one example of automation inside a role that isn't an automation role.
- ☐ You can compare centralized, embedded, and hub-and-spoke teams, including each one's typical failure.
- ☐ You can follow a business request through the roles in a sensible order, and explain why prediction comes after a clear question and trusted data.
- ☐ You can say, for your target role, what AI assistants speed up and what stays your responsibility.
- ☐ You've placed yourself on the map, and can say which role you're aiming for next.

Recap

- **Orientation** comes before tools: know what each role does before choosing what to learn.
- Every data job answers one of **four questions**: what happened; what will happen, and why; how data moves, gets served, and gets put to work; and how the whole system should be designed.
- The book follows **ten roles**: data analyst, business analyst, BI developer, analytics engineer, data scientist, ML engineer, data engineer, AI engineer, automation analyst / RPA developer /

integration engineer, and data architect. Judge roles by what they produce, not by title.

- **SQL** is the one skill every role on the map uses.
- The field grows like a **tree**: a shared trunk (foundations and the analyst core), a branch point (Part III), two peer branches (science and engineering), and a top where they rejoin (production AI and architecture).
- The **automation and integration track** removes manual steps and connects systems, using spreadsheet automation, low-code flows, **RPA**, APIs, and **reverse ETL**. Every data role includes some automation.
- Data teams are **centralized**, **embedded**, or **hub-and-spoke**; each trades consistency, business context, and speed differently, and each has a typical failure.
- Following one request through every role shows the right order: clarify, answer, share, standardize, supply trusted data, predict, then act, with the architect designing the whole.
- **AI assistants** speed up tasks with a checkable answer; context, judgment, and accountability stay with you. You're responsible for every number you deliver.

Practice exercises

Warm-up

1. For each request, name which of the four questions it mainly answers. (a) "How many orders did we ship late last month?" (b) "How much stock of Storage Box 25L should we make for October?" (c) "Can the website's new orders reach the ERP without anyone re-typing them?" (d) "Should the warehouse, the CRM, and the finance system share one customer ID, and who should own it?" (e) "Did the 5% hospitality discount actually increase orders, or would they have grown anyway?"
2. Which role from section 7.2 is each person closest to, whatever their title? (a) Priya spends her week building tested SQL tables that define revenue, margin, and active customers for every dashboard in the company. (b) Arjun connects the e-commerce platform to the ERP through APIs and makes sure no order is loaded twice. (c) Sana meets department heads, maps how purchase approvals work today, and writes the requirements for a new approval system. (d) Karthik keeps a demand-forecasting model running, monitors its accuracy, and retrains it when it drifts.
3. A company's finance and sales teams each have their own analysts, who report to their department heads. There's no central data team. Which structure is this, and what problem would you expect to hear about in meetings?

Core

4. In the section 7.6 result, Home Plus shows "none" for both its latest order and its days since that order. What do those gaps mean, and why would it be a mistake to leave Home Plus off Anita's list?
5. Check by hand that City Needs Store's `days_since_last_order` of 284 is correct. Its last order was on 22 March 2025, and "today" is 31 December 2025.
6. Anita can make two calls this afternoon. From the five customers in section 7.6, which two should she call, and why? Use the order counts given in the analyst's notes.

7. Meera spends 25 minutes each working day copying overdue invoices from the finance system into emails for sales reps. Assuming 250 working days a year, how many hours is that per year? About how many 40-hour working weeks? Which role's skills would remove this work, and name one way they might do it.
8. Riverstone has grown. It now has two analysts in sales, one in finance, and one in operations, all reporting to their department heads. In the monthly review, sales and finance report different revenue totals for the same month. Recommend a team structure, and describe the first two things the new setup should do.
9. Rewrite this vague request as a precise question a data analyst could answer, in the style of section 7.6, step 1: "Are our hotel customers happy?" List at least three clarifying questions you'd ask first.
10. For each automation, say whether an API integration or an RPA bot is the better choice, and why.
(a) Copying new website orders into the ERP, which has a documented API. (b) Downloading a monthly statement from an old supplier portal that only offers a login page and a "Download" button, with no API.

Stretch

11. Finance says "inactive" should mean *no non-cancelled order in 90 days, including customers who never ordered*. Using only the section 7.6 result, how many customers would be on finance's list, and which one drops off? Which role would fix the problem of two definitions, and how?
12. An AI assistant's list shows four quiet customers, not five. Without looking at any code, give three reasons the list could differ (for example, never-ordered customers dropped, cancelled orders counted as activity, a different "today"), and how you'd check each against the result table.
13. Trace a new request through the roles: "Our deliveries to Chennai keep arriving late. Why, and can we stop it?" Name the roles you'd involve, in order, what each produces, and which roles you'd leave out for now, with reasons.

Think about it (no query needed)

14. Why does a data architect need to have walked at least partway down both the science and the engineering branches? Give one concrete decision from section 7.6, step 10, that would go wrong without that experience.
15. Some people argue that automation shouldn't be a separate track, because every role automates. Others argue it deserves its own roles. Make the best case for each view in two or three sentences, then say which fits a company like Riverstone today.

Key terms

orientation · track · role · data analyst · business analyst · BI developer · analytics engineer · data scientist · ML engineer · data engineer · AI engineer · automation analyst · RPA developer · integration engineer · data architect · dashboard · model · pipeline · BI (business intelligence) · MLOps · LLM (large language model) · SQL · data governance analyst · data steward · data quality analyst · RPA (robotic process automation) · reverse ETL (data activation) · Daily Sales Flash · centralized team · embedded (decentralized) team · hub-and-spoke (federated) team

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 8, The Career Tree: How Skills Unlock Roles**, turns this map into tiers in the book's part order, with a skills matrix per role, a day in the life of each role, how to decode a job description, the reader pathways table, and entry routes for freshers, career switchers, and internal moves.
- **Chapter 9, How Expertise Actually Forms**, is honest about the timeline, and shows how to practice, build a portfolio, find feedback, and get through plateaus.
- **Chapter 12** teaches you to write section 7.6's four-step method as a query yourself, first on the mini database (section 12.15). **Chapter 13** improves the rule itself, comparing each customer's silence with their own usual ordering rhythm (Pattern 6).
- **Chapters 19 and 20** start the automation thread in earnest: spreadsheet automation, and reports delivered by email and on schedule. **Chapter 25** covers the business analyst track, including finding automation opportunities.
- The later steps of section 7.6 each have their own chapter: **Chapter 32** builds tested, shared definitions like the analytics engineer's; **Chapter 56** keeps models running, as the ML engineer does; **Chapters 55 and 58** build AI assistants with the safeguards in step 9.
- **Chapter 51** teaches reverse ETL, and **Chapters 51, 58, and 63** take automation to the engineering, AI, and architect levels; Chapter 63 also covers owning and governing every automation. **Chapter 66** returns to building and structuring data teams.
- **Part VIII, Chapter 68, How Data Hiring Works**, explains the interview rounds for each of these roles. Every role in this chapter has matching question banks in Chapters 70–82.

Answers to practice exercises

1. (a) Question 1, what happened: it describes last month. (b) Question 2, what will happen: it's a forecast. (c) Question 3, how data moves and gets put to work: it's an integration between two systems. (d) Question 4, how the whole system should be designed: shared IDs and ownership across systems are architecture decisions. (e) Question 2, *why*: it asks whether the discount *caused* the increase, which needs an experiment or a careful causal analysis. The common wrong answer is Question 1, because it sounds like "what happened to orders". Describing that orders rose is Question 1; proving the discount caused it is Question 2.

2. (a) Priya is an **analytics engineer**: tested, shared definitions used by every report. (b) Arjun is an **integration engineer**: API connections and duplicate-safe loading. (c) Sana is a **business analyst**: process mapping and requirements. (d) Karthik is an **ML engineer**: keeping a model running, monitored, and retrained. Titles at their companies might be anything; the outputs tell you the role.
3. It's an **embedded** (decentralized) structure. Expect the typical failure from the table in section 7.5: the departments' numbers disagree ("why does finance's revenue differ from sales?"), and similar reports get built more than once.
4. The gaps mean Home Plus has no non-cancelled orders at all, so there's no latest order date and nothing to count days from. They aren't zero, and they aren't a mistake: they mean "no value here". Leaving it off would be a mistake because a customer who signed up and never ordered is the easiest kind to miss and often the easiest to win: someone showed interest and nobody followed up. That's why the business analyst's requirement in step 1 explicitly included never-ordered customers, and why the analyst's method starts from every customer and keeps those with no order at all.
5. From 22 March to 31 March is 9 days. April 30, May 31, June 30, July 31, August 31, September 30, October 31, November 30, December 31. Total: $9 + 30 + 31 + 30 + 31 + 31 + 30 + 31 + 30 + 31 = 284$ days.
✓ The common slip is counting 22 March itself, which gives 285; days *since* the last order start counting the day after it.
6. **Sunrise Caterers and Om Sai Provisions**. Each placed four orders at a regular rhythm (roughly every five to six weeks) and then went silent for 204 and 162 days, so something has probably gone wrong, and they were worth ₹1,14,072.50 together in 2025. City Needs Store and Tasty Tiffins ordered only twice each, so their silence may be their normal pattern. Home Plus is worth a call too, but as a sales lead to convert rather than a customer to win back, so it can go to a sales executive. A reasonable alternative answer puts Home Plus second, if Anita's priority is new business; what matters is giving a reason.
7. $25 \times 250 = 6,250$ minutes per year. $6,250 \div 60 \approx 104.2$ hours, and $104.2 \div 40 \approx 2.6$ working weeks. This is **automation analyst** (or integration engineer) work, and an analyst who automates could do it too. Possible approaches: a query or saved report that lists overdue invoices by rep, delivered as a scheduled email with each rep's list in the email body (Chapter 20); or a low-code flow in Power Automate or n8n that runs every morning (Chapter 20); or, at a later stage, overdue flags pushed straight into the CRM as tasks (Chapter 51).
8. A **hub-and-spoke** structure fits: the analysts stay close to their departments, but a small hub (even one person to begin with) owns shared definitions and the platform. First two things: (1) agree one written definition of revenue with sales and finance (for example, net of discount, excluding cancelled orders, by order date or by invoice date, but one of them), and build it once as a shared table; (2) point both departments' reports at that shared table and reconcile the old numbers so everyone understands why they differed. Moving everyone into a fully centralized team would also fix consistency, but it would lose the business context the embedded analysts already have.
9. Clarifying questions could include: What does "happy" mean to you: ordering more, complaining less, paying on time, or a survey score? Which customers count as "hotel": the Hospitality segment, which also includes caterers? Over what period? Compared with what: last year, or other segments? Who will act on the answer, and what decision is it for? One precise version: **"For Hospitality customers, compare the**

number of orders, revenue, and support complaints per customer in the last six months of 2025 with the six months before, and list customers whose orders fell by more than half." Any precise version is acceptable if it names the group, the measures, the period, and the comparison.

10. (a) API integration. A documented API is designed for systems to exchange data; it's faster, and it doesn't break when a screen's layout changes. **(b) RPA bot** is reasonable here, because there's no other way in. Treat it as fragile: monitor it, and ask the supplier whether a data feed or an emailed statement is available, so the bot can be retired.

11. Finance's rule keeps customers with more than 90 days since their last order, plus never-ordered customers: Home Plus (never), City Needs Store (284), Sunrise Caterers (204), and Om Sai Provisions (162). That's **4 customers. Tasty Tiffins drops off**, at 66 days. An **analytics engineer** would fix the problem of two definitions: agree one definition (or two clearly named ones, such as "quiet 60" for sales follow-up and "inactive 90" for finance), build it once as a tested, documented table, and point every report at it. The wrong fix is to let each team keep its own hidden rule and argue in meetings.

12. Likely reasons: **(1) Never-ordered customers were dropped.** The assistant started from orders instead of from every customer, so a customer with no orders never appears. Check: is Home Plus on its list? **(2) Cancelled orders were counted as activity.** A customer whose only recent order was cancelled would look active. Check: find which of the five is missing, then look for a cancelled order after 1 November for that customer. **(3) A different "today" or a different edge.** Counting from the day the assistant ran instead of 31 December 2025, or keeping "60 days or more" instead of "over 60 days", moves customers near the line. Check: is Tasty Tiffins, the closest to the line at 66 days, the one missing? Ask the assistant which date it counted from and which rule it kept. In every case, reconcile by putting the two lists side by side and investigating the customer that differs. Here, reason (1) is the most likely, because a never-ordered customer like Home Plus is exactly the kind of row that's easy to lose without noticing.

13. A sensible order: **Business analyst** first, to define "late" (against the promised date? by how many days?) and find who needs the answer. **Data analyst** next, to measure how often Chennai deliveries are late, compared with other cities, and by delivery partner, route, product, or day of the week, and to suggest the likely causes. **BI developer**, if it needs watching every week, to add an on-time delivery page to the operations dashboard. **Automation or integration engineer**, if delivery status has to be copied from the delivery partners' systems by hand, to connect those systems. Leave out the **data scientist** and **ML engineer** for now: prediction is premature until the cause is understood and the data is reliable; they might come in later to predict late deliveries before dispatch. The **data architect** isn't needed for one analysis, though they'd care if delivery data turned out to be missing from the platform. Other reasonable orders are fine if each step depends on the one before.

14. Because the architect's decisions sit where the branches meet, and each branch fails in different ways. Example from step 10: *"What happens if the CRM write-back fails halfway?"* Without engineering experience, an architect might not know that retries can create duplicate tasks unless each write can be safely repeated. Another example: *"Where does the risk score officially live?"* Without science experience, they might not realize the score needs to be stored with the model version and date that produced it, or nobody can later explain why a customer was flagged. An architect who has never met these problems tends to design systems that look clean on paper and break in production.

15. *For a separate track:* automation at scale needs specialist knowledge of APIs, integration platforms, RPA, error handling, and governance, and companies with many systems benefit from people who do it full time and own the automations. *Against:* the best automations come from people who understand the work being automated, so every role should automate its own repeated tasks, and a separate team can become a bottleneck. For a company like Riverstone today, most automation should come from analysts who automate their own reports (as in the story), with a specialist integration role added when many systems need connecting. Any answer that weighs both sides and ties the choice to the company's size is acceptable.

Chapter 8. The Career Tree: How Skills Unlock Roles

Chapter at a glance

You will learn to: use the "keys and doors" idea to decide what to learn next · name the seven tiers of the career tree and the roles each one unlocks · read a skills matrix to see what each role really needs · picture a normal working day in each of the ten roles · decode a job description line by line and score your own fit · read salary figures carefully, and check them yourself · choose a reading pathway through this book for your goal · plan an entry route as a fresher, a career switcher, or through an internal move.

Before you start: Chapter 7 (the four questions, the ten roles, and how teams are organized).

Time needed: 3–4 hours, including the exercises. Allow another 2–3 hours for the project, which uses real job postings.

Tools: a spreadsheet (Excel or Google Sheets) or paper, and a job portal for the project. Nothing to install.

Practice data: a fictional job description from Riverstone Supplies, the skills matrix in section 8.3, and salary figures published by PayScale and Indeed for India, as retrieved in September 2026. Every calculation shown is checked.

Why this matters

"What should I learn next?" is the most common question in every data community, and it's usually asked backwards. People ask about tools in isolation: *Should I learn Spark? Is Tableau worth it? Do I need deep learning?* The useful question is about **doors**: *which job would this skill let me apply for that I can't apply for today?*

Asked backwards, the question leads to months of learning things no employer will ask you about yet, while the three skills that would get you shortlisted stay half-learned. Asked the right way round, it turns an endless list of topics into a short, ordered plan with a job at the end of each step.

This chapter gives you that plan. It arranges the ten roles from Chapter 7 into a tree of tiers, shows exactly which skills each role needs, lets you see what each job looks like on an ordinary Tuesday, and teaches you to read a job description the way a hiring manager wrote it. By the end, you'll be able to name your next door, and the chapters of this book that open it.

In plain English

Picture an office building where every room is a job. Each room has a locked door, and each lock needs a particular set of keys.

- Each **key** is a skill: SQL, a spreadsheet, a BI tool, Python, statistics, the ability to write a clear requirement.
- Each **door** is a role you could be hired into. When you hold the keys a door needs, you become a credible applicant: someone a hiring manager would take seriously.
- The building has **floors**. The ground floor's keys open the first rooms. Higher floors need the lower keys *and* some new ones. Nobody takes the lift straight to the top.
- On one floor, two corridors branch off in different directions, one toward science and one toward engineering. They're on the same floor: neither is "above" the other.
- You **keep your keys** as you climb. A data scientist still uses SQL every week.
- A **job description** is the notice on a door, listing the keys it needs. Some are required; some are "nice to have".

The building is the **career tree**. The floors are **tiers**. The rest of this chapter is its floor plan.

8.1 Skills are keys; roles are doors

One idea sits at the center of this chapter: **learn skills for the doors they open, not because they're popular**.

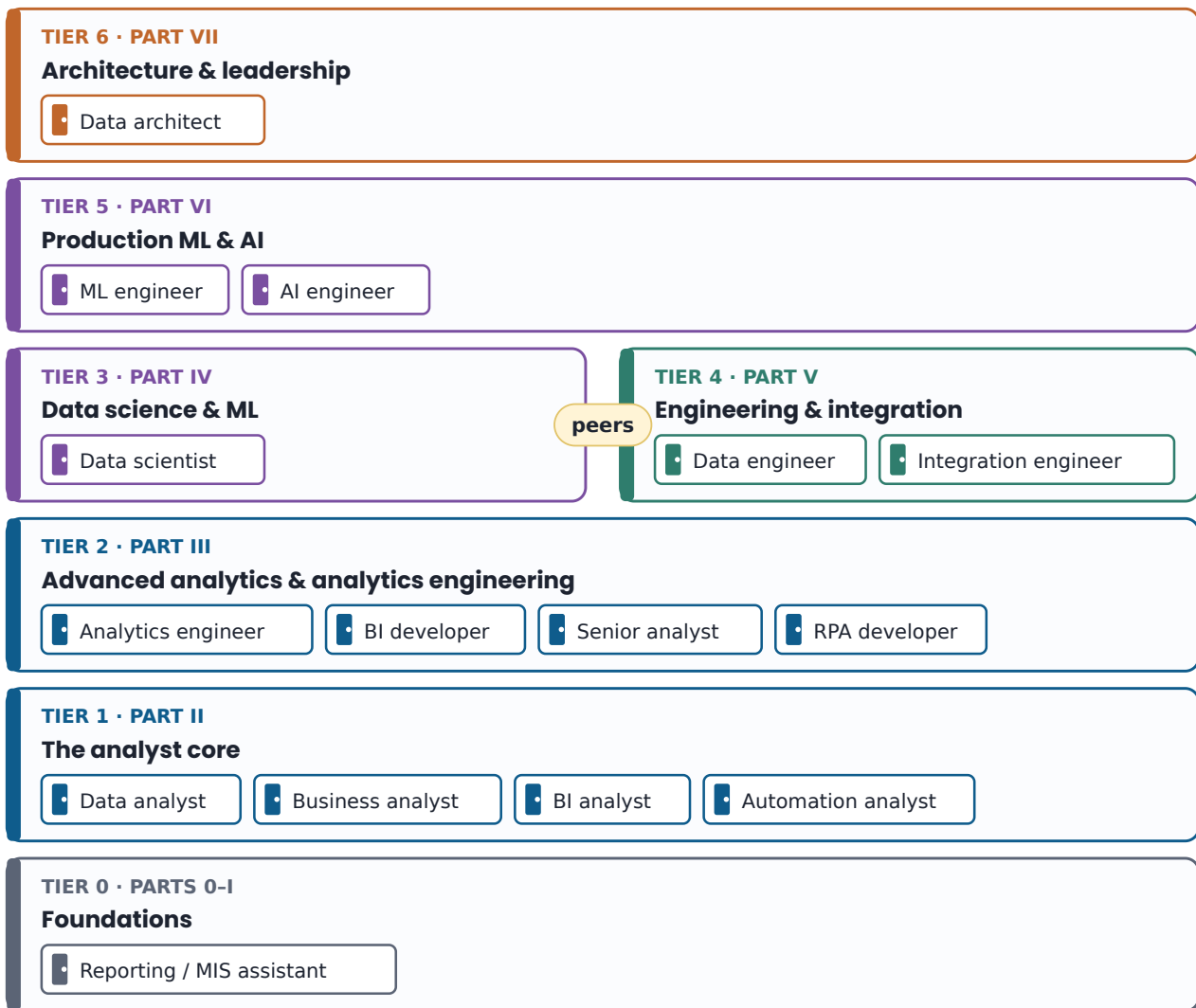
That idea changes how you study in three ways.

- **It gives every topic a reason.** You don't learn "data science" in the abstract. You learn the set of skills that makes you a credible applicant for *data scientist*, and each skill is a key you can see yourself adding.
- **It tells you when to stop.** When you hold the keys a door needs, you're ready to apply. You'll keep learning on the job, but you don't need to finish the whole book first.
- **It protects you from fashion.** A tool that's trending this year might open no door you want. A skill that appears in every job description for your target role is worth more than ten trending ones.

A **credible applicant** is someone who meets a role's must-have skills well enough to pass a screening and hold their own in a technical interview. It doesn't mean knowing everything, and it doesn't mean experience alone. Part VIII (from Chapter 68) shows what each interview round checks.

8.2 The tiers, and what each unlocks

The career tree has seven tiers, numbered from 0. They match the parts of this book, in order, so the book is the tree.



Read from the bottom up. Each role sits at the first tier that makes you a credible applicant; you keep every skill below it.

Figure 8.1 – The career tree as tiers and doors. Tiers 3 and 4 sit side by side: they're peers, and you can climb either one first. Names in the tiers that aren't among Chapter 7's ten roles (reporting/MIS assistant, BI analyst, senior analyst) are common entry titles for those roles.

Tier 0: Foundations (Parts 0 and I). What data is, how computers store it, how a business runs on it, numbers without fear, thinking like an analyst, and the map you're reading now. *Unlocks:* reporting assistant and MIS (management information system) roles, where you prepare regular reports mostly by hand. Few people stay here long, and the skills matter mainly because everything above rests on them.

Tier 1: The analyst core (Part II). Excel and Google Sheets in depth, SQL, cleaning data, visualization, Power BI, Python for analysis, spreadsheet and report automation, statistics, business metrics, working with stakeholders, and the professional toolkit. *Unlocks:* **data analyst**, **business analyst**, **BI analyst** (an analyst who mainly builds reports and dashboards), **automation analyst**, and the domain versions: sales, finance, marketing, operations, supply chain, and HR analyst. This is the single biggest unlock in the field. If you master only one tier, make it this one.

Tier 2: Advanced analytics and analytics engineering (Part III). Advanced SQL and data modeling, Python written as maintainable software, experiments and causal inference, dbt, the computer science you need, and the command line. *Unlocks:* **senior analyst, analytics engineer, BI developer** (who designs the data models and semantic layers behind dashboards, not only the visuals), and **RPA developer**. Tier 2 is the branch point: the skills here feed both branches above it.

Tier 3: Data science and machine learning (Part IV). The math under the models, the machine learning workflow, supervised and unsupervised learning, honest evaluation, forecasting, text, recommendations, and a first look at deep learning. *Unlocks:* **data scientist**, and specialist roles such as forecasting analyst or applied scientist.

Tier 4: Engineering and integration (Part V). Ingestion, pipelines and orchestration, data quality, distributed compute, warehouses and lakehouses, streaming, data activation and APIs, and the cloud. *Unlocks:* **data engineer, integration engineer**, and platform roles.

Tiers 3 and 4 are peers. They sit at the same height on the tree. The book teaches science first because many analysts move toward it first, but plenty of excellent careers go from tier 2 straight into engineering. Tier numbers match the book's part numbers, so the figure shows the two branches side by side.

Tier 5: Production ML and AI (Part VI). Deep learning in depth, generative AI and large language models, building AI applications, MLOps, LLMOps, intelligent automation, and industry cases. *Unlocks:* **ML engineer, AI engineer**, and MLOps and ML platform roles. Most people arrive here from tier 3 (with engineering added) or from tier 4 (with machine learning added).

Tier 6: Architecture and leadership (Part VII). Designing whole systems, trade-offs, architecture patterns, automation architecture and governance, security and privacy, the economics of data platforms, data strategy, and leading as an architect. *Unlocks:* **data architect**, ML or AI architect, and the management ladder up to head of data or chief data officer.

Where the automation roles sit

The automation and integration track from Chapter 7 doesn't fit neatly on one floor, because automation grows with every tier. Its roles appear at three levels:

Role	Tier	Why there
Automation analyst	1	Spreadsheet automation, scheduled reports, and low-code flows are analyst-core skills (Chapters 19–20)
RPA developer	2	Needs maintainable, tested automation and command-line habits (Part III), plus RPA tools
Integration engineer	4	Needs APIs, pipelines, data quality, and reverse ETL (Chapters 45–51)

How the tree works: three rules

Three rules make the tree work, and the rest of the chapter depends on them.

1. **You keep every skill below your tier.** Climbing doesn't mean leaving skills behind. They accumulate.

2. **A role sits at the first tier that makes you credible for it.** You don't need tier 6 to apply for a tier 1 job. Learn up to the door you want, apply, and keep climbing on the job.
3. **Pick your door, then learn backward.** Choose the role you want next, find its tier, and learn the keys that door needs. Ignore everything higher for now. It will still be there when you arrive.

Simplification note. Real hiring is messier than a tree. A small company might hire a "data scientist" who mostly does tier 1 work, and a large bank might require tier 2 skills for an analyst post. The tiers describe where a role's *must-have* skills are taught, which is what you need for planning. Section 8.5 shows how to read what a particular employer really wants.

8.3 The skills matrix: what each role really needs

The tiers tell you roughly where a role sits. The skills matrix tells you exactly which keys each door needs. It lists 17 skills against the ten roles, and marks each skill as **core** (needed to be hired), **useful** (helps, often listed as a plus), or **not needed** to get in.

Skill	DA	BA	BI	AE	DS	MLE	DE	AIE	AUT	ARC
Business and domain sense	●	●	○	○	●	○	○	○	●	●
Communication and storytelling	●	●	○	○	●	○	○	○	○	●
Spreadsheets (Excel, Sheets)	●	●	○	-	○	-	-	-	●	○
SQL	●	○	●	●	●	○	●	○	○	●
BI and dashboards	●	○	●	○	○	-	-	-	○	○
Statistics	○	-	-	-	●	○	-	○	-	-
Python	○	-	-	○	●	●	●	●	●	○
Spreadsheet and report automation	○	○	○	-	-	-	-	-	●	-
Requirements and process mapping	○	●	○	○	-	-	-	○	●	●
Data modeling (incl. dbt)	-	-	●	●	-	-	●	-	-	●
Machine learning	-	-	-	-	●	●	-	○	-	○
Pipelines and orchestration	-	-	-	○	-	●	●	○	○	●
Cloud and infrastructure	-	-	-	○	○	●	●	●	○	●
APIs and integration	-	-	-	-	-	○	●	●	●	●
LLMs and AI applications	-	-	-	-	○	○	-	●	○	○
Git and version control	-	-	○	●	○	●	●	●	○	○
System design and governance	-	-	-	○	-	○	○	○	○	●

Key: ● core · ○ useful · - not needed to get in. DA data analyst · BA business analyst · BI BI developer · AE analytics engineer · DS data scientist · MLE ML engineer · DE data engineer · AIE AI engineer · AUT automation analyst, RPA developer, or integration engineer · ARC data architect.

	DA	BA	BI	AE	DS	MLE	DE	AIE	AUT	ARC
Business and domain sense	●	●	○	○	●	○	○	○	●	●
Communication and storytelling	●	●	○	○	●	○	○	○	○	●
Spreadsheets (Excel, Sheets)	●	●	○	—	○	—	—	—	●	○
SQL	●	○	●	●	●	○	●	○	○	●
BI and dashboards	●	○	●	○	○	—	—	—	○	○
Statistics	○	—	—	—	●	○	—	○	—	—
Python	○	—	—	○	●	●	●	●	●	○
Spreadsheet and report automation	○	○	○	—	—	—	—	—	●	—
Requirements and process mapping	○	●	○	○	—	—	—	○	●	●
Data modeling (incl. dbt)	—	—	●	●	—	—	●	—	—	●
Machine learning	—	—	—	—	●	●	—	○	—	○
Pipelines and orchestration	—	—	—	○	—	●	●	○	○	●
Cloud and infrastructure	—	—	—	○	○	●	●	●	○	●
APIs and integration	—	—	—	—	—	○	●	●	●	●
LLMs and AI applications	—	—	—	—	○	○	—	●	○	○
Git and version control	—	—	○	●	○	●	●	●	○	○
System design and governance	—	—	—	○	—	○	○	○	○	●

● core: needed to be hired
○ useful: helps, often a plus
— not needed to get in

DA data analyst · BA business analyst · BI BI developer · AE analytics engineer · DS data scientist
MLE ML engineer · DE data engineer · AIE AI engineer · AUT automation / integration · ARC data architect

Figure 8.2 — The same matrix as a heatmap. Read down a column to see a role; read across a row to see which doors a skill helps open.

Reading the matrix

Read down a column to see what a role needs. The data analyst column has five core skills: business sense, communication, spreadsheets, SQL, and BI and dashboards. That's the tier 1 core. The data engineer column has seven: SQL, Python, data modeling, pipelines, cloud, APIs and integration, and Git. The data architect column is the darkest of all, with nine core skills, because the architect's job is to understand how every part fits.

Read across a row to see which doors a skill helps open. Three rows are worth noticing.

- **SQL** is marked core or useful for all ten roles. This is the promise from Chapter 7 in matrix form.
- **Machine learning** is core for only two roles, data scientist and ML engineer. If neither is your target, it can wait.
- **Python** is core for five roles (data scientist, ML engineer, data engineer, AI engineer, and the automation and integration roles) and useful for three more. It's the most important skill to add after the analyst core.

Compare two columns to see how far apart two roles are. Data analyst and BI developer share seven skills that each role needs at least a little of; data analyst and data engineer share only four (business sense, communication, SQL, and Python). That's why analyst-to-BI-developer is a short move and analyst-to-data-engineer takes longer.

Watch out: "useful" is not "optional forever." A useful skill often decides between two otherwise equal candidates, and it's usually the next key you'll need on the job. Treat the useful cells in your target column as your second list, after the core ones.

Try it. Pick the role you're most curious about. Count its core skills, and tick the ones you could use at work today. The unticked ones are your next keys.

8.4 A day in the life of each role

A matrix tells you what a job needs. It doesn't tell you what the job *feels* like. Here's an ordinary working day in each role. The people and companies are fictional; the shape of each day is typical, though every company differs.

Data analyst

Pooja works in the sales team of a consumer goods distributor. At 9:30 she checks that the morning's sales report refreshed and that yesterday's total matches the billing system. At 10:00 the regional manager asks why the West region missed its target; she spends two hours in SQL and a spreadsheet, finds that one large wholesaler delayed an order to next month, and writes a four-line answer. After lunch she builds a comparison of two discount schemes for Friday's review. At 4:30 a colleague reports a strange number on the dashboard, and she traces it to a customer entered twice. Most of her day is questions, SQL, checking, and explaining.

Business analyst

Aditya works for a hospital chain that is replacing its appointment system. His morning is a workshop with front-desk staff, mapping how appointments are booked, changed, and cancelled today, including the paper register nobody mentioned in the first meeting. He turns the notes into a process map and eight user stories, each with acceptance criteria. After lunch he pulls last quarter's cancellation data with a short SQL query to show that most cancellations happen by phone, which changes the design. The day ends with a call with the software vendor to agree what "reschedule" means. Most of his day is conversation, documents, and small, sharp pieces of analysis.

BI developer

Lakshmi works in a bank's reporting team. She starts by checking the overnight dataset refreshes in Power BI; one failed because a source table was renamed, so she fixes the query. Her main task is a new branch-performance dashboard: she designs the data model, writes measures, and makes sure the numbers match the finance team's official figures to the rupee. In the afternoon she reviews a colleague's report for slow visuals and confusing labels. Before leaving, she sets row-level security so branch managers see only their own branch. Her day is modeling, measures, performance, and access.

Analytics engineer

Rohan works at an online retailer. His morning starts with a failing data test: an "orders" model suddenly has duplicate order IDs. He finds that a source system began sending corrections as new rows, fixes the model, and adds a test so it can't happen silently again. Then he meets the marketing and finance analysts to agree a single definition of "new customer". He writes the change in dbt, documents it, and opens a pull request that a colleague reviews. In the afternoon he helps an analyst turn a 300-line query into three reusable models. His day is SQL, tests, Git, and agreeing definitions.

Data scientist

Meenal works for an insurance company. Her project is predicting which policyholders are likely to not renew. In the morning she checks whether a promising feature is really *leakage*: information the model wouldn't have at prediction time. It is, so she removes it, and the model's accuracy drops to a more believable level. After lunch she presents results to the retention team, explaining in plain language what the model can and can't do, and proposes a test on one region before any wider rollout. She ends the day writing up the experiment design. Her day is Python, statistics, careful doubt, and explanation.

ML engineer

Farhan works at a logistics company that uses a model to estimate delivery times. His morning alert says the model's errors have grown over the past week; he finds that a new warehouse opened and the model has never seen its routes. He schedules retraining with the new data and adds a monitoring check per warehouse. In the afternoon he packages a colleague's new model as a service, writes deployment tests, and sets up a way to roll back if it misbehaves. His day is code, monitoring, deployment, and reliability.

Data engineer

Sneha works for a fintech company. At 8:00 she checks the overnight pipelines; one loaded zero rows from the payments system because an API credential expired, so she renews it and reruns the load before analysts arrive. She spends the morning building a new pipeline that brings support tickets into the warehouse, with checks for missing fields and duplicates. After lunch she meets the payments team to agree a *data contract*: which fields they'll send, in what format, and how they'll warn her before changing it. Her day is pipelines, failures, quality checks, and agreements with system owners.

AI engineer

Karan works for a company that sells industrial parts. He's building an assistant that answers sales reps' questions from product manuals and price lists. In the morning he runs the assistant against a set of 150 test questions with known answers, finds that it invents a warranty period for one product line, and changes how documents are retrieved so the right manual is used. After lunch he adds a rule that any price in an answer must come from the price list, with a link. He ends the day reviewing how often reps click "not helpful". His day is Python, retrieval, evaluation, and guardrails.

Automation analyst or integration engineer

Nisha works in the operations team of a manufacturer. Her morning goes on a report that three planners rebuild by hand every Monday: she replaces it with a Power Query refresh and a scheduled flow that emails each planner their section in the body of an email. After lunch she investigates why yesterday's purchase orders didn't reach the supplier portal; an API call failed and nobody was alerted, so she adds retries and a failure email to the team inbox. She finishes by updating the automation inventory: what runs, when, who owns it. Her day is removing manual steps and making sure automations fail loudly, not silently.

Data architect

Vivek works for a retail group with six brands. His morning is a design review: a team wants to add a new tool for customer data, and he asks how it will match customer IDs with the existing warehouse, who will own the matching rules, and what it will cost in two years. After lunch he writes an *architecture decision record*: a one-page note of a decision, the options considered, and why one was chosen. He spends an hour with the security team on who may access sales data by region, and ends the day sketching next year's platform roadmap for the head of data. His day is questions, trade-offs, documents, and influence without direct authority.

Real-life example: the title you'll actually see. Many of these people's job titles don't match the role names above. Pooja's title might be "MIS executive", Nisha's "process excellence analyst", Lakshmi's "senior reporting analyst". Search job portals by the work ("Power BI", "dbt", "Power Automate") as well as by title.

8.5 Decoding a job description

A **job description** (JD) is written by a hiring manager, often edited by HR, and read by hundreds of applicants and, first, by software that filters CVs. Reading one well means separating what the manager needs from what got copied in from last year's posting.

Here is the full posting that Anita Rao and Meera Iyer wrote in Chapter 7's story. It's fictional, but it's typical of a real posting for a tier 1 role.

Data Analyst (Sales Analytics and Automation) Riverstone Supplies · head office, hybrid · 1–3 years' experience (freshers with strong projects considered)

About the role. You'll be Riverstone's first dedicated data analyst, working with the sales team and reporting to the Sales Head. You'll turn our sales and customer data into lists, dashboards, and reports that the team uses every week, and automate the reports we currently build by hand.

What you'll do

- Build and run a weekly list of at-risk customers for the sales reps.
- Build and maintain the sales dashboard in Power BI.
- Agree definitions of key numbers, such as "active customer" and net revenue, with the finance team.
- Automate recurring reports, including the Daily Sales Flash.
- Answer ad-hoc questions from sales leadership, clearly and on time.

Must have

- SQL: joins and aggregation; window functions a plus.
- Excel or Google Sheets: lookups and pivot tables.
- A BI tool, Power BI preferred.
- Experience automating a recurring report, with any tool.
- Clear written communication in English.

Nice to have

- Python (pandas).
- VBA or Google Apps Script.
- Forecasting or customer analytics.
- Experience with a CRM or ERP.

Reading it line by line

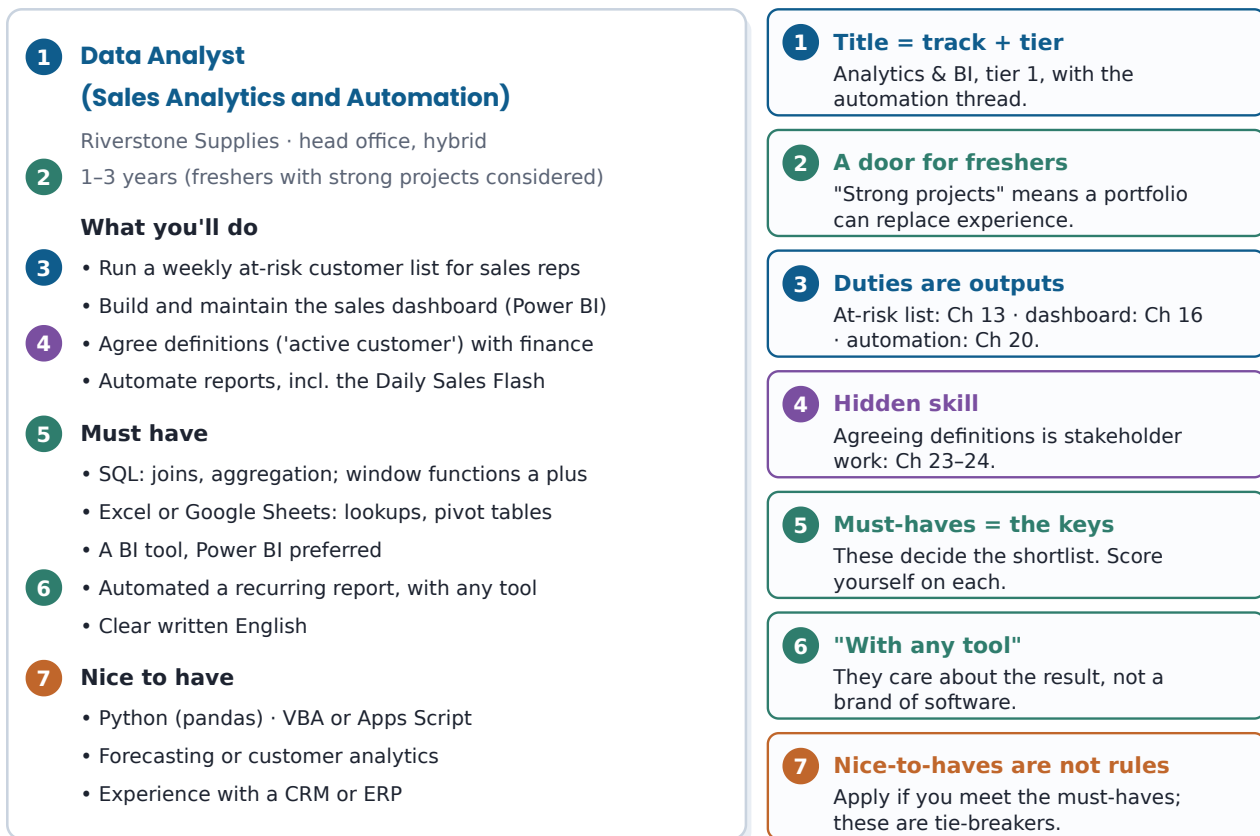


Figure 8.3 – Decoding a job description. Every line tells you something about the job, and a few lines tell you what the manager is worried about.

Work through a JD in this order.

1. **Title: find the track and the tier.** "Data Analyst" puts it on the analytics and BI track at tier 1. "Sales Analytics and Automation" tells you the domain (sales) and a thread (automation). The words in brackets often say more than the title.
2. **Experience: read the exception.** "1–3 years" is a guide; "freshers with strong projects considered" is a door. The manager is saying a portfolio can stand in for experience. Chapter 9 and Chapter 27 show how to build one.
3. **Duties: translate each into an output.** An at-risk customer list is Chapter 13's Pattern 6. A Power BI dashboard is Chapter 16. Automating the Daily Sales Flash is Chapters 19–20. When every duty maps to something you've built, you can talk about it in the interview.
4. **Find the hidden skill.** "Agree definitions with the finance team" isn't a tool. It's stakeholder work: negotiating a shared meaning across departments (Chapters 23–24). Hiring managers put lines like this in because they've felt the pain.
5. **Must-haves: these are the keys.** They decide the shortlist. Be strict with yourself here.
6. **Watch the qualifiers.** "Window functions a plus" means basic SQL is required and advanced SQL is a bonus. "Power BI preferred" means another BI tool is acceptable. "With any tool" means they care about the result: a recorded macro, an Apps Script trigger, or a Python script all count.

7. **Nice-to-haves: tie-breakers, not rules.** If you meet the must-haves, apply. Nobody expects every nice-to-have.

Scoring your fit

A quick, honest way to judge whether to apply is to score yourself. For each skill in the JD, give yourself **0** (can't do it yet), **1** (have done it with help, or in a course), or **2** (have done it on real or realistic data without help). Must-haves count double, because they decide the shortlist.

Take Farah Khan, a sales executive at Riverstone who is interested in this role (you'll meet her properly in "In the real world").

Skill	Type	Farah's score	Weight	Points
SQL	Must	1	2	2
Excel or Google Sheets	Must	2	2	4
A BI tool	Must	0	2	0
Automated a recurring report	Must	0	2	0
Written communication	Must	2	2	4
Python	Nice	0	1	0
VBA or Apps Script	Nice	1	1	1
Forecasting or customer analytics	Nice	0	1	0
CRM or ERP experience	Nice	2	1	2
Total				13

The maximum is 28: five must-haves at 2 points $\times$ 2 weight = 20, plus four nice-to-haves at 2 points $\times$ 1 weight = 8. Farah scores 13 of 28, or 46.4%. More useful than the total is her **must-have coverage**: her raw scores on the must-haves add up to $1 + 2 + 0 + 0 + 2 = 5$ out of a possible 10, which is 50%.

Reading it. Farah isn't ready to apply today, but she's much closer than she feels. Her gaps are specific: SQL to working level (Chapters 12–13), a BI tool (Chapter 16), and one automated report she can show (Chapters 19–20). Her strengths, spreadsheets, communication, and CRM experience, are exactly the ones that are hard to teach.

Simplification note. This scoring is a planning tool, not a formula employers use. Its value is that it forces you to be specific about each gap, and to see which gaps matter most.

Red flags in job descriptions

Some postings tell you more about the company than the job. Watch for these patterns.

- **A tool list longer than the duties.** Fifteen tools, from Excel to Kafka to TensorFlow, for a fresher role usually means nobody decided what the job is.
- **A senior title with junior pay or junior duties.** "Data scientist" whose duties are data entry and weekly MIS reports.

- **Every tier at once.** "Build dashboards, pipelines, ML models, and the cloud platform" for one person is four jobs.
- **No outputs at all.** If you can't tell what you'd produce in your first three months, ask in the interview.
- **Unpaid "test projects" that look like real work.** A reasonable take-home task is short and clearly an exercise; Chapter 82 shows what good ones look like.

None of these is always a reason to walk away. A small company with a messy JD might be a great place to learn. But they're questions to ask before you accept.

Interview extra point. Bring your decoded JD to the interview. When asked "Why this role?", name two duties from the posting, say what you've built that matches each, and ask one question about the hidden skill ("How are definitions like 'active customer' agreed today?"). Chapter 68 explains why this lands well with hiring managers.

8.6 What the roles pay, and how to read salary figures

Pay matters, and you deserve real numbers. You also deserve an honest warning: **salary figures online are rough, vary widely by source, and go stale within a year.** Treat everything in this section as a way to read salary data, with examples, not as a promise of what you'll earn.

Two kinds of salary source

Salary websites build their figures in two main ways.

- **Self-reported salaries.** People enter their own pay, title, and experience. PayScale works this way. The figures reflect real pay, but only from people who chose to report it.
- **Job postings.** The site collects the salaries advertised in job ads. Indeed's salary pages work this way. The figures reflect what employers advertise, which may differ from what people accept.

Because they measure different things, the two can disagree for the same role, and neither is wrong.

One role, as published in 2026

Here is one role, data analyst, as a worked example of reading a salary page. The figures are from PayScale's India page for the title. **Base salary** is fixed yearly pay before bonuses; **total pay** adds bonuses and similar extras.

Role (PayScale title)	Salary profiles	Average total pay, under 1 year	Average total pay, 1-4 years	Base salary, 10th-90th percentile	Average base salary
Data Analyst	2,389	₹4,13,462	₹5,65,999	about ₹2,89,000 to ₹10,00,000	₹5,77,472

As retrieved September 2026. Source: PayScale India, "Data Analyst Salary in India", page updated 10 July 2026. PayScale rounds its percentile figures (it shows "₹289k" and "₹1m"), so that column is approximate. Its chart also marks the average base salary, rounded to ₹577k, as the median. Figures change often;

check the live page.

In Indian terms, a data analyst's average total pay in the first year works out to about ₹4.1 lakh, and the average base salary to about ₹5.8 lakh.

A **percentile** tells you where a value sits in a sorted list. The 10th percentile is the value below which 10% of reported salaries fall; the 90th percentile is the value below which 90% fall. So "about ₹2,89,000 to ₹10,00,000" means the middle 80% of data analyst profiles reported base pay in that range. The **median** is the middle value: half earn less, half earn more. An **average** (the mean) adds every salary and divides by how many there are, so a few very high salaries can pull it up; that's why salary pages often show both.

Reading the table carefully

Sources disagree. For data analysts, Indeed's India page (based on 605 salaries from job postings, updated 20 September 2026) showed an average of ₹6,29,019 a year. PayScale's average base salary for the same title was ₹5,77,472. The difference is ₹51,547, with Indeed about 8.9% higher. That's not an error: one counts advertised pay across all experience levels, the other counts reported base salaries.

Some roles don't have reliable figures under their own title yet. Analytics engineer, AI engineer, and integration engineer are newer titles, and many people doing that work are listed under older ones (data engineer, software engineer, BI developer). Business analyst figures mix IT business analysts, finance business analysts, and more. For these, look at live job postings for the work you'd do, not only the title, and compare several sources.

Chapter 68 compares pay across roles, and how pay grows with experience, when you're preparing for the job search.

CTC and in-hand pay

Indian offers are usually stated as **CTC** (cost to company): everything the employer spends on you in a year. CTC often includes the employer's provident fund contribution, variable or performance pay that isn't guaranteed, and sometimes insurance, gratuity, or one-time bonuses. Your **in-hand** (take-home) pay is what reaches your bank account each month after income tax, your own provident fund contribution, and other deductions. In-hand pay is always lower than $CTC \div 12$, and how much lower depends on how the offer is structured and on tax rules at the time. Always ask for the breakup, and ask which parts are fixed.

Watch out: what moves pay. City, company type, industry, and your specific skills can move pay more than the job title does. Two "data analyst" offers in the same month can differ widely. Compare offers on fixed pay, the work you'll do, and what you'll learn, not on the headline CTC alone. This is general information, not financial advice.

How to check salaries yourself

1. Look up the role on at least two sources that use different methods (for example PayScale and Indeed), and note the date each page was updated.
2. Filter by city and experience where the site allows it, and write down the number of salaries behind each figure. A figure built from 20 salaries means much less than one built from 2,000.
3. Read five live job postings for the role in your city and note any advertised ranges.
4. Ask people in the role, respectfully, what range is typical. Most people will share a range, not their own salary.
5. Treat the result as a range, and update it every six months.

8.7 Reader pathways: your route through this book

Not everyone climbs to the top, and you shouldn't read every chapter with the same care. This table suggests a route for each goal. "Read fully" means work through the chapters, exercises, and projects. "Skim" means read to understand the ideas and vocabulary. "Interview chapters" lists the Part VIII chapters to prepare with.

Goal	Read fully	Skim	Interview chapters
Complete beginner, exploring	Parts 0, I	Part II (first half)	—
Data analyst	Parts 0, I, II (all)	Part III (Ch 28, 30)	68, 69, 70, 71, 72, 73, 75, 78, 81, 82
Business analyst	Parts 0, I; Ch 10–16, 19–27	Ch 17–18	68, 69, 70, 71, 75, 76, 78, 81
BI developer	Parts 0, I, II; Ch 28, 32	Ch 45–49, 51, 63	68, 69, 70, 71, 77, 78, 81
Analytics engineer	Parts 0–III	Ch 45–49, 51	68, 69, 71, 72, 77, 81
Automation / integration engineer	Parts 0, I; Ch 10–20, 25, 29, 34, 45–47, 51, 58, 63	Ch 52, 55	68, 69, 70, 71, 72, 76, 77, 78, 81, 82
Data scientist	Parts 0–IV	Part V; Ch 53–56, 58	68, 69, 71–75, 79, 81, 82
Data engineer	Parts 0–III, V	Part IV (Ch 35–39); Ch 56, 63	68, 69, 71, 72, 77, 78, 81, 82
ML / AI engineer	Parts 0–VI	Part VII	68, 69, 71, 72, 74, 77, 78, 79, 81, 82
Data / ML architect	Everything	—	68, 69, 77, 78, 79, 80, 81
Already an analyst	Skim Parts 0–II; start fully at Part III	—	Per target role

Two things are worth noticing. First, **every route reads Parts 0 and I fully**, including this chapter, and every route except the beginner's goes through the analyst core. Second, the **automation and integration route** is the most spread out: it takes the spreadsheet and report automation chapters from Part II, the software habits from Part III, the integration chapters from Part V, and the automation chapters from Parts VI and VII.

The automation thread, tier by tier

Whichever route you take, automation grows with you. At each tier, the same business need, getting the right data to the right people and systems without manual work, is met with better tools.

Tier	What automation looks like at this level	Taught in
Foundations	Seeing the manual steps hidden in a business process	Ch 3, 5
Analyst core	Refreshable spreadsheets; recorded macros, VBA, Office Scripts, and Google Apps Script; scheduled scripts; reports delivered by email, including in the email body; dashboard subscriptions and alerts; low-code flows	Ch 11, 16, 18, 19, 20
Business analyst	Mapping processes, finding and prioritizing automation opportunities, writing automation requirements	Ch 25
Advanced analytics and analytics engineering	Tested, version-controlled, maintainable automations; scheduled dbt jobs; command-line scheduling	Ch 29, 32, 34
Engineering and integration	Orchestrated pipelines, quality gates before delivery, pushing data back into business systems (reverse ETL, APIs, webhooks), integration platforms, RPA where there's no API	Ch 45–47, 51
Production ML and AI	Models and AI agents inside business workflows, with human approval where it matters	Ch 56, 57, 58
Architecture	Designing the whole source-to-action flow as one system, choosing the right tool for each job, and governing every automation	Ch 60, 63
Interviews	Spreadsheet automation questions; automation, integration, and report-delivery questions and design cases	Ch 70, 78

8.8 Entry routes: fresher, career switcher, internal move

Most people enter the tree at tier 1. How they get there depends on where they start.

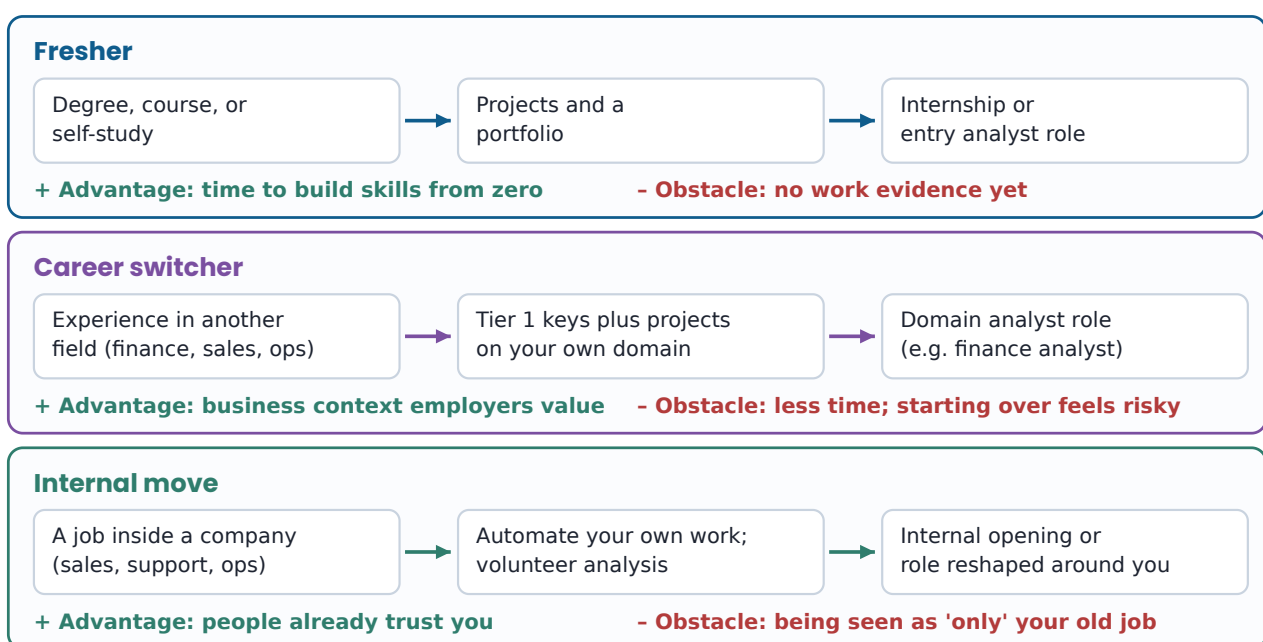


Figure 8.4 – Three entry routes. They end at the same door; what differs is the evidence you bring.

The fresher

A **fresher** is someone applying for their first job, usually straight from a degree or course. Your challenge is evidence: the employer can't see work you've done, so you have to show it.

- **Build the tier 1 keys properly**, not a little of everything. SQL, a spreadsheet, a BI tool, and one automated report beat a certificate in ten tools.
- **Make projects that look like work**. Use realistic data (the Riverstone datasets, public government data, or a business you know), answer a real question, and write up what you'd tell a manager. The projects at the end of every chapter in Part II are designed for this, and Chapter 27 turns them into a portfolio.
- **Take internships seriously**, including short or unpaid-but-fair ones at small companies, where you often get real data and real responsibility.
- **Apply to the doors with exceptions**. Postings that say "freshers with strong projects considered" are written for you.

The career switcher

A **career switcher** moves into data from another field: accounts, sales, operations, teaching, engineering, healthcare. Your challenge is time, and the feeling of starting over. You aren't starting over.

- **Your domain is your advantage**. An accountant who learns SQL and Power BI is a strong candidate for *finance analyst*, because they already understand what the numbers mean. Aim first for a data role in your own domain.
- **Build projects on your own domain's problems** (with made-up or public data, never your employer's confidential data). A former logistics coordinator's delivery-delay analysis tells a hiring manager more than a generic movie-ratings project.
- **Rewrite your experience as data work**. "Prepared monthly MIS reports for 12 branches" is analysis experience. Say what you produced and what decision it supported.
- **Plan in months, not weeks**. Chapter 9 is honest about how long this takes alongside a job.

The internal move

An **internal move** is changing into a data role inside the company where you already work. For many people this is the fastest route, and it's how many data teams in mid-sized companies begin.

- **Automate your own work first**. The report you rebuild every week is your first project, and the hours you save are visible to your manager.
- **Volunteer for analysis nobody owns**, such as a question your manager keeps asking, a list that's always out of date, or a number two teams disagree on.
- **Tell your manager what you're aiming for**. Managers can't support a move they don't know about, and they often know about openings first.
- **Expect the "old job" problem**. People may keep seeing you as "the person from sales". Evidence helps: a dashboard people use, an automation that saved hours, a problem you solved with data.

Real-life example: Meera's route. Meera Iyer (Chapter 1) joined Riverstone as a sales coordinator, not as an analyst. By Chapter 7 she was the person everyone asked for numbers, because she asked good questions about the data and fixed the reports she inherited. That's an internal move in progress: the job title hasn't changed yet, but the work already has.

8.9 Using the tree: pick your door, then learn backward

Put the chapter together and planning your next step takes four moves.

1. **Pick one door.** Choose the role you want next, not the one you want in ten years. Use Chapter 7 and section 8.4 to check that you'd enjoy the day, not only the title.
2. **Find its keys.** Take the role's column in the skills matrix (section 8.3), then check it against five real job descriptions for that role in your city (section 8.5).
3. **Score yourself and list the gaps.** Use the scoring in section 8.5. Order the gaps: must-haves first, then the useful skills that appear most often in the postings.
4. **Map gaps to chapters and projects.** Use the pathways table (section 8.7) to find where each gap is taught, and plan one project that shows each must-have skill.

Then apply when your must-have coverage is strong, and keep climbing on the job. When you're settled, pick the next door.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Learning tools because they're trending	Many half-finished courses; no door you're ready for	Pick one door and learn its keys first (section 8.9)
Trying to learn the whole tree before applying	Two years of study, no applications	A role sits at the first tier that makes you credible; apply when you get there
Treating tiers 3 and 4 as a ladder	Feeling you must do data science before engineering (or the reverse)	They're peers; choose by the work you enjoy
Skipping the analyst core	Models or pipelines built on questions you can't frame or data you can't check	Build tier 1 properly; every branch rests on it
Reading "nice to have" as "must have"	Never applying, because you don't meet every line	Score must-haves strictly; treat nice-to-haves as tie-breakers
Reading "must have" as "nice to have"	Many applications, few shortlists	Close must-have gaps before applying widely
Searching only by job title	Missing roles called "MIS executive" or "process excellence analyst"	Search by the work and tools too (section 8.4)
Comparing offers by headline CTC	Accepting the "bigger" offer and getting less in hand	Ask for the breakup; compare fixed pay and the work
Treating an online salary figure as a promise	Disappointment, or turning down fair offers	Use ranges from two sources, note the date and sample size (section 8.6)

Mistake	Symptom	Fix
Building generic projects	Portfolio looks like everyone else's	Use realistic business questions, ideally in your own domain
Switchers hiding their old career	CV looks like a fresher's	Rewrite past work as data work; your domain is an advantage
Waiting for an internal role to be announced	Openings filled before you hear of them	Automate your work, volunteer analysis, and tell your manager your goal

In the real world: Farah's internal move

Farah Khan had been a sales executive at Riverstone for four years. She knew the hospitality customers better than anyone: which caterers ordered before wedding season, which hotel buyer needed a call before Diwali. When Anita pinned the new *Data Analyst (Sales Analytics and Automation)* posting on the notice board, Farah read it three times and then walked past it.

Meera noticed. At lunch she asked, "Are you going to apply?"

"I'm a sales executive," Farah said. "I use Excel. That's not the same thing."

"Let's check," Meera said, and pulled the posting up on her laptop. They went through it the way section 8.5 describes. Farah scored herself: 2 for spreadsheets ("I run my whole customer list in Sheets, with lookups and a pivot"), 2 for written communication, 2 for CRM experience. Then 1 for SQL, because she'd done an online course but never used it on real data; 1 for Apps Script, because she'd once recorded a macro to format her weekly report; and 0 for a BI tool, automating a report, Python, and forecasting.

Farah looked at the total, 13 of 28, and shrugged. "Less than half."

"Look at the must-haves instead," Meera said. "You're at half of those already. And look at *which* half you have. Anita can teach someone Power BI in a few months. She can't teach someone which caterers go quiet in the monsoon, and why."

They turned the gaps into a plan. Meera was careful not to promise a timeline; Farah had a full-time job, and the honest answer was months, not weeks.

- **SQL to working level:** Chapters 12 and 13 of this book, practiced on Riverstone's own practice databases, and then on questions from her own customers.
- **A BI tool:** Chapter 16, building a dashboard of her own region's customers.
- **One automated report she could show:** her weekly customer follow-up list, rebuilt as a Google Sheet that refreshes itself and emails her the list every Monday (Chapter 19).

Farah also asked the question she'd been avoiding. "What happens to my sales targets while I'm doing this?"

"Talk to Anita," Meera said. "Tell her you're aiming for the role. She'd rather grow someone who knows the customers than hire a stranger."

The conversation with Anita was shorter than Farah expected. Anita didn't promise her the job. The posting would stay open, and she'd interview other candidates on the same must-haves. But she agreed that Farah could spend Friday afternoons helping Meera with the at-risk customer list, and she asked Farah to bring the Monday email to the next sales review once it worked.

Six weeks later, the email worked. It wasn't sophisticated: a query, a sheet, and a time-driven trigger. But every Monday at 9 a.m. it listed Farah's customers who hadn't ordered in their usual rhythm, and in the first month it caught two she would have missed. At the sales review, Vikram asked if his team could have the same email.

What made this work.

- **Farah scored herself against the real posting**, not against a vague idea of "being technical". Her gaps became three specific items.
- **She used her domain as an advantage**, and built her first project on her own customers.
- **She made her goal visible**, and turned an internal move from a hope into an agreement about what evidence would count.
- **Nobody over-promised**. The role wasn't guaranteed, and the timeline was honest. The evidence she built would count here or anywhere else.

Tools

- **A spreadsheet** (Excel or Google Sheets) for your skills matrix and gap score, or paper if you prefer.
- **Two or more job portals**. Save postings as PDFs or screenshots; they disappear.
- **Two salary sources that use different methods**, such as PayScale (self-reported) and Indeed (job postings), plus your city filter. Note the date on every figure you record.
- **Companion file**: `skills_matrix.xlsx`, the matrix in section 8.3 as a spreadsheet you can filter and extend.

The project: your door plan

Goal: a two-page plan for your next role, grounded in real job postings. It's the most useful document you'll write in Part I, and a good one makes interviews easier, because you'll already know what the employer wants and what you can show.

Option A: your own target. Use the role you want next, in the city or remote market you'll apply to.

Privacy reminder. If you're planning an internal move, don't include confidential details about your employer, colleagues, or internal postings in anything you share publicly.

Option B: Farah's plan. Use the Riverstone posting in section 8.5 and Farah's scores, and write the plan she'd bring to Anita.

Steps

1. **Pick one door.** Write the role and one sentence on why, using the day in the life in section 8.4.
2. **Collect five job descriptions** for that role. For Option B, use the Riverstone posting plus four real postings for similar roles.
3. **Decode each one** with the seven steps in section 8.5. Highlight must-haves in one color and nice-to-haves in another.
4. **Build a skills table** in a spreadsheet: skills down the side, postings across the top. Mark each skill as must, nice, or absent in each posting. Add a column counting how many postings list each skill as a must-have.
5. **Score yourself** 0, 1, or 2 on every skill, and calculate your score and must-have coverage for the posting you like most.
6. **List your gaps in order:** must-haves that appear in most postings first.
7. **Map each gap to chapters and one project** that would prove the skill, using the pathways table in section 8.7.
8. **Check pay** for the role using the steps at the end of section 8.6, and record the range, sources, dates, and sample sizes.
9. **Choose your entry route** (section 8.8) and write the three things you'll do differently because of it.

Stretch goals

- Repeat steps 4–6 for a second, higher door, and mark which gaps the two doors share.
- Ask one person in your target role to review your plan, and note what they'd change.
- Set a date, three months out, to re-score yourself against the same postings.

You've got it when...

- ☐ You can explain "skills are keys, roles are doors" and why it changes what you learn next.
- ☐ You can name the seven tiers in order, say which part of the book teaches each, and explain why tiers 3 and 4 are peers.
- ☐ You can place each of the ten roles at the first tier that makes someone credible for it.
- ☐ You can read your target role's column in the skills matrix and list its core skills.
- ☐ You can describe an ordinary day in at least three roles, and say which you'd enjoy.
- ☐ You can decode a job description into title, duties as outputs, hidden skills, must-haves, and nice-to-haves.
- ☐ You can score your fit and calculate your must-have coverage.
- ☐ You can read a salary table with percentiles, and explain why two sources disagree.
- ☐ You can explain CTC versus in-hand pay.
- ☐ You can choose a reading pathway through this book for your goal.
- ☐ You've written down your next door and its three most important gaps.

Recap

- **Skills are keys; roles are doors.** Learn skills for the roles they unlock, not because they're popular.
- The **career tree** has seven **tiers** that match the book's parts: foundations (0), the analyst core (1), advanced analytics and analytics engineering (2), data science (3), engineering and integration (4), production ML and AI (5), and architecture and leadership (6). **Tiers 3 and 4 are peers.**
- A role sits at the **first tier that makes you a credible applicant**, and you keep every skill below it.
- The **skills matrix** shows each role's **core** and **useful** skills. SQL helps open every door; machine learning is core for only two.
- A **day in the life** matters as much as the matrix: choose work you'd enjoy on an ordinary day.
- **Decoding a job description** means finding the track and tier, turning duties into outputs, spotting hidden skills, and separating **must-haves** from **nice-to-haves**. **Must-have coverage** is the most useful fit score.
- **Salary figures** differ by method (self-reported versus job postings), date, sample size, city, and company. Read **percentiles** and **medians**, compare two sources, and know **CTC** from **in-hand** pay.
- **Reader pathways** show which parts of this book to read fully or skim for each goal, and automation grows with every tier.
- There are three **entry routes**: **fresher**, **career switcher**, and **internal move**. They end at the same door with different evidence.

Practice exercises

Warm-up

1. List the seven tiers in order, with the part of the book that teaches each. Which two tiers are peers?
2. Place each role at the first tier that makes someone credible for it: (a) automation analyst, (b) analytics engineer, (c) ML engineer, (d) data architect, (e) business analyst, (f) integration engineer.
3. Using the skills matrix in section 8.3: (a) For which roles is machine learning a core skill? (b) For which roles is Python a core skill? (c) Which skill is core or useful for every role?

Core

4. Classify each line from a job posting as a must-have, a nice-to-have, a hidden skill, or a signal about who can apply: (a) "Experience with Tableau or Power BI." (b) "Exposure to Python is an advantage." (c) "You'll work with regional managers to agree monthly targets." (d) "Final-year students with internship experience may apply." (e) "Strong SQL is essential."
5. Rahul Mehta, from Riverstone's sales team, scores himself against the Riverstone posting in section 8.5. Must-haves: SQL 0, spreadsheets 2, BI tool 1, automated a report 1, written communication 1. Nice-to-haves: Python 1, VBA or Apps Script 0, forecasting 2, CRM or ERP 2. Calculate his total score out of 28, the percentage, and his must-have coverage. Which single gap would you tell him to close first, and why?

6. Using the data analyst row in section 8.6: (a) Say in one sentence what "about ₹2,89,000 to ₹10,00,000, 10th–90th percentile" tells you. (b) Calculate how much higher average total pay is for data analysts with 1–4 years than for those under a year, as a percentage. (c) PayScale's chart marks ₹577k as the median. What does a median tell you that an average doesn't?
7. A friend says: "Indeed says data analysts earn ₹6,29,019 on average, but PayScale says ₹5,77,472. One of them must be wrong." Explain why both can be right, giving two differences between the sources.
8. Explain the difference between CTC and in-hand pay in three sentences, as you would to a friend comparing two offers.
9. Using the reader pathways table, list what someone aiming to be a BI developer should read fully, what they should skim, and which interview chapters they should prepare with.

Stretch

10. Here's a posting: "Data Scientist (Fresher). Must know Excel, SQL, Power BI, Tableau, Python, R, Spark, Hadoop, Kafka, AWS, Azure, GCP, TensorFlow, PyTorch, and LLMs. Duties: daily data entry of invoices and preparing weekly MIS reports for the accounts team." List at least three red flags, say what the job really is (role and tier), and write two questions you'd ask before applying.
11. Neha Kulkarni, one of Riverstone's sales executives, has strong Excel skills and three years of customer contact, and wants to become a data analyst. Recommend an entry route and write a five-step plan using section 8.8 and section 8.9. Include which chapters of this book close which gaps.
12. Using the skills matrix, count the skills that data analyst and BI developer both need at least some of (core or useful in both columns), and do the same for data analyst and data engineer. What do the two counts suggest about which move is shorter, and which skills make the difference?

Think about it (no calculation needed)

13. The tree says a role sits at the *first* tier that makes you credible. Why is that more useful for planning than placing each role at the highest tier its best practitioners reach?
14. Could someone skip tier 1 and go straight into data engineering, for example from a software engineering job? What would they gain, and what would they risk missing?

Key terms

key · door · credible applicant · career tree · tier · MIS (management information system) · BI analyst · senior analyst · skills matrix · core skill · useful skill · leakage · data contract · architecture decision record · job description (JD) · must-have · nice-to-have · hidden skill · must-have coverage · self-reported salary · base salary · total pay · percentile · median · CTC (cost to company) · in-hand pay · reader pathway · fresher · career switcher · internal move

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 9, How Expertise Actually Forms**, is honest about how long each tier takes, and shows how to practice deliberately, build a portfolio as you learn, find feedback and mentors, and get through plateaus.
- **Part II, The Analyst (Chapters 10–27)**, is tier 1: the keys for data analyst, business analyst, BI analyst, and automation analyst. **Chapter 27** turns your projects into a portfolio.
- **Chapters 23–24** teach the stakeholder skills behind "hidden skill" lines in job descriptions; **Chapter 25** covers the business analyst track in depth.
- **Chapter 66** returns to careers from the other side of the desk: building and hiring for data teams.
- **Part VIII, Chapter 68, How Data Hiring Works**, explains the interview rounds for each role, CVs, portfolios, and referrals. **Chapter 81** covers behavioral questions and offer conversations, including negotiating on CTC. **Chapter 82** shows what fair take-home assignments look like.

Answers to practice exercises

1. Tier 0, foundations (Parts 0 and I); tier 1, the analyst core (Part II); tier 2, advanced analytics and analytics engineering (Part III); tier 3, data science and machine learning (Part IV); tier 4, engineering and integration (Part V); tier 5, production ML and AI (Part VI); tier 6, architecture and leadership (Part VII). **Tiers 3 and 4 are peers.** The common wrong answer puts data engineering at tier 3; the tier numbers follow the book's parts, and tiers 3 and 4 are peers.
2. (a) Automation analyst: tier 1. (b) Analytics engineer: tier 2. (c) ML engineer: tier 5. (d) Data architect: tier 6. (e) Business analyst: tier 1. (f) Integration engineer: tier 4. The trap is (a) and (f): both are on the automation track, but at different tiers, because an integration engineer needs the pipeline and API skills of Part V.
3. (a) Data scientist and ML engineer. (b) Data scientist, ML engineer, data engineer, AI engineer, and the automation and integration roles. (c) SQL, which is core for six roles and useful for the other four.
4. (a) Must-have, with a qualifier: either BI tool is acceptable. (b) Nice-to-have: "an advantage". (c) Hidden skill: negotiating targets with managers is stakeholder work, not a tool. (d) Signal about who can apply: a door for students with internship evidence. (e) Must-have: "essential".
5. Must-have points: $(0 + 2 + 1 + 1 + 1) \times 2 = 5 \times 2 = 10$. Nice-to-have points: $1 + 0 + 2 + 2 = 5$. Total **15 of 28 = 53.6%**. Must-have coverage: $5 \text{ of } 10 = 50\%$, the same as Farah's, even though his total is higher, because his extra points come from nice-to-haves. Close **SQL** first: it's a must-have, he scores 0, and it's the one skill every role on the tree uses. His forecasting score won't get him shortlisted without it. The common wrong answer is to celebrate the higher total; must-haves decide the shortlist.
6. (a) The middle 80% of data analysts who reported their pay earn a base salary between about ₹2.89 lakh and ₹10 lakh a year: 10% earn less than the lower figure and 10% more than the upper one. (b) $(₹5,65,999 \div ₹4,13,462 - 1) \times 100 = 36.9\%$ higher. (c) The median is the middle salary: half earn less, half

earn more. An average can be pulled up by a few very high salaries, so the median is often the better picture of a typical salary. Here PayScale's average and median round to the same ₹577k, so the few high earners aren't pulling the average far.

7. Both can be right because they measure different things. (1) **Method:** Indeed's figure comes from salaries advertised in job postings; PayScale's comes from salaries people report themselves. (2) **What's counted:** the PayScale figure quoted is average base salary, while advertised figures may include other pay. The sources also differ in **sample** (605 postings versus 2,389 profiles), **date** (updated 20 September 2026 versus 10 July 2026), and the mix of experience and cities. The gap is ₹6,29,019 – ₹5,77,472 = ₹51,547, and $₹51,547 \div ₹5,77,472 = 8.9\%$, which is small next to the range within either source.

8. A sample answer: "CTC is everything the company says it spends on you in a year, including things like the employer's provident fund share and bonuses that aren't guaranteed. In-hand pay is what actually reaches your bank account each month, after tax and your own deductions, so it's always less than CTC divided by 12. When you compare offers, ask for the breakup and compare the fixed, monthly in-hand amount, not only the CTC headline."

9. **Read fully:** Parts 0, I, and II, plus Chapters 28 and 32. **Skim:** Chapters 45–49, 51, and 63. **Interview chapters:** 68, 69, 70, 71, 77, 78, and 81.

10. Red flags: (1) a **tool list far longer than the duties**, covering every tier from spreadsheets to cloud to LLMs; (2) a **senior title with junior duties**: invoice data entry and MIS reports aren't data science; (3) a **fresher role expected to know tools from tiers 1–5**, which nobody entering the field has. The real job is closest to a **reporting or MIS assistant (tier 0 to tier 1)**, with some data entry. Useful questions: "What would I produce in my first three months?" and "Which of the listed tools are used by the team today?" Other good questions ask about who the role reports to, or whether there's a path into analysis. The job might still be a reasonable first step if the pay and learning are fair, but not under the belief that it's a data science role.

11. Recommended route: an **internal move**. Neha already works at Riverstone, knows its customers, and has the posting in section 8.5 in front of her, as Farah did. If no role opens there, the same plan works as a **career switcher** route into a sales analyst role elsewhere, with her customer knowledge as her advantage. A five-step plan: (1) **Pick the door:** data analyst, ideally sales analytics. (2) **Find the keys:** the data analyst column (business sense, communication, spreadsheets, SQL, BI) checked against five postings. (3) **Score and list gaps:** likely SQL (Chapters 12–13), a BI tool (Chapter 16), and one automated report (Chapters 19–20); her spreadsheets and business sense are strengths. (4) **Build evidence on her own domain:** automate her weekly customer report and build a dashboard of her region, using non-confidential or practice data if she'll show it publicly. (5) **Make it visible:** tell her manager her goal, volunteer for an analysis nobody owns, and re-score herself in three months. Any plan that orders must-have gaps first and ties each to a project is acceptable.

12. **Data analyst and BI developer: 7 shared skills** (business and domain sense, communication and storytelling, spreadsheets, SQL, BI and dashboards, spreadsheet and report automation, requirements and process mapping). **Data analyst and data engineer: 4** (business sense, communication, SQL, and Python). Analyst to BI developer is the shorter move: the analyst already has most of what's needed,

and the main new core skill is data modeling. Analyst to data engineer needs several skills the analyst column doesn't have at all: data modeling, pipelines and orchestration, cloud, APIs and integration, and Git.

13. Because it tells you when you can apply. If each role were placed where its best people end up, a data analyst would sit at tier 6 (some senior analysts have architect-level judgment), and every beginner would conclude they need years of study before applying for anything. Placing roles at the first credible tier gives each door a clear, reachable set of keys, and the rest can be learned on the job, which is how most people climb.

14. Yes, it's possible, and common. A software engineer already has much of what data engineering needs: Python, Git, cloud, APIs, and software habits. They'd gain speed. They'd risk missing the tier 1 skills that make data engineering *useful*: business sense, understanding what the numbers mean, knowing what analysts need, and the habit of checking a total against a source. A data engineer without them can build fast, reliable pipelines that deliver the wrong numbers on time. The fix is to learn the analyst core alongside the move, not to skip it.

Chapter 9. How Expertise Actually Forms

Chapter at a glance

You will learn to: estimate how long a stage of your learning will take, from hours and your real weekly schedule · explain why a long timeline is an advantage, not a punishment · combine the three ingredients of expertise: study, projects, and feedback over time · design deliberate practice sessions instead of hours of passive study · build a portfolio from the work you do while learning · find feedback, peers, and mentors, and ask for help in a way people say yes to · recognize a plateau in your own practice log and change what you do about it.

Before you start: Chapter 8 (the career tree, and choosing your next door).

Time needed: 2–3 hours, including the exercises. The project runs alongside your learning for 12 weeks.

Tools: a spreadsheet or notebook for a practice log. Nothing to install.

Practice data: Farah Khan's fictional 12-week SQL practice log, and the time estimates printed at the start of Chapters 12 and 13. Every calculation shown is checked.

Why this matters

Chapter 8 told you *what* to learn and in what order. This chapter is about *how long* it takes, and *how* the learning turns into ability. It's the chapter that protects you from two opposite mistakes.

The first mistake is quitting too early. In month four, you'll have worked hard, finished chapters, and still feel slow when you open a real dataset. If you believe the promise that anyone can become a data scientist in twelve weeks, that feeling will look like failure. It isn't. It's the normal middle of learning something difficult.

The second mistake is stopping too soon. In year two, you'll be good at your job, and the work will feel comfortable. Comfort is pleasant, and it's also the point where many people stop improving without noticing.

Both mistakes come from not knowing how expertise forms. Once you know, you can plan realistically, practice in a way that works, collect evidence as you go, and recognize a plateau as a signal to change your practice rather than a reason to give up.

In plain English

Think about learning to cook.

- You can read recipes and watch videos. That's **study**. It teaches you what should happen, but on its own it doesn't get dinner on the table.

- You start cooking real meals, with real ingredients that don't behave like the video. The dal sticks to the pan; the rice is undercooked. That's a **project**, and it's where study turns into skill.
- Your family tells you the dal needs less salt. An aunt watches you and says you're adding the tadka too early. Over months, you cook the same dishes many times and start adjusting without thinking. That's **feedback and time**, and it's what turns skill into judgment.
- Some evenings you cook what you already know. That's comfortable, and you don't get better. Other evenings you try one dish that's slightly beyond you, and pay attention to exactly what went wrong. That's the difference between **naive practice** and **deliberate practice**.
- After a few months, your cooking stops improving for a while, however often you cook. That's a **plateau**. The fix is usually to change what you practice, not to cook more of the same.
- If you photograph your best dishes and write down what you changed, you end up with a record of what you can do. That's a **portfolio**.

Everything in this chapter is a version of learning to cook, applied to data work.

9.1 The honest timeline

Let's be plain about time, because a great deal of advertising is not.

Reaching real expertise in this field takes years, not weeks. Becoming a job-ready analyst (tier 1) is a matter of months of steady work for most people who start from zero. Each tier above adds more study, and the top of the tree isn't mainly about study at all. Architects earn their judgment by designing real systems, watching some of them fail, and being responsible when something broke on a Friday night.

No book, course, or bootcamp changes that arithmetic. A course can make your study efficient and your projects well chosen. It can't give you the years of feedback that turn skill into judgment. Anyone who promises to compress a decade into ten weeks is describing a brochure, not a career.

Estimating your own timeline

"Months" and "years" are vague, and vague plans get abandoned. So estimate with numbers you can check.

Every chapter in this book starts with a *Time needed* estimate. Chapter 12, *Databases & SQL Foundations*, says 19–23 hours of reading and practice. Chapter 13, *SQL for Real Analysis*, says 15–20 hours. Together, that's **34–43 hours** to work through the book's two core SQL chapters properly, with exercises.

Now divide by the hours you can *really* give each week, not the hours you wish you had:

Hours per week	Weeks for Chapters 12 and 13
6	$34 \div 6 = 5.7$ to $43 \div 6 = 7.2$ weeks
10	$34 \div 10 = 3.4$ to $43 \div 10 = 4.3$ weeks

At 6 hours a week, a realistic amount alongside a full-time job, the SQL chapters alone take about six to seven weeks. Part II has eighteen chapters. Working through all of it at that pace is a matter of many months, and that's before the extra practice that makes SQL fluent rather than familiar. Chapter 6's hours table uses the same Time needed lines, so your plan and this chapter agree.

Watch out: chapter hours are not fluency hours. Finishing Chapters 12 and 13 means you can write the queries they teach. Being quick and confident on a messy real dataset takes more: many more questions, answered on data you didn't design. Plan for the chapter hours, then plan for practice beyond them. Section 9.4 shows how to make those hours count.

Try it. Write down the hours you gave to learning last week, counted truthfully (not the hours you had planned). Multiply by four. That's your realistic monthly budget, and the number to plan with.

9.2 Why the long timeline is good news

Here's the reframe that makes the long timeline an advantage instead of a punishment.

Because it's slow, it's defensible. A skill anyone can pick up in a weekend, everyone picks up in a weekend, and it earns no premium. The very thing that makes this career demanding is what makes it valuable. The years are the moat around your work.

Because it compounds, early effort pays for decades. SQL you learn in month one is still in use in year ten. Chapter 8's skills matrix showed SQL helping to open every door on the tree. Foundations don't expire; they become what everything else stands on.

Because it's layered, you're employable the whole way up. You don't wait years for the payoff. Finish tier 1 and you can be hired as an analyst. Finish tier 4 and you can be an engineer. Every tier is a real job, not only a checkpoint, so you earn while you climb.

9.3 The three ingredients of expertise

Expertise in data work isn't made from study alone. It's an alloy of three ingredients, and leaving any one out gives a weak result.

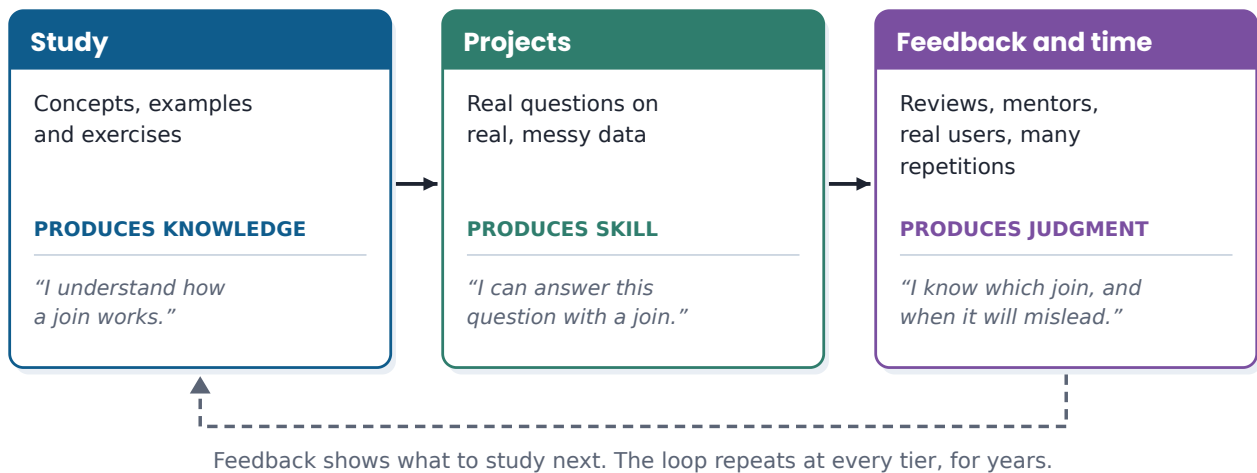


Figure 9.1 – The three ingredients, and what each produces. Most people who stall have plenty of the first and too little of the other two.

1. **Study** gives you **knowledge**: the concepts, from books like this one. It's necessary, but on its own it produces people who can *talk* about data and not *do* it.
2. **Projects** turn knowledge into **skill**: building real things with real, messy data, where nothing matches the example. This is the ingredient most people shortchange, and the one that matters most.
3. **Feedback and time** turn skill into **judgment**: code reviews, mentors, stakeholders who question your numbers, and many repetitions. It can't be rushed.

The three form a loop. Feedback shows you what you don't understand, which tells you what to study next, which you then use on the next project.

People who plateau are usually over-supplied with the first ingredient. They read endlessly and build rarely, because reading feels productive and building feels exposing. Don't be one of them. **Study enough to start building, then build.**

9.4 Deliberate practice

Not all practice is equal. Two people can each spend 100 hours "doing SQL" and end up far apart, because of *how* they spent those hours.

What deliberate practice is

The idea of **deliberate practice** comes from research on expert performers, most associated with the psychologist K. Anders Ericsson, whose studies of musicians in the early 1990s compared how the best and the merely good had practiced. Deliberate practice is practice designed specifically to improve performance, rather than practice that repeats what you can already do.

It has five features.

- **A specific goal.** Not "do some SQL", but "work out, by hand, which customers in a list of 20 have never ordered".
- **Difficulty slightly beyond your current ability.** Hard enough that you might fail, not so hard that you can't tell why.
- **Full attention,** for a limited time. Short, focused sessions beat long, distracted ones.
- **Immediate feedback.** You find out quickly whether you were right, and *why* you weren't.
- **Repetition with refinement.** You try again, changed by what the feedback told you, and you come back to it later.

Naive practice is the opposite: doing what's comfortable, without a goal or feedback, and counting the hours.

	Naive practice	Deliberate practice
Goal	“Do some SQL tonight”	“Build a monthly total of your spending log and check it against the sum of every row”
Difficulty	What already feels comfortable	Slightly beyond what you can do unaided
Attention	Half-watching a video, phone nearby	Full focus for a short, fixed block
Feedback	None, or “it ran, so it’s right”	Compare with a known answer; find why it differs
Repetition	Move on after one success	Redo it tomorrow without looking; vary it
Record	Nothing written down	A log: what was hard, what to try next

Same hours, different results: deliberate practice spends them where you’re weakest, and checks every attempt.

Figure 9.2 – Naive and deliberate practice. The hours can be identical; what you do in them is not.

What the research does and doesn't say

You may have heard that expertise takes "10,000 hours". That number was popularized by Malcolm Gladwell's book *Outliers* (2008), drawing on Ericsson's research. Ericsson himself later objected that the number was an average from one group of musicians, not a threshold, and that the *kind* of practice mattered far more than the count.

Later research also found that practice isn't the whole story. A 2014 meta-analysis by Brooke Macnamara, David Hambrick, and Frederick Oswald, combining many earlier studies, found that the amount of deliberate practice explained 26% of the differences in performance in games, 21% in music, 18% in sports, 4% in education, and less than 1% in professions. The authors concluded that deliberate practice is important, but not as important as had been argued. Researchers still disagree about how to measure practice fairly, so treat the exact percentages with care.

Two honest conclusions follow for you.

- **Practice matters, and its quality matters most.** Nobody in the research became good without a great deal of practice.
- **Practice isn't everything, especially at work.** In professions, what you practice on (real problems, real consequences, real feedback) and who you learn *from* matter a great deal. That's why projects and feedback sit beside practice in the three ingredients, and why the rest of this chapter covers them.

Simplification note. The research above studied chess, music, sport, education, and various professions, not data work specifically. Its lessons transfer as sensible guidance, not as measured facts about analysts.

Deliberate practice for data skills

Here are practice methods that fit the five features, and how this book supports them.

Method	What you do	Where the book helps
Predict, then run	Before running a query or formula, write down the result you expect. Then run it and explain any difference.	<i>Predict-the-result</i> prompts in exercises; the "trap" examples in Chapters 12–13
Rebuild without looking	The day after a worked example, rebuild it from the question alone, without the book open.	Every "plain question first" example
Retrieval before rereading	Before rereading a section, write down what you remember. Then check.	<i>Recap</i> and <i>Key terms</i> sections
Spaced repetition	Return to a skill after a few days, then after a week or two, instead of all at once.	<i>Warm-up</i> exercises, and later chapters that reuse earlier skills
Vary the problem	Answer the same question on a different table, or a different question with the same technique.	<i>Core</i> and <i>Stretch</i> exercises; the pattern library in Chapter 13
Hunt for your mistakes	Keep a list of every mistake you make and its cause. Review it weekly.	<i>Common mistakes</i> tables in every chapter
Explain it aloud	Explain a result to someone else, or to a notebook, as if to a manager.	"What to tell Anita" sections

Research on learning has repeatedly found that testing yourself (retrieval) and spreading practice over time (spacing) produce more durable learning than rereading and cramming, even though rereading *feels* more productive. That feeling is the trap: the comfortable method feels like progress precisely because it asks so little of you.

Watch out: AI assistants can turn practice into watching. If an assistant writes the query every time you're stuck, you practice reading answers, not writing them. Use the assistant *after* your own attempt: to compare, to explain a difference, or to suggest a harder variation. Chapter 6 covers learning with AI assistants without letting them think for you.

9.5 Building a portfolio as you learn

A **portfolio** is a small collection of work that shows what you can do. For a fresher or career switcher, it often matters more than a certificate, because it answers the question a hiring manager really has: *can this person do the work?*

The most common mistake is treating the portfolio as something you build *after* learning. Build it *while* you learn. Every chapter project in this book is designed to become a portfolio piece, and Chapter 27 turns the best of them into a finished analyst portfolio.

What makes a good portfolio piece

A strong piece reads like a short piece of real work, not a tour of a tool. It fits on one page, with the details linked.

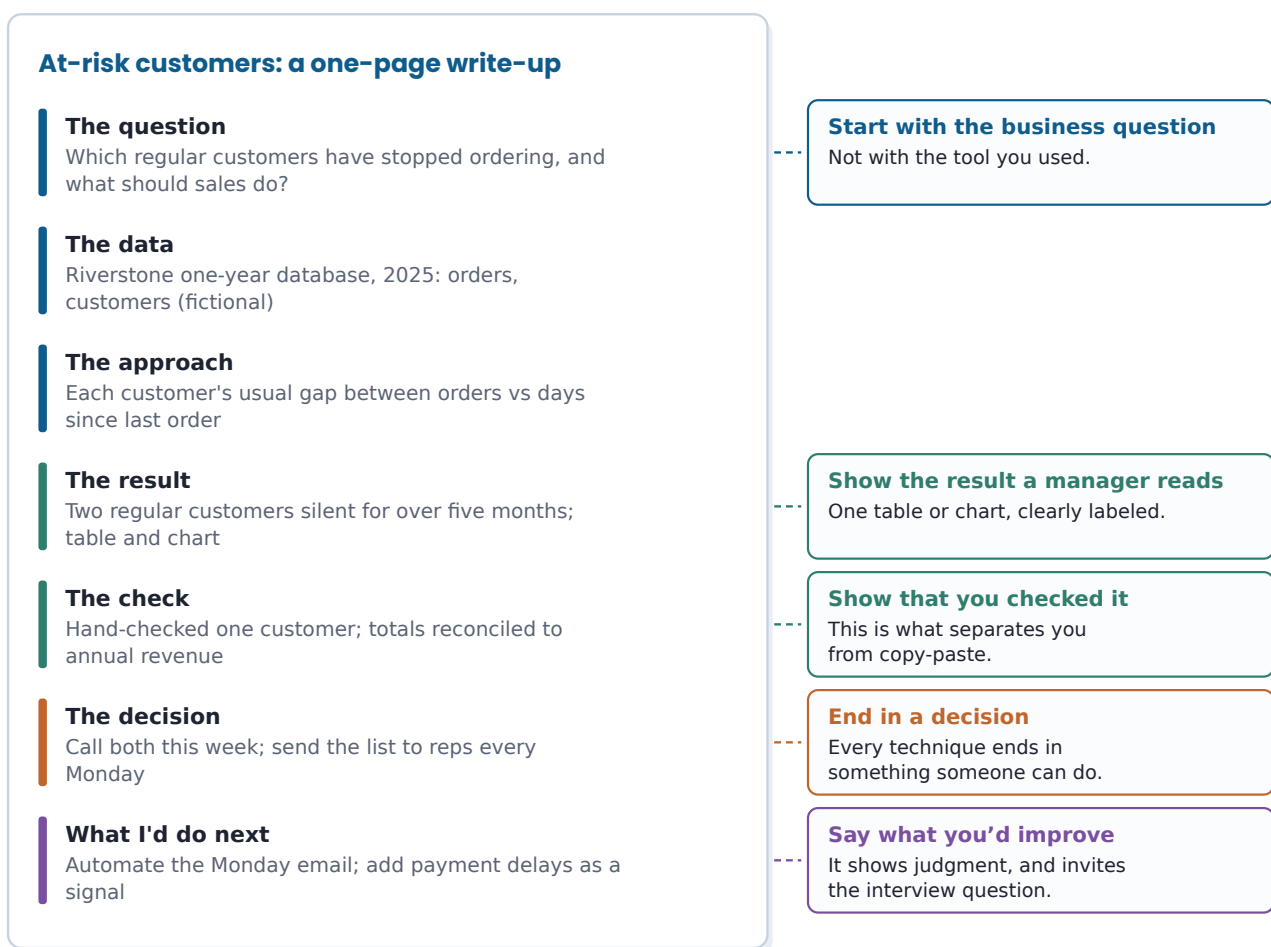


Figure 9.3 – The anatomy of a portfolio piece. The example uses Chapter 13's at-risk customer pattern on the Riverstone one-year database.

1. **The question.** One sentence a manager would ask, in business words.
2. **The data.** What you used, where it came from, and its limits.
3. **The approach.** How you answered it, in plain words, with the code linked.
4. **The result.** One clear table or chart.
5. **The check.** How you know it's right: a hand-checked row, a reconciled total.

6. **The decision.** What someone should do because of it.
7. **What you'd do next.** How you'd improve or automate it.

A portfolio that grows with you

Stage	Pieces worth building	From chapters
Foundations	A classified personal dataset; a process map of a real workflow	1, 3, 7
Analyst core	A cleaned dataset with a data-quality note; a SQL analysis answering three business questions; a dashboard; one automated report	12, 13, 14, 16, 20, 27
Advanced analytics	A tested dbt project; an A/B test analysis	30, 32
Science or engineering	An end-to-end model with honest evaluation, or a monitored pipeline	44, 46-47
Architecture	A design document with decision records	60, 63

Practical rules

- **Quality over count.** Three pieces that each answer a real question beat ten tutorial copies.
- **Use realistic or public data,** never your employer's confidential data. The Riverstone datasets, public government data, and your own records (anonymized) are all fine.
- **Put it where people can open it in one click:** a GitHub repository with a clear README, a shared folder, or a simple web page.
- **Keep the write-up short,** and make the first paragraph readable by a manager who never opens the code.
- **Update it as you climb.** Retire early pieces when better ones replace them.

9.6 Finding feedback and mentors

Feedback is the ingredient you can't give yourself entirely. You can check whether a query runs; it takes someone else to tell you that the question was wrong, or that your chart hides the point.

Where feedback comes from

Different sources give different kinds of feedback. Use several.

Source	What it's good for	Limits
Answer keys and known totals	Is this result correct? Every exercise in this book has a worked answer.	Only for problems with a known answer
The data itself	Reconciling to a trusted total, hand-checking a row	Doesn't tell you if the question was right
AI assistants	Explaining an error; reviewing your code after you've tried	Can be confidently wrong; not a substitute for checking
Study partners and peer groups	Swapping solutions, explaining to each other	Peers may share your blind spots
Online communities	Specific, well-asked technical questions	Slow; answers vary in quality
Colleagues and stakeholders at work	Whether your analysis helps real decisions	You have to show work early, before it's polished
Code review	Readability, correctness, habits	Needs a team or a willing reviewer
Mentors	Direction, judgment, career choices, what to learn next	Their time is limited; their experience is one path

Asking for help well

People say yes to questions that respect their time. A good request for help has five parts.

1. **What you're trying to do**, in one sentence.
2. **What you tried**, with the exact code or formula.
3. **What happened**, with the exact error or output.
4. **What you expected**, and why.
5. **A small example** someone can reproduce, with invented or public data.

Compare "My totals are wrong, please help" with "I'm totalling my spending log by category. My Food total is ₹1,230, but when I add the Food receipts by hand I get ₹1,380. I expected them to match. Here are the eight Food rows." The second question is often answered in minutes, and writing it frequently reveals the answer before you send it.

Finding a mentor

A **mentor** is someone further along the path who gives you occasional guidance. You don't need a famous one. Someone two or three years ahead of you in your target role is often more useful, because they remember the steps.

- **Start with a specific, small request**, not "will you be my mentor?" For example: "Could I have 20 minutes to ask how you moved from reporting into analytics engineering?"
- **Look close to home first**: senior colleagues, former classmates, alumni of your college, people who answered your questions in a community.
- **Do the work between conversations**. Arrive with what you tried since last time, and one specific question. Mentors keep helping people who use the help.
- **Give something back**. Share a useful article, thank them with a result ("your suggestion about the Monday email worked"), and help people behind you. Explaining to others is also excellent

practice.

- **Have more than one.** Different people help with different things: one with SQL habits, another with career decisions.

Real-life example: feedback before it's finished. Many new analysts polish a dashboard for two weeks before showing anyone, and discover in the first meeting that it answers the wrong question. Experienced analysts show a rough version after a day: "Is this the question you meant?" Early feedback feels exposing, and it saves weeks.

9.7 Handling plateaus

Almost everyone who learns a difficult skill hits **plateaus**: stretches where effort continues and visible progress stops. This section comes early on purpose: you'll meet your first plateau long before the end of the book.

What a plateau looks like in data

Farah Khan, the Riverstone sales executive from Chapter 8, kept a simple log while working through the SQL chapters. Each week she recorded her practice minutes and took a **weekly check**: ten new problems at the same level of difficulty, solved without help, scored out of 10. A fixed check matters, because it measures progress on comparable problems, not on whatever she happened to practice.

Week	1	2	3	4	5	6	7	8	9	10	11	12
Practice minutes	180	200	210	220	240	260	270	240	240	250	240	250
Weekly check (of 10)	3	4	5	6	6	6	6	6	7	8	8	9

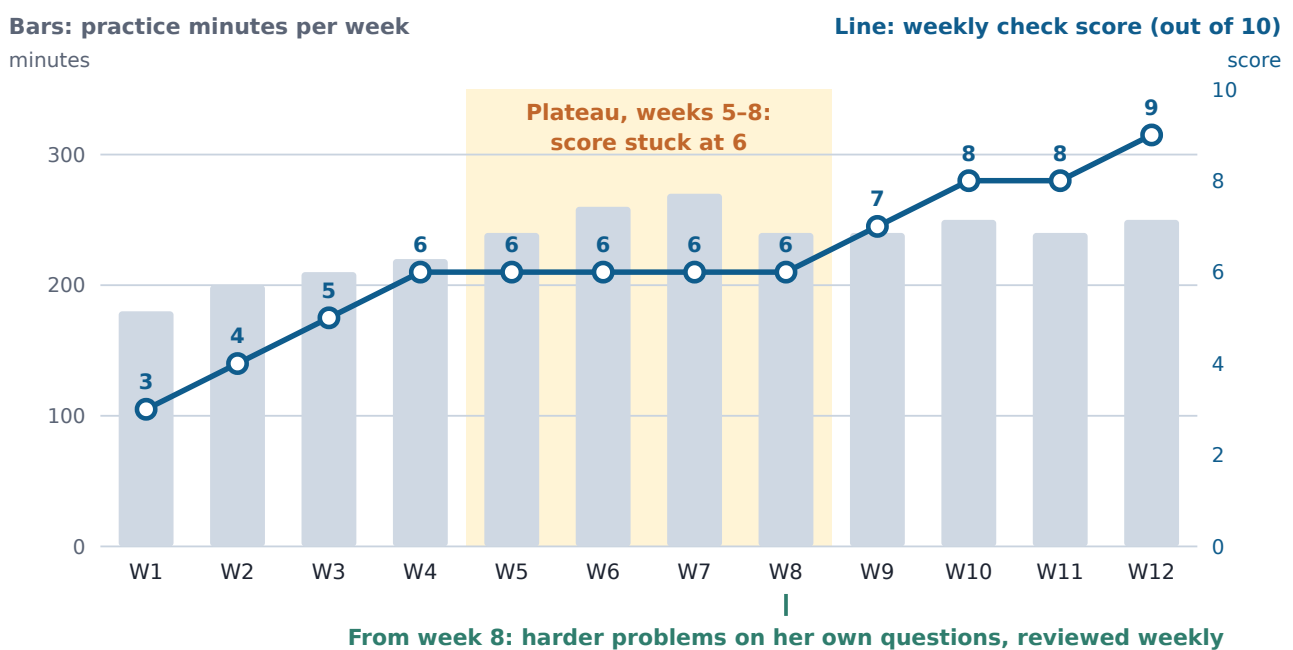


Figure 9.4 — Farah's practice log. During the plateau she practiced more, not less. The score moved again only when she changed how she practiced.

Reading it.

- In weeks 1–4, Farah practiced 810 minutes ($180 + 200 + 210 + 220$), and her score rose from 3 to 6.
- In weeks 5–8, she practiced 1,010 minutes ($240 + 260 + 270 + 240$), and her score didn't move. She responded the way most people do: she practiced *more*, from 220 minutes in week 4 to 270 in week 7, an increase of 22.7% ($50 \div 220$), with no gain.
- In week 8, she changed *what* she practiced (the story in "In the real world" explains how). In weeks 9–12, she practiced 980 minutes ($240 + 250 + 240 + 250$), slightly less than in the plateau, and her score rose from 6 to 9.

Over the 12 weeks, she practiced 2,800 minutes ($810 + 1,010 + 980$), about 46.7 hours ($2,800 \div 60$): slightly more than the 34–43 hours the book estimates for Chapters 12 and 13. That's consistent with section 9.1: the chapter hours get you through the material, and fluency takes a little more.

Simplification note. Farah's log is fictional and deliberately tidy, to make the pattern clear. Real logs are noisier: a bad week, a holiday, a harder check. Look for a flat stretch over several weeks, not a single low score.

Why plateaus happen

- **The practice became comfortable.** Early on, everything is new, so any practice is deliberate. Later, you drift toward problems you can already solve, because solving them feels good.
- **The difficulty is wrong.** Problems that are too simple teach nothing; problems far too hard give no usable feedback.
- **You're missing one underlying idea.** A single misunderstanding (how NULLs behave, what a row represents) can block a whole family of problems.
- **Tool-hopping.** Switching to a new course or tool every few weeks resets you to the beginner stage, which feels like progress but isn't.
- **Life.** Work pressure, illness, family, and fatigue are real. Some flat weeks are rest, not failure.

What to do about a plateau

1. **Measure before you worry.** Without a fixed check, you can't tell a plateau from a feeling. Start logging.
2. **Change the practice, not only the amount.** More of the same rarely breaks a plateau. Raise the difficulty, switch to your own real questions, or add a new kind of feedback.
3. **Find the missing idea.** Review your mistakes list: do several errors share one cause? Go back to that section and rebuild its examples without looking.
4. **Get outside feedback.** Ask a peer or mentor to watch you solve one problem aloud. They'll often spot a habit you can't see.
5. **Teach it.** Explaining a topic to someone else exposes the parts you only half understand.
6. **Rest deliberately.** A planned week off, followed by a return to the same check, is better than grinding tired.

7. **Don't switch doors because of a plateau alone.** Choose a different role because you'd enjoy its work more (Chapter 8), not because the current skill has gone flat for a month. Every door has its plateaus.

9.8 What this means for how you use this book

Every remaining chapter gives you study *and* a project, on purpose, because the two belong together. Treat the projects as the real curriculum and the prose as the briefing before each one.

In practice, that means:

- **Do the exercises before reading the answers**, and predict results before running code.
- **Keep a practice log** from Part II onward: minutes, a weekly check, and your mistakes list.
- **Turn at least one project per part into a portfolio piece** with the seven-part write-up.
- **Show your work early** to at least one other person each month.
- **Expect plateaus**, and treat them as instructions to change your practice.

Be patient with the third ingredient, feedback and time. It's the one this book can't hand you. Only the work, and the years, can.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Believing a timeline advertised by a course	Feeling like a failure in month three or four	Estimate from chapter hours and your real weekly hours (section 9.1)
Planning with the hours you wish you had	Plans that collapse in the second week	Count last week's real hours; plan with those
Studying without building	Can explain a technique, can't answer a question with it	Start a small project as soon as you've learned enough to begin
Rereading instead of testing yourself	Material feels familiar but won't come back without the book	Write what you remember first, then check (retrieval practice)
Counting hours instead of designing practice	Many hours, little change	Use the five features of deliberate practice (section 9.4)
Letting an AI assistant solve every hard step	Can read solutions, can't write them	Attempt first; use the assistant to compare and explain
Building the portfolio "later"	Nothing to show when a good posting appears	Turn chapter projects into one-page pieces as you go
Portfolio of tutorial copies	Every piece looks like everyone else's	Start each piece with a real business question and end with a decision
Asking vague questions for help	Slow or no answers	Use the five-part request: goal, attempt, result, expectation, small example
Waiting until work is polished to show it	Weeks spent on the wrong question	Show a rough version early and ask "is this what you meant?"

Mistake	Symptom	Fix
Asking a stranger "will you be my mentor?"	Polite silence	Ask a specific, small question; do the work between conversations
Practicing more to break a plateau	More minutes, same results (Figure 9.4)	Change the difficulty, the problems, or the feedback
Switching courses or tools at every plateau	Always a beginner at something	Stay with the skill; change the practice

In the real world: Farah's week seven

By the seventh week of her SQL practice, Farah had stopped enjoying it.

She'd started well. The Riverstone practice databases made sense to her, because she knew the customers behind the rows. Her weekly check had climbed from 3 to 6 in four weeks. Then it stuck. Week 5: 6. Week 6: 6. In week 7 she practiced 270 minutes, the most yet, including two late nights after a long sales day, and scored 6 again.

On Friday afternoon, while they worked on the at-risk list, she told Meera she was thinking of switching to a Power BI course instead. "Maybe SQL isn't for me. I've plateaued."

"Can I see your log?" Meera asked.

Farah opened the sheet. Meera looked at the minutes column, then at the problems Farah had been practicing. Nearly all of them were from the warm-up and core exercises of Chapter 12, which Farah had now done two or three times each.

"You're not bad at SQL," Meera said. "You've become very good at these exercises. You know the answers. Your check has new problems, and that's where you're stuck."

She asked Farah to solve one of the week-7 check problems aloud. Farah got halfway and stopped: the problem needed customers with no orders, and her answers kept leaving out customers who had never ordered. Meera recognized it immediately; she'd tripped over the same thing when she started. "Every one you missed last week has the same shape, doesn't it?"

Farah checked her mistakes list. Eight of the twelve problems she'd got wrong in weeks 5 to 7 involved customers or products that didn't appear in another table. It wasn't a plateau in SQL. It was one idea, how to keep the rows that have no match in the other table, that she'd been working around instead of learning.

They changed three things.

- **Harder, real problems.** Instead of repeating chapter exercises, Farah wrote five questions a week about her own customers, the kind Anita actually asked, and answered them on the practice data.
- **One missing idea, fixed properly.** She went back to the section that teaches it, rebuilt that section's examples without looking, and did its trap examples until she could predict every result. (Chapter 12, section 12.10, teaches this exact idea.)
- **Weekly feedback.** Every Friday, she showed Meera one query and explained it aloud, in the ten minutes before they started on the at-risk list.

She kept her minutes about the same, but the late nights stopped.

Week 8's check was 6 again, and Farah nearly gave up on the new plan. Week 9 was 7. Week 10 was 8. By week 12 she scored 9, having practiced slightly fewer minutes in weeks 9 to 12 than during the plateau. And one of her "own questions" (*which hospitality customers ordered before last year's wedding season but not this year?*) turned into the first portfolio piece she was proud of, written up on one page with the check and the decision.

What made this work.

- **Farah had a log with a fixed check**, so the plateau was a pattern she could see, not a feeling.
- **Meera looked at how Farah practiced**, not only how much.
- **The fix was a change of practice**: harder, real problems; one underlying idea fixed; weekly feedback. Not more hours, and not a new course.
- **The practice produced evidence**. The problems she practiced on became a portfolio piece.

Tools

- **A practice log**, in a spreadsheet or notebook. Columns: date, minutes, what you practiced, weekly check score, mistakes and their causes.
- **A weekly check**: a fixed set of new problems at a steady difficulty. The exercises in later chapters, used for the first time, work well; don't reuse problems you've already practiced.
- **A place for your portfolio**: a GitHub account (Chapter 26 shows how to use it), a shared folder, or a simple free web page.
- **A timer**, for short focused practice blocks.
- **Companion file**: `practice_log_template.xlsx`, with Farah's 12 weeks already filled in and a chart that updates as you add your own weeks.

The project: a 12-week learning system

Goal: a simple system that makes your learning measurable, turns it into evidence, and brings you feedback. Start it now and run it alongside Part II.

Option A: your own learning. Use the next skill on your door plan from Chapter 8.

Privacy reminder. If any practice uses data from your job, keep it private and anonymize anything you show others. Use practice or public data for portfolio pieces.

Option B: Farah's next 12 weeks. Plan Farah's next skill after SQL, Power BI (Chapter 16), using her door plan and this chapter.

Steps

1. **Set your weekly budget.** Count last week's real learning hours. Multiply by 12 for your 12-week budget, and compare it with the *Time needed* of the chapters you plan to cover. Adjust the plan,

not the arithmetic.

2. **Build your log** with the columns in Tools.
3. **Design your weekly check:** ten new problems at a steady level, or a timed task (for example, "build this chart from raw data in 30 minutes").
4. **Plan your practice blocks** using at least three methods from the table in section 9.4.
5. **Choose one portfolio piece** to build during the 12 weeks, and outline its seven parts now.
6. **Name your feedback sources:** one peer or study partner, one community, and one person further along the path you'll ask a specific question.
7. **Run it for 12 weeks.** Every Sunday, fill in the log and take the check.
8. **At week 6 and week 12,** chart minutes and scores in the style of Figure 9.4, and write three sentences: what's working, whether you see a plateau, and what you'll change.

Stretch goals

- Keep a mistakes list and group mistakes by cause at week 6. Which single idea would fix the most?
- Explain one topic to someone else each month, and note what you couldn't explain.
- Publish your portfolio piece and ask two people for feedback on the write-up, not the code.

You've got it when...

- ☐ You can estimate how long a stage of your learning will take from chapter hours and your real weekly hours.
- ☐ You can explain why a long timeline is defensible, compounding, and layered.
- ☐ You can name the three ingredients of expertise and what each one produces.
- ☐ You can describe the five features of deliberate practice and turn a vague goal into a specific one.
- ☐ You can say what the research on practice does and doesn't show, including why "10,000 hours" is misleading.
- ☐ You practice by predicting results and rebuilding examples without looking, not by rereading.
- ☐ You can outline a portfolio piece in the seven parts, and you've started one.
- ☐ You can write a request for help that's likely to be answered quickly.
- ☐ You've identified at least two feedback sources and one possible mentor.
- ☐ You keep a practice log with a fixed weekly check, and can recognize a plateau in it.
- ☐ You know what to change when you hit a plateau, and why "practice more" usually isn't it.

Recap

- **Expertise takes years, not weeks.** Estimate your own timeline with numbers: chapter hours divided by the hours you can really give each week.
- The long timeline is an advantage: skills that take years are **defensible**, they **compound**, and every tier is a real job, so you're employable as you climb.

- Expertise combines three ingredients: **study** (knowledge), **projects** (skill), and **feedback and time** (judgment). Most people who stall have too much of the first.
- **Deliberate practice** has a specific goal, difficulty slightly beyond your ability, full attention, immediate feedback, and repetition with refinement. **Naive practice** counts hours.
- Research shows practice matters but isn't everything; the popular **10,000-hour** rule misreads it. In professions, real problems and real feedback matter a great deal.
- **Retrieval** and **spacing** beat rereading and cramming, even though rereading feels more productive.
- Build a **portfolio** while you learn: a few one-page pieces with a question, data, approach, result, check, decision, and next step.
- Get **feedback** from several sources, ask for help with a five-part request, and find **mentors** with small, specific asks.
- A **plateau** is flat results despite steady effort. Measure it with a fixed **weekly check**, then change the practice rather than only adding hours.

Practice exercises

Warm-up

1. Name the three ingredients of expertise, and what each one produces.
2. Rewrite each vague practice goal as a specific, deliberate one: (a) "Get better at Excel." (b) "Practice SQL this weekend." (c) "Learn Power BI."
3. Classify each activity as naive or deliberate practice, and say why: (a) rereading Chapter 12 for the third time; (b) writing down the expected output of a query before running it; (c) watching a two-hour tutorial while replying to messages; (d) rebuilding yesterday's worked example from the question alone.

Core

4. Arjun can study 45 minutes a day, five days a week. How many hours will he have after 8 weeks? Is that enough to work through Chapters 12 and 13 at the book's estimate of 34–43 hours? If not, how many hours short of the lower estimate is he?
5. Compare two plans over a year: (a) 20 minutes every day of the year; (b) one 3-hour session every Sunday for 52 weeks. Calculate the hours in each. Which plan has more hours, and which would you recommend for learning SQL? Give a reason from section 9.4.
6. In Farah's log, her weekly check score went from 3 in week 1 to 9 in week 12. By what percentage did her score increase? Then explain why her *minutes* in weeks 5–7 are not evidence that she was working badly.
7. Rewrite this request for help using the five-part structure from section 9.6: "Pivot table not working in Google Sheets, totals are wrong, help!!" Invent reasonable details.
8. Outline a portfolio piece in the seven parts from section 9.5, for this question: "Which products sell best to hospitality customers, and should we change what we promote to them?"

Stretch

9. A learner's weekly check scores over ten weeks are 4, 5, 5, 6, 6, 6, 6, 5, 6, 6, while their practice minutes rose every week. Describe what the log shows, list two likely causes from section 9.7, and propose three specific changes.
10. Design one week of deliberate practice (five 40-minute sessions) for someone who has finished Chapter 4 and struggles with percentage points versus percent change. For each session, give the goal, the method, and the source of feedback.
11. A friend says: "*Deliberate practice only explains a small share of performance in professions, so practice doesn't matter for data jobs.*" Using section 9.4, explain what's wrong with that conclusion, and what the research does suggest.

Think about it (no calculation needed)

12. Why might a plateau be *more* likely after several months of learning than in the first few weeks?
13. A career switcher has 8 hours a week and wants to spend all of them on courses until they "know enough" to start a project. What would you tell them, and why?
14. What can a mentor give you that an AI assistant can't, and what can an AI assistant give you that most mentors can't?

Key terms

honest timeline · study · project · feedback · knowledge · skill · judgment · deliberate practice · naive practice · 10,000-hour rule · meta-analysis · retrieval practice · spaced repetition · portfolio · portfolio piece · mentor · plateau · practice log · weekly check · mistakes list

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Part II, The Analyst (Chapters 10–27)**, is where you'll use this chapter first. Start your practice log with Chapter 10.
- **Chapter 6, Planning Your Learning**, turns this chapter's timeline into hours and weeks for your own plan, and covers learning with AI assistants without letting them think for you.
- **Chapter 12, section 12.10**, teaches how to keep the rows that have no match in another table, the idea that stalled Farah.
- **Chapter 26** covers Git and GitHub, where your portfolio can live; **Chapter 27** turns your projects into a finished analyst portfolio.
- **Chapter 83, The Long Game**, returns to learning over a whole career, including the plateaus of later years.
- **Part VIII, Chapter 68, How Data Hiring Works**, shows how portfolios are read in hiring, and **Chapter 81** helps you turn your projects and plateaus into strong behavioral interview answers.

Answers to practice exercises

1. **Study** produces knowledge (understanding the concepts). **Projects** produce skill (being able to do the work on real, messy data). **Feedback and time** produce judgment (knowing which approach to use, and when it will mislead).

2. Sample answers; any specific, checkable goal is acceptable. (a) *"Build a pivot table of Riverstone revenue by month and category, and reconcile its grand total to the total in the source data."* (b) *"Answer three questions about customers with no orders on the practice data, predicting each result first."* (c) *"Build a one-page Power BI report with revenue by month and a slicer by segment, from the Chapter 16 dataset, in 60 minutes."* The common weak answer names a topic ("learn pivot tables") instead of a task with a checkable result.

3. (a) Naive: rereading feels productive but doesn't test recall or give feedback. (b) Deliberate: a specific prediction with immediate feedback when the result differs. (c) Naive: divided attention and no feedback. (d) Deliberate: retrieval (rebuilding from memory) with feedback (compare with the worked example).

4. $45 \times 5 \times 8 = 1,800$ minutes = **30 hours**. That's **not enough** for 34–43 hours: he's **4 hours short** of the lower estimate. He could extend the plan by about one more week, or add a longer weekend session. The point of the exercise is to adjust the plan to the arithmetic, not to hope.

5. (a) $20 \times 365 = 7,300$ minutes $\approx$ **121.7 hours**. (b) $3 \times 52 =$ **156 hours**. Plan (b) has more hours, but plan (a) is usually better for learning SQL, because practice spread over many days (spacing) and frequent short, focused sessions produce more durable learning than one long weekly block, where attention fades in the third hour. The best answer might combine them: short daily practice plus a longer weekly project session. Either recommendation is acceptable with a reason from section 9.4.

6. From 3 to 9 is an increase of 6 points: $(9 \div 3 - 1) \times 100 =$ **200%**. Her rising minutes in weeks 5–7 (240, 260, 270) aren't evidence of bad work: she was putting in *more* effort. The log shows that the *kind* of practice had stopped working (she was repeating familiar exercises and working around one missing idea), not that she wasn't trying. The common wrong reading is that a plateau means low effort.

7. A sample answer: *"(Goal) I'm building a pivot table in Google Sheets that totals revenue by month. (Attempt) I selected the data down to row 200 and added Month as rows and Revenue as values, summarized by SUM. (Result) The March total is ₹48,200, but adding the March rows by hand gives ₹52,700. (Expectation) I expected the two to match. (Example) Here's a copy of the sheet with invented data, 12 rows, that shows the same difference."* Writing this often reveals the cause, for example rows outside the selected range or revenue stored as text in some rows.

8. Sample outline. **Question:** which products sell best to hospitality customers, and should promotion change? **Data:** Riverstone one-year database (fictional), orders and products for 2025, hospitality segment only; note it excludes returns and tax. **Approach:** revenue and quantity by product for hospitality customers, compared with all customers; share of hospitality revenue per product. **Result:** one table of products ranked by hospitality revenue, with each product's share compared with its share overall. **Check:** hospitality product revenues add up to total hospitality revenue; one order hand-checked. **Decision:** promote the products where hospitality's share is high but sales are still

small, and stop promoting products hospitality customers rarely buy. **Next:** repeat by quarter to see seasonality, and send it to the hospitality sales executive monthly. Any outline with all seven parts, a check, and a decision is acceptable; the analysis itself is built in Part II.

9. The log shows a **plateau**: scores rose from 4 to 6 by week 4, then stayed at 5–6 for six weeks while practice minutes kept rising. Likely causes: practice has become **comfortable** (repeating problems already mastered), or a **missing underlying idea** is blocking a family of problems; tool-hopping or fatigue are also possible. Three changes: (1) review the mistakes list and look for a shared cause, then rebuild that section's examples without looking; (2) replace repeated exercises with new, harder problems, ideally the learner's own real questions; (3) get outside feedback, for example solving one problem aloud for a peer each week, and reduce minutes if fatigue is part of it.

10. Sample week (any design with specific goals, deliberate methods, and a feedback source is acceptable). **Monday:** goal: explain the difference between a percentage-point change and a percent change in one paragraph; method: write it from memory, then check against section 4.2; feedback: the chapter text. **Tuesday:** goal: for five pairs of rates (for example, a market share that rises from 40% to 50%), write down both changes before working them out; method: predict, then calculate ($50 - 40 = 10$ points; $10 \div 40 =$ a 25% rise); feedback: the calculator. **Wednesday:** goal: rebuild section 4.2's worked example from its question alone; method: rebuild without looking; feedback: compare with the chapter's result. **Thursday:** goal: find three news sentences that report a change in a rate (an interest rate, an unemployment rate, a market share) and rewrite each one with both the points and the percent change; method: vary the problem; feedback: a study partner checks the arithmetic. **Friday:** goal: explain aloud why a loan rate going from 8% to 9% is a rise of 1 point but a 12.5% increase ($1 \div 8$) in the interest you pay; method: explain it to a peer or record it; feedback: the peer's questions, and a note of anything you couldn't explain.

11. The friend misreads the finding. The meta-analysis measured how much of the *differences between people* was explained by the *amount* of deliberate practice they reported; a small share in professions doesn't mean practice is unimportant. Everyone in those professions had already practiced a great deal, which shrinks the differences practice can explain, and on-the-job learning is hard to measure as "deliberate practice". The research suggests that practice matters and its quality matters most, and that in professional work other things also matter: working on real problems, getting real feedback, and learning from experienced people. That's why the three ingredients include projects and feedback, not practice alone.

12. Because early on, everything is new, so almost any practice is at the right difficulty and gives feedback. After a few months, you can already solve the familiar problems, so it's natural to drift toward practice that feels comfortable, and a single missing idea can block progress on the harder problems you now meet. Fatigue and competing commitments also build up over months.

13. Start a small project now, alongside the courses. Knowledge without projects doesn't turn into skill, and it's hard to know what "enough" is until a real problem shows you what's missing. With 8 hours a week, a split such as 5 hours of study and practice and 3 hours on a small project on their own domain gives them all three ingredients, and the project becomes a portfolio piece. The feeling of "not knowing enough" never fully goes away; it's a reason to build, not to wait.

14. A **mentor** can give judgment from experience: which skills matter in your company or city, which role fits you, how to handle a stakeholder, when your question is the wrong question, introductions to people, and encouragement from someone who has been through the same plateaus. An **AI assistant** can give instant, patient explanations at any hour, many variations of practice problems, and a quick first review of code, without using up anyone's time. The assistant can be confidently wrong and doesn't know your situation; the mentor's time is limited and their experience is one path. Use both, and check both.